

残酷共学 · 增量笔记

自 2026-08-10 23:01 BJT 后新增
143 篇

本次增量 · 带索引

点击目录条目可跳转

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-11 23:01 (BJT)

目录

主题章节 → 笔记索引（点击可跳转）

01 AMM机制（29 篇）	8
笔记 001: 1\、固定费恒定 0.25%，无 min fee、无规模折扣——4 档金额费率完全一致（0.250	8
笔记 002: Day 5 打卡 鏈上套利殘酷共學	8
笔记 003: 显示毛利只回答“在两个页面上，参考价格相差多少”。净收益回答“在给定期限、路径和时间内完成所有动作后	8
笔记 004: 略读了panoptic关于把uni v3 lp model成期权的文章，里面很多数学而且我没有期权基	9
笔记 005: 兩池套利模型筆記	14
笔记 006: AMM ;	21
笔记 007: D04 常数乘积 AMM	31
笔记 008: 今天弄清了模拟执行在换币流程里是干什么、要准备什么。	31
笔记 009: 学习记录	32
笔记 010: Day 06 : DEX 套利	32
笔记 011: Universal Router & Permit2	33
笔记 012: 2026-08-11 共学 Day 7 第一周总结 投入约 0.5 小时	34
笔记 013: Day 6 打卡 鏈上套利殘酷共學	34
笔记 014: 案例：Binance Alpha 与链上搬砖——以 dappOS（DOS）为例	35
笔记 015: 新增代币对池子的方向是否可行	36
笔记 016: Day 6（2026-08-11）：跨 CEX/DEX 比價工具——設計日	37
笔记 017: 学习笔记：AMM 价格影响计算（滑点数学）	38
笔记 018: TVL 是池内全部资产的总价值；active liquidity 才是当前价格位置参与报价的流动性参	39
笔记 019: 一、背景与核心思路	40
笔记 020: 真实滑点是「凹曲线」，不是直线——线性模型结构性失配。	40
笔记 021: Day 07 · 2026-08-11 · 第一周复盘：成本模型 v1	40
笔记 022: Day 7 2026.08.11 纯内存报价上线验收 + 报价延迟 6.5x 优化	41
笔记 023: D05 流动性深度、价格冲击与滑点	42
笔记 024: Day 5 DOS 跨链搬砖案例研究	43

笔记 025: 每日学习笔记 CEX-DEX套利报价系统	43
笔记 026: Day 7 · 2026-08-11 · 第一周复盘：从“看见价差”到“算清可执行性”	44
笔记 027: 学习记录	45
笔记 028: 3. 流动性深度	45
笔记 029: 【Day 7 打卡】2026/8/11	46
02 LI.FI与跨链 (33 篇)	47
笔记 030: 链上套利残酷共学 · Day 5 心得	47
笔记 031: 一、RPC 选型：小白别自建，别全付，混合最香	47
笔记 032: 今日打卡	48
笔记 033: Day 6 · 设计可验证的套利信号状态机	50
笔记 034: 链上套利学习笔记 Day 6：成本模型 —— 把直觉变成公式	52
笔记 035: 使用 LI.FI 获取并拆解路线报价	53
笔记 036: August 10, 2026 · Day 6 / 21 · 时间分配：先动手 60min → 后概	54
笔记 037: LI.FI 路由基础	56
笔记 038: markdown	57
笔记 039: 8月9日（周日）—— 拥抱 Agent：Hermes AI Agent 入门与实操	57
笔记 040: 昨天的重点是读真实路径：一条交易从哪里发起，经过哪些 Router、DEX、池子、桥和中间资产，最后	60
笔记 041: D5 学习进展：LI.FI 路径净值与执行证据边界	60
笔记 042: 	61
笔记 043: 今日学习心得：选择 USDT 作为稳定币跨链观察方向	62
笔记 044: 1,000+ Partners in LI.FI's Ecosystem Can Now Acces	62
笔记 045: 套利赛道分析	62
笔记 046: 链上套利与跨链开发	63
笔记 047: Day 7 学习笔记（2026-08-11）	64
笔记 048: 链上套利残酷共学 · Day 6 心得	64
笔记 049: 链上套利残酷共学 · Day 7 打卡（2026-08-11）	65
笔记 050: Day 7（8/11）第一周收官学习总结，素材来自今日 51 位同学精华：	66

笔记 051: Day 7 链上套利残酷共学	66
笔记 052: Day 7 补齐库存再平衡后, 完整净 Edge 仍为负	67
笔记 053: Day 7 · 给"等待"标个价, 然后收口 Week 1	68
笔记 054: 今天算是第一周基础部分的收尾, 开始把前几天学到的跨链、Gas、滑点、RPC 和 API 放到一个真实	69
笔记 055: Day 6 续学: LI.FI Route 费用口径与严格净收益	69
笔记 056: 今天完成了一条从报价证据到接受/拒绝结果的 Paper Trading 流水线。	70
笔记 057: 理解跨链流动性	71
笔记 058: 今天主要学习的是套利机会真正出现之后, 应该怎么处理。	72
笔记 059: 2026-08-11 补打 把 LI.FI Quote / Route 落成可追溯 SQLite\	73
笔记 060: 今天重温了一篇讲加密高频交易的文章 (High-Frequency Trading in Crypto	74
笔记 061: 打卡笔记: LI.FI 路径发现与套利机会实操	74
笔记 062: 链上套利 Phase 1 Review: 套利机会地图	75

03 MEV与抢跑 (12 篇) 76

笔记 063: 今日研究资金费率套利 (现货多+永续空、delta中性收正funding) 的真实执行细节: 并非无风险,	77
笔记 064: Day 7: 全自动套利系统架构 v0.6	77
笔记 065: MEV 是通过决定交易是否被包含、以及以什么顺序执行, 在常规区块奖励和 Gas 费之外可提取的最大价	81
笔记 066: D7 (8/11) 第一周复盘: 完整演练「发现→算净收益→判断」	81
笔记 067: 参考	82
笔记 068: 【8月11日 学习打卡】链上套利 Day 7: 清算速度优化 (事件驱动+预构建+mempool)	83
笔记 069: 链上套利残酷共学 · Day 7: CoW Protocol 批量竞价与 Solver	83
笔记 070: Day 7: 把停止线扩展为候选机会状态机。	85
笔记 071: 今日打卡: 三角套利与 MEV	85
笔记 072: 1Meme池与交易传播 (Mempool & Propagation)	88
笔记 073: 普通用户 DEX 交易规避三明治 (夹子) 攻击完整方案	89
笔记 074: 2026.08.11	90

04 做市与LP (1 篇) 91

笔记 075: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责	91
---	----

05 其他 (1 篇)	91
笔记 076: 有个反直觉的观察：噪声带宽和流动性不是正相关。	92
06 套利框架 (46 篇)	92
笔记 077: 2026.08.10	92
笔记 078: Day 6 笔记：认识半自动对冲套利工具 —— taoli.tools 与 QQQ.BOT	93
笔记 079: TUT事件下，资金费率套利：高资金费率不是单纯的收益机会，也可能是拥挤交易和逼空风险的前兆。	95
笔记 080: 第四天 净收益、模拟与执行决策	96
笔记 081: 今天继续学习 DEX-CEX 套利，重点了解了资金准备的问题。	98
笔记 082: 【2026-08-10 学习打卡】	98
笔记 083: 对冲腿先死了，套利仓就成了单边赌局	99
笔记 084: 今天的学习重点，是把前几天对跨链路径和交易成本的零散观察，整理成一套可以重复使用的研究流程。	99
笔记 085: 实现一次加密套利，系统性的准备和极致效率：	100
笔记 086: 今天主要学习了跨链路径与成本验证	100
笔记 087: 学习内容参考< https://x.com/brucexu_eth/status/2086652810	100
笔记 088: 探讨在 cex-dex 套利中我一直存疑的问题，Day 5：【持仓与对冲风险管理】	106
笔记 089: 之前想到的价差雷达、回测工具、路径模拟和风险面板，看起来像不同项目，实际更像同一套系统里的功能。真正	107
笔记 090: 这是一次明显的原子套利：	107
笔记 091: Delta 中性资金费率套利实现规划	108
笔记 092: 链上套利残酷共学打卡 - 2026-08-10 (Day 6)	108
笔记 093: 套利共学 新手必读链上套利的第一性原理：从价差到可执行净利润\	109
笔记 094: Solana MEV 套利案例分析：一笔 swap 获利 1.73万U	109
笔记 095: 2026-08-08 學習打卡	110
笔记 096: 监控bsc和币安价格差异的功能这样设计：	110
笔记 097: Day 7 — CEX-DEX 套利深度解析	111
笔记 098: TUT (Tutorial) 近期“插针”事件总结与验证	112
笔记 099: Day 6 (8/10) 机会的统计学：频率、幅度、半衰期、竞争	113

笔记 100: 今天学习帖子：《老孙带你搭建价值百万的金融终端系统4:套利监控模块设计思路》 \	113
笔记 101: https://github.com/ZeroNullNone/onchain_arbitrage_	114
笔记 102: 昨天看到群里讨论清算套利和原子套利，说前者更值得开始做，我以为这两个是不一样的，但其实并不是。下面是	114
笔记 103: markdown	115
笔记 104: 今天研究期权 Put-Call Parity（看涨-看跌平价）套利，重点不是预测 BTC/ETH 涨	115
笔记 105: 今天把一个很容易混淆的地方拆开了：看到两个市场之间有价差时，那个价差只能先叫“毛价差”。	116
笔记 106: 今天做了什么：D5 定 2 个候选假设方向并写成可证伪假设卡进证据库：OPP-005 Aave v3	116
笔记 107: Day 7 - 第一周复盘	116
笔记 108: 8 月 11 日打卡 先确认不同市场交易的是不是同一个风险敞口	117
笔记 109: 链上套利共学 · Day7 打卡笔记（08/11）	118
笔记 110: 接入并完善 Lighter、Hyperliquid、Variational 三个平台的 BTC/ET	119
笔记 111: Day06 数据工具与信息源	120
笔记 112: 【8/11 链上套利共学打卡】	121
笔记 113: DEX 间套利（Inter-DEX Arbitrage）	121
笔记 114: Day 7 从价差到净收益：AMM 线性化估算 + 成本模型逐项落地	122
笔记 115: 今天我完成了受控候选路径的首次批量 Paper Trading 验证。	122
笔记 116: 链上套利残酷共学 2026-08-11 打卡	123
笔记 117: 携手 AI 学套利	123
笔记 118: 今天继续学习 DEX-CEX 套利，尝试把一笔交易的成本完整列出来。	124
笔记 119: 套利基础学习打卡 什么是 Basis？从基差理解现货与合约套利	124
笔记 120: Cex-Dex 套利熔断监控：\	129
笔记 121: 永续合约学习日报	129
笔记 122: 8月11日 套利系统真正的地基：状态一致性	132

07 学习计划（3 篇） 133

笔记 123: Day 4 The Graph、GraphQL 与历史链上数据	133
笔记 124: 调整学习计划，解码 Vivek 样本 6389... 的 Raydium → Orca 两腿。结果：	156
笔记 125: Day 5 RPC、Node 与 Direct On-chain State	157

08 实盘与数据 (11 篇) 185

笔记 126: Day 6 — Hyperliquid HIP-3 发现 + 四平台实测 + 方法修正	185
笔记 127: Day 6 - 2026-08-11 : 推翻完整集主线, 建立证据型共学仓库	187
笔记 128: Day 6 Funding 现金流与纸上账本	189
笔记 129: Day 7 (8/11 周二) 完整打卡 · 「看起来赚了≠真的赚了」认知闭环日	189
笔记 130: 阅读相关清算论文	190
笔记 131: 2026-08-11	194
笔记 132: Day 7 打卡 Paper Trading 回测: 六种套利策略 30 天模拟表现与可行性排名	195
笔记 133: 2026年8月11日打卡日记	197
笔记 134: 从验证到最小执行原型——套利策略学习笔记	200
笔记 135: 蒸馏了群友分享的資料, 看看如何加入已有研究中	209
笔记 136: 一、任务拆解	211

09 风险与安全 (7 篇) 212

笔记 137: 今日套利共学总结 基础设施、原子套利与 AI 安	212
笔记 138: LI.FI 學習筆記 (二)	215
笔记 139: 什么LP值得做	221
笔记 140: 个人量化交易系统开发中的可靠性、审计与部署思路学习笔记	221
笔记 141: 第七天学习记录: 今天把 Robinhood Chain 的 Feed V4 机会观察从“看到正返回”	229
笔记 142: 今天继续实际操作 Haas BOT, 并完成了链上套利交易。	229
笔记 143: 2026-08-10\链上套利共学群日报.md	229

笔记 001: 1\ 固定费恒定 0.25%，无 min fee、无规模折扣——4 档金额费率完全一致 (0.250)

作者：Mevin Wang | 时间：2026-08-10 15:01 UTC | 字数：393 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

- 1\ 固定费恒定 0.25%，无 min fee、无规模折扣——4 档金额费率完全一致 ($0.2501\% \pm 0.0007\%$) ；
- 2\ Gas 是固定成本 \$0.014，占比随规模反比下降——\$100 时 0.0137%，\$5000 时 0.0003%，小额最吃亏但绝对值可忽略；
- 3\ 到手率与规模无关 (恒定损失 0.3216%)，不存在「越大越划算」的临界点——因为成本结构是比例费为主；
- 4\ 路径会漂移：opt→arb 方向 eco/polymerStandard 随机切换，默认 quote 可能给出 18 分钟的执行路径——「费用最优 ≠ 综合最优」，执行时间是 Day 3 模型的 capital (资金占用) 成本，Day 6 要正式处理；
- 5\ 稳定币 4 组合全部 0 滑点 (toAmount == toAmountMin)，与 Day 4 判断一致。

笔记 002: Day 5 打卡 | 鏈上套利殘酷共學

作者：roy328line | 时间：2026-08-10 15:11 UTC | 字数：920 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 打卡 | 鏈上套利殘酷共學

今日實作：v3 池接入 tri_scanner

① fee 換算邏輯分叉

v2/Sushi：bps (/10,000)，30 → 0.3%

v3：pips (/1,000,000)，500 → 0.05%

踩坑：v3 fee 如果也用 /10,000，500 會被算成 5% (高估 100x)，完全過濾掉 v3 機會。

② v3 虛擬儲備推算

v3 沒有 reserve0/1，從 sqrt_price_x96 + liquidity 推算：

$$\text{sqrtP} = \text{sqrt_price_x96} / 2^{96}$$

$$\text{virtual_r0} = \text{liquidity} / \text{sqrtP}$$

$$\text{virtual_r1} = \text{liquidity} \cdot \text{sqrtP}$$

在 current tick 附近、Q << pool depth 時，近似誤差 < 1%，足夠快篩用。

③ ev_model.py 重構

- 新增 simulate_tri_arb()、optimal_size_tri() (數值最佳化，三腿無閉式解)

- compute_ev() 修正：net_after 不再 clamp (虧損如實保留，避免 scanner 失去分辨力)

掃描結果

池數：2,098 → 3,043 (+945 v3)

三角路徑：92 → 7,990 (86x 成長)

net_star > \$5.5：19 次/block

但 19 個「機會」全是流動性幻覺：

- Group 1：WETH→USDC→長尾 token→WETH，net ≈ \$61,000，Q ≈ 49 WETH → Q 遠超池實際深度

- Group 2：net ≈ \$3,000-4,000，Q 極小 (3~6 WETH) + 100 bps 高費率池

根本原因：optimal_size_tri 沒有對 Q 做池深度上界約束，在極小流動性的長尾 token 池上會算出荒謬的正 net。

明日待辦 (Day 6)

- 加過濾條件排除流動性幻覺：MIN_Q_STAR = 50、MIN_POOL_TVL = 100,000

- 接 TVL 資料來源

笔记 003: 显示毛利只回答“在两个页面上，参考价格相差多少”。净收益回答“在给定规模、路径和时间内完成所有动作后

作者：Y-del-W | 时间：2026-08-10 15:21 UTC | 字数：1762 | 评分：77

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

显示毛利只回答“在两个页面上，参考价格相差多少”。净收益回答“在给定期限、路径和时间内完成所有动作后，账户真实多了多少”。两者之间隔着成本、执行和风险。

| 层级 | 计算对象 | 常见错误 |

| :----- | :----- | :----- |

| 显示价差 | 参考卖价 参考买价 | 把中点价当成交价。 |

| 可交易毛利 | 某规模下的预计卖出所得 预计买入成本 | 忽略多跳、Gas 和最低收到量。 |

| 预计净收益 | 可交易毛利 所有确定/估计成本 | 把未知项目填 0。 |

| 风险调整后结果 | 预计净收益 失败与延迟的预期损失 | 把“可能失败”当成不会发生。 |

成本

| 成本/风险 | 何时出现 | 最常见的低估方式 |

| :----- | :----- | :----- |

| DEX / 协议费 | 每次 swap、路由或协议动作。 | 只计算第一跳；忘记多跳。 |

| Gas | 提交、批准、执行、失败都可能消耗。 | 只看一条链或只看成功交易。 |

| 价格冲击 | 你的规模推动 AMM 曲线或吃掉订单簿。 | 用小额报价外推大额。 |

| 滑点/报价过期 | 报价到打包期间状态改变。 | 把预估输出当最终输出。 |

| 桥费与交付时间 | 跨链路径的桥接环节。 | 只看桥费，不看等待窗口。 |

| 失败交易 | 最低收到量、Gas 不足、竞争等导致失败。 | 把失败概率与失败 Gas 设为 0。 |

| 资金占用 | 资金在路径中锁定或无法同时使用。 | 忽略时间与机会成本。 |

| 安全/操作缓冲 | 合约、桥、资产版本、人工失误。 | 以为所有可量化风险都被报价覆盖。 |

不确定

| 标签 | 含义 | 第一天式处理 |

| :----- | :----- | :----- |

| 确定 | 页面或协议明确给出，或已知固定。 | 直接纳入。 |

| 估计 | 随网络、规模、市场状态变化。 | 使用保守值，并记录来源与时间。 |

| 未知 | 尚未找到数据，或无法确认。 | 不当作 0；暂停、加缓冲或直接否决。 |

案例

| 项目 | 金额 (USDC) | 说明 |

| :----- | :----- | :----- |

| 显示/预计毛利 | +180 | 10,180 10,000 |

| 买入与卖出 DEX 费 | 60 | 两腿合计 |

| Gas (两笔) | 18 | 按当时估计 |

| 价格冲击与滑点缓冲 | 70 | 按目标规模保守预留 |

| 失败交易预期成本 | 12 | 失败概率 × 可能损失的简化估算 |

| 预计净收益 | +20 | 180 60 18 70 12 |

笔记 004: 略读了panoptic关于把uni v3 lp model成期权的文章，里面很多数学而且我没有期权基

作者：allen123wzh | 时间：2026-08-10 15:26 UTC | 字数：7032 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

略读了panoptic关于把uni v3 lp model成期权的文章，里面很多数学而且我没有期权基础，看的云里雾里。资料主要源于：

1. Panoptic whitepaper <https://arxiv.org/pdf/2204.14232> (<https://arxiv.org/pdf/2204.14232>)
2. 推特 <https://x.com/guil_lambert>
3. Medium <<https://lambert-guillaume.medium.com/>>

文章指出，一个在极窄区间内的LP可以建模成 在区间价位卖出一张put。如果是宽区间，相当于连续卖出了从区间下沿到上沿的一系列put。其中我问了该如何建模range为190-220的nvda/usdc LP与卖出190 put和220 call的区别，是否LP fee可以当作是期权的权利金。AI回答如下：

不完全等同。你这个 NVDA/USDC、190–220 的 Uni v3 LP，最容易误解的地方就在这里：

它不是 short 190 put + short 220 call。

更准确地说，它相当于在 190–220 整个区间内卖出了一条连续分布的 short Gamma / option strip，同时持有一定的 NVDA/USDC 现货头寸。

Panoptic 论文里所谓“LP≈short option”，最直接的情况其实是极窄、接近 single-tick 的 range：如果你把 liquidity 几乎集中在一个价格 (K)，它才会越来越接近一个 strike = (K) 的 covered-call / cash-secured-put payoff。论文 Figure 1 就明确把 single-tick LP 描述为 covered-call-like。

用你这个 190–220 LP 看，就很直观

假定 NVDA 当前 \$205，价格单位是 USDC/NVDA。

你的 LP 会这样变化：

NVDA < 190：LP 100% 是 NVDA

190 < NVDA < 220：随着 NVDA 上涨，LP 不断卖 NVDA、换成 USDC

NVDA > 220：LP 100% 是 USDC

所以它的 Delta 不是在 190 和 220 两个点突然跳变，而是在 190→220 整段区间连续下降：

```
\[
\Delta(S)
=====
L\left(\frac{1}{\sqrt{S}}\right.
-----
\frac{1}{\sqrt{220}}\backslash
\right)
]
```

而区间内：

```
\[
\Gamma(S)
=====
-\frac{L}{2S^{3/2}}\backslash
<0\backslash
]
```

也就是说，190–220 内每个价格点都有 negative Gamma exposure。

而如果你真的卖：

```
\[
\text{short 190 put}+\text{short 220 call}\backslash
]
```

这是一个典型 short strangle。忽略权利金，它在 190–220 中间区域 payoff 是平的，只有跌破 190 或涨破 220 后才开始产生期权损益。

这和 LP 完全不同：

short 190P + short 220C：

```
P/L
-----
190 220
/\
/\
----- S
190–220 Uni v3 LP：
Value
```

```
—
·
·
·
·
·
```

190 220

LP 的曲率恰恰主要发生在 190–220 内部。

更严格地用期权语言说，你这个 LP 甚至可以写成一个很漂亮的静态复制关系。

设下限：

$$P_{a=190}$$

上限：

$$P_{b=220}$$

那么 LP 的价值函数可以写成：

$$V(S) = \begin{cases} L \left(\frac{1}{\sqrt{190}} - \frac{1}{\sqrt{220}} \right) S, & S \leq 190 \\ L \left(2\sqrt{S} - \sqrt{190} - \frac{S}{\sqrt{220}} \right), & 190 < S < 220 \\ L(\sqrt{220} - \sqrt{190}), & S \geq 220 \end{cases}$$

它甚至可以精确地理解成：

$$\boxed{\int_{190}^{220} \frac{L}{2K^{3/2}} \text{Call}(K) dK}$$

也就是说：

持有一部分 NVDA + 卖出从 strike 190 到 220 的连续一篮子 call。

不是：

$$Put(190) - Call(220)$$

而更像：

$$-\text{Call}(190) - \text{Call}(191) - \text{Call}(192) - \cdots - \text{Call}(220)$$

当然每个 strike 的权重并不相等，而大约按照：

$$\frac{1}{K^{3/2}}$$

变化。

这就是为什么 Lambert 后来会用 Gamma Transform 的视角来描述 Uni v3 : liquidity range 本质上是在一段 strike 空间里部署 Gamma，而不是只在上下两个边界部署两个 option。(Medium (https://lambert-guillaume.medium.com/gamma-transforms-how-to-hedge-squeeth-using-uni-v3-da785cb8b378?utm_source=chatgpt.com "Gamma transforms: How to hedge squeeth using Uni V3"))

那你的第二个问题：LP 手续费是不是“权利金”？

这个理解方向上是对的，但也不能严格画等号。

Panoptic/Lambert 的核心思想就是：

$$\boxed{\text{LP fees} \approx \text{short option premium}}$$

论文明确说，Uni v3 LP 在价格穿过 liquidity 区域时不断积累交易费，因此可以把这些 fees 视为随着时间增长的 short-option premium。Panoptic 更进一步，把 option buyer 向 seller 支付的 streaming premium 与对应 Uni v3 liquidity 本应产生的 fees 挂钩。

所以你的 190–220 LP：

$$\int_0^T \text{LP fees}_t dt \approx \text{Option-like premium earned}$$

这个直觉是对的。

但它不是：

$$\text{Premium}_{190P} + \text{Premium}_{220C}$$

原因正是前面说的：你卖的根本不是这两个期权。

更好的表述应该是：

$$\boxed{\text{190–220 LP Fees} \approx \text{190–220 区间内整条 short-Gamma strip 的 streaming premium}}$$

而且 native Uni v3 里手续费不是期权买家付给你的，而是 spot swap trader 付的。Panoptic 才通过协议机制，把 liquidity relocation 与这些 fee flows 转化为 option buyer/seller 之间的 streaming premium。论文对此专门区分了 LP reward 与 option premium。

这里还有一个非常重要、而且对你实际做 NVDA LP 最有用的直觉。

假设 NVDA 一直在：

$$200 \rightarrow 205 \rightarrow 198 \rightarrow 208 \rightarrow 202$$

来回震荡。

你的 LP 会不断：

NVDA 涨 → 卖 NVDA

NVDA 跌 → 买 NVDA

NVDA 涨 → 再卖 NVDA

每次交易都收手续费。

这就是典型：

$$\boxed{\text{short Gamma} + \text{collect premium}}$$

如果 realized volatility 不大，但交易量巨大：

$$\text{Fees} \gg \text{Gamma loss}$$

]

LP 很赚钱。

但如果 NVDA :

$\backslash \backslash$
 $205 \rightarrow 230 \backslash$
]

单边暴涨，你会一路：

$\backslash \backslash$
 $205 \rightarrow 210 \rightarrow 215 \rightarrow 220 \backslash$
]

不断把 NVDA 卖掉，到 220 后变成 100% USDC。

于是：

$\backslash \backslash$
 $220 \rightarrow 230 \backslash$
]

这 \$10 涨幅你基本不再参与。

这就是你的 short Gamma / covered-call-like cost。

反过来：

$\backslash \backslash$
 $205 \rightarrow 180 \backslash$
]

你会一路买 NVDA，到 190 以下变成 100% NVDA，然后继续承担下跌。

所以你可以把它理解成：

$\backslash \backslash$
 $\boxed{\backslash \text{NVDA } 190\text{--}220 \text{ LP}}$
=====

$\backslash \text{持续卖出区间内的波动率} \backslash$
 $\} \backslash$
]

你赚的是：

$\backslash \backslash$
 $\boxed{\backslash \text{swap fees / Theta-like income} \backslash}$
]

你付出的则是：

$\backslash \backslash$
 $\boxed{\backslash \text{negative Gamma / adverse selection} \backslash}$
]

这比“卖 190P + 220C”这个模型准确得多。

还有一个很有意思的结果：如果你把 190–220 不断缩窄，比如 204.9–205.1，那么刚才那条连续 option strip 就逐渐塌缩到 ($K \approx 205$) 附近。这时才越来越接近 Lambert/Panoptic 所说的一个单 strike 永续 covered call / cash-secured put。

所以可以记一个很简洁的对应关系：

$\backslash \backslash$
 $\boxed{\backslash \begin{array}{ccc} \backslash \text{single tick LP} \&\approx\& \backslash \text{single-strike short option} \backslash \\ \backslash \text{wide-range LP} \&\approx\& \backslash \text{strip of short options across strikes} \backslash \\ \backslash \text{LP fees} \&\approx\& \backslash \text{streaming option premium} \backslash \end{array} \backslash}$

$$\end{array}}\backslash$$

而不是：

$$\boxed{190-220 \text{ LP}}$$
$$\text{eq short } 190P + \text{short } 220C$$

并且列出了大致学习计划：

不需要重新从“什么是 call/put”这种程度开始。我建议做成 10 个 session，每次约 1–2 小时：

1. Session 1：Uni v3 position 数学。重新推一遍 $x(S), y(S), V(S)$ ，理解 L 、 sqrtPrice 、 lower/upper tick 。最终自己能解释“为什么价格上涨 LP 自动减少 ETH”。官方文档先用于确认 concentrated liquidity 与 in-range mechanics。
2. Session 2：LP $\rightarrow$ covered call / short put。精读 Lambert 第一篇 Uniswap V3 LP Tokens as Perpetual Put and Call Options，自己画 $V(S)$ 和 covered-call payoff。然后用 put-call parity 证明 covered call = cash-secured short put。
3. Session 3：从 payoff 升级到 Greeks。对 $V(S)$ 求一阶、二阶导数，重点真正搞懂：

$\Delta = \text{inventory}, \Gamma < 0$

然后画 $\pm 1\%$ 、 $\pm 5\%$ 、 $\pm 20\%$ range 的 Delta/Gamma。到这里以后，“IL”会开始显得只是表象。

4. Session 4：为什么 centered LP 像 short straddle。读 Synthetic Options and Short Calls in Uniswap V3。学习 underlying + put/call 的 synthetic relationships，然后理解为什么对称 ATM LP 是“局部 short straddle-like”，但不要把二者当严格相同的产品。

5. Session 5：Range = Gamma / effective DTE。精读 How to Create Perpetual Options in Uniswap v3，重点只看 K, r, Tr, Δ 。自己算一次：同样 ETH spot， $\pm 2\%$ 、 $\pm 5\%$ 、 $\pm 10\%$ 、 $\pm 20\%$ range 分别对应怎样的 effective DTE。

6. Session 6：Fees $\rightarrow$ Theta $\rightarrow$ volatility。这是最重要的一课。读 Panoptic whitepaper Section III。理解 streaming premium：

$P = \int \theta(S) dt$

再理解 fee-implied volatility。最终能回答：“一个 100% APR 的 LP 究竟是在卖多贵的 volatility？”

7. Session 7：IV vs RV + LVR。把传统的“fee APR vs IL”框架彻底升级成：

Theta earned Gamma / LVR / adverse-selection cost

研究什么时候 LP 相当于 $IV > RV$ 的 profitable short vol，什么时候实际上是在太便宜地卖 volatility。Lambert 后期 X 的思路也更接近这种 framing。

8. Session 8：Gamma Transform。精读 Lambert 的 Gamma Transforms: How to hedge squeeze using Uni V3。理解一个 tick 是一个 negative-Gamma basis，然后理解为什么多个 liquidity ranges 可以“编译”一个 payoff。这是整个系列里数学上最漂亮的一篇。

9. Session 9：最后才完整读 Panoptic whitepaper。此时重新读 Section II–III，你会发现 put/call、fixed gamma、streaming premium 基本都已经认识；再读 margin、pool utilization、multi-leg tokenization，理解 Panoptic 如何把这种 financial primitive 产品化。

10. Session 10：真正做一个 LP/options notebook。选一个真实 Uni v3 pool，输入 $S, Pa, Pb, L, \text{fee}, \text{volume}$ ，输出：

V, Δ, Γ , fee income, fee implied vol

笔记 005: 兩池套利模型筆記

作者：corn15 | 时间：2026-08-10 15:26 UTC | 字数：3697 | 评分：84

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

兩池套利模型筆記

1. 基本概念

假設有兩個同交易對的 AMM Pool：

$$\text{Pool A: } (x_A, y_A)$$

$$\text{Pool B: } (x_B, y_B)$$

皆符合 Uniswap V2 類型的 Constant Product：

$\backslash$

$xy=k$

]

若定義：

(X = USDC)

(Y = ETH)

則 ETH 價格為：

$\backslash$

$P=\frac{x}{y}$

]

若：

$\backslash$

$P_A < P_B$

]

代表 Pool A 的 ETH 比較便宜，可以：

$X \rightarrow Y$ (Pool A 買 ETH)

$Y \rightarrow X$ (Pool B 賣 ETH)

2. 單池 Swap 公式

假設投入：

$\backslash$

q

]

個 (X)。

若手續費為 (f)，定義：

$\backslash$

$\gamma=1-f$

]

則實際參與定價的 input 為：

$\backslash$

γq

]

輸出的 (Y)：

$\backslash$

$\boxed{\Delta y=}$

$\frac{\gamma q y}{x + \gamma q}$

$\}$

]

Uniswap V2 若 fee = 0.3%：

$\backslash$

$\gamma=0.997$

]

3. 兩池套利 Output

先在 Pool A：

$\backslash$

$X \rightarrow Y$

]

得到：

$\backslash$

$\Delta y=$

$$\frac{\gamma_A q y_A}{x_A + \gamma_A q}$$

再將 (Δy) 放入 Pool B :

$$Y \rightarrow X$$

最終得到 :

$$\boxed{X_{\text{out}}(q)}$$

$$\frac{q \gamma_A \gamma_B x_B y_A}{x_A y_B + q \gamma_A (y_B + \gamma_B y_A)}$$

4. Profit Function

套利收益 :

$$\boxed{\pi(q) = X_{\text{out}}(q) - q}$$

因此 :

$$\boxed{\pi(q) = \frac{q \gamma_A \gamma_B x_B y_A}{x_A y_B + q \gamma_A (y_B + \gamma_B y_A)} - q}$$

套利問題就是 :

$$\max_q \pi(q)$$

5. 是否存在套利 ?

若交易量非常小, 套利成立的條件為 :

$$\boxed{\gamma_A \gamma_B \frac{y_A}{x_A} > 1}$$

]

整理：

$\backslash\backslash$

$\backslash\boxed{\backslash$

$\backslash\gamma_A\backslash\gamma_Bx_By_A>x_Ay_B\backslash$

$\}\backslash$

]

若用價格表示：

$\backslash\backslash$

$P_A=\frac{x_A}{y_A}\backslash$

]

$\backslash\backslash$

$P_B=\frac{x_B}{y_B}\backslash$

]

則：

$\backslash\backslash$

$\backslash\boxed{\backslash$

$\backslash\frac{P_B}{P_A}>\backslash$

$\backslash\frac{1}{\gamma_A\gamma_B}\backslash$

$\}\backslash$

]

如果兩池 fee 都是 0.3%：

$\backslash\backslash$

$\backslash\gamma_A=\gamma_B=0.997\backslash$

]

則至少約需要：

$\backslash\backslash$

$\backslash\frac{P_B}{P_A}>1.006\backslash$

]

也就是約 0.6% 價差才有套利空間。

6. 最佳套利量

Profit 最大的位置：

$\backslash\backslash$

$\backslash\frac{d\pi}{dq}=0\backslash$

]

最後可得到：

$\backslash\backslash$

$\backslash\boxed{\backslash$

$q^\wedge\backslash$

====

$\backslash\frac{\backslash$

$\backslash\sqrt{\backslash$

$\backslash\gamma_A\backslash\gamma_Bx_By_Ax_Ay_B\backslash$

$\}\backslash$

$-x_Ay_B\backslash$

$\}\backslash$

$\backslash\gamma_A(y_B+\gamma_By_A)\backslash$

$\}\backslash$

$\}\backslash$

]

其中：

$$q^*$$

就是最佳投入量。

重點：

最佳套利量不是「能投入多少就投入多少」。

因為你的交易本身會改變 Pool 價格。

7. 為什麼有最佳套利量？

套利時：

Pool A

買走 (Y)：

$$x_A \uparrow, \quad y_A \downarrow$$

所以：

$$P_A \uparrow$$

Pool B

賣入 (Y)：

$$x_B \downarrow, \quad y_B \uparrow$$

所以：

$$P_B \downarrow$$

因此：

Pool A Pool B

低價格 高價格

↑ ↓

↑ 套利 ↓



套利會逐漸把兩池價格拉近。

最佳點就是：

$$\boxed{\text{Marginal Profit} = 0}$$

也就是：

$$\frac{d\pi}{dq} = 0$$

8. Gas Cost

假設 Gas 換算成 (X) 為：

$$G$$

則淨收益：

```

\[\
\boxed{\
\pi_{\{\text{net}\}}(q)
=====
X_{\{\text{out}\}}(q)-q\cdot G\
}\
]

```

若 Gas 與交易量無關，則：

```

\[\
q^{\
}

```

不會改變。

但需要確認：

```

\[\
\boxed{\
\pi(q^{\})>G\
}\
]

```

才值得執行。

9. Bot 的基本流程

兩池套利 Bot 可以拆成：

```

Pool Reserves
↓
判斷是否存在套利
↓
判斷套利方向
↓
計算最佳 q
↓
模擬 Swap
↓
扣除 Gas / Fee
↓
Profit > 0 ?
↓
執行交易

```

核心流程：

```

\[\
\boxed{\
Opportunity\
\rightarrow\
Direction\
\rightarrow\
Size\
\rightarrow\
Execution\
}\
]

```

10. Atomic Arbitrage

兩次 Swap 通常需要放在同一筆 Transaction：

```

X
↓
Pool A

```

↓
Y
↓
Pool B
↓
X

Smart Contract 大致邏輯：

```
function arbitrage() {
  swapA(X -> Y);
  swapB(Y -> X);
  require(finalX > initialX);
}
```

若最後無利可圖：

revert

因此整筆套利具有 Atomic 性質。

11. 最重要的三個公式

單池 Swap

$$\boxed{\Delta y = \frac{\gamma \Delta x, y}{x + \gamma \Delta x}}$$

兩池 Profit

$$\boxed{\pi(q) = \frac{q \gamma_A \gamma_B x_B y_A}{x_A y_B + q \gamma_A (y_B + \gamma_B y_A)} - q}$$

Optimal Input

$$\boxed{q^* = \frac{\sqrt{\gamma_A \gamma_B x_B y_A x_A y_B}}{x_A y_B + \gamma_A (y_B + \gamma_B y_A)}}$$

核心直覺

兩池套利最重要的概念不是背公式，而是：

```
\[\boxed{\text{你的套利交易本身會改變 Pool 價格}}\]
```

因此套利的本質是：

找到價差

↓

用交易吃掉價差

↓

交易造成 Price Impact

↓

直到再交易一點已經沒有額外收益

即：

```
\[\boxed{\frac{d\pi}{dq}=0}\]
```

這個位置就是最佳套利量。

笔记 006: AMM ;

作者：A9xYau | 时间：2026-08-10 15:31 UTC | 字数：4751 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

AMM ;

订单簿；

流动性深度；

价格冲击；

DEX-CEX 套利结构；

库存套利。

但是如果真想真正理解链上套利系统，还需要继续掌握一些更深入的概念。

这些概念决定了：

一个套利机会是否能够被发现、是否能够执行、是否能够稳定盈利。

下面介绍：

1. 套利机会来源；
2. 套利路径 (Route) ；
3. 原子套利 (Atomic Arbitrage) ；
4. 闪电贷套利 (Flash Loan Arbitrage) ；
5. 滑点保护；
6. Gas 与执行成本；
7. MEV 与抢跑；
8. 套利机器人的核心流程；
9. 一个真实套利案例。

一、套利机会到底来自哪里？

回答：

套利机会并不是随机出现的，它通常来自以下几类市场失衡。

1. 流动性不均衡 (Liquidity Imbalance)

这是最常见原因。

例如：

Uniswap :

ETH/USDC 池 :

ETH:

1000

USDC:

3000000

价格 :

3000 USDC。

但是突然有人 :

大量买 ETH。

池子变化 :

ETH:

900

USDC:

3300000

价格上涨。

此时 :

另一个 DEX :

PancakeSwap :

仍然 :

3000。

于是 :

出现 :

Uniswap:

3300

PancakeSwap:

3000

套利者 :

PancakeSwap 买入。

Uniswap 卖出。

2. 信息传播速度不同 (Information Latency)

不同市场 :

收到信息速度不同。

例如 :

Binance :

有人大额买 ETH。

价格 :

立即上涨。

但是 :

某个小 DEX :

还没有交易。

价格 :

仍然较低。

机器人 :

利用时间差套利。

3. 市场参与者不同

CEX：

主要是：

高频交易机构；

做市商；

普通交易者。

DEX：

主要是：

DeFi 用户；

流动性提供者；

链上机器人。

交易行为不同：

导致价格暂时不一致。

二、什么是套利路径（Arbitrage Route）？

回答：

套利路径指：

资金从哪里进入，经过哪些交易池，最后如何回来。

简单套利：

只有两个节点：

USDC

↓

ETH

↓

USDC

复杂套利：

可能：

USDC

↓

ETH

↓

WBTC

↓

USDC

甚至：

USDC

↓

ETH

↓

ARB

↓

USDT

↓

USDC

套利机器人本质是在寻找：

收益最高的交易路径。

例如：

有三个池：

ETH/USDC

ETH/BTC

BTC/USDC

机器人计算：

如果：

10000 USDC

经过：

USDC → ETH → BTC → USDC

最终：

得到：

10120 USDC。

那么：

利润：

120 USDC。

这实际上就是：

图搜索问题（Graph Search）。

三、什么是原子套利（Atomic Arbitrage）？

回答：

原子套利是链上套利最重要的概念之一。

意思：

所有交易必须：

在同一个区块、

甚至同一笔交易中完成。

例如：

借钱

↓

DEX1买入

↓

DEX2卖出

↓

还钱

必须：

全部成功。

如果任何一步失败：

整个交易回滚。

优点：

没有持仓风险。

例如：

普通套利：

买 ETH

↓

等待转账

↓

卖 ETH

期间：

ETH可能暴跌。

原子套利：

买

卖

完成

没有价格暴露。

四、什么是 Flash Loan 套利？

回答：

Flash Loan（闪电贷）是实现原子套利的重要工具。

特点：

无需本金。

但是：

必须：

同一笔交易归还。

例如：

你发现：

Uniswap：

ETH:

3000

SushiSwap：

ETH:

3020

但是：

你没有资金。

流程：

向 AAVE 借：

100万 USDC

↓

Uniswap 买 ETH

↓

SushiSwap 卖 ETH

↓

得到：

100.6万 USDC

↓

归还：

100万 USDC

↓

剩余：

6000 USDC

如果失败：

整个交易取消。

五、什么是滑点保护（Slippage Protection）？

回答：

滑点是套利最大的风险之一。

例如：

机器人发现：

ETH：

3000

准备买：

100 ETH。

但是提交交易后：

别人先交易。

池子价格变化：

ETH：

3020。

如果没有保护：

机器人：

高价买入。

套利失败。

因此交易通常设置：

最大接受滑点。

例如：

最多接受：

0.3%

超过：

交易失败。

六、Gas 为什么决定套利胜负？

回答：

链上套利竞争本质上是：

速度竞争。

例如：

一个套利机会：

利润：

1000 USDC。

同时：

100 个机器人发现。

谁先进入区块？

取决于：

Gas。

例如：

机器人 A：

Gas:

30 Gwei

机器人 B：

Gas:

50 Gwei

矿工/验证者：

通常优先：

Gas 更高交易。

结果：

B 成功。

A 失败。

所以：

套利系统必须动态调整：

Gas Price。

七、什么是 MEV 抢跑？

回答：

因为交易进入区块前：

会进入：

Mempool（交易池）。

别人可以看到：

你的交易。

例如：

你发现：

某 DEX：

价格异常。

提交套利交易。

机器人看到：

你的交易。

于是：

提前：

复制你的交易。

提高 Gas。

抢在你前面执行。

这叫：

Front-running。

因此专业套利者通常使用：

Flashbots；

Private RPC；

Builder Bundle。

避免公开暴露交易。

八、套利机器人的完整工作流程

回答：

一个真实套利机器人通常如下：

数据采集



↓
套利机会发现

↓
路径搜索

↓
成交模拟

↓
利润计算

↓

风险过滤

↓

构造交易

↓

提交执行

↓

监控结果

九、真实套利案例：Uniswap 与 Binance ETH 套利

回答：

下面用一个接近真实交易逻辑的案例说明。

假设：

时间：

2024 年某次 ETH 波动期间。

市场出现短暂价格差。

第一步：发现价格差

机器人实时监控：

DEX：

Uniswap ETH/USDC

发现：

ETH:

2990 USDC

Binance：

ETH/USDT

发现：

ETH:

3010 USDT

表面价差：

3010 - 2990

=20美元

第二步：检查是否可执行

机器人不能直接交易。

需要模拟：

DEX 买入模拟

准备：

买：

100 ETH。

查看 Uniswap Pool：

发现：

如果买100 ETH：

平均成交价格：

2995。

成本：

2995 × 100

=299500 USDC

CEX 卖出模拟

查看 Binance Order Book。

卖：

100 ETH。

订单深度：

平均成交：

3008。

收入：

3008×100

=300800 USDT

第三步：计算成本

收益：

300800

-

299500

=

1300美元

扣除：

DEX手续费：

100美元

CEX手续费：

30美元

Gas：

40美元

总成本：

170美元。

最终：

净利润：

1130美元

第四步：执行方式

如果采用传统搬砖：

Uniswap买ETH

↓

转Binance

↓

卖出

风险：

等待时间太长。

所以专业方式：

提前准备库存：

Uniswap钱包：

USDC

Binance：

ETH

同时：

执行：

DEX：

买ETH

CEX：

卖ETH

完成套利。

第五步：套利后的状态

套利后：

DEX：

ETH增加。

USDC减少。

CEX：

ETH减少。

USDT增加。

需要：

后续：

进行资金再平衡。

十、从这个案例可以学到什么？

回答：

这个案例看似简单：

低买高卖。

但是实际包含：

1. AMM 定价；
2. CEX OrderBook 深度；
3. 流动性分析；
4. 滑点计算；
5. 手续费计算；
6. Gas成本；
7. 库存管理；
8. 执行速度；
9. MEV竞争。

真正的套利系统不是：

发现价格差

↓

买卖

而是：

发现价格异常

↓

模拟真实成交

↓

计算净利润

↓

判断风险

↓

最快执行

↓

管理库存

总结：

DEX-CEX 套利的核心思想可以总结为：

套利不是寻找价格差，而是寻找扣除所有成本后仍然可以执行的价格不平衡。

未来继续深入学习时，重点应该掌握：

1. Uniswap V2/V3 定价公式；
2. CEX OrderBook 深度模拟；
3. 套利路径搜索算法；
4. Flash Loan；
5. MEV；
6. Flashbots；
7. 套利机器人的系统架构。

这些知识组合起来，才构成真正的链上套利系统。

笔记 007: D04 | 常数乘积 AMM

作者：averywexler | 时间：2026-08-10 15:32 UTC | 字数：623 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D04 | 常数乘积 AMM

AI总结：AMM 不是给你一个固定价格，而是让交易沿一条曲线改变池子储备；交易越大，平均成交价通常越差。

今日术语

储备量：池中两种资产各有多少；

不变量：协议要求交易前后保持的数学关系；

$xy=k$ ：常数乘积 AMM 的基础关系；

池内瞬时价格：当前两边储备比例表达的价格；

交易费：每次交换被 LP 或协议收取的费用。

参考材料

Uniswap v2 Pricing (<https://developers.uniswap.org/docs/protocols/v2/concepts/pricing>)

Uniswap v2 Whitepaper (<https://app.uniswap.org/whitepaper.pdf>) 前两节

例子

池中有 100 ETH 和 200,000 USDC，暂时忽略费用：

1. $k = 100 \times 200,000 = 20,000,000$ ；
2. 投入 20,000 USDC 后，USDC 储备变成 220,000；
3. 新 ETH 储备为 $20,000,000 \div 220,000 \approx 90.9091$ ；
4. 交易者得到约 9.0909 ETH；
5. 平均成本约 \$2,200/ETH，而交易前池价是 \$2,000。

笔记 008: 今天弄清了模拟执行在换币流程里是干什么、要准备什么。

作者：catcashx | 时间：2026-08-10 15:42 UTC | 字数：604 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天弄清了模拟执行在换币流程里是干什么、要准备什么。

在干什么

模拟执行：在节点（或本地分叉）上，按「假如现在发出这笔交易」把合约逻辑完整跑一遍，结果不写进真链。

用来确认：会不会失败回滚、大概耗多少 gas、相关地址余额会怎么变。

它不是再算一遍纸面进出，而是让 EVM 真的跑一遍合约代码，只是不提交状态。

和路径预算比较不同：预算可以只在脚本里用池子数字推演；模拟执行必须能对上真实的合约调用。

需要什么

1. 一笔完整的交易描述：谁发起 (from)、调哪个合约 (to)、调用数据 (data)，以及可选的附带原生币数量。
2. 链上已有的合约：代币、池子/AMM，常用还有路由或自研执行器。换币模拟就是调这些已部署地址，不是临时现编合约。
3. 正确的调用意图：to + data 要对应要走的兑换路径。乱填不能代表你的路径，多半直接失败。
4. 节点接口：如 `eth_call` / `eth_estimateGas`，在指定区块状态下仿真执行一次并返回结果。
5. 业务条件仍会被检查：余额、授权、路径和最小输出等。接口公开可试，条件不够一样回滚——模拟就是为了在不上链时发现这一点。

边界

模拟用的是某一时刻的状态；真发时状态可能已变，通过不等于真发一定成功。模拟通常不需要私钥；真签名上链才需要。

模拟执行 = 对真实合约按完整交易描述仿真执行一次、不落链，用来过滤走不通或成本不划算的候选。

笔记 009: 学习记录

作者：capromka | 时间：2026-08-10 15:43 UTC | 字数：438 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习记录

今天继续学习LP套利者相关知识，同时学习群友笔记和群内分享，对 LP、套利和价格变化之间的关系 有了进一步理解。

主要收获

AMM 本身不知道外部市场价格，套利者通过池内外价差，把 AMM 价格拉回市场价格附近。

当资产上涨时，套利者会从池子里买走上涨资产，LP会被动卖出上涨资产；资产下跌时则相反。

套利者对 AMM 很重要，但对 LP 来说又存在一定博弈：套利带来交易量和手续费，同时也可能让 LP 跑输单纯持币。

套利者并不是无条件套利，只有当价差能够覆盖手续费、Gas、滑点、Price Impact 等成本并留下利润时才会执行。

今日总结

LP 提供流动性，交易推动价格变化，套利者负责纠正价格，而 LP 通过手续费获得补偿。三者共同构成了 AMM 的运行机制。

今天对“LP为什么会产生无常损失，以及套利者到底在赚谁的钱”有了更直观的理解。

笔记 010: Day 06 : DEX 套利

作者：FireLoveInIce | 时间：2026-08-10 15:54 UTC | 字数：2049 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 06 : DEX 套利

笔记链接：<https://github.com/FireLoveInIce/21DaysCoLearnNote/tree/main/day06/note>

今日目标

理解同一 DEX 内价差产生的原因。

区分“存在价差”和“能够盈利”。

掌握包含手续费、Gas、滑点和安全边际的利润判断方法。

知道套利执行需要的数据、条件和主要风险。

一、什么是 DEX 内套利

同一个 DEX 中，不同资金池、费率池或交易路径可能对同一资产给出不同价格。

例如：

text

Pool A : 1 ETH = 3000 USDC

Pool B : 1 ETH = 3030 USDC

套利路径：USDC → Pool A 买 ETH → Pool B 卖 ETH → USDC

价差通常来自：

大额交易使某个池的价格暂时偏移。

不同池的流动性深度、费率和状态不同。

V3 集中流动性、V4 Hook 或动态费率造成报价差异。

套利交易尚未将各池价格重新拉近。

常见路径包括同交易对双池、不同费率池，以及 USDC → WETH → USDT → USDC 这类多跳循环。

二、利润如何判断

看到价格差不代表可以盈利。交易会改变池中价格，必须使用实际输入金额逐跳报价。

对于 V2 恒定乘积池：

text

$\text{amountInAfterFee} = \text{amountIn} \times (1 - \text{fee})$

$\text{amountOut} = \text{reserveOut} \times \text{amountInAfterFee}$

$\text{reserveIn} + \text{amountInAfterFee}$

V3/V4 涉及 Tick、流动性区间和 Hook，应固定 blockTag，使用 Quoter 或完整交易模拟获得实际输出。

统一计价资产后：

text

毛利润 = 最终输出 - 初始投入

净利润 = 毛利润

- Gas 成本

- 闪电贷费用

- Builder/优先费

- 其他执行成本

可执行利润 = 净利润 - 滑点与状态变化安全边际

只有 可执行利润 > 最低目标利润 时，才应把它视为候选机会。

交易规模也不能无限增大：规模越大，价格影响越大，最终会吞噬价差。实际要寻找的是“净利润最高的输入规模”，可以先扫描多组输入金额，再在最优区间细化搜索。

三、执行条件与主要风险

执行条件：

所有报价和池状态来自同一个区块。

路径有效、流动性充足，且滑点和 Gas 可控。

能在一笔交易中原子完成；任何一步失败时整笔回滚。

提交前已模拟完整交易，而不只是单独调用 Quoter。

最容易失败的地方：

使用旧状态，交易上链前机会已经消失。

被抢跑或交易排序改变。

遗漏手续费、Gas、优先费等成本。

Token decimals、取整或报价方向错误。

交易规模过大，价格影响吞噬利润。

V4 Hook 改变费用、资产流或执行结果。

模拟成功，但真实执行时状态变化导致回滚。

四、复盘问题与答案

1. 哪个成本影响最大？

没有固定答案。小额交易通常受 Gas 影响最大，大额交易通常受价格影响和滑点影响最大；竞争激烈时，抢跑、排序和状态变化风险可能最关键。因此必须针对每条路径和输入规模单独计算。

2. 判断机会至少需要哪些数据？

池地址或 PoolKey、Token 顺序和 decimals。

同一 blockTag 下的池状态、手续费和实际报价。

套利方向、输入金额与每一跳输出。

Gas 使用量、Gas 价格及其统一计价后的成本。

最低目标利润与安全边际。

V4 场景还需要 Hook 地址、权限、状态和真实 hookData。

3. 这个场景适合自动化吗？

适合，而且多数机会持续时间很短，通常必须自动化：

text

监听池状态 → 快速筛选路径和规模 → 精确报价

→ 模拟完整交易 → 检查净利润 → 原子提交

未知 Hook、无法模拟、状态不一致或安全边际不足的路径，不应进入自动执行器。

五、今日动手任务

[] 选择一个同 DEX 双池或循环路径。

[] 固定历史区块，记录池状态和实际报价。

[] 扫描至少 5 个输入金额，找出净利润最高的规模。

[] 记录毛利润、全部成本、净利润和失败风险。

完成标准：能够解释价差来源，写出净利润公式，并说明为什么“最大价差”不等于“最大利润”。

作者：xsh | 时间：2026-08-11 01:37 UTC | 字数：340 | 评分：60
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Universal Router & Permit2

Universal Router

Universal Router v1.2

Docs

<https://uniswap.org/blog/permit2-and-universal-router>

Profile

0xef1c6e67703c7bd7107eed8303fbe6ec2554bf6b

Permit2

衍生于 EIP-2612

允许在不同的应用程序中共享和管理代币审批

有时间限制

....

Universal Router

整合Permit2

支持v2/v3聚合

支持多个ERC-20 & ERC-721聚合交易

笔记 012: 2026-08-11 | 共学 Day 7 | 第一周总结 | 投入约 0.5 小时

作者：wuyuandao | 时间：2026-08-11 03:14 UTC | 字数：627 | 评分：62
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-11 | 共学 Day 7 | 第一周总结 | 投入约 0.5 小时

第一周回顾：

Day 1：Hermes 安装 + Telegram 连通

Day 2：7 大套利类型全景 + Bruce 8 项成本公式 + Uniswap 池子实操

Day 3：AMM 第一性原理 ($xy=k$ 、滑点非线性、无常损失)

Day 4：8 项成本模型实战 (Arbitrum \$10k 场景跑通，净利仅 \$31.66)

Day 5：LI.FI API 配置 (5 条真实跨链 Quote，发现正常市场无套利空间)

Day 6：The Graph + DexScreener 数据查询 (确认 DexScreener/LI.FI/CoinGecko 三板斧方案)

工具箱现状：

已配齐 LI.FI API、DexScreener、CoinGecko、Hermes Agent。taoli.tools 和 QQQ.BOT 尚未深入，The Graph 子图迁移中需等待。

核心认知：

正常市场没有躺赚的套利——主流币价差不到 0.1%，稳定币未脱锚，LI.FI 费先吃掉 0.25%。真实机会转瞬即逝，必须用脚本和 Agent 才能抓住。第一周地基已打好，第二周上脚本和案例深挖。

困难：The Graph 生态迁移导致子图不可用，编程零基础需持续靠 Hermes 辅助。

下一步：第二周——深度案例研究 + 训练专属套利 Agent + 数据采集脚本。

笔记 013: Day 6 打卡 | 链上套利残酷共学

作者：roy328line | 时间：2026-08-11 04:15 UTC | 字数：716 | 评分：64
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 打卡 | 链上套利残酷共学

今日實作

tri_scanner.py 加入双重過濾機制 (排除流動性幻覺)：

1. 已知 token 白名單：三條腿的 token 都必須在 TOKEN_PRICES 內，長尾 token 路徑直接跳過

2. MIN_POOL_TVL = \$50,000：每條腿的估算 TVL 都需達標，三腿取最小值做門檻

新增 run_tri_scanner.py：啟動腳本，自動從 .env 讀取 KEY，預設 SCAN_SECONDS=1800、MIN_POOL_TVL=50000。

深挖 WETH→RAIL→USDT→WETH 幻覺案例

顯示 net：約 \$61,000

根本原因：v3 虛擬儲備高估 9 億倍

為什麼會高估 9 億倍？

$L/\sqrt{P}$ 公式假設流動性均勻分布，但 $v3$ 的流動性只在 $[\text{tickLower}, \text{tickUpper}]$ 範圍內有效。

RAIL/USDT pool 距 tick 上界僅 1 tick，真實可用的 USDT 約等於 \$0，

但公式算出的虛擬儲備卻是天文數字。

今日最重要的一件事

$v3$ 的 $L/\sqrt{P}$ 虛擬儲備公式，忽略了 tick range 邊界。

當 current tick 靠近邊界時，誤差可達億倍——這不是精度問題，是模型假設根本不成立。

正確做法（Day 7 實作）：

$\text{virtual}_0 = L (1/\sqrt{P}_{\text{current}} - 1/\sqrt{P}_{\text{upper}})$

$\text{virtual}_1 = L (\sqrt{P}_{\text{current}} - \sqrt{P}_{\text{lower}})$

才能算出在當前 tick 範圍內實際可用的儲備量。

笔记 014: 案例：Binance Alpha 与链上搬砖——以 dappOS (DOS) 为例

作者：QQQ | 时间：2026-08-11 06:02 UTC | 字数：3768 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

案例：Binance Alpha 与链上搬砖——以 dappOS (DOS) 为例

一、背景与案例边界

2026 年 8 月 10 日，dappOS 的 DOS 上线 Binance Alpha。该资产是 BSC 上的 BEP-20 代币，合约为 0xB0f09ea9ae0515C3551080D4a745C8115aA30e37。市场讨论的路径是：在 Alpha 以较低价格买入 DOS，提至 BSC，再在 PancakeSwap 的 DOS/USDT 池卖出；这不是跨链桥套利，而是同一 BSC 资产在 Alpha 订单簿与链上 AMM 之间的短暂定价不同步。

需要先把它当作“事件驱动、可执行性受限的价差交易”，而不是一个固定收益产品。Alpha 交易对当时仅提供限价订单；低价限价单不等于立刻成交，也不等于已经锁定链上卖出价。

二、机制：为什么会有价差，又为什么会很快消失

上线早期，两端流动性和参与者不同步：Alpha 的挂单、链上 AMM 初始流动性、机器人报价、用户的提币和链上卖出速度都不同。若 Alpha 的可成交买价低于链上“实际可卖出价”，套利者就会买入、提币、卖出，持续买高 Alpha、卖低链上，直到两端价格收敛。

这里最容易误判的是把“最新成交价”或 AMM 页面中间价当成可成交价格。真正可执行的比较应是：Alpha 买入订单的成交均价（含手续费）对比链上输入相同 DOS 数量后的预估到手 USDT（已含池费和价格影响）。

三、DOS 的事后数据：机会存在于最初窗口，不存在于收敛之后

Alpha 记录显示，DOS 于 2026-08-10 09:00 UTC（北京时间 17:00）开始交易。首小时波动极端：开盘约 0.10 USDT，最高约 0.7136 USDT，最低约 0.10 USDT，收盘约 0.2271 USDT。这样的波动说明当时可能出现过较大瞬时价差，但也说明“买入后等提币”并非无风险。

可取得连续主池 K 线的北京时间约 20:00 以后，两端已经高度同步：Alpha 约 0.363 → 0.433 → 0.398 USDT；PancakeSwap 主池约 0.362 → 0.435 → 0.400 USDT。8 月 11 日北京时间约 13:00 的快照中，Alpha 最新成交约 0.5280 USDT，链上主池约 0.5288 USDT，表面差仅约 0.15%。这小于任何严肃的执行安全垫，不能作为入场信号。

四、实操路径：按“能成交、能提、能卖”三项依次验证

1\、资产核验：从 Binance 提币页复制合约尾号并在 BscScan 核对，确认链、合约、代币符号和链上交易池完全一致；DOS 的目标为 BSC，不应误提到不支持的地址或链。

2\、Alpha 买入：读取最优卖盘/限价可成交量，设置只在可接受均价以内成交的限价单。若未成交，机会不存在；不要为了追单把价格抬到链上可卖价之上。

3\、提币可用性：先确认 BSC 提币开关、最小提币量、提币费和预计到账时间。此次截图显示提币费为 0.037 DOS、最低提币量为 0.074 DOS；这只是当时界面参数，实盘前必须重新读取。

图：DOS 的 BSC 提币参数截图（用户提供）

4\、链上卖出：在 PancakeSwap 主池输入“预计到账的 DOS 数量”，以实际到手 USDT 而非图表价格计算；设置最小接收额、准备 BNB gas，并确认代币未出现卖出税、黑名单、暂停交易或异常授权限制。

5\、复盘记录：保存下单时间、Alpha 成交均价、到账数量/时间、链上报价、最终成交、gas、提币费和净利润。没有这组记录，就无法知道收益来自价差还是赌对方向。

五、合理的实际收益分析：先算净利润，再谈套利

单次净利润 (USDT) = 链上实际卖出到手 USDT - Alpha 实际买入成本 - Alpha 手续费 - 提币费折算 USDT - BSC gas 失败/重试成本 - 价格波动缓冲。

DOS 收敛后 0.15% 的“表面价差”举例：若名义买入 1,000 USDT，毛差只有约 1.5 USDT。哪怕不计 Alpha 费用，链上池费、价格影响、提币费与 gas 也足以吞掉这 1.5 USDT；而且 Alpha 最新成交价不是你一定能买到的价格，链上中间价也不是你一定能卖到的价格。因此该报价下的合理预期净收益应视为负值，而不是“低风险赚 0.15%”。

把安全垫拆开更实用：链上卖出池费 + 预计滑点 + Alpha 费用 + 提币费/名义金额 + gas/名义金额 + 延迟期间的价格波动缓冲。新币上线时，延迟风险通常最大。实际入场阈值至少要显著高于这些项目之和；对于无对冲、需提币的单向路径，建议先以不少于 1%–2% 的保守安全垫做观察阈值，再用小额试单校准，而不是把它当作固定规则。

六、仓位与风险：它不是严格无风险套利

主要风险包括：限价单部分成交；提币临时暂停或排队；链上流动性不足造成大额滑点；价格在到账前快速反转；AMM 报价被 MEV/抢跑影响；合约、假池、卖出税或授权风险；以及交易所、钱包、地区与合规限制。

因此仓位应由“在最坏可接受卖出价格下仍可承受的亏损”决定，而不是由屏幕上的价差百分比决定。新币首日不应把资金一次性打满：先做极小额的全链路测试，到账并成功卖出后才有资格评估是否放大；同时设置单币最大敞口、单笔最大亏损、最短剩余有效时间和中止条件。

七、监控：从看图升级为可执行报价监控

每 2–5 秒采集 Alpha 的可买入报价和可成交数量；每 2–5 秒模拟 PancakeSwap 对同等 DOS 数量的卖出到手 USDT；同时采集提币状态、提币费、BSC gas、池子流动性和最近 1/5/15 分钟波动。核心指标不是 last price，而是：

Alpha 买入 VWAP：指定金额实际吃单后的平均买入价；

链上卖出净报价：指定 DOS 卖出后实际收到的 USDT；

净边际：扣除全部已知成本后的净利润率；

可执行容量：在净边际仍高于阈值时最多能做多少金额；

时间风险：提币预计耗时 × 最近短周期波动率。

告警条件可设为：净边际高于阈值、可执行容量高于最低仓位、BSC

提币可用、gas

不异常、主池流动性足够、过去几分钟无异常跳价。条件任一失效就撤销未成交订单或停止新开仓。

八、自动化路线：先半自动，后全自动

阶段 1（人工执行）：脚本只负责推送“净边际、容量、成本明细、建议最大仓位”，人手确认下单、提币与链上卖出。先积累至少若干次小额完整记录。

阶段 2（半自动）：自动创建/撤销

Alpha

限价单；成交后立即提醒人工确认提币；链上端由钱包弹窗确认交易。适合先验证各平台限额、延迟和实际滑点。

阶段 3（全自动）：需要交易所 API 权限、白名单地址、热钱包/BSC gas 管理、DEX 路由模拟、交易广播、状态机、持仓对账和熔断。它应是“事件驱动执行系统”，而不是每秒钟盯两个价格的脚本。

最小系统模块：事件日历（新 Alpha 上线）→ 合约/池子发现与白名单 → Alpha 深度与链上报价采集 → 成本/滑点模拟 → 信号与仓位引擎 → 下单/提币/卖出状态机 → 失败重试与熔断 → 交易日志和复盘报表。

九、下一次如何抓住机会：上线前准备清单

订阅 Binance Alpha 上新公告；上线前确认正确合约、目标链、主流池地址与是否允许提币。

提前在 BSC 钱包准备少量 BNB，并预先验证钱包能接收该链上的代币。

预设小额试单额度和最大总额度，不在上线后临时决策。

同时打开 Alpha 限价簿和链上“卖出报价”界面；只记录可执行报价，不记录展示价。

第一次成交后，先完成一次小额提币与卖出；把真实耗时、费用和滑点写回模型。

只有当连续多个采样点仍满足安全垫，且容量足够时才考虑分批加仓；一旦净边际收敛或提币状态变化，立即停止。

十、资料与核验入口

Binance Alpha 机制说明 (<https://academy.binance.com/en/articles/what-is-binance-alpha>)；DOS BSC 合约核验 (<https://bscscan.com/token/0xB0f09ea9ae0515C3551080D4a745C8115aA30e37>)；PancakeSwap DOS/USDT 主池报价 (<https://dexscreener.com/bsc/0xcbeaad783ffd2cea125e9d9b2ec21e639d20fa59>)。历史数据为 2026-08-10 至 2026-08-11 的公开接口快照与 K 线汇总，只用于解释该案例，不构成对未来收益的承诺。

笔记 015: 新增代币对池子的方向是否可行

作者：hellingercheri-gif | 时间：2026-08-11 08:32 UTC | 字数：1673 | 评分：90

来源：<https://intensivecollearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

新增代币对池子的方向是否可行

答案是：可行，但“新增池”只是候选机会入口，不是套利条件。

一个新池要产生原子套利，至少需要：

新池中的资产

- 在其他池子、兑换合约或赎回机制中有价格锚
- 能走出一条回到起始资产的路径
- 扣除价格冲击、手续费、gas和builder费用后仍为正

不同类型的新池，价值完全不同

| 新池类型 | 可行性 | 判断 |

| :----- | :- | :----- |

- | 已有主流资产进入新 DEX | 高 | 有明确外部价格锚，最值得研究 |
- | 同一资产新增费率档 | 较高 | 初始价格或流动性分布可能不合理 |
- | 项目从旧池迁移到新池 | 较高 | 迁移期间容易出现流动性割裂 |
- | 新包装资产/LST/LRT 池 | 中高 | 可利用铸造、赎回或底层资产定价 |
- | 全新代币同时建立多个池 | 中低 | 有闭环，但安全风险很高 |
- | 全新代币只有一个池 | 很低 | 没有第二价格或赎回出口，不是套利 |
- | 极低流动性新池 | 很低 | 账面价差大，但容量不足以覆盖 gas |
- | 带未知 v4 hook 的池 | 高风险 | hook 可改变兑换结果，必须精确模拟 |

单纯监听建池事件并不构成优势

Uniswap v2 Factory	会公开发出 PairCreated , Uniswap v3	会发出 PoolCreated , v4	会发出 Factory
Initialize。任何人都能低成本监听这些标准事件。Uniswap v2			Factory
(https://github.com/Uniswap/v2-core/blob/master/contracts/UniswapV2Factory.sol)、	Uniswap v3		Factory
(https://github.com/Uniswap/v3-core/blob/main/contracts/interfaces/IUniswapV3Factory.sol)			

而且：

- PairCreated 不代表池里已有流动性；
- v3 创建、初始化和添加流动性可能是不同操作；
- v4 还需读取 hook、动态费率和有效 tick 流动性；
- 等到事件确认后再计算，可能已经晚了一整个区块。

已有实证研究发现，sniper bot 会直接扫描 mempool 中的首次加池交易，并争取在同一块紧随其后执行。也就是说，“监听新池”本身已经是成熟且高度自动化的赛道。USENIX : Token Spammers, Rug Pulls, and Sniper Bots (<https://www.usenix.org/conference/usenixsecurity23/presentation/cernera>)

新币池最大的成本是逆向选择

新池中大量“机会”实际上来自：

- 只能买不能卖的 trapdoor/honeypot token
- 动态买卖税
- 黑名单、白名单和交易开关
- maxTx、maxWallet
- 可升级代理和 owner 特权
- transfer/rebase/reflection 等非标准行为
- 假币地址和同名代币
- 流动性可瞬间撤走
- 恶意 hook、router 或回调

研究已经观察到大量允许买入但阻止卖出的 token；有效检测通常需要“买入—卖出模拟”和合约语义检查结合，而不能只看池子报价。Trapdoor Token 研究 (<https://arxiv.org/abs/2309.04700>)

笔记 016: Day 6 (2026-08-11)：跨 CEX/DEX 比價工具——設計日

作者：HenryChang | 时间：2026-08-11 08:58 UTC | 字数：1224 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 (2026-08-11)：跨 CEX/DEX 比價工具——設計日

今天沒有寫程式，做的是比寫程式更花腦的事：把「查一個幣兌在各交易所的價格」這個簡單需求，一路推演成一個有完整規格與實作計畫的系統設計。

今天做了什麼

1. 需求擴充：原本只想做 CLI（給定 USDC/USDT，列出 Binance/Bybit/OKX 與 Uniswap/PancakeSwap/Curve 的價格，並算「賣 A 實拿多少 B」），中途決定升級為類 CMC/Coinglass 的網頁比價儀表板。
2. 關鍵決策（皆已寫入 spec）：
 - 幣種完全通用：alias 表 + 任意 ERC-20 地址（鏈上讀 symbol/decimals）
 - 跨鏈如實呈現：PancakeSwap 查 BSC，每筆報價標注所在鏈
 - CEX 用 orderbook 逐檔吃單模擬、DEX 用鏈上 quoter——兩邊都是「真實可成交」口徑，比較才公平
 - 費用全計入：taker fee、池費、gas（獨立欄位 + 扣 gas 淨值排序）
 - 更新頻率：依各來源速率上限取 ~70% 安全額度做最大化輪詢
3. 三方案比較後拍板：DEX 資料源選「純鏈上直查」為主體（準確度最高、零 API key、延續前幾天手發 eth_call 的寫法），只把最繁瑣的 Curve pool 探索外包給官方免 key API，報價仍問鏈上合約——把 80% 的程式碼風險外科手術式拆掉。
4. 學到的架構判斷：「速率最大化輪詢」= 長駐程序 → 不適合 serverless（Vercel functions 短命），上雲要選容器平台。另一個對稱的洞察：鏈上資料的新鮮度天花板是出塊速度，同一個塊內重複拉取是純浪費——所以排程器對鏈上來源用「區塊感知」策略，先廉價探測新塊再重拉整批。
5. 依賴分層想清楚了：httpx 是對外「發」請求的客戶端（引擎用），fastapi/uvicorn 是對內「收」請求的伺服器（只有 server 進入點用）；uv 按進入點各自解析 PEP 723 依賴，跑 CLI 根本不需要裝 fastapi。分層不是潔癖，是讓引擎之後能直接搬進 bot/。

順手鋪路

collector 會把每筆價差落 JSONL——正好是 Day 5 結尾規劃的「價差超出費率走廊的頻率統計」的資料來源，儀表板跑起來的同時就在攢回測素材。

明日計畫

執行 P1：pricer/ 引擎（3 CEX + 5 DEX venue）+ CLI，用 subagent 逐 task 實作，過驗收 1-5（含與 Day 4 amm_price_scan.py 同池讀數交叉驗證）。

笔记 017: 学习笔记：AMM 价格影响计算（滑点数学）

作者：journeyanhk | 时间：2026-08-11 09:51 UTC | 字数：2108 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习笔记：AMM 价格影响计算（滑点数学）

完成原计划 Day 3-4：深入理解 AMM 恒定乘积公式 $xy=k$ 的价格影响，编写 Python 滑点计算器并用 BSC 链上实时数据验证。

核心概念

1. 恒定乘积公式 $xy=k$
 - 无论发生什么交易，池子储备量的乘积恒定
 - 买入 token0 $\rightarrow x$ 减少 $\rightarrow y$ 必须增加维持 $k \rightarrow$ 价格上涨
 - 价格影响是数学必然，不是市场噪声
2. 价格影响 vs 滑点
 - 价格影响：你的交易本身抬高价格（ $xy=k$ 决定，可精确计算）
 - 滑点：实际成交价 vs 预期价之差（价格影响 + 别人抢先 + 延迟）
 - 有限规模报价已含价格影响，滑点是报价后的额外偏差
3. 价格影响非线性
 - 双曲线 $y=k/x$ 的特性：token 越少越稀缺，价格加速上涨
 - 交易占比 1% $\rightarrow$ 影响 ~1%；占比 10% $\rightarrow$ ~11%；占比 50% $\rightarrow$ ~100%
 - 不能线性外推小交易的价格影响
4. 池子深度决定一切
 - 同样买 10 WBNB：深池(7000 WBNB) 滑点 0.39%，浅池(234 WBNB) 滑点 4.73%
 - 深度差 30 倍 $\rightarrow$ 价格影响差 12 倍
 - 浅池的便宜报价 = 幻觉，一买价格就崩

实操成果

- 编写 04_slippage_calculator.py 滑点计算器（买入/卖出双向）
- 修正手续费方向 bug（V2 手续费从输出侧扣除）
- BSC 链上实时验证（区块 115,270,681）：
- PancakeSwap（25亿池）：\$1,000 买入滑点 27bps，\$10,000 买入 49bps
- BiSwap（8400万池）：\$1,000 买入 97bps，\$10,000 买入 796bps（16倍差距！）
- 多跳闭环模拟：PancakeSwap 买 10 WBNB → BiSwap 卖，净亏 4.94%
- 静态价差看似 0.3%，两边价格影响把利润吃光还倒亏

关键收获

1. 浅池报价 = 陷阱：便宜的价格一买就崩，真实套利要盯深池
2. 小资金在深池滑点可控：\$1000 在 PancakeSwap 仅 27bps，是优势
3. 必须逐腿真实规模模拟：第一步成交后池子状态已变，不能用静态报价（7步法第3步）
4. 0.5% 毛价差不够覆盖滑点：BiSwap 买 97bps + PancakeSwap 卖 ~25bps ≈ 122bps，毛价差需 >1%
5. LP 选深池：手续费多 + 无常损失可控；浅池容易被大单砸穿

课后思考题要点

价格影响非线性来自反比例函数数学特性

\$1000 资金、0.5% 毛价差（600 vs 597）：实际执行亏 ~1.25%，滑点覆盖全部价差

浅池"价差多"是假机会，深池"小价差"才是可执行的肉

下一步

Day 5-7：动手写双 DEX 价差监控脚本

集成滑点计算器：报价时自动算真实可成交价和净利润

从 BSC 开始（Gas 低、试错成本小）

补充学习：资金费率套利研究 + 数据验证

基于青蛙老师文章「入门套利策略保姆级教程：BTC 资金费率套利 488 天净赚 9.57%」，完成策略理解，并用币安官方 API 独立验证全部数据。

策略原理（Delta 中性）

左手：自有资金买 BTC 现货（不借币）；右手：BTCUSDT U本位永续 1x 做空相同数量

BTC 涨跌被对冲抵消，净赚多头定期支付的资金费（每 8 小时结算）

本质：赚市场多空失衡的钱，不赌方向

数据验证结果（币安官方 API，2019-09-10 至 2026-08-11）

拉取 7,582 条结算记录，与青蛙老师数据完全一致：

- 正费率占比 85.58%，负费率 14.42%（完全一致）
- 84 个月中 6 个月为负，最差月 2020-03 为 -0.5319%（完全一致）
- 2026-02~04 连续三个月为负（一致）

额外发现

年度毛费率波动极大：2021 +30.61%（牛市顶峰）→ 2026 +1.28%（熊市）

7 年累计毛费率 +80.56%，简单年化毛 11.64%

青蛙老师净年化 7.16% → 成本约吃掉 4.5 个点（四笔手续费+滑点+基差）

当前 2026-08 费率极薄（年化约 2%），熊时期实盘性价比低

结论

1. 青蛙老师数据真实可靠，文章无夸大
2. 资金费率套利是"时间套利"，与链上"空间套利"互补，Delta 中性是核心认知
3. 当前熊时期费率 2% 左右，适合作为认知学习而非重仓实盘
4. 验证脚本本身是很好的 Python 实操（分页拉取 API + 数据分析）

笔记 018: TVL 是池内全部资产的总价值；active liquidity 才是当前价格位置参与报价的流动性参

作者：Montaire | 时间：2026-08-11 11:57 UTC | 字数：342 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

TVL 是池内全部资产的总价值；active liquidity 才是当前价格位置参与报价的流动性参数。
Tick 是价格刻度，tick spacing 只限制 LP 区间边界；符合间隔不代表该 Tick 已 initialized。
sqrtPriceX96 表示精确价格；Tick 相同不代表价格完全没变。

区间内交易会沿当前 AMM 曲线移动价格；跨 initialized tick 时再应用 liquidityNet 更新活跃流动性。
即使没有跨边界，大交易仍可能产生明显价格影响。
QuoterV2 的 initializedTicksCrossed 是压缩 Tick 位图的 Gas 相关计数，不能直接当作实际越过的 LP 边界数量。

笔记 019: 一、背景与核心思路

作者：anyi | 时间：2026-08-11 12:59 UTC | 字数：748 | 评分：66
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、背景与核心思路

在DeFi的高频套利中，AMM（自动做市商）的非线性定价模型（如Uniswap V2的恒定乘积公式）计算复杂，难以满足实时性要求。解决方案是：在小额交易的前提下，将非线性的价格曲线在当前价格附近进行线性化处理，从而大幅简化计算，同时保证精度。

二、线性近似的优势

计算高效：线性模型简单，适合实时估算和高频交易。
策略优化：能快速计算单笔套利的最优交易量。
通用性强：可推广至不同AMM模型，降低开发成本。
便于多币种套利：能快速评估多个交易对间的价格偏差。

三、公式推导（以Uniswap V2为例）

- 无偏差汇率：即交易量趋近零时的理论汇率，需根据买卖方向及手续费率（如0.3%）分别计算。
- 近似偏差率：即价格曲线在当前点的斜率，通过对AMM定价公式求导得到，买入和卖出方向的斜率不同。
- 线性近似公式：最终得到价格与交易量之间的线性关系：实际成交价格 = 无偏差汇率 + 近似偏差率 × 交易量。

四、复杂AMM模型的通用解法

对于难以直接推导的复杂模型，可采用基于询价的通用方法：
步骤：分别对买入和卖出方向，各取两个小额交易量进行询价，获得对应的成交价格。
建模：利用两点确定一条直线，计算出该方向的无偏差汇率和近似偏差率。
优势：仅需四次询价，成本低；不依赖AMM内部机制，适用于集中流动性等复杂模型。

五、总结价值

线性近似方法能提升决策速度、降低滑点风险、节约开发成本，并能有效支持多币种、跨平台的复杂套利策略

笔记 020: 真实滑点是「凹曲线」，不是直线——线性模型结构性失配。

作者：Freetrip | 时间：2026-08-11 13:15 UTC | 字数：532 | 评分：74
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

真实滑点是「凹曲线」，不是直线——线性模型结构性失配。

看 CAKE：\$50k→\$200k 名义额翻 4 倍，impact 只从 0.357% 涨到 0.832%（约 2.3 倍），不是 4 倍。每 \$1k 的边际冲击随规模下降（\$1k 档 0.055%/k，\$200k 档 0.004%/k）。原因是聚合器在大额时接入更多池，有效深度变大。线性占池比例×系数 在某一档校准好，换个档就偏——大额高估、或小额低估，取决于系数锚在哪。

流动性好的腿，滑点几乎可忽略，成本由 0.25% 池费主导。
WBNB 到 \$200k 才 0.088%。

真正该用单池 $x \cdot y = k$ 、而非 LI.FI/线性系数的，是迷因币。

LI.FI 这条凹曲线只在它路由得到的深池成立。回测的目标是单池、浅、LI.FI 覆盖弱的新迷因币——那里 $\text{impact} = \Delta X / X$ 的闭式解精确成立且冲击很大。所以正确校准姿势是：

深腿（USDT/WBNB 等）：用 LI.FI 实测，系数其实 $\ll 1$ （多池摊薄）。

迷因币腿：直读那个 Pancake 单池的 getReserves，套 $\text{impact} = \text{交易额} / \text{输入侧储备}$ ，系数 ≈ 1 ，别用 LI.FI 也别用线性外推。

笔记 021: Day 07 · 2026-08-11 · 第一周复盘：成本模型 v1

作者：twone | 时间：2026-08-11 13:24 UTC | 字数：2011 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 07 · 2026-08-11 · 第一周复盘：成本模型 v1

本周回顾

Day	内容	关键产出
1	链上套利是什么、怎么做	机会类型地图 + 净收益公式 + 五环节闭环
2	AMM 定价原理	滑点手算；「滑点 $\approx$ 交易量占池深比例」
3	Gas、延迟、MEV	最小可行规模；价差窗口 vs 执行链路耗时
4	跨链桥与 LI.FI	LI.FI API 调通；桥费 = 价差引力下限
5	环境搭建	RPC / DefiLlama 实测可用；钱包与 The Graph 待办
6	报价采集脚本 v1	脚本 + 首批数据：L2 间 USDC 摩擦单边 0.25%

成本模型 v1

预期净收益 = 价差收益

- Gas（主网主导，L2 可忽略）
- 协议费（DEX 每笔 $\sim 0.3\%$ ，套利双边 $\times 2$ ）
- 桥费 / 聚合费（L2 间跨链主导，实测 0.25% 单边）
- 滑点与价格冲击（ $\approx$ 交易量 / 池子有效深度）
- 延迟成本（价差窗口 vs 出块/桥确认时间）
- 失败交易成本（失败率 $\times$ 单次 Gas）
- 资金占用成本（慢速桥 / 再平衡期间的敞口）

本周最重要的认知：成本结构是分场景的，不存在一个通用的“手续费率”。同样一笔套利，主网上 Gas 是头号成本，L2 跨链时桥费是头号成本，大额进小池子时滑点是头号成本。用错场景的成本假设，等于没算。

「屏幕价差」vs「可执行净收益」判别清单

看到一个价差 X% 时，按顺序过这五道筛子，任何一道不过就放弃：

1. 规模筛：价差收益是否大于该链的最小可行成本（主网约 \$5 $\sim$ 20，L2 约 \$0.3）？不够就直接忽略
2. 成本筛：X 是否显著大于全链路成本？基准值：DEX 间同链 $> 0.7\%$ （双边手续费 + 滑点 + Gas）；L2 跨链 $> 0.5\%$ + 滑点（实测往返摩擦 0.5%）
3. 深度筛：目标规模占池子有效深度的比例是否远小于 X？（Day 2：滑点 $\approx$ 占比；v3 要看当前 tick 附近的有效深度，不是 TVL）
4. 速度筛：我的发现-验证-执行链路耗时是否小于价差的历史存活窗口？跨链还要把桥的确认时间算进敞口
5. 竞争筛：这个机会是否长期稳定存在？长期存在的“价差”大概率是成本下限（如桥费 0.25%）而非机会；能赚钱的机会通常存活几秒到几个区块

本周实测锚点（用数据校准的直觉）

L2 间 USDC 转移摩擦：单边 0.25%（LI.FI 固定费主导），100 $\sim$ 10,000 美元量级无规模效应

L2 Gas：\$0.007 $\sim$ 0.03，可忽略；主网拥堵时复杂套利 Gas 可达 \$20+

聚合器路由会随规格切换桥（100U 走 18 分钟的 polymerStandard，大额走 7 秒的 eco）——成本数据必须绑定 tool 和耗时才有意义

第一周结论

第一周的假设——“稳定收益的现实路径是先跑通被动/半自动工具流”——得到支持：光是搞清成本结构，就已经能否决掉绝大多数屏幕上的假信号。但本周也暴露了一个事实：目前采集到的是“成本数据”，还不是“价差数据”。成本模型已经能用，缺的是另一侧——两端的实时价格。这是第二周工具篇要解决的第一个问题。

第二周计划修正

Day 8 拆案例时，优先用成本模型 v1 去套，检验模型是否漏项

报价脚本 v2 提前提上日程：加入 DEX 现货价对比，让采集从“成本”扩展到“价差 - 成本 = 净额”

补齐欠账：The Graph Studio API Key 注册、隔离小额钱包创建（本周内必须完成）

参考资料

本周全部笔记：notes/2026-08-05-day01.md $\sim$ notes/2026-08-10-day06.md

成本数据来源：data/quotes_2026-08-10.csv

笔记 022: Day 7 | 2026.08.11 | 纯内存报价上线验收 + 报价延迟 6.5x 优化

作者：xzone911 | 时间：2026-08-11 13:34 UTC | 字数：1911 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 | 2026.08.11 | 纯内存报价上线验收 + 报价延迟 6.5x 优化

全路线零 RPC：线上验收

322 份 WSOL 两跳 + 687 份三跳生产 summary, 100% quote_source=offline_snapshot。rpc_fallback=0, quote/state selection error=0, fatal=0。WSOL/USDC、2/3/4 hop 所有入口的 Rpc 参数和 fallback 分支已从生产代码移除。缺帧只能 fail-closed。

报价延迟：312ms → 48ms (6.5x)

不是靠加线程，是靠减少无效计算和重复解码：

| 优化 | p50 | p90 | 候选组合 |

|-----|-----|-----|-----|

| 旧版 100 档全量 | 312,937us | 611,527us | 3,776 |

| DLMM 每帧只解码一次 + 方向缓存 | 207,911us | 463,225us | — |

| 17 点探针替代 100 档粗搜 | 65,599us | 158,669us | 800 |

| 方向分拆 + 两 endpoint 覆盖 | 48,175us | 109,559us | 800 |

关键技巧：DLMM 的 pool/bin/bitmap/mint 每帧只解码一次，按 swap 方向保存 bin arrays。两跳 fast lane 用 17 个均匀探针(含端点)+ 两轮 5 等分细化，替代 100 档全量。候选组合从 3,776 减到 800——不是丢了机会，是丢了不会产生不同结果的冗余排列。

延迟真相：build 只花了 66 微秒

以前日志里的 build=1,008ms 把 tip-floor、mint/ATA 查询、余额检查、blockhash 获取、ALT 地址加载全算进去了。

| 阶段 | 旧版标注 | 实际耗时 |

|-----|-----|-----|

| 纯交易构造(签名+序列化) | build=1,008ms | build=20–57μs |

| RPC 预热(tip/余额/blockhash/ALT) | 被算进 build | preparation=636μs |

| 网络提交+leader 调度 | submit=49ms | submit=34ms |

20–57μs 的纯构造基准来自 5,000 次本地迭代(4 hop、ALT、inline tip、签名、序列化)。p50=20μs, p99=33μs, max=57μs。

STONK 两跳：首份四段微秒样本

路线：DLMM → DAMM v2 两跳，输入 11,900,000 lamports，报价毛利 +66,878，eligible=true。data/send/accepted slot 均为 438460524，chain slot 438460527(+3)。preparation=636us，build=66us，RPC submit=34,314us，Jito accept=155,387us。结果 Custom(13) ProfitTooLow，扣 19,000 fee。

从 data 到 chain 的 3 个 slot(~1.2 秒)中，build+submit 只占 34ms，剩下 1.1 秒在等 leader 调度和池状态被竞争者改变。

通知精简

正毛利 signal 只有显式 eligible=true 才进 Telegram。bundle 提交回执、terminal、tracking timeout、Jito/链上失败改为 log-only。保留：普通 RPC 提交通知、真正成功执行的复盘、fatal/panic/landed execution error 报警。

今日认知

把 build 从 1 秒压到 20 微秒不是优化——是把计时搞对了。真正的端到端延迟结构：

端到端 = state_age(状态帧新旧) + prep(后台缓存命中) + pure_build(20–57μs) + submit(~34ms) + leader_gap(data→chain, 1–3 slots)

prep 和 leader_gap 是可压的，build 和 submit 已经是物理极限附近。

笔记 023: D05 | 流动性深度、价格冲击与滑点

作者：averywexler | 时间：2026-08-11 13:40 UTC | 字数：693 | 评分：71

来源：<https://intensivecolearn.org/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D05 | 流动性深度、价格冲击与滑点

深度、价格冲击和滑点有关，但不是同一个概念；只看池子总 TVL 也可能误判当前价格附近的可成交深度。

术语

流动性深度：当前价格附近能以较小价格变化成交的数量；

价格冲击：你的交易改变池子状态或吃掉订单，让平均成交价变差；

滑点：从看到报价到实际执行时，状态和市场变化造成的额外差异；

滑点容忍度：允许实际结果比预期差多少，超出通常回滚；

集中流动性：LP 只在选定价格区间提供资金；

Tick：集中流动性中划分价格区间的离散边界。

参考材料

Uniswap：Concentrated

Liquidity

(<https://developers.uniswap.org/docs/get-started/concepts/liquidity-providers/concentrated-liquidity>)

Uniswap Glossary (<https://developers.uniswap.org/docs/get-started/concepts/glossary>)：查 price impact 与 slippage。

具体例子

同样投入 20,000 USDC：

池 A : 100 ETH / 200,000 USDC , 得到约 9.0909 ETH ;
池 B : 1,000 ETH / 2,000,000 USDC , 得到约 9.9010 ETH。
两池初始价格相同，但池 B 在这个规模下更深。

笔记 024: Day 5 | DOS 跨链搬砖案例研究

作者 : Oxbin | 时间 : 2026-08-11 13:41 UTC | 字数 : 658 | 评分 : 63
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 | DOS 跨链搬砖案例研究

今天做了什么

今天拿 DOS 做了一个真实的跨链搬砖案例。

先核验了 DOS 在 Ethereum 和 BSC 两条链上的官方合约，确认两边通过 LayerZero V2 OFT / OFTAdapter 实现 1:1 跨链。:contentReference\{oaicite:0\}{index=0}

同时梳理了两边主要流动性池：

ETH : Uniswap V3 DOS/USDT

BSC : PancakeSwap V3 DOS/USDT

并开始设计跨链搬砖监控器，不再只比较页面价格，而是对相同数量 DOS 计算两边真实买卖报价。

今天的认识

直接：

ETH 买入 → 跨链 → BSC 卖出

会面临跨链延迟，DOS 的 LayerZero 路径实测通常需要约 1-3 分钟，价差可能在到账前消失。:contentReference\{oaicite:1\}{index=1}

所以更合理的方式是：

两边提前放好资金 → 同时买卖锁定价差 → 之后再通过 LayerZero 做库存再平衡。

代码进展

今天整理了 DOS 跨链套利监控器 v0.1 的需求，准备实现：

ETH / BSC 双边实时 Quote

相同 DOS 数量比较

Gas 和再平衡成本

双向净利润计算

达到阈值后报警

第一版只读，不自动交易

明日计划

先把监控器真正跑起来，验证程序出现的正价差到底是不是可执行机会。

笔记 025: 每日学习笔记 | CEX-DEX套利报价系统

作者 : cawood | 时间 : 2026-08-11 14:05 UTC | 字数 : 609 | 评分 : 52
来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

每日学习笔记 | CEX-DEX套利报价系统

今天进一步简化了CEX-DEX套利的实现思路：白名单Token + 单池 + 单CEX交易对 + Block级链上报价。

核心不是每个区块对300个DEX市场逐个调用报价接口，而是在本地维护池状态。启动时全量读取一次池子的reserve、liquidity、tick等状态；之后订阅新区块和Swap/Mint/Burn等事件，只增量更新发生变化的池。

Quote Engine基于本地状态直接计算不同交易规模下的可执行价格，例如1k、5k、10k、50k USDT，而不是只维护一个spot price。这样300个市场每个Block的计算量其实很小。

CEX端通过WebSocket持续维护盘口和可成交价格，再与DEX本地Quote比较：

CEX可卖价 DEX可买价 Gas 手续费 滑点缓冲 = 套利Edge

系统关键模块可以简化为：

Blockchain Event → Pool State Cache → Local Quote Engine → CEX Price → Spread Engine → Execution

最重要的认知是：链上报价不应该依赖实时RPC查询，而应该把链上状态同步成本地行情系统。
300个市场本身不是瓶颈，真正的复杂度来自多池、多路径、多链和mempool竞争。

笔记 026: Day 7 · 2026-08-11 · 第一周复盘：从“看见价差”到“算清可执行性”

作者：rslofty | 时间：2026-08-11 14:25 UTC | 字数：2880 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 · 2026-08-11 · 第一周复盘：从“看见价差”到“算清可执行性”

一句话总结

第一周没有找到一个可以直接宣称“值得实盘”的套利机会，但建立了一条比较完整的判断链：先用公开数据发现偏差，再把流动性、滑点、协议费、Gas、桥费、资金费率和执行延迟都放进模型，最后才判断净收益是否为正。对我来说，本周最大的收获不是某个具体价差，而是确认了：屏幕上的价差只是线索，扣除全部成本后还能留下来的，才是机会。

1. 本周做了什么

Day 1：先打通数据和工具链

只读 LI.FI Quote API，整理了跨链报价的采集流程，并记录费用、Gas、路由步骤、延迟和原始 response。通过不同链、资产和金额的 quote 对比，先建立“报价 → 成本 → 净结果”的观察框架。

这一天也确认了几个基础边界：跨链执行不是原子的，桥的等待时间本身就是风险；token 地址不能凭记忆填写，必须先从 token 列表或官方数据源取得；同一个报价不能只看 toAmount，还要看 fee、gas 和 route steps。

Day 2：调查 Hyperliquid xyz 美股永续

通过 Hyperliquid 公共 info API 读取 HIP-3 的 xyz 市场，观察美股永续合约的 mark、oracle、funding、成交量和 1 小时 K 线，再和 Yahoo Finance 报价做对照。

得到的判断是：美股盘前、盘后存在理论价差，但通常很难覆盖 taker fee、funding、跨链资金移动和执行延迟；真正值得继续研究的是美股休市期间 oracle 可能冻结，而永续市场仍在交易所形成的 mark/oracle 偏离。不过，这需要更长时间序列和更可靠的盘后真实股票成交数据，不能只凭一个时间点下结论。

Day 3：把“价差”拆成不同套利机制

补看套利机制 Demo，重点理解五类路径：Atomic Arbitrage、Inventory Arbitrage、Convergence / Carry、Protocol Incentives 和 MEV / Order Flow。

其中最重要的一点是：三角套利不能只把三个报价相乘。报价是边际价格，有限金额会沿着 AMM 曲线移动；每一腿都会改变后续一腿的输入和价格。另一个重要认识是，跨链预置库存加后置调仓，和等待跨链完成后再执行，风险结构完全不同。

Day 4：把净收益漏斗写成闭式解

把两个恒定乘积池串联成一个等效池，推导最优交易规模、最大毛利、最小可盈利价差和最小可盈利池深度。核心模型可以概括为：

text

净收益 = 毛利 - Gas - 其他执行成本

并且在两池费率为 0.3% 时，中间价差首先要超过约 0.6% 才有正毛利；加入 Gas 后，真正的门槛还会更高。这个结论比“扫几个金额看结果”更有用，因为它可以在扫描之前先排除明显不可能的池子。

Day 5 / Day 6：把闭式解搬到 V3，并做报价链路验证

V3 的集中流动性不能直接当成固定的 V2 储备。流动性 L、当前价格和 tick 区间共同决定了某个价差带内可以承受的输入量，因此需要用“推到 tick 边界前能消化的输入量”定义 V2 等效深度 $E_{x,eff}$ 。

随后继续验证 LI.FI 的 quote 形状，重点查看不同金额下的 fee、gas 和 route 变化。过程中也确认了一个容易误判的地方：feeCosts 为空不代表没有费用，费用可能落在 step 的 estimate、gasCosts 或桥/DEX 的实际执行字段里，不能只看一个字段就判定“免费”。

2. 本周最重要的三个认识

认识一：深度比价差更硬

价差越大不一定越好。如果池子很浅，稍微放大交易规模就会把价格推回去，最终的实际成交价差会明显小于屏幕上的报价。一个机会至少要同时看：价差、流动性、交易规模、价格冲击和费用。

认识二：最优规模不是越大越好

在 AMM 中，交易规模增加会带来更多毛利，但也会带来更大的价格冲击。存在一个最优规模；超过这个规模，新增输入带来的收益会迅速下降。Day 4 的闭式解和 Day 3 的步进扫描，本质上是在用两种方法回答同一个问题。

认识三：执行层决定理论能不能落地

跨链桥的非原子性、quote 有效期、Gas 波动、路由变化、失败交易成本和资金预置，都会让“理论套利”变成“执行系统问题”。因此，策略判断不能停留在价格公式，必须把真实执行约束放进同一个模型。

3. 本周的负面结论

没有签名或广播任何真实链上交易。

没有把单次 API quote 当成可实现利润。

没有完成 30 天的 mark/oracle 长时间序列回测。

没有证明 Hyperliquid 美股永续与真实股票之间存在稳定、可净赚的套利窗口。

没有证明当前 LI.FI 报价矩阵能直接支持套利执行；目前仍是只读研究和成本验证。

这些“没有证明”本身也是结果：它们防止我把研究样本、模拟数据或理论公式误写成收益记录。

4. 下一周计划

1. 把 V2 / V3 的 net_pnl 计算统一成一个小工具：输入池深度、费率、价差、Gas 和规模，输出毛利、净收益及 break-even。

2. 继续采集同一条路径的多金额 quote，记录 quote 时间、route、fee、gas、状态和失败原因，观察费用曲线是否稳定。

3. 给 E_x,eff 增加 tick 跨越判断，避免用单一等效深度覆盖多个价格区间。

4. 对 Hyperliquid xyz 只做 paper track：先积累 mark/oracle/funding 的时间序列，再决定是否值得继续做周末 gap 假设。

5. 保持只读和模拟边界，暂不连接钱包、不签名、不广播交易。

证据与可复跑入口

LI.FI 文档：<https://docs.li.fi/api-reference/introduction>

LI.FI Quote API：<https://li.quest/v1/quote>

Hyperliquid 公共 API：<https://api.hyperliquid.xyz/info>

本周已记录的 Day 1–Day 6 笔记：同一项目下的历史 check-in，可从 API /api/v1/me/check-ins 复核。

安全边界

本周全部工作以公开 API、文档、报价读取、数学推导和 paper research 为主；没有使用私钥、助记词或钱包签名，也没有进行真实资金操作。

笔记 027: 学习记录

作者：capromka | 时间：2026-08-11 14:32 UTC | 字数：513 | 评分：58

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习记录

今天主要学习了 滑点、Price Impact 与流动性深度。

主要收获

理解了 Price Impact 和 Slippage 的区别：Price Impact 是自己的交易把池子价格推偏了；Slippage 是等待链上执行期间价格发生变化。

明白了 Slippage Tolerance

不是实际手续费，而是自己愿意接受的最大价格偏差；设置越高，交易越容易成功，但也可能接受更差的价格。

学习了流动性深度：池子越深，同样金额的交易对价格影响越小。

对 V2 / V3 有了进一步理解：V2 看整体储备，V3 更重要的是看当前价格附近的活跃流动性，不能只看总 TVL。

了解到不同池型的特点：V2、V3、StableSwap、Infinity，以及它们分别适合什么场景。

今日总结

做 LP 不能只看 TVL 和 APR，还要看真实的流动性深度、交易量和 Price Impact；而做交易时，也要分清 Price Impact、Slippage 和 Slippage Tolerance。

笔记 028: 3. 流动性深度

作者：JackChang-01 | 时间：2026-08-11 14:39 UTC | 字数：605 | 评分：60

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

3. 流动性深度

流动性深度回答的是：

如果我交易这么多，价格会不会被明显推高或压低？

订单簿中的深度

看当前价格附近有多少买单和卖单。

挂单多、数量大：深度好。

挂单少、价格档位稀疏：深度差。

AMM中的深度

主要看：

池子储备规模；

当前价格附近的有效流动性；

你的交易额占流动性的比例。

例如：

用1万USDC交易一个价值1亿美元的深池，影响可能很小。

用1万USDC交易一个只有5万美元的浅池，价格可能剧烈变化。

所以：

“有流动性”不等于“有足够深度”。

4. 价格冲击

价格冲击是：

因为你的交易数量太大，导致实际成交价格偏离交易前价格。

简单理解：

$\lfloor \text{价格冲击} \approx \frac{\text{你的交易量}}{\text{市场有效深度}} \rfloor$

这不是精确公式，但能帮助理解关系：

交易量越大 → 冲击越大。

深度越好 → 冲击越小。

价格冲击和滑点的区别

价格冲击：你的交易自己推动了价格。

滑点：最终成交价和预期成交价之间的全部差异。

滑点可能来自：

价格冲击；

市场价格变化；

区块等待；

其他交易抢先成交；

MEV或三明治攻击。

因此可以把它理解为：

$\lfloor \text{滑点} = \text{价格冲击} + \text{等待期间的其他价格变化} \rfloor$

笔记 029: 【Day 7 打卡】2026/8/11

作者：ed11555 | 时间：2026-08-11 14:43 UTC | 字数：738 | 评分：74

来源：<https://intensivecolearn.org/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 7 打卡】2026/8/11

今日學習：讀取合約數據

完成內容

1. 理解 ABI (Application Binary Interface) 概念
2. 學習 view/pure 函數的調用方式
3. 使用 web3.py 讀取 Uniswap V2 WETH/USDC 池的 reserves
4. 實踐任務：讀取 WETH/USDC 合約數據

關鍵發現

- Uniswap V2 池 reserves 數據可以通過 getReserves() 免費讀取
- 合約 ABI 是與智能合約交互的「說明書」
- USDC 6位小數，WETH 18位小數，價格計算需注意
- view/pure 函數不消耗 Gas，適合免費讀取鏈上數據

實作代碼

python

from web3 import Web3

```
w3 = Web3(Web3.HTTPProvider('https://eth.llamarpc.com'))
pool = w3.eth.contract(
address=w3.to_checksum_address("0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc"),
abi=PAIR_ABI
)
reserve0, reserve1, timestamp = pool.functions.getReserves().call()
```

學習狀態

- 理解 ABI :
- 成功讀取 reserves :
- 計算 WETH 價格 :
- 完成實踐任務 :

明日計劃

Day 8 : 價格監控腳本 + 价差計算

02 LI.FI与跨链 (33 篇)

笔记 030: 链上套利残酷共学 · Day 5 心得

作者 : kako | 时间 : 2026-08-10 15:12 UTC | 字数 : 564 | 评分 : 59

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 5 心得

今日主题 : LI.FI (<https://li.fi/>) 快速上手

今天进入了跨链实操环节, 动手用

LI.FI

(<https://li.fi/>) 查询跨链报价。这一步是从“看别人怎么交易”转向“自己探索可行路径”的过渡。

第一天学跨链套利时就知道跨链成本很高——桥费、两端Gas、延时风险, 但今天通过 LI.FI (<https://li.fi/>) 把几条不同路径的报价并列对比后, 才真正感受到“路径选择”本身就是一门学问。同一条链出发、同一条链到达, 走不同的桥和不同的中间链, 最终到账数量和耗时可以差出好几个档次。

LI.FI (<https://li.fi/>) 这类聚合器的价值在于把碎片化的流动性整合到一个查询界面里。如果靠自己手动去一个个桥网站查报价再手工对比, 不仅效率低下, 而且价差可能在你对比的过程中已经消失了。聚合器通过同时查询多条路径, 帮套利者抢到了“比较”这个环节的时间优势。

动手跑了三条路径并截图存档的过程也让我意识到, 报价查询只是跨链套利的第一步。查到的报价是理想状态下的估算值, 真正执行时还要考虑目标链的 Gas 费是否足够、跨链桥的当前拥堵状况、以及最坏情况下交易失败后资金被卡住的处理方案。这些现实因素在报价面板上是看不到的。

笔记 031: 一、RPC 选型 : 小白别自建, 别全付, 混合最香

作者 : jinbuge101-art | 时间 : 2026-08-10 15:14 UTC | 字数 : 3427 | 评分 : 88

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、RPC 选型 : 小白别自建, 别全付, 混合最香

群友吵“QuickNode 49 还是 78”其实没抓重点。真实分层是这样的 :

| 场景 | 推荐方案 | 月成本 | 说明 |

| :----- | :----- | :----- | :----- |

| 你现在 Day 13 阶段 (CCXT 抓 CEX + 偶尔 eth_call 试跑) | 公共 RPC (Chainlist 找) + CCXT 内置 | 0 元 | 限速 1req/s 无所谓, 你本来就不是高频 |

| 写脚本监控 Uniswap V3 slot0 / 批量 staticcall 探测 hot pool | QuickNode / Alchemy 免费档或 x402 按量付 (QuickNode 钱包付 USDC, 免费层 100 万 credit/月) | 0~10 元 | 不用绑信用卡, 够你跑教学脚本 |

| 真要多链 (ETH+BSC+Base+Arb+Solana) 常驻哨兵 | QuickNode 多链套餐 (现最低档约 \$78/月) 或 Helius (Solana 专用) | ~500 元 | 比自建 VPS 省心, VPS 自建还要管同步、磁盘、被 slash |

| 冲 MEV / 抢块 / 私有 Bundle | 自建 Reth + bloXroute BDN + Flashbots Protect | 服务器 \$200+/月 + 极客人力 | 你现阶段绝对别碰 |

结论：测试阶段0 元起步就行。等你脚本真能稳定报出“扣完成本为正”的机会，再花那 78 刀。群友说“200 美金/月测试成本”是把 VPS+多链RPC+小额资金占用全算进去了，对单人学习者偏贵。

二、多链资金：66% 的跨链套利机器人根本不实时跨链

这是群聊里最有价值但“自留资金最快，各链都留资金”的论文背书：

跨域 MEV 研究追踪 9 条链 24 万笔套利，66.96% 用已在两条链预置的库存，不碰桥。

正确姿势：ETH 上留一点 ETH/USDC，BSC 上留一点 BNB/USDT，Base 留一点 USDC。发现价差→两边双边下单→事后再用低价时段做库存再平衡（rebalance）。

临时跨链=送死：LI.FI 实测 1000 USDC 跨 ETH→Arb 固定吃 25 bps + Gas $\approx$ 27.5 bps，表面价差小于 30 bps 全亏。

你之前写的 day12_loss_calculator.py 逻辑完全对，把“临时跨链”模式跑一遍就会放弃这条路。

三、GitHub 仓库怎么用 (qiaopengjun5162/onchain-arbitrage-colearn)

你贴的链接解析失败了，但搜索确认这是共学营的公开知识库，含：

482 篇笔记按 9 主题索引（AMM/ LI.FI/ MEV/ 框架/ LP/ 实盘等）

8 个监控哨兵（上市公告、OI 异动、清算、跨所价差、资金费率拥挤度…）

Python + Rust 双实现，Day01 克隆 MEV sandwich-rs，Day02 写 LI.FI Go 后端+Next.js 看板

给你的用法（别全读）：

1. 只把 8 个监控哨兵 里跨所价差哨兵和资金费率拥挤度哨兵的 Python 版扒出来，和你你 Day12 的 CCXT 脚本做合并。
2. lifi/ 目录下的 quote 请求代码直接抄，把你 Day12 脚本里的“假想成本”替换成真实 LI.FI /quote 返回的 toAmountMin。
3. Rust 的 mev.rs、三明治机器人、别看——那是给有自建节点的人写的。

四、BSC 那个 token (0xb0f09ea9ae0515c3551080d4a745c8115aa30e37) 案例怎么“复盘”而不“接盘”

群友问“项目方加错池子拉到 1.5，被撸 50w u，怎么赚到”——这种 case 的本质是：

1. 不是套利，是 MEV backrun + JIT 流动性：项目方/巨鲸大单砸进浅池，价格瞬间偏离相邻池子，机器人用闪电贷在低价池吃、高价池吐，或者在大单前加流动性吃手续费。

2. 你作为无节点、无币、无合约的小白，三种合法参与方式：

LP 费率监控（群友说“我吃了点 lp 手续费”）——写脚本监控该池 feeGrowthGlobal 突增，下次类似新币开盘时，提前在相邻主流池（如 WBNB/USDT）挂窄区间 V3 LP，赚 JIT 流量费，不赚价差。

失败交易反推：用 BscScan API 拉该 token 当天所有 reverted swap tx，看机器人 gas 消耗和 staticcall 路径，练“从失败 tx 学成本结构”的本事（这是笔记里提的方法论）。

CEX 侧慢半拍：等链上乱战结束、CEX 上该币出现滞后价差时，用你 Day12 的 CCXT 脚本吃 1~2 分钟级别的收敛，不碰链上。

3. 绝对别做：没币想“撸”、想仿写合约、想用公共 RPC 抢 1.4 狙击手的单——那 1.4 进去的两个狙击手是带 Jito Bundle 的，你公共 RPC 看到的时候他们已经结算完了。

五、论文圈那句黑话翻译（群友装 X 你不用怕）

Graves 预计收入低：arXiv 2507.13023 测 19 个 CEX-DEX searcher 19 个月抽 2.338 亿刀，但净留 9010 万，剩下 1.437 亿分给 builder；Graves 是其中小角色，链上那条腿单算当然薄。

做市商负手续费：Wintermute 这类和 CEX 签返佣协议，挂单返费，表面负 cost；但利润来自订单流垄断+CEX 返佣+MEV 分成，不是你我能碰的。

CEX-DEX 被巨头垄断：Cross-Exchange-Dex，Wintermute + SCP/beaverbuild 占 75% 份额，新人进去就是给 builder 交保护费。

论文去哪找：arxiv.org 搜 “CEX-DEX extracted value” \、Google Scholar 镜像、让 AI 帮搜标题再你去 arxiv 核对，别信 ResearchGate 付费墙，群友也是 arxiv+谷歌搜的。

笔记 032: 今日打卡

作者：elipitic | 时间：2026-08-10 15:16 UTC | 字数：2634 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡

打卡日期 (UTC+8)：2026-08-10

今日学习主题

今天的重点不再是单纯记录 Li.Fi 跑通了哪些报价，而是进一步分析一个更关键的问题：

为什么这条路径看起来有利润，但实际上套不上。

也就是说，今天的学习重点是把“表面套利”拆开，理解它为什么在真实执行层面不成立。

一、今日分析对象

今天继续围绕前一天跑出来的 Li.Fi 报价结果做复盘，重点看两条 round-trip 路径：

1. Ethereum → Arbitrum → Ethereum
2. Ethereum → Base → Ethereum

在表面上，某些单跳报价尤其是方向，会给人一种“美元价值似乎增加了”的感觉，但在把路径完整串起来之后，结果并没有形成真实套利。

二、为什么“套不上”

1. 单跳报价不等于完整套利路径

Li.Fi 返回的是某一跳、某一时刻、某一输入规模下的报价。

问题在于：

- 套利判断不是只看 单跳
- 而是要看 完整往返路径
- 还要把每一跳的 fee、gas、滑点与数量损耗都一起算进去

所以今天最先确认的一点是：

单跳看起来有优势，不代表完整路径有正收益。

2. token 数量先天已经在衰减

今天分析里最关键的一点是：

- 1000 USDC 跨链过去，实际到手通常不是 1000
- 当前样本中多条路径到手只有 997.5 USDC 左右

如果往返两次都出现类似损耗，那么仅 token 数量层面就已经在缩小：

第一次：1000 → 997.5

第二次：997.5 → 再被扣一次

这意味着：

还没算 gas 之前，token 数量本身就已经不是平的。

所以很多“看起来有价差”的路径，实际上在 token 数量维度已经不成立。

3. Li.Fi 固定费用会重复吃掉利润

今天的分析还明确了一个很重要的点：

- Li.Fi 的固定费用已经被嵌入 toAmount
- 也就是说，表面输出数量已经不是“毛值”，而是被扣过一层费用后的结果

当你做 round-trip 的时候：

- 去程扣一次
- 回程再扣一次

这会导致：

即使两端价格差存在，只要价差不够厚，很容易被双边费用直接吃光。

4. gas 成本会把最后一层表面利润打掉

今天继续核对后，发现 gas 虽然单跳看起来不高，但在完整路径里不能忽略：

Ethereum 侧 gas 明显更高

回程还要再付一次 gas

gas 是真实成本，不会像“表面美元估值”那样给你错觉

所以今天更清楚地理解到：

很多表面套利不是死在路径逻辑，而是死在最后那一点真实 gas 成本上。

5. 不同链上的 USD 标价可能制造“假利润”

这是今天最关键的收获之一。

在 Li.Fi 的返回结果里，不同链上的同名稳定币可能会显示出不同的 priceUSD。比如：

- Ethereum 上的 USDC 一个价格
- Base 上的 USDC 另一个价格

这会造成一个视觉错觉：

- 你会看到某一跳的 toAmountUSD 好像比 fromAmountUSD 更高
- 于是误以为路径里产生了利润

但今天分析后确认：

这很可能只是元数据估值差异，不是你真实能兑现的可成交利润。

如果没有一个外部可成交退出价格来验证，这种“利润”大概率只是：

- 标价差
- 估值差
- 不是可执行套利

6. quote 是独立的，不是原子执行的

今天还明确了一点：

- 去程 quote 和回程 quote 是分别获取的
- 它们不是链上原子地一起保证执行
- 中间还会受到时间、流动性变化、路由切换和竞争的影响

所以就算你现在拿到两条看上去还不错的 quote：

- 等你真的执行第二跳时，条件可能已经变了
- 真正的返程未必还能按刚才那样成交

这意味着：

报价可以拿来作为候选分析，但不能直接当成无风险套利证明。

三、今天的结论

今天我真正搞清楚的是：这条路径之所以“套不上”，核心不是一个原因，而是多个因素叠加：

1. 单跳报价不能代表完整路径
2. token 数量在往返过程中已经衰减
3. Li.Fi 固定费用双边重复扣除
4. gas 成本进一步吃掉边际利润
5. 不同链的 USD 标价差制造了表面利润错觉
6. quote 之间不是原子执行，真实执行风险更大

所以今天最重要的学习成果不是“找到套利”，而是：

确认了为什么这类看似漂亮的路径，其实只是伪套利。

四、这对本期共学的意义

今天这部分分析非常重要，因为它把我从“会看报价”推进到了“会拆穿表面套利”。

这意味着后续在判断机会时，我不能只问：

- 有没有价差

而必须继续问：

- token 数量是否真的在增长？
- 双边费用后还剩什么？
- gas 之后还有没有正收益？
- 这个利润是不是只是链上元数据估值差？
- 第二跳执行时条件还在不在？

这才是更接近真实套利研究的方法。

五、下一步计划

接下来我会继续推进：

1. 把 Li.Fi 报价分析标准化，不只看 toAmountUSD，而是优先看 token 数量、费用和 gas
2. 引入外部可成交价格，避免被链上元数据估值误导
3. 继续比较不同金额规模下 route 是否切换，观察路径稳定性
4. 逐步建立“伪套利识别 checklist”，提高机会筛选质量

六、今日完成标准检查

- [x] 完成了对 Li.Fi 往返路径的复盘分析
- [x] 识别出“表面套利”不成立的多个原因
- [x] 明确了费用、gas、USD 标价差与非原子执行风险的影响
- [x] 把今天的重点从“跑报价”升级成“解释为什么套不上”

一句话总结

今天最重要的收获不是发现了套利，而是明确理解了：一条路径为什么看起来有利润、但在真实执行里根本套不上。

笔记 033: Day 6 · 设计可验证的套利信号状态机

作者：Junfan Chen | 时间：2026-08-10 15:32 UTC | 字数：2255 | 评分：90

Day 6 · 设计可验证的套利信号状态机

日期 2026-08-10

主题 让 collector 不只输出数字，而是给出信号所处阶段和拒绝原因

今天的推进

前几天已经逐步明确：

同币种跨链要用 token 数量口径，避免把 priceUSD 偏差当成收益；

AMM 定价必须用整数并与链上 Router 做精确校准；

策略筛选应使用 toAmountMin，而不是乐观的 toAmount；

字段缺失不能静默变成零；

RFQ/intent 与 AMM 的风险模型需要分开。

今天继续把这些原则收敛成一个信号状态机。目标是让每个候选机会都有可追踪的生命周期，而不是只保存一条“收益为正”的记录。

1. 为什么需要状态机

单个正 bps 只能说明某个时刻出现了一个数字，不能回答：

数据是否完整；

报价是否仍有效；

成本是否扣全；

最差成交结果是否仍为正；

信号是否重复出现；

最终失败在发现、验证还是执行阶段。

如果不记录这些状态，后续回溯很容易把“报价成功”误当成“交易可执行”，也无法统计信号从发现到消失的真实寿命。

2. 候选信号的六个状态

text

observed → 看到原始价差

normalized → 统一 token、decimals、方向与计价口径

costed → gas、协议费、桥费等已纳入

qualified → 以 toAmountMin 计算后仍满足阈值

stale → 报价过期、区块变化或边际消失

rejected → 数据不完整、成本为负或风险条件不满足

只有 qualified 能进入进一步验证。其余状态同样需要保留，因为失败样本能告诉我们策略的真实边界。

3. 每次状态转换都要有二值判据

延续 Day 2 的经验，状态转换不能依赖模糊判断：

observed → normalized

必须满足：

token 地址与 decimals 已确认；

from/to 方向一致；

同币种路径使用 token 数量口径；

原始 payload 已保存。

normalized → costed

必须满足：

所有已知费用都有明确值；

null、missing 和真实 0 已区分；

gas 已按本次本金换算成 bps。

costed → qualified

必须满足：

text

executable_bps > threshold_bps

slippage_band_bps < gross_edge_bps

quote_age_ms < max_quote_age_ms

并且 RFQ/intent 与 AMM 使用各自的附加条件。

4. 拒绝原因必须结构化

后续不只输出一段错误文本，而是保存稳定枚举：

```
text
missing_cost
oracle_skew
negative_after_cost
band_over_edge
stale_quote
block_mismatch
route_unknown
rfq_expired
parse_contract_changed
```

这样才能统计：

哪一类假机会最多；
哪个链对主要被 gas 吃掉；
哪种路由经常因报价过期失败；
哪些失败来自数据契约，而不是市场本身。

5. 从单点收益转向信号寿命

后续 collector 对同一路径和资金档位生成稳定的 signal_key，例如：

```
text
from_chain / to_chain / from_token / to_token / amount_bucket / route_type
```

并记录：

```
text
first_seen_at
last_seen_at
qualified_count
consecutive_qualified
peak_executable_bps
latest_reject_reason
```

一个边际只出现一次，可能只是噪声；连续多个采样周期保持 qualified，才说明它可能具有可重复性。

6. 当前结论

套利信号的核心不应是“当前收益多少”，而应是：

它经过了哪些验证、在最差成交口径下是否仍成立、持续了多久，以及最后为什么失效。

状态机可以把发现、成本验证、时效性和失败复盘串成同一条证据链。这样未来无论做回测、告警还是小额验证，都不会把不同质量的样本混在一起。

下一步

- [] 为候选记录增加 status、reject_reason 和状态时间戳
- [] 设计稳定的 signal_key
- [] 统计连续 qualified 次数和信号寿命
- [] 对 RFQ/intent 增加报价有效期与成交结果字段
- [] 对 AMM 增加区块一致性、滑点带宽和失败 gas 字段
- [] 只对可重复、证据完整的信号触发告警

今天的收获是：一个好的

collector

不是机会排行榜，而是证据生成器。它既要证明为什么值得继续，也要清楚记录为什么应该放弃。

笔记 034: 链上套利学习笔记 Day 6：成本模型 —— 把直觉变成公式

作者：倪响 | 时间：2026-08-10 15:39 UTC | 字数：1954 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习笔记 Day 6：成本模型 —— 把直觉变成公式

2026-08-07 | 残酷共学 Day 6

前五天的碎片拼起来

每天学了一种成本，今天把它们合在一起：

来源	成本项	特点
Day 1	Price Impact	随交易量超线性增长
Day 2	双向 Impact + 双向手续费	DEX 间套利两边都要算
Day 3	三向 Impact + 三笔 Gas	三角套利更贵
Day 4	桥费 + 跨链协议费 + 双链 Gas	跨链固定成本高，但 Impact 低
Day 5	闪贷费 + 清算奖励递减	清算协议的进入成本

统一公式

不管什么类型的套利，净利润都等于同一件事：

净利润 = 理论收益 - Price Impact - DEX 手续费 - 协议/桥费 - Gas

每一项展开：

理论收益：按屏幕价格算出来的“假设无损”收益。比如两个 DEX 价差 1%，1000U 进去，理论收益 \$10。但这是从来拿不到的。

Price Impact：你的交易推高买入价格、压低卖出价格。套利金额越大，Impact 越严重。跨链套利的 Impact 通常小一些（跨链流动性更分散）。

DEX 手续费：Uniswap V2 0.3%，每步交易都收。N 步套利就是 $N \times 0.3\%$ 。

协议/桥费：跨链特有。LI.FI 固定费 $\sim \$1$ ，桥协议费约 0.1-0.3%。同链套利没有这一项。

Gas：主网约 \$5-20/笔。N 笔交易就是 N 倍。Layer 2 的 Gas 便宜很多（Arbitrum 上 $\sim \$0.02-0.05$ /笔）。

套利可行性判断

有了成本模型，判断逻辑很简单：

if 理论收益 > Price Impact + 手续费 + 桥费 + Gas:

 可以做

else:

 不值得

但这个判断里最不稳定的变量是两件事：

1. Gas 波动：EIP-1559 下 base fee 随网络拥堵剧烈波动，上一秒 \$5 下一秒可能 \$20
2. MEV 风险：即使算出来利润为正，交易在 mempool 里可能被抢跑，实际执行结果完全不一样

所以成本模型的价值不在于“精确算出一个数字然后马上交易”——而在于快速过滤。99%

的价差信号在成本计算这一步就被否掉了，省下大量时间。

五种套利的成本对比

拿 1000U 本金为例，在 Arbitrum 上做一个估算：

套利类型	步数	Price Impact	手续费	Gas	总成本	需要的最小价差
DEX 间（同链）	2	约 0.2-0.5%	0.6%	$\sim \$0.05$	1-2%	$> 1.5\%$
三角套利	3	约 0.3-0.6%	0.9%	$\sim \$0.08$	1.5-2%	$> 2\%$
跨链（LI.FI）	3-4	约 0.1%	0.3%	$\sim \$3(L1)$	0.5-1%+固定	看固定费

一个感受：对个人来说，跨链和清算的成本结构比 DEX 间套利更友好。DEX 间和三角的变量太多、波动太快，成本模型算出来的“利润”大概率在执行时就不存在了。

共学打卡：建立了统一的套利成本框架——理论收益减五项成本。这个框架的价值不在于精确计算，而在于快速过滤——99% 的价差信号在成本这一步就挂了。Arbitrum 上小额 DEX 间套利的综合成本约 1-2%，跨链虽然固定成本高但有时间窗口。

笔记 035: 使用 LI.FI 获取并拆解路线报价

作者：Marco | 时间：2026-08-10 15:45 UTC | 字数：731 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

使用 LI.FI 获取并拆解路线报价

第五天的学习重点是理解如何使用 LI.FI 获取路线报价，并拆解一条跨链或跨协议交易路线的具体组成。对于套利研究来说，路线报价不是一个简单的最终输出数字，而是一组需要逐项核对的执行信息。

一条完整路线通常包含源链、目标链、输入资产、目标资产、输入金额、预计输出金额，以及中间经过的桥、DEX 或其他协议。不同路线可能在费用、预计耗时、流动性和执行步骤上存在明显差异，因此不能只比较最终到账数量，还要理解资产究竟经过了哪些协议。

分析 LI.FI 报价时，至少需要关注几个方面：第一是输入和输出资产是否准确，避免把不同合约地址或包装资产误认为同一种资产；第二是路线中的桥费、DEX 手续费和 Gas 成本；第三是预计交易时间和跨链确认时间；第四是报价的时间戳、有效性以及重新请求后是否仍然成立；第五是交易数据和签名要求，确认报价信息不等于已经执行交易。

对于套利判断，建议固定源链、目标链、资产和金额，在不同时间获取多组报价，再比较路线差异。记录时应保存原始响应、路线步骤、费用拆分、预计到账数量、预计耗时和证据链接。只有把毛价差扣除全部成本，并考虑延迟和失败风险后，才能判断路线是否具备研究价值。

今天的关键知识点是：LI.FI 可以帮助发现和比较跨链路线，但聚合器的推荐路线不等于套利机会，也不等于成交保证。真正有价值的分析，是把一条复杂路线拆成可理解、可计算、可复核的步骤。

下一步可以固定一组源链、目标链、资产和金额，在三个不同时间点获取报价，建立路线对照表。第六天将进一步建立价格和路线的正常状态基线，区分真实机会与短暂报价波动。

今日小结：先看懂路线，再计算成本；先验证报价，再判断套利。

笔记 036: August 10, 2026 · Day 6 / 21 · 时间分配：先动手 60min → 后概

作者：clare ma | 时间：2026-08-10 15:46 UTC | 字数：3653 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

August 10, 2026 · Day 6 / 21 · 时间分配：先动手 60min → 后概念 40min → 打卡 20min

学了什么

LI.FI route 是执行前报价，不是真实交易结果。

toAmount 是预计到账数量，toAmountMin 是最低到账保护边界，不是桥费。

feeCosts 已经从预计到账数量中扣除；gasCostUSD 是独立的 Gas 估算。

executionDuration 是用户侧预计填充时间，不等于 Relayer 收回资金的最终结算时间。

跨链流程中，用户到账和 Relayer 获得最终结算是异步的。

Across 的主要角色：

Relayer：先垫付目标链资产；

LP：提供流动性；

Dataworker：提出结算 bundle；

任何人都可以挑战 bundle；

如果没有 Relayer 及时填充，可以使用 LP 资金进行 slow fill。

路线选择存在不同目标：Polymer 更偏向预计到账数量，Across 更偏向速度；CHEAPEST 和 FASTEST 不是同一个指标。

做了什么

因为 LI.FI Widget 页面需要登录，本次改用 LI.FI Advanced Routes API 做只读查询。

固定路径：

Ethereum → Arbitrum

USDC → USDC

分别查询：

100 USDC = 100000000 raw units

1000 USDC = 1000000000 raw units

没有连接钱包，没有使用真实资金，没有签名，没有提交交易。

记录并比较：

toAmount

toAmountMin

feeCosts

gasCostUSD

executionDuration

tool

route 标签

使用 JSON 中的 USDC 价格 1.0003394947 USD/USDC，将 Gas 从 USD 近似换算为 USDC，以便比较总成本。

得到了什么结果

100 USDC 的 AcrossV4 报价

toAmount : 99734564 raw = 99.734564 USDC

toAmountMin : 99734564 raw = 99.734564 USDC

LIFI Fixed Fee : 0.250000 USDC

Relayer fee : 0.009975 USDC

Relayer gas fee : 0.005461 USDC

feeCosts 合计 : 0.265436 USDC

gasCostUSD : 0.0402 USD

executionDuration : 2，属于报价估计

验证关系：

100 99.734564 = 0.265436 USDC

这个差额正好等于三项 feeCosts 合计，说明 feeCosts 已经从预计到账数量中扣除；Gas 没有包含在 toAmount 中。

Relay 报价中的最低到账边界

Relay 路线曾返回：

toAmount : 99.718804 USDC

toAmountMin : 99.619086 USDC

差额 : 0.099718 USDC

差额相对于预计到账约为 0.1%

这个差额是最低到账保护空间，不是桥费，也不是已经发生的真实滑点。

Across 中 toAmount 与 toAmountMin 相同，也不能因此证明真实执行滑点为 0，因为本次没有执行交易。

1000 USDC 的 Polymer 与 Across 对比

Polymer：

toAmount : 997.500000 USDC

toAmountMin : 997.500000 USDC

LIFI Fixed Fee : 2.500000 USDC

gasCostUSD : 0.1330 USD

executionDuration : 1080

标签：RECOMMENDED / CHEAPEST

Gas 换算后估算总成本：2.6329548625 USDC

Across：

toAmount : 997.394862 USDC

toAmountMin : 997.394862 USDC

LIFI Fixed Fee : 2.500000 USDC

Relayer fee : 0.099750 USDC

Relayer gas fee : 0.005388 USDC

feeCosts 合计 : 2.605138 USDC

gasCostUSD : 0.0542 USD

executionDuration : 2

标签：FASTEST

Gas 换算后估算总成本：2.6593196056 USDC

在这组 1000 USDC 报价下：

Polymer 的估算总成本略低；

Across 的预计速度更快；

不能因为 Polymer 更便宜，就断言它在所有情况下都更好；

不能因为 Across 更快，就断言它的风险和成本更低。

“LI.FI Fixed Fee”并不是绝对金额固定，而是按输入金额的 0.25% 收取：100 USDC 时为 0.25 USDC，1000 USDC 时为 2.5 USDC。

遇到什么失败 / 卡点

一开始尝试使用 LI.FI Widget，但页面需要登录；后来切换为只读 API。

一开始把 toAmountMin 误解为桥费，后来区分为最低到账保护边界。

一开始把报价损耗叫作滑点，后来区分为：

feeCosts：报价中已经扣除的费用；

gasCostUSD：独立的 Gas 估算；

真实滑点：报价到实际执行之间的变化，本次没有执行，无法观察。

一开始把 executionDuration 理解成完整结算时间，后来理解为用户侧快速填充时间；Relayer 资金回收还要经过后续 bundle 结算。

1000 USDC 时 LI.FI 的路线选择发生变化，说明路线会随着规模和目标函数变化。

API quote 没有提供真实失败交易成本、真实到账延迟和用户资金占用成本。

八项成本状态：

1. 价差收益：本次不是 DEX 套利交易，没有形成可执行价差收益。
2. Gas：已量化，但只是报价估算。
3. 协议 / 服务费：已量化。
4. 桥费：Across 没有单独名为 Bridge Fee 的字段，部分桥接相关成本以 Relayer fee 和 Relayer gas fee 展示。
5. 价格冲击：Across 路线没有显示 DEX swap 步骤，未观察到 AMM 价格冲击；不能泛化为真实价格冲击为 0。
6. 延迟成本：有 executionDuration 估计，但真实到账延迟和最终结算延迟尚未量化。
7. 失败交易成本：本次没有执行交易，尚未量化。
8. 资金占用成本：知道 Relayer 资金会在最终结算前被占用，但本次没有量化成用户可执行的成本。

下一步验证什么

Day 7 进入“MEV：你的对手是谁 + 周复盘”：

学习公开 mempool、三明治攻击、私有 mempool 和 bundle；

理解为什么公开提交的裸套利交易可能被抢跑；

整理第一周词汇表；

建立候选机会成本核查清单；

继续只做观察、计算和 Paper Trading，不连接真实资金、不签名、不提交交易。

官方资料：

LI.FI Advanced Routes API：

<<https://docs.li.fi/api-reference/advanced/get-a-set-of-routes-for-a-request-that-describes-a-transfer-of-tokens>>

Across Actors：

<<https://docs.across.to/introduction/actors>>

Across Security Model：

<<https://docs.across.to/introduction/security>>

笔记 037: LI.FI 路由基础

作者：adurey | 时间：2026-08-10 15:49 UTC | 字数：1034 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 路由基础

阶段：阶段二：使用 LI.FI 发现路径与机会 今日问题：LI.FI 聚合了哪些跨链流动性和路由信息？

今日目标

阅读 LI.FI 官网和文档，写出一次报价需要的关键输入。

学习记录

LI.FI 一次报价的关键输入

LI.FI 单步报价接口：GET <https://li.quest/v1/quote>

6 个必填参数

1. fromChain：来源链，填写 Chain ID 或链 key
2. toChain：目标链，填写 Chain ID 或链 key
3. fromToken：卖出/支付的代币，填写合约地址或符号
4. toToken：希望收到的代币，填写合约地址或符号
5. fromAmount：输入数量，必须包含代币精度 例如 USDC 有 6 位小数：1 USDC = 1000000
6. fromAddress：发送方钱包地址

常用可选参数

toAddress：收款地址；不填则默认等于 fromAddress

slippage：最大滑点，如 0.005 表示 0.5%

order：选择 CHEAPEST 或 FASTEST

allowBridges / denyBridges：限定跨链桥

allowExchanges / denyExchanges：限定 DEX/聚合器

maxPricelmpact：最大价格影响

preset=stablecoin：稳定币路线优化预设

最小请求示例

GET https://li.quest/v1/quote

?fromChain=1

&toChain=137

&fromToken=USDC

&toToken=USDC

&fromAmount=1000000

&fromAddress=0xYourWalletAddress

一句话总结：报价需要明确“从哪条链、用什么币、付多少、由谁支付，以及到哪条链、换成什么币”。

笔记 038: markdown

作者：y-cat-van | 时间：2026-08-10 15:52 UTC | 字数：350 | 评分：53

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

学了什么

昨天的模型是“一次性算一个场景”，今天要把它变成可复用的引擎，核心是两条工程原则（借我做原型的老规矩）：

CONFIG 与逻辑分离：所有业务参数（各链 Gas、池费率、桥费、滑点经验值、资金年化）集中成一个 CONFIG 字典。改市场/改场景 = 改 CONFIG，计算逻辑一行不动。

纯计算函数：evaluate(opportunity) 无副作用，输入一个机会字典，输出净收益 + 判断。这样它既能被脚本直接测，以后也能干净地被 Agent 调用。

概念上，这就是把“我脑子里的成本清单”外化成一个机器可以反复执行的判断流程 —— Agent 的本质不是聊天，是把一套判断固化下来重复跑。

笔记 039: 8月9日（周日）—— 拥抱 Agent：Hermes AI Agent 入门与实操

作者：Cool-CoCo | 时间：2026-08-10 15:58 UTC | 字数：4328 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月9日（周日）—— 拥抱 Agent：Hermes AI Agent 入门与实操

今天把重点放在了 Hermes AI Agent 上。之前只是听别人提过，今天自己动手研究了一下，发现它确实有可能改变我的研究方式。

一、为什么研究需要 Agent？

之前的研究流程是：

发现一个异常 → 手动搜索资料 → 查看数据 → 记录结果

这种模式有几个问题：

1. 效率低：需要手动重复操作。
2. 容易遗漏：无法 24 小时监控。

3. 不可重复：每次研究都是“一次性”的，没有沉淀。

而 Agent 可以帮助实现：

- 1. 信息收集与整理：自动搜索网页、读取文档。
- 2. 多来源数据查询：同时查询多个链、多个 DEX 的数据。
- 3. 研究任务拆解：把一个复杂任务（比如“评估 Arbitrum 和 Optimism 之间的套利机会”）拆解成多个子任务（查询价格、查询费用、计算成本）。
- 4. 工具调用：帮我执行终端命令、调用 API。
- 5. 定期监控与提醒：设置定时任务，监控特定指标，在条件满足时通知我。

本质上，Agent 的作用是让研究从“一次性的手动行为”变成“持续运行的自动化系统”。

二、认识 Hermes AI Agent

它是什么？

Hermes Agent 是一个开源、自进化的 AI 智能体，由 Nous Research 开发。它不像 ChatGPT 那样每次对话都重置，而是拥有持久记忆，能记住你的偏好、项目背景，甚至能从过往任务中学习，自动生成“技能”（Skills），越用越顺手。

核心特性

表格

特性	说明
持久记忆	跨会话保存信息，重启不丢失。
技能系统	完成任务后自动总结经验，生成可复用的技能文件，下次直接调用。
多平台接入	通过 Gateway，可以接入 Telegram、Discord、飞书等平台，随时随地使用。
灵活部署	支持本地、Docker、云端服务器。
多模型支持	支持 OpenRouter、Anthropic、DeepSeek、Kimi 等 20+ 模型提供商，自由切换。

和普通 AI 助手的区别

表格

特性	普通 AI 助手	Hermes Agent
记忆能力	会话级，重启丢失	跨会话持久记忆
学习能力	固定能力，无法自我提升	自动创建技能，越用越聪明
使用方式	网页/APP 界面	CLI、Telegram、飞书等多平台
部署方式	云端服务	本地部署，数据完全可控

三、快速上手：安装与配置

考虑到我是在 Windows 上，但安装 WSL2 有点麻烦，我决定先试试在已有环境（macOS/Linux 虚拟机或 WSL2）上安装。如果实在不行，就先用云服务器。

我的步骤：

- 1. 打开终端。
- 2. 运行一键安装命令：

bash

```
curl -fsSL https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
```

脚本会自动安装 Python、Node.js 等依赖。

- 3. 重新加载 Shell 配置：

bash

```
source ~/.bashrc # 或 source ~/.zshrc
```

- 4. 验证安装：

bash

```
hermes --version
```

看到版本号即成功。

配置模型提供商

- 1. 运行 hermes setup 启动配置向导。
- 2. 选择 Kimi / Moonshot 作为提供商。

- 3. 输入API Key。
- 4. 其他配置按默认。

四、第一次对话

启动 Hermes：

```
bash
hermes
```

或者使用更现代的 TUI 界面：

```
bash
hermes --tui
```

看到欢迎横幅后，尝试了一些具体任务 比如：

- “帮我搜索今天关于 AI Agent 的最新新闻，列出 5 条。”
- “读取当前目录下的 README.md 文件，并总结其核心内容。”
- “帮我写一个 Python 脚本，用于计算两个 DEX 之间的价格差。”

五、Agent 在套利研究中的可能应用场景

结合之前学习的跨链和路由知识，我设想未来可以让 Agent 帮我做这些事：

- 1. 监控价差：让 Agent 定时查询 LI.FI 的 API，对比不同链上同一资产的价格，如果价差超过阈值，就通知我。
- 2. 检查路径变化：让 Agent 定期检查特定跨链路径（如 Across 从 Arbitrum 到 Optimism）的成本和时间，如果发现异常（比如桥费用突然降低），就记录下来。
- 3. 生成研究报告：让 Agent 根据我收集的数据，自动生成一份包含图表和结论的每日或每周报告。
- 4. 学习并创建技能：如果我发现了一个有效的套利流程，可以让 Agent 把它存成一个技能，下次遇到类似情况直接“一键执行”。

六、实操入门与跟随演练

演练1：第一次调用 LI.FI API（获取报价）

目标：用最直接的方式，感受一下 LI.FI 返回的数据是什么样的。

- 1. 打开浏览器，访问 <https://docs.li.fi/>。
- 2. 找到 API Reference，查看 Quote 接口。
- 3. 复制示例请求。LI.FI 文档通常提供了一个可以直接在浏览器中打开的 URL 示例。
- 4. 修改参数。比如，将源链改为 Arbitrum，目标链改为 Optimism，源资产改为 USDC，目标资产改为 USDC.e，金额改为 100。
- 5. 在浏览器中打开这个 URL。看到一大段 JSON 数据。
- 6. 找到关键字段：toAmount（预计收到）、toAmountMin（最低收到）、gasCosts、feeCosts。
- 7. 记录数据：把这次请求的 URL、返回的 JSON（或关键字段）复制到一个新的 Markdown 文件中，命名为 evidence/d8-1-lifi-first-quote.md。

产出：一个包含第一次成功获取 LI.FI 报价的证明文件。

演练2：设计我的第一个测试矩阵

目标：实践“固定测试矩阵，每次只改变一个变量”的原则。

- 1. 确定变量：源链、目标链、资产、金额。
- 2. 设计测试案例：

表格

测试ID	源链	目标链	源资产	目标资产	金额	变化变量
1	Arbitrum	Optimism	USDC	USDC	100	初始基准
2	Arbitrum	Base	USDC	USDC	100	目标链
3	Arbitrum	Optimism	ETH	ETH	0.1	资产类型
4	Arbitrum	Optimism	USDC	USDC	1000	金额大小

- 1. 执行测试：使用演练1的方法，逐个获取测试1-4的报价，并记录到一个表格中。
- 2. 观察结果：比较测试1和2，看看目标链从 Optimism 换成 Base，跨链成本有什么变化。比较测试1和3，看看稳定币和原生资产在费用上有什么不同。

产出：一个包含 4 个测试案例的 Markdown 表格，以及你对结果的初步观察。

七、今日总结

今天最大的收获是，我开始理解“元认知”层面的东西——研究套利，不仅仅是在研究具体的链和协议，更是在研究如何建立一套高效、可重复的研究方法。Agent

是这套方法中的关键一环，它能把我从繁琐的重复劳动中解放出来，让我把精力集中在更重要的判断和决策上。

下一步，我准备就是要开始动手，先跑通一个最小的 Agent 用例，或者完成一次 API 调用，用具体的“证据”来驱动学习。

笔记 040: 昨天的重点是读真实路径：一条交易从哪里发起，经过哪些 Router、DEX、池子、桥和中间资产，最后

作者：海椰 Bareerah | 时间：2026-08-10 16:25 UTC | 字数：1029 | 评分：70
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

昨天的重点是读真实路径：一条交易从哪里发起，经过哪些 Router、DEX、池子、桥和中间资产，最后输出什么。

今天我把注意力放到了另一个问题上：

工具到底能帮我到哪一步？

在链上套利里，很容易看到各种监控工具、提醒工具、报价工具和机器人产品。它们看起来都在帮你发现机会，但我现在更想先拆清楚它们的边界：

它监控的是价格、池子、跨链路径、清算，还是 MEV？

它给的是提醒、分析、报价，还是会代替执行？

它有没有把 Gas、滑点、价格冲击、桥费、等待时间和失败成本算进去？

它是否需要连接钱包、授权合约、托管资金或导入私钥？

它没有替用户承担哪些风险？

这个拆解对小白阶段很重要。

因为工具给出的“机会”，不等于用户最终能执行到这个结果。

报价可能过期，路径可能变化，池子深度可能不够，Gas 可能突然升高，跨链等待时间可能让价差消失。更关键的是，如果工具需要授权、签名或自动执行，那就已经进入了完全不同的风险层级。

所以我现在给自己定了一个规则：

Week 2 只把工具当作研究对象，不把工具当作收益来源。

我会看它监控什么、提醒什么、展示什么数据，但不会连接钱包，不会授权，也不会复制任何自动交易逻辑。

同时，我也开始把 Hermes 的角色限定得更清楚。

Hermes 不是“告诉我能不能赚钱”的工具，而是一个只读研究助理。它更适合帮我做这些事：

检查一条路径里有没有漏掉成本项

区分确定事实、推测信息和待验证信息

帮我找反证，而不是只证明机会成立

把候选信号归类为“继续观察 / 需要更多证据 / 放弃”

复盘我为什么误判了一条路径

今天最大的收获是：链上套利学习里，工具感和边界感同样重要。

工具能提高信息获取效率，但不能替我承担私钥风险、授权风险、执行失败、滑点扩大、桥接延迟和资金占用。

如果我不理解这些边界，工具越强，反而越容易让我误以为自己看懂了机会。

下一步我会继续做三件事：

1. 拆解 taoli.tools 和 QQQ.BOT 的公开功能边界，只记录已核验信息。
2. 用 Paper Trading 表记录候选信号，重点写“为什么放弃”。
3. 让 Hermes 每次都输出证据、反证和缺失值，不允许把缺失数据补成 0。

今天的结论：

链上套利不是工具给一个提醒，我就跟着做。真正要训练的是：看见信号以后，能不能拆清楚工具没告诉我的成本和风险。

笔记 041: D5 学习进展：LI.FI 路径净值与执行证据边界

作者：Qun Wang | 时间：2026-08-11 02:38 UTC | 字数：1929 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D5 学习进展：LI.FI 路径净值与执行证据边界

今日问题

怎样固定一份 LI.FI route response，识别已嵌入与未嵌入成本，把不同 token 的输入、输出、Gas 和额外 fee 统一计价，并避免把报价状态写成实际成交或盈利？

使用的一手资料或区块/数据

2026-08-10T05:15:32+00:00 捕获的 POST <https://li.quest/v1/advanced/routes> 完整 request/response。
路径：Arbitrum 42161 USDC → Optimism 10 DAI；输入 10000000 raw = 10 USDC；slippage 0.005；排序 CHEAPEST。
LI.FI OpenAPI、Request Routes/Quotes、Get Quote 与 Status Tracking 官方文档。
本地 bc 1.07.1 任意精度十进制计算。

做了什么

固定 capture time、完整 request、raw response、route IDs 与 OpenAPI SHA-256。
用 jq 提取 4 条 route 的预计输出、最低输出、外部 fee 与时长。
对 squid route 区分已嵌入 LI.FI Fixed Fee、未嵌入 Gas receiver fee 与执行 Gas。
把 USDC、DAI 与 ETH 成本统一到条件单位 USD-at-quote-time，并用 bc 保留精确小数。
计算正屏幕毛差、负 expected net 与负 minimum net。
用 COMPLETED / PARTIAL / REFUNDED / FAILED 情景练习 publisher status、transaction receipt 与实际 token 到账的证据边界。
完成退出卡和一组新数字迁移复测，保留所有提示与错误轨迹。

结果或失败

固定 squid route 的：

```
screenSpread = 0.0058734563069491845320600962 USD-at-quote-time
expectedNet = -0.0381514857607893925879399038 USD-at-quote-time
minimumNet = -0.1479647519949303306611858524 USD-at-quote-time
```

因此屏幕输出价值虽略高于输入，仍不足以覆盖执行 Gas 与未嵌入外部 fee。included = true 的 LI.FI fee 已反映在 route 结果中，不能再次相减。

学习过程并非一次全对：曾混淆 token 数量与 USD 价值、Gas receiver fee 与执行 Gas、源侧输入与 receiving 资产、历史 Transfer 与当前状态，也曾把负条件净值过度写成“不能盈利”。新数字下能独立算出 -0.02 USD-at-quote-time 并避免重复扣费；状态记录经针对性补全为 0.0049 WETH on Base (8453), pending。

证据链接或本地路径

```
evidence/0005-lifi-route/routes-response.json
evidence/0005-lifi-route/README.md
evidence/0005-lifi-route/worksheet.md
learning-records/0005-d5-lifi-route-net-value.md
reference/0006-lifi-route-net-value.html
```

下一假设

如果在同一固定高度同时保存 RPC、索引器与聚合器观察，并显式记录高度、延迟、派生过程和冲突，就能判断哪些字段是链上状态、哪些是发布方报告、哪些可能因素引滞后而不一致。

风险认识的变化

报价只证明聚合器返回过估算，不证明路径执行。源链成功也不证明 receiving 或 refund 终态；PARTIAL 必须记录实际 token，REFUNDED 不能预设退款链。没有 receipts、Transfer/余额、实际 Gas/fee 与明确价格时点/来源，就不能声称实际盈利。

笔记 042:

作者：wmroar | 时间：2026-08-11 06:59 UTC | 字数：450 | 评分：75

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

桥摩擦时段性假设被证伪：Across 摩擦恒定 0.0101%，无时段差异

新发现：Curve 池失衡 5 天从未消失（均值 +0.06%）——结构性状态

1. Curve 深度验证

\$1M 规模失衡收益仍 +0.063%（深度无限）

但变现腿成本更高 → 定位修正：失衡是叠加信号，不能独立收割（价差精确停在变现路径成本上）

1. 告警系统上线 + 首次实战

阈值告警（Curve 失衡 >0.1%、peg 偏离 >0.3%）→ 自动链上交叉验证 → 告警 + 证据包

首战即教学：LI.FI 报 USDT -0.54% → 链上验证发现是报价源噪声 → 假信号记录为负样本 → 交叉验证自动化内置

1. 多链扩展（跨链方向的核心信号）

发现 Alchemy key 通多链，ARB 2pool 链上验证（差点栽在“以为是 3pool”）
链间价差轨上线：主网 vs ARB 的 USDT 相对价差每 15 分钟自动计算，>0.1% 告警

笔记 043: 今日学习心得：选择 USDT 作为稳定币跨链观察方向

作者：Chic | 时间：2026-08-11 07:50 UTC | 字数：1010 | 评分：62
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日学习心得：选择 USDT 作为稳定币跨链观察方向

今天学习重点是从“概念和工具学习”转向选择一个具体方向做持续观察。我选择了 USDT 作为第一个观察对象，因为自己平时使用最多的就是 USDT，尤其是在 CEX 和钱包之间转账时更有实际经验。

今天首先明确了一个核心认知：链上转账不是简单地“转 USDT”，而是要同时确认币种、网络 and 地址。之前自己就遇到过类似问题：以为只要换成 USDT 就可以，但后来发现对方要求的是 Optimism 网络，而我换成的是 Ethereum 主网版本。这说明“币种对”不等于“网络对”。以后真实转账前必须确认：对方要什么币、在哪条链、地址是否支持该网络，并且最好先小额测试。

接着学习了原生资产和桥接资产的区别。原生资产可以理解为发行方直接在某条链上发行的版本；桥接资产则是通过桥从一条链转到另一条链后生成的代表资产。桥接资产除了要考虑发行方和链本身，还要额外考虑桥风险、流动性、能否兑回等问题。比如看到 USDC.e，就应该先怀疑它是桥接或包装版本，而不是直接当成原生 USDC。

今天还设计了 USDT 跨链 Quote/Route 观察表。这个表的目的是不是马上找实盘机会，而是记录 USDT 在不同链之间移动时的真实成本和路径稳定性。表格字段包括：时间、源链、目标链、起始资产、目标资产、是否原生/桥接、投入金额、toAmount、toAmountMin、Gas、feeCosts、执行时间、Route 经过的桥和 DEX、是否有中间资产、保守净收益、主要风险和结论。

通过练习也再次强化了一个判断原则：保守净收益要优先用 toAmountMin 来计算，而不是只看预计收到的 toAmount。如果投入 1000 USDT，toAmountMin 只有 995，再扣 Gas 和 feeCosts，那么保守结果就是负收益，应该“不执行，仅记录观察”。如果目标资产还是 USDC.e 这类疑似桥接资产，还要额外标注资产版本风险。

今天最大的收获是，我开始把自己的真实转账经验和套利观察结合起来。稳定币跨链并不是简单比较哪里多收到一点，而是要同时看币种、网络、资产版本、桥、费用、滑点、执行时间和保守净收益。接下来准备继续学习如何选择具体观察路线，比如 Ethereum、Arbitrum、Optimism、Base、BSC、Tron 等链之间的 USDT 路径。

笔记 044: 1,000+ Partners in LI.FI's Ecosystem Can Now Access Tokenized Stocks

作者：Yolandatsai | 时间：2026-08-11 07:56 UTC | 字数：1055 | 评分：57
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1,000+ Partners in LI.FI's Ecosystem Can Now Access Tokenized Stocks

LI.FI is a full-stack, intent-based execution engine powering same-chain and cross-chain token transfers. Rather than navigating routes manually, users express a desired outcome, and a competitive network of professional solvers handles execution automatically for best-price, gasless, and near-instant execution. LI.FI has processed \$80B+ in volume across 100M+ transfers, powering infrastructure for platforms like MetaMask, Robinhood Wallet, and Binance.

1,000+ partners across LI.FI's global network can now seamlessly access and integrate 438+ tokenized U.S. stocks and ETFs-including Tesla, NVIDIA, Apple, and broad-market ETFs like QQQ and SPY-directly within their existing wallets and apps. Ondo Stocks tokenizes U.S. public securities, bringing exposure onchain in the form of fully backed, DeFi-compatible tokens. Every asset is backed by underlying securities held with one or more U.S. broker-dealers, with daily verification and institutional-grade investor protections.

笔记 045: 套利赛道分析

作者：Ox2077 | 时间：2026-08-11 08:47 UTC | 字数：442 | 评分：58
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利赛道分析

| 赛道 | 机会状态 | 竞争 | 技术要求 | 资金要求 |

| :----- | :---- | :- | :-- | :----- |

CEX跨所搬砖	基本消失	极高	低	中
DEX原子套利	有	极高	很高	低~中
DeFi清算套利	有结构性机会	中高	高	低~中 (可Flashloan)
xStock/股票Token套利	早期	较低	中高	中
跨链套利	较弱	高	很高	高

笔记 046: 链上套利与跨链开发

作者: Joneloo | 时间: 2026-08-11 10:11 UTC | 字数: 1510 | 评分: 84

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利与跨链开发

一、市场结构

以太坊本质是卖排序权 (Greenfield) : 区块空间按「纳入市场」(EIP-1559 基础费) 和「位置市场」(交易顺序决定价值) 双维度定价, 后者是 MEV 根源。供应链四层: 订单流 → 路由 (公开 mempool/私有 RPC/OFA) → 构建者 → 中继+验证者。构建者只需胜过次优报价, 独有订单流是最大护城河。

二、稳定币脱锚 (msUSD 案例)

- 根因: 预言机延迟 (18.5% 时间出价区) → 套利者超发 → 约 30% 无抵押
- 教训: 脱锚从供应端开始 (单日铸币 3 倍于流通量); 单一价格源会掩盖真相 (\$0.96 vs 共识 \$0.73); 分扩张型 (超发) / 收缩型 (挤兑)

三、路由聚合 (LI.FI 实测)

- 同路线 17 条路由到账差 1%; 热门对 1s vs 冷门对 360s; 规模决定桥选择
- 机会 = 净到账, 不是单项成本; 速度是最强定价因子

四、实战案例范式

Aave 清算: EOA 提交 + 共享执行合约 + RFQ 回补 + BuilderNet 竞价; 链上毛机会全付给 builder, 真利润在事后退款 (链下不可见)。

Robinhood 三角: WETH → 中间币 → USDG → WETH 原子闭环; 55% 失败率靠低 gas 高频覆盖。

共同范式: 三角色分离 (提交/执行/归集); 链上可见 ≠ 真实利润; 原子执行失败只亏 gas。

五、跨链开发实战 (Stargate Lite Pro / LayerZero OFT)

目标: 纯前端实现 Sui WBTC → Ethereum WBTC。

完整链路: quote 拿 nativeFee → 构建含全序列的 PTB → dry-run → 钱包签名广播。

关键知识:

- oft::send() 返回的 Call 对象不能丢, 必须继续走 endpoint → ULN → executor → DVN 全路由并确认
- sendMoveCall(tx, sender, sendParam, coinProvided, nativeFee, zroFee, refundAddress, validators, maxSimulationTimes)
- 配置 (package/oftObject/oappObject/EID) 从成功交易反推, 不靠猜

十大坑 (精华):

1. UnusedValueWithoutDrop: PTB 中 split 的中间 coin 必须被消费, 直接传原始 coin, 别手动先 split
2. 一个 import 报错 → 整个前端瘫痪, 先看 Console 第一条红错
3. Sui RPC CORS → 换 publicnode
4. ethers 用 vite import 而非 CDN
5. Sui 缺 id 字段 → option value="undefined"
6. 重复 let src 声明报错
7. wallet-standard 依赖缺失 → 清 lock 重装
8. 钱包连接: getWallets → dapp-kit → 退化检测 window.slush 注入
9. API 名不确定 → grep 本地 SDK 的 .d.ts
10. 先 dry-run 成功再弹签名

六、贯穿主线

1. 排序权 = 利润根源 (位置市场)
2. 结构信号先于价格 (供应端、链上事件)
3. 数据层最脆弱 (预言机、单一价格源)
4. 真利润在链下 (refund、私有订单流、归集账户)
5. 原子性是安全底线: 失败只亏 gas, 不留单边仓位

笔记 047: Day 7 学习笔记 (2026-08-11)

作者: Azhan | 时间: 2026-08-11 10:12 UTC | 字数: 1681 | 评分: 80

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 学习笔记 (2026-08-11)

今天学了什么

1. DOS 跨链套利案例 (今天的主线)

群里说 DOS 可以跨链搬砖赚钱 (EVM 和 BSC 之间)。我一开始判断"这是陷阱"——结果判断错了,被群里的一份调研报告打脸。报告用链上实测证明:

DOS 是真实的 LayerZero V2 OFT 代币 (ETH 官方 0x951f..., BSC 桥合约 0xb0f0...)

桥真实存在,双向 1:1,桥费 ETH→BSC 约 \$0.02、BSC→ETH 约 \$0.69

当时 CEX 折价 vs BSC DEX 溢价 6-7%,确实有窗口 (但 1-2 天就被抹平了)

我错在哪 (要记住):

- 用了不可靠的 RPC (eth.llamarpc 报错,我误读成"合约 27 字节空壳")
- 看 LI.FI 报"无路径"就以为"没桥"——聚合器不认识新代币 ≠ 不存在
- 找到"证据"后锁定结论,后面所有信息都在强化错误 (确认偏误)

2. 学会了"自己动手验证" (最重要)

不依赖记忆、不依赖报告、不依赖中间商,直接跟合约对话:

字节码探测法:拉合约代码 → 提取所有函数 selector (PUSH4 指令) → 逐个 eth_call 调用 → 看返回值

我亲手验证了 BSC DOS 确实是 OFT (探测到 LayerZero V2 Endpoint 地址 + OFT V2 的 decimalConversionRate=1e12)

这个过程中还发现:我记忆的 OFT selector 是错的 (V1 和 V2 不一样)——更证明不能靠记忆

3. Sui 生态调研 (有 SUI 现货所以关心)

SUI 现货各所价格一致 (\$0.6928),无价差

Sui 生态币 (US/WAL/DEEP) CEX vs DEX 价差 <0.5%,被手续费吃掉

Sui 目前没有 DOS 式的"新币窗口"——生态币都已上线一段时间,价格被套利抹平

群里对 Sui 关注少 = 竞争少 (正面) 但信号源少 (负面)

做了什么

- 完整验证了 DOS 跨链案例 (从误判到亲手验证真伪)
- 写了 2 个工具:
- scripts/verify_token.py:代币真伪 6 步验证 (字节码探测法,不依赖记忆)
- scripts/check_spread.py:跨市场价差查询 (事件触发筛子)
- 建了跨链套利工具库:notes/crosschain-toolkit.md (RPC 池/官方工厂地址/OFT 接口/桥/防错规则)
- 存档了 LayerZero OFT 官方文档 (220KB) 和跨链套利指南
- 调研了 Sui 生态 (结论:目前无窗口,保持事件驱动)

得到什么结果

方法论升级:验证金标准 = 字节码探测法 (直接跟合约对话)

工具就位:以后遇到跨链机会,3 秒查价差 + 自动验证真伪

认知确立:不依赖记忆/报告/中间商,多个源头交叉验证,结论要能被推翻

遇到什么失败 (教训)

DOS 误判:初判"陷阱"是错的 (数据源不可靠 + 确认偏误)——但这次失败价值巨大,逼出了"验证优先"原则

check_spread.py 自动找币会张冠李戴:--symbol US 自动找到 Tether! 已修复 (强制显示 id + 警告)

记忆的接口签名不可靠:OFT V2 selector 记错了——以后接口验证必须查 ABI/官方文档

下一步验证什么

Sui 生态保持"事件驱动"关注 (有新币上线消息才查)

如果群里有新的跨链机会消息:先 check_spread.py 看价差 → verify_token.py 验证真伪 → 再决定

费率套利主线继续积累数据 (cron 正常跑)

笔记 048: 链上套利残酷共学 · Day 6 心得

作者: kako | 时间: 2026-08-11 10:14 UTC | 字数: 632 | 评分: 56

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 6 心得

今日主题：报价对比实验

今天不是“查一条报价”，而是“横向对比一堆报价”——固定三个金额档位（1k、10k、100k USDT），在同一条跨链路径上记录报价、Gas、桥费和滑点的差异。

这个实验做下来最直观的感受是：跨链成本结构完全不是线性的。1k 的时候，桥费和固定 Gas 成本占比极高，有时候光手续费就吃掉 3%~5%，小资金跨链基本是给基础设施打工。到了 10k，固定成本占比被摊薄，单位成本明显下降，看起来开始“划算”了。但到了 100k，滑点突然飙升——大额资金在目标链 DEX 上兑换时，直接把池子价格推上去了，实际到账数量反而比按 1k 报价线性外推的少一大截。

把三档数据并列对比时，发现了一条意外的规律：某些路径在 1k 时报价很差（固定费用太高），但到了 10k 反而成为最优路径；而另一些路径在 1k 和 10k 都表现不错，到 100k 却因为目标链流动性不足而大幅恶化。这意味着“最优路径”不是一个固定答案，而是随着交易金额动态变化的。

今天自查问题的答案也清晰了：同一条路径在 1k 和 100k 的报价差异，本质反映的是成本结构的非线性和流动性的深度限制。小单主要被固定成本（Gas、桥的基础费）压着，大单则被可变成本（滑点、比例手续费）主导。一个成熟的套利者必须根据每笔交易的实际金额动态选择路径，而不是记住一条“最优路径”反复用

笔记 049: 链上套利残酷共学 · Day 7 打卡 (2026-08-11)

作者：kelthuz4d | 时间：2026-08-11 10:14 UTC | 字数：2982 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 7 打卡 (2026-08-11)

今日一条线：当一回“阅卷人”——拉取今天（Day 7，第一周收官）全部 30 篇打卡，挑出 8 篇最有技术含量的精读，并验证平台是否提供可分享的单篇链接（结论：不提供，已实测，改为本地导出全文）。

一、今日 30 篇打卡概览

今天明显从“学概念”转向“用数据回测” / 证伪”。30 篇里硬核度高的集中在：清算论文、策略回测、事件搬砖案例、网格幸存者偏差、清算演练、CEX-DEX 架构、插针猎杀验证。我精读了其中 8 篇全文（starkxun 9695字 / ChineseCatMan 5128字 / QQQ 3768字 / Oxmigueleth 1862字 / pennchester 1389字 / Paxon 1688字 / peanutwen00 1007字 / Sansho2 735字）。

二、8 篇优质打卡精读结论

1. starkxun — 清算论文对照（最重磅）：用 AFT 2025 论文《Measuring CEX-DEX Extracted Value》（19个月 / 720万笔 / \$233.8M 提取值）校验自己结论。被作废的 3 条：清算人不是只有 2~3 个（Aave V3 有 183、Morpho 159）；清算中位不是 \$10 尘埃而是 \$20,266；全市场不是 30 天 \$640 万而是三年 \$1.458B/月、极端 48 小时 \$12 亿。被印证的 1 条：“价值被排序权拿走”——Wintermute 给构建者 66%、SCP 81%、整合渠道 ~90%，甚至有 searcher (Graves) 净亏 204%。核心认知：这不是清算特有，是 PBS/MEV 的跨赛道普适税。还坐实了之前那台 0x8d64 清算机器人 = Wintermute 的猜测。
2. ChineseCatMan — 六策略 30 天回测排名：用“报价后 N 秒真实价格”回测（非报价时，否则高估）。结果：稳定币脱锚 30 天零信号；LINK 跨链（快桥）唯一正夏普 0.62 但仅 21 个信号；WETH 跨链夏普 0.38“不具吸引力”；DEX 内部 / 三角套利夏普为负。→ 印证“主流路线结构性为负，脱锚是黑天鹅捕猎”。
3. QQQ — dappOS (DOS) 搬砖案例（最实操）：8/10 DOS 上 Binance Alpha，链上 AMM 不同步产生价差。收敛后 Alpha 0.5280 vs 链上 0.5288，表面差仅 0.15%，名义 1000U 毛差 1.5U——池费+滑点+提币+gas 足以吞掉 → 净收益应视为负。安全垫：无对冲单向路径入场阈值 $\geq 1\sim 2\%$ ，先极小额全链路试。
4. Paxon Qiao — 网格幸存者偏差：用 OKX BTC/USDT 1h 真实行情回测“晒单 1 月赚 1300U”。13 个月下跌月 8 个亏 7 个（88% 亏损率），月收益与 |月涨跌| 相关 -0.48；0 手续费仍净亏 -\$1122（库存浮亏 > 震荡毛利，网格固有不对称）；10x 杠杆下 -10.2% 月即爆仓。
5. peanutwen00 — 清算演练：Base morpho 清算替还 \$50,000 按 92 折拿 ETH，毛利 \$4,347 (8.7%)。两个纠偏：① 数量级检查（是 \$4347 不是 \$434）；② 优先费是“让出去的利润”非“买必中保险”，应设出价上限 = 毛利最低要求利润，超了放弃。判断：不做（拼基建，与 starkxun 实证一致）。
6. pennchester — CEX-DEX 深度解析（离实盘最近）：本质是信息套利（CEX ms 级 vs DEX 12s 区块级时间差）。订单簿前 5 档 \$100K 仅 1.24bp 滑点；最佳订单类型 IOC；提币延迟 5~30min 是最大挑战 → 解法是双边预持仓（NetExposure=0 锁定价差非方向）；含永续对冲需 0.8%+ 价差才盈利；每 500ms 延迟损失利润 10~25%，超 2 秒变负。
7. Oxmigueleth — TUT 插针事件验证：8/9 TUT 各所插针（Bitget \$1.00 / Gate \$0.70 / 币安 \$0.305），1h 爆仓 \$3500 万。验证“资金费率陷阱 + 跨所流动性猎杀”组合拳：币安做多 / Bitget-Gate 做空吃费率 → 暴力插针爆空单 → 砸盘收割多单。结论：meme/小市值永续跨所资金费率套利 $\approx$ “送人头”给控筹方。
8. Sansho2 — Week1 研究模型 v1：把一周压成决策状态机——强制可成交 Quote 口径、跨链 ~25bps 地板、路径差与双边毛利分列、失败分 Gas/单腿级、延迟拆老化、五步硬闸门。主战场排序：P1 Funding/基差中性 + LI.FI 尺子；同链最优路由作执行边；裸跨链微利 / 裸 LP / MEV 不做。

三、一个元动作：链接导出（踩坑记录）

用户要"把优质打卡链接发出来"。我实测确认：ICL 不提供单篇打卡的公开链接——API 每条只有整个共学项目的 webUrl；前端路由 /check-ins/{id}、?checkin={id} 用浏览器打开只渲染项目落地页（且未登录 GitHub 连列表都不显示，属学员私享内容）。最终改为把 8 篇全文导出到本地 /tmp/aug11_checkins/.md（含 author / checkinId / program 注释，登录后可在 UI 用 ID 定位原文）。这本身也是个"平台能力边界"的验证练习。

四、今日核心结论（Day 7 共识比 Day 4 更冷）

1. 回测说话：六策略、网格，都用真实历史把"看起来行"打成"夏普为负 / 幸存者偏差"。
2. 跨赛道普适结构：starkxun 论文印证"价值被排序权拿走"不是清算独有，是 MEV/PBS 的普遍税。
3. 散户现实路径收敛：裸跨链微利 / 裸 LP / MEV 不做；P1 是 Funding/基差中性 + LI.FI 尺子；事件驱动（脱锚/新币上线）要 1~2% 安全垫 + 极小额试单。
4. 防坑升级：TUT 事件证明"跨所资金费率套利"在控筹 meme 上就是猎杀场。

五、下一步

- A. 按 ChineseCatMan 思路升级 lifi_spread_probe.py——加时间戳快照 + 延迟成本模型（带"报价后 N 秒真实价格"的回测，这是离"可证明可行"最近的改进）；补 integrator 对照列修掉 ~50bps 偏差；专扫长尾/新池。
- B. 把"今日 8 篇优质打卡"整理成可复用的《第一周硬核笔记索引》。
- C. 跑一遍 CEX-DEX 单边预持仓的只读测算骨架（pennchester 路线）。

笔记 050: Day 7（8/11）第一周收官学习总结，素材来自今日 51 位同学精华：

作者：cjdaocoin | 时间：2026-08-11 10:47 UTC | 字数：620 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7（8/11）第一周收官学习总结，素材来自今日 51 位同学精华：

1. 清算论文对照（kelthuz4d/starkxun）：用 AFT 2025 论文《Measuring CEX-DEX Extracted Value》（19个月/720万笔/\$233.8M）校验认知——清算人不是只有2~3个（Aave V3 有183个），清算中位也不是 \$10 尘埃。用论文数据证伪自己的直觉，是这周学会的最硬核方法。
2. 跨链案例验证（Azhan）：DOS 在 EVM/BSC 间桥费极低（\$0.02/0.69），CEX 折价 vs DEX 溢价 6-7% 有窗口但 1-2 天被抹平。教训：聚合器报"无路径"≠不存在桥，必须字节码探测法直接跟合约对话验证，且要防确认偏误。
3. 滑点数学（journeyanhk）： $xy=k$ 下价格影响非线性——交易占比 1% 影响 ~1%，占比 10% 影响 ~11%，占比 50% 影响 ~100%；深池(7000 WBNB)滑点 0.39% vs 浅池(234 WBNB)4.73%，浅池的便宜报价是幻觉。
4. 成本结构非线性（kako）：1k/10k/100k 三档报价对比，小单被固定成本压、大单被滑点主导，"最优路径"随金额动态变化。

今日收获：学会用论文数据证伪直觉、用字节码探测法验证合约真实性、用 $xy=k$ 精确计算价格影响。第一周从"看价差"升级到"算真实成本"。

笔记 051: Day 7 | 链上套利残酷共学

作者：HashFlow | 时间：2026-08-11 11:09 UTC | 字数：1018 | 评分：71

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 | 链上套利残酷共学

日期：2026.08.11（周二·缓冲日）

主题：LI.FI 平台全局认知

内容

没有学新套利知识，而是系统性地熟悉 LI.FI 这个平台的全貌。之前只用 /v1/quote 一个端点，今天把 28 个端点、三个产品模块、多种使用模式都过了一遍。

LI.FI 自身定位

llms.txt 定义："routing and execution layer for cross-chain liquidity, payments, swaps, and yield." 定位是执行层——不是分析工具，拿到的 transactionRequest 应该直接签名发送。但 cache-control: no-store 的实时数据特性对分析端也有价值。

交易执行后

GET /v1/status 追踪跨链交易。四种状态 PENDING → DONE/FAILED, DONE 有三种子结果（COMPLETED/PARTIAL/REFUNDED）。响应中 feeCosts 字段区分 included/excluded 费用——Relay fee 从金额中扣，protocol fee 额外支付。

产品矩阵

Composer : swap + 协议存入合为一笔交易 (toToken 设为金库地址即自动触发)

Earn : 收益发现 API (earn.li.fi) → Composer 执行存入

Intents/Solvers : 意图执行替代路由执行, Solver 竞价填充, 不指定路由

三种使用模式

Quote (单步) vs Routes (多步对比)

自己签名 (控制优先) vs Relayer (体验优先)

普通 swap vs Composer (swap + 存入)

认知更新

DEX Screener 有 30 秒缓存 (max-age=30, cf-cache-status: HIT) ——发现的价差至少过期半秒到半分钟

LI.FI 无缓存 (no-store, BYPASS) ——实时但响应需 0.7-1.0 秒

机器人响应速度 1-2 区块 (Base 上 2-4 秒) ——手工套利在时间维度上无优势

下一步

Day 8 (周三) : 跨链套利实操

笔记 052: Day 7 | 补齐库存再平衡后, 完整净 Edge 仍为负

作者: Courtney Snyder | 时间: 2026-08-11 11:17 UTC | 字数: 2026 | 评分: 90

来源: <https://intensivecollearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 | 补齐库存再平衡后, 完整净 Edge 仍为负

网站进展:

截至 8 月 11 日, 活动页面新增了链上套利公开社群 <<https://t.me/arbitrageicl>>, 并已评出 8 月 7 日的三篇优秀笔记。前两篇分别强调 LI.FI 约 0.25% 服务费的硬下限, 以及交易规模/价格冲击对最优路径的影响, 这与我当前的成本建模方向一致。报名人数增至 692。历史公开笔记仍停在 Day 4; 8 月 9 日完成的 Day 5 本地实验没有发布, 今天按活动第 7 天继续推进, 不把未提交草稿写成已打卡。

今天完成:

1. 补齐预置库存的双资产再平衡

新增 rebalance_sampler.py。在每个候选中, 先构造买入链 USDC → WETH、卖出链 WETH → USDC 两条现货报价, 再并发请求 WETH 从买入链回补到卖出链、USDC 从卖出链返回买入链。完整候选由 4 个 LI.FI 报价组成, 仍使用合成地址, 只读、不签名、不发送交易。

2. 统一净收益成本口径

现货与跨链协议费、桥费、滑点已经反映在各腿 toAmountMin 中, 不重复扣减; 另外扣除四腿 Gas、WETH 跨链后的库存缺口、10 bps 失败缓冲和按 10% 年化折算的资金占用成本。假设 Token 已预授权、USDC=1 美元; WETH 缺口按两条可执行价格中较高者估值。

3. 增加可直接用于筛选的门槛

新增 minimum_pre_rebalance_edge_bps: 表示现货阶段至少需要多大的正 Edge, 才能在再平衡和风险成本后仍保留 20 bps。这样后续监控可以先比较实时 Edge 与门槛, 再决定是否继续请求完整路径。

4. 测试与真实采样

新增 test_rebalance_sampler.py, 覆盖再平衡缺口/Gas 扣减、低净 Edge 拒绝, 以及超时和慢速路线拒绝; 全部测试共 12 个, 全部通过。

真实采样时间: 2026-08-11 19:14:29 至 19:14:42 (UTC+8)。

范围: 100 / 1,000 / 10,000 USDC, Base 买 → Arbitrum 卖及反方向, CHEAPEST; 共 6 个候选、24 个报价, 成功 6, 失败 0。

输出:

data/day7/lifi_rebalanced_paper_trades_20260811T111442Z.jsonl

data/day7/lifi_rebalanced_paper_trades_20260811T111442Z.csv

本轮数据发现:

再平衡前的保守现货 Edge 已为 -135.8929 至 -150.3196 bps, 没有出现正价差。

完整净 Edge 进一步降至 -196.5282 至 -221.2991 bps。100 USDC 两个方向净亏约 2.17 / 2.21 USDC; 1,000 USDC 约 20.63 / 21.07 USDC; 10,000 USDC 约 196.53 / 210.96 USDC。

单次再平衡成本为 0.5700–50.6351 USDC。若要在完整成本后保留 20 bps, 现货阶段至少需要 80.6107–93.0848 bps 的正 Edge; 小额档因固定 Gas 占比更高, 门槛最高。

三档金额之间出现 3 次路由签名变化, 说明金额仍是缓存与监控的必要维度。

其中 1,000 USDC 的 Base 买 → Arbitrum 卖路径报价窗口为 5.209 秒, 超过 5 秒门槛; 其余 5 条在 3.54–4.48

秒内。所有路线预计最长 7 秒。

Paper Trading 判定：

本轮 0/6 通过。全部候选先因净 Edge 低于 20 bps 被淘汰，其中一条同时因报价过期被淘汰。当前模型已覆盖主要库存再平衡与风险缓冲，但仍只是报价模拟，不代表真实成交或收益。

今天的核心认识：

Day 5 只证明现货两腿本身为负；今天进一步证明，即使使用预置库存避免把桥放进机会执行窗口，事后的库存恢复仍约增加 50–70 bps 成本。LI.FI 服务费、两腿保守滑点和再平衡形成了较高硬门槛，当前 Base ↔ Arbitrum 的 WETH/USDC 候选距离可执行条件很远。

明日行动：

进入第二阶段：对 100 / 1,000 / 10,000 USDC 同时比较 CHEAPEST 与 FASTEST，多轮记录 minimum_pre_rebalance_edge_bps、路由切换和淘汰原因；把成本拆成 DEX/LI.FI、桥、Gas、库存缺口、失败缓冲和资金占用六类，形成可聚合的数据表。继续只读验证，不使用真实资金。

Paper Trading

笔记 053: Day 7 · 给"等待"标个价,然后收口 Week 1

作者：99hansling | 时间：2026-08-11 12:22 UTC | 字数：2143 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 · 给"等待"标个价,然后收口 Week 1

这篇在讲什么

昨天(Day 6)发现跨链桥的手续费已经被竞争压到 1 bps 量级——快桥和慢桥之间只差不到万分之一。当时留了一个问题:既然手续费这么低,跨链套利的成本到底在哪?

今天的答案是:成本在等待的时间里,不在你付的手续费里。具体来说,等你跨链的那 20 分钟里,资产价格可能漂移的幅度,远远大于你省下的那 1 bps 桥费。

给"等待"算一笔账

有一种标准的金融方法,可以把"等待"换算成"价格风险":用资产的年化波动率,按等待时长的平方根缩放。

打个比方:以太坊(ETH)一年的价格波动大约 60%。如果你要等 20 分钟(1200 秒)才到账,这 20 分钟内 ETH 价格可能漂移多少?按公式算,一倍标准差的漂移大约 37 bps(千分之 3.7)。如果往返两腿各等 18 分钟,合计 36 分钟,漂移大约 50 bps。

而昨天测到的快慢桥报价差只有 1 bps。

换句话说:你为了省 1 bps 桥费而多等 36 分钟,但这 36 分钟里价格可能漂移 50 bps——你省下的还不够覆盖你冒的风险。这就是"成本在时间里不在费用里"的数学证明。

跨链闭环实测:同名稳定币的零点校准

为了验证"同名资产跨链到底亏多少",跑了一次完整闭环:把 USDC 从 Arbitrum 跨到 Base,再跨回来,看最终到手多少。

本金	用什么桥	总等待	到手/发出	净亏
\$100	eco + eco	14 秒	1.00000000	3.1 bps
\$1,000	eco + eco	14 秒	1.00000000	0.3 bps
\$10,000	eco + eco	14 秒	1.00000000	0.03 bps
\$100,000	eco + eco	14 秒	1.00000000	0.003 bps
\$500,000	polymer + polymer	2160 秒	1.00000000	0.0009 bps

USDC 跨到 Base 再跨回来,到手和出发完全一样($R = 1.00000000$),唯一的亏损来自 Gas 费,而且 Gas 随规模摊薄——\$100 亏 3 bps, \$50万只亏 0.0009 bps。

这说明同名稳定币跨链本身几乎零成本(桥费被竞争压没了)。但这不能当成套利机会——因为 USDC 两端是同一种资产,没有价差可套。

它是一个"零点校准":证明仪器能正确测出"零",不是瞎的。

真正有价差的资产(比如 ETH)跨链时,上面算的那 50 bps 漂移风险才会变成真实成本。

Week 1 死亡路径清单

本周用数字杀死了的路径,每条带具体死因:

| 路径 | 死因(带数字) |

|---|---|

| 同链 AMM 套利(USDCWETH,\$1000) | 严格口径净亏 102 bps,因为双腿各承担 0.5% 滑点下限,合计 100 bps 的承保成本直接吃掉全部边际 |

| 稳定币对套利(USDCUSDT,\$1000) | 严格口径净亏 100 bps,即使是低冲击的稳定币对也跨不过双腿滑点下界——死因是承保口径,不是 Gas |

| 不带 integrator 的同链套利 | 净亏 154 bps,比带 integrator 多亏 62 bps,因为每腿多一笔 25 bps 的固定费。一个参数就能差 62 bps |

| 非同名资产跨链闭环(两腿各 18 分钟) | 延迟漂移风险约 50 bps,而快慢桥费差只有 1 bps——省下的桥费远不够覆盖等待期间的价格风险 |

这四条的共同结论:屏幕上看得见的静态价差,扣掉成本后全是负的。

这不是"没找到机会",而是"用数字证明了这些路走不通"。

Week 1 整体状态

闭环记录:20 条(8/7 的 4 条 + 8/10 的 8 条 + 8/11 的 8 条),满足课表下限

扫描器:能跑、能存原始响应、能同时输出三种口径(乐观/混合/严格)

至少一条用数字杀死的路径:有,上面四条都是

诚实地说,还有几个限制没解决:Day 5 的稳定币对照跑在另一条链上

(不是严格的同市场实验);原始响应存在本地但不入 Git(fresh clone 不能重放)。

这些保留到 Week 2,不拿"达到 20 条"掩盖。

一句话收口

Week 1 证明了:静态价差是负的,桥费可以忽略,真正的成本是等待期间的价格风险。

这跟项目的核心论点完全一致——所有非负的数字都不产生于价格发现场所,

机会在"对手方被迫"的场景里(清算、funding),不在公开 AMM 的价差里。

笔记 054: 今天算是第一周基础部分的收尾,开始把前几天学到的跨链、Gas、滑点、RPC 和 API 放到一个真实

作者: Yannis | 时间: 2026-08-11 12:34 UTC | 字数: 743 | 评分: 56

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天算是第一周基础部分的收尾,开始把前几天学到的跨链、Gas、滑点、RPC 和 API 放到一个真实场景里,重点看 LI.FI 的 Quote 和 Route 到底在告诉我什么。之前我对聚合器的理解基本停留在“帮我找一条比较好的跨链路径”,今天继续拆以后发现,一条 Quote 其实已经包含了很多以后判断套利机会需要的信息。

先区分了一下 Quote 和 Route。我的理解是,Quote 更像是在条件已经比较明确的情况下问:“如果我现在拿这么多 A,从这条链换到另一条链的

则更偏向完整路径,里面可能不只是一次简单兑换,而是包含源链 DEX、跨链桥、目标链 DEX,甚至中间资产。也就是说,我在页面上看到的“一次跨链兑换”,底下实际上可能是一串连续动作。

比如我想把 Arbitrum 上的 USDC 变成 Base 上的 ETH,最终路径不一定是 USDC 直接跨过去变 ETH。有可能先在 Arbitrum 做一次 Swap,再通过某个桥转移资产,最后到 Base 上再 Swap 一次。对普通用户来说,只需要关心最后能收到多少;但如果是在研究套利,这中间每一步都值得拆,因为任何一段都可能吃掉利润。

这也让我重新理解了“最优路径”。以前会下意识觉得聚合器给出的最优路线就是最便宜的一条,但实际上还需要考虑交易金额、预计到账数量、Gas、执行时间、桥和 DEX 的流动性等条件。同样一条跨链路线,输入 100 USDC 和 10000 USDC 时,最合适的路径完全可能不同。小额交易容易被固定 Gas

成本影响,大额交易又更容易产生价格冲击,所以套利策略不能简单记住一条“最好用的桥”,而是应该每次根据当前状态重新报价。

笔记 055: Day 6 续学: LI.FI Route 费用口径与严格净收益

作者：Flxxx | 时间：2026-08-11 12:44 UTC | 字数：1241 | 评分：79

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 续学：LI.FI Route 费用口径与严格净收益

今天继续完成 Day 6 的机会发现支线，重点从“看 LI.FI 的 CHEAPEST 排序”推进到“用统一口径审核每条 route 的严格净收益”。我完成了 Arbitrum USDC -> Base USDC、1,000 USDC、order=CHEAPEST 的 /advanced/routes A/B 复测：版本 A 不传 integrator，版本 B 传 integrator=jumper.exchange。

这次 baseline 返回 16 条 route，jumper.exchange 返回 15 条；前 5 条 route 的工具路径顺序一致。但 top route eco 的费用呈现不同：baseline 的 toAmount / toAmountMin 为 997.5 USDC，feeCosts 中有约 2.4984 USDC 的 LI.FI Fixed Fee，Gas 约 0.0142 USDC；jumper.exchange 的 top route toAmount / toAmountMin 为 1,000 USDC，feeCosts 为空，Gas 约 0.0094 USDC。

今天厘清了一个重要口径：feeCosts 不能机械地从 toAmount 再扣一次。baseline 中 1000 - 997.5 与约 2.4984 USDC 的固定费基本一致，说明该费用已经反映在到账金额中。对币种 route，我采用：

text

strictNet = toAmountMin - fromAmount - gasCostUSD - 未包含的额外成本

因此 baseline top route 的 strictNet 约为 997.5 - 1000 - 0.0142 = -2.5142 USDC。这不是套利机会，而是数据完整后的明确 reject。

同时明确了 CHEAPEST 的职责边界：它只是 LI.FI 的候选路由排序，不是完整套利策略的净收益排序。正确流程是对同一次请求中的每一条 route 分别提取自己的 toAmountMin、fromAmount、Gas、费用口径、报价时间和 integrator，计算 routeStrictNet 后重排；缺少 toAmountMin、Gas、token/decimals 或费用归属无法判断的候选，标记 NO-GO，不能作为正收益报警。

今天没有发现或执行任何实盘套利。下一步是把 100 和 10,000 USDC 加入同样的 A/B 复测，并把 route 严格成本排序与两端 DEX 买卖、事后调仓成本组合成完整策略的 strategyStrictNet。

笔记 056: 今天完成了一条从报价证据到接受/拒绝结果的 Paper Trading 流水线。

作者：rakisa | 时间：2026-08-11 13:11 UTC | 字数：1756 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天完成了一条从报价证据到接受/拒绝结果的 Paper Trading 流水线。

原计划实时调用 LI.FI，但本地网络执行没有获得权限批准。我没有绕过限制，而是读取第四天保存的六份真实 LI.FI 原始报价做离线回放。因此今天使用的是真实历史报价，但不是今天的实时行情。

流水线做了什么

text

读取去程原始报价

- 使用去程toAmountMin作为回程输入
- 使用回程toAmountMin作为最终保护到账
- 计算两腿Gas和最低净结果
- 检查收益、时间和执行开关
- 输出接受或拒绝
- 保存完整审计日志

全程没有连接钱包、没有签名、没有广播交易。

三条Paper Gate规则

第一，最低净结果必须大于0：

text

最低净结果

- ≈ 回程toAmountMin
- 初始USDC
- 去程Gas
- 回程Gas

LI.FI的fee已经包含在到账量里，所以没有重复扣除。

第二，预计往返时间不能超过30秒。30秒只是本次实验的可配置门槛，不是市场通用标准。

第三，执行功能必须保持关闭。即使某次结果为正，也只能标记为PAPER_CANDIDATE_NON_ATOMIC，不能签名或下单，因为跨链两腿不是原子交易。

三档历史回放结果

初始金额	最终最低到账	两腿Gas	最低净结果	预计时间	决策
100 USDC	99.500625	\$0.0305	-0.529875 USDC	14秒	拒绝
1,000 USDC	995.006250	\$0.0363	-5.030050 USDC	1,087秒	拒绝
10,000 USDC	9,950.062500	\$0.0305	-49.968000 USDC	14秒	拒绝

三档全部返回REJECT_NEGATIVE_NET。

100和10,000 USDC虽然满足30秒时间要求，但最低净结果为负。

1,000 USDC不仅最低净结果为负，回程还走了Polymer Standard，预计1,080秒，往返共1,087秒，同时违反收益和时间两条规则。程序在主状态中记录负收益，审计检查中也保留了超时失败。

为什么用toAmountMin

官方文档说明toAmountMin是包含滑点后的最低结果。因此我没有使用更乐观的toAmount做闭环，而是：

text

去程toAmountMin → 回程输入

回程toAmountMin → 最终保护到账

这样计算出来的Paper结果更保守，也更适合做执行前过滤。

今天新增的审计能力

程序生成了8条JSONL审计事件，记录：

历史数据来源和日期；

每档金额读取了哪些原始文件；

每条规则是否通过；

净结果和预计时间；

拒绝状态与文字原因；

最终0候选、3拒绝、0签名、0广播。

这意味着以后即使程序拒绝一条路线，也能解释是因为负收益、超时还是执行开关关闭，而不是只返回一个false。

安全测试

我另外用明确标记的合成数据测试了正收益、负收益和超时三个分支，共7项断言：

正收益且时间合格，只能成为非原子Paper候选；

负收益必须拒绝；

超时必须拒绝；

任何测试都不能获得真实执行资格。

7项全部通过。合成测试数据没有混入上面的历史报价结果。

今天的结论

今天没有发现套利机会，但把之前分散的报价、成本和决策逻辑串成了一条可审计流水线。三档历史闭环都被正确拒绝，而且每个拒绝都有数据和规则依据。

我认为一个研究系统首先应该证明自己会拒绝错误机会，然后才有资格讨论自动执行。下一步是在获得只读网络权限后，用同一套流水线处理实时报价，继续保持钱包、签名和广播关闭。

笔记 057: 理解跨链流动性

作者：hello16 | 时间：2026-08-11 13:35 UTC | 字数：1250 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

理解跨链流动性

学习：LI.FI

LI.FI 是什么？\

LI.FI 是一个跨链流动性聚合与路由/执行层（不是单一桥）。它把分散在不同链上的：

桥 (Bridges)

DEX / DEX Aggregators

Solvers (意图求解器)

其他流动性来源

统一成一个接口，通过智能订单路由（Smart Order Routing）帮用户找到「价格 + 速度 + Gas + 执行成功率」综合最优的路径，并负责执行与监控。

核心价值：把「跨链 + 换币 + 多步操作」的复杂性全部抽象掉，开发者/交易者只需一次集成，就能访问 60+ 链上的流动性。

理解：Router（路由执行流程）

你给的简化图是正确的核心骨架，实际执行时通常会更细一些。完整逻辑如下：

Source Chain（源链）

↓

【可选】源链 DEX Swap（如果源资产不是桥友好资产，先换成可桥的资产，如 USDC）

↓

Bridge（跨链桥）

↓

【可选】目标链 DEX Swap（桥过来的资产如果不是目标资产，再换成最终想要的资产）

↓

Destination Chain（目标链）

更精确的 LI.FI 端到端流程（官方架构）：

1. dApp / 用户发起请求\

告诉 LI.FI：「我想从 A 链的 Token X 换成 B 链的 Token Y，金额是 Z」。

2. LI.FI API（链下聚合与路由层）

同时查询多家桥、DEX Aggregator、Solver

计算多种可能路径（价格、Gas、时间、成功率）

返回最优报价（Quote）和可执行交易数据

3. 用户确认并提交交易\

交易发送到源链上的 LI.FI Diamond Contract（主入口合约）。

4. 链上执行（Diamond + Facet 合约）

Diamond 合约根据路由，把调用转发到对应的 Facet 合约（Bridge Facet / DEX Facet / Solver Facet）

Facet 再调用真正的桥合约或 DEX 合约完成操作

5. 跨链完成后\

资产到达目标链，若需要还会在目标链再做一次 Swap，最终资产打到用户地址。

关键认知点（套利相关）：

路由不是固定的「Bridge → DEX」，而是动态组合：可能「Swap → Bridge → Swap」、也可能「只 Bridge」、也可能走 Solver。

真正影响净收益的变量：桥费、源链/目标链 Gas、滑点、桥延迟、失败率、资金占用时间。

LI.FI 有内置监控与 Fallback：某条路径失败时会尝试重新路由，这既是优势，也意味着机会窗口会被快速抹平。

笔记 058: 今天主要学习的是套利机会真正出现之后，应该怎么处理。

作者：Oxgen3 | 时间：2026-08-11 13:40 UTC | 字数：737 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要学习的是套利机会真正出现之后，应该怎么处理。

之前更多是在研究怎么发现机会、怎么判断价差、怎么计算成本。今天开始思考一个问题：

看到一个信号，并不代表这个机会真的能做。

所以如果以后监控到一个套利信号，第一步不是马上下单，而是先快速检查几个东西：

这个价差是不是真实存在的

当前的成交深度够不够

扣掉 Gas、手续费和滑点之后还有没有利润

两边的交易能不能正常执行

跨链的话，时间延迟和资金占用能不能接受

有没有合约、桥或者交易失败的风险

如果条件不满足，就直接放弃；如果不确定，就先观察或者模拟验证，不急着用真实资金测试。

今天也重新认识到复盘的重要性。

一笔交易最后赚钱了，不一定说明策略是对的；一笔交易亏了，也不一定说明策略完全没用。更重要的是把每次机会记录下来，看看：

什么情况下出现？\
当时为什么判断可以做？\
最后实际执行结果怎么样？\
哪个成本超出了预期？\
这种机会以后还能不能重复出现？
这样慢慢积累下来，才能区分真正有规律的机会和偶然出现的噪声。

今日最大的收获

前面几天一直在学习：

发现机会 → 分析路径 → 计算成本 → 建立策略

今天进一步理解了后面的：

捕获信号 → 验证 → 执行 → 记录 → 复盘 → 优化

感觉这才算是一个比较完整的套利闭环。

接下来准备把前面学到的 AMM、DEX、跨链、LI.FI、Hermes、成本模型这些东西串起来，慢慢整理成自己的链上套利知识图谱，同时筛选几个值得继续验证的策略方向。

先把流程跑通，再追求收益；先证明机会可以重复，再考虑放大。

笔记 059: 2026-08-11 补打 | 把 LI.FI Quote / Route 落成可追溯 SQLite\

作者：mirrornight2026 | 时间：2026-08-11 14:11 UTC | 字数：1540 | 评分：76

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-11 补打 | 把 LI.FI Quote / Route 落成可追溯 SQLite\

\

补记上一节的学习。前一节已经拿到 LI.FI 的真实只读 Quote 和 Advanced Routes；这次没有继续堆 API，而是先解决一个更基础的问题：怎样把报价完整、精确、可追溯地保存下来，避免后面计算利润时拿错字段或覆盖证据。\
\

我最初不理解为什么 \amount_raw\ 要存成 TEXT。真实例子里：\
\

\toAmountMin_raw = 99391824751519684052

\USDC decimals = 18

\

两者必须一起使用，才能还原为 \99.391824751519684052 USDC\。同一个 \amount_raw = 1000000\，在 6 decimals 下是 1 Token，在 18 decimals 下却是 0.0000000000001 Token，所以不能只凭 symbol 猜精度。\
\100 USDT\ 的 raw 数量也已经超过 SQLite 64 位 INTEGER 上限；若改用浮点数，真实 raw 值往返后会少 5588 个最小单位。我的新规则是：raw 数量用 TEXT 原样保存，计算时再转成任意精度整数或 Decimal，展示时才除以 10^{decimals} 。\
\

随后把真实脱敏证据落成了 SQLite，而不是只画表结构。最终包含：\
\

\- 3 条 observations：两腿 Quote 和一次 Advanced Routes 观察；\
\

\- 3 个 tokens：BSC USDT、USDC、BNB；\
\

\- 8 条 routes、10 条 route steps；\
\

\- 17 条 cost items。\
\

原来预估只有 16 条成本，测试却发现 Sushiswap 同时有两项 fee：约 0.2487 USDT 的 LI.FI Fixed Fee，以及约 0.0997 USDC 的 Sushi Protocol Fee。它们虽然都是 fee，但 Token 不同，不能直接相加 \amount_raw\；必须分别保留 Token、decimals 和原始数量，需要比较时再按同一时间点换算到统一计价单位。Gas 仍作为 external cost 单独记录。\
\

另一个修正是：\collected_at_utc\ 表示报价采集时间，不是 \executionDuration\。本次两腿 Quote 相隔 3 秒，第二腿与六条 Advanced Routes 相隔 529 秒。即使工具和路线名称相同，不同时间取得的 Quote 也必须新增 observation，不能用新值覆盖旧值。六条 Advanced Routes

来自同一次响应，适合相对比较；但它们仍只是候选报价，不是六条路线同时发生的链上 Fill。\
\

验证结果是 6 项自动化测试全部通过，SQLite integrity check 为 ok，外键问题、raw 格式错误和危险观察均为 0，来源文件 SHA-256 全部匹配。全程只读，没有连接钱包、授权、签名或广播。\
\

我现在的规则是：每次新 Quote 都 INSERT 新 observation；Token 数量保留 chain、address、decimals 和 raw；每项 fee/Gas 各占一行；任何比较都必须带采集时间和来源。下一步是在这套标准化数据上计算净收益下界，并加入报价新鲜度与 UNKNOWN 门禁，继续保持 paper-only。

笔记 060: 今天重温了一篇讲加密高频交易的文章 (High-Frequency Trading in Crypto)

作者：gang | 时间：2026-08-11 14:21 UTC | 字数：632 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天重温了一篇讲加密高频交易的文章 (High-Frequency Trading in Crypto，作者是个做了7年加密量化基金的人)，虽然讲的是 CEX 跨交易所套利，但里面的成本思路和链上套利是相通的，记几个印象深的点：

- 1\ 屏幕上的价差 $\neq$ 实际净收益。作者 \$100k 本金做跨所价差，月毛收入 \$72k 看着很美，但光手续费就 \$12k/月，加上 VPS、监控、开发维护，净利大概 \$58k。这跟链上完全对应：Gas、滑点、桥费、失败交易的损耗都得先算进成本模型，再谈机会成不成立。
 - 2\ 延迟直接决定命中率。他的实测数据：总延迟 <50ms 时命中率 82%，>150ms 只剩 31%。链上节奏虽然不一样，但本质相同——机会窗口就那么几百毫秒到几个区块，执行路径短的人吃到。另外他花了 \$30k 把订单路由从 800 μ s 优化到 200 μ s，命中率零提升，因为瓶颈在交易所处理那 5-10ms。教训是：先找真正的瓶颈，别在不是瓶颈的地方烧钱。
 - 3\ 单一套利策略的边际利润会被手续费压到接近零，真正赚钱的是同一套基础设施支撑的多个策略组合。映射到链上，我的理解是：别只盯一种价差结构，DEX 间、跨链、三角、清算这些机会应该尽量共用同一套监控和执行系统，摊薄成本。
- 接下来打算动手做的：先用 LI.FI 比较几条跨链路径的报价，把 Gas + 桥费 + 滑点都扣掉，看看还有没有真实价差存在，先建立成本模型再谈策略。

笔记 061: 打卡笔记：LI.FI 路径发现与套利机会实操

作者：Wave | 时间：2026-08-11 14:30 UTC | 字数：1654 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

打卡笔记：LI.FI 路径发现与套利机会实操

链上套利残酷共学 · 模块 2 · 2026-08-11

今天做了什么

围绕「使用 LI.FI 发现路径与机会」完成完整实操：理解跨链聚合与路由 $\rightarrow$ 实测不同链/资产/规模的报价 $\rightarrow$ 评估套利机会可行性 $\rightarrow$ 加入 Builders 社群。

一、LI.FI 聚合与路由原理

架构：dApp $\rightarrow$ LI.FI API $\rightarrow$ Diamond 合约 (EIP-2535) $\rightarrow$ Facet $\rightarrow$ 桥/DEX/求解器。聚合 35+ 桥、35+ DEX、意图求解器，跨 60+ 链

quote vs routes：quote 返回单条最优路径（可直接签名执行）；routes 返回多条候选对比

成本构成：LI.FI Fixed Fee 0.25%（固定比例）+ 桥自身费 + Gas + 滑点（默认 0.5%）

关键参数：slippage / allowBridges / allowExchanges / integrator（避免重复计费）

二、实测报价对比（关键发现）

稳定币跨链：损耗恒定 0.25%

| 路由 | 损耗 | 工具 |

|---|---|---|

| ETH $\rightarrow$ ARB | 0.250% | eco |

| ETH $\rightarrow$ BASE | 0.250% | eco |

| ARB $\rightarrow$ BASE | 0.250% | eco |

1K/10K/100K/1M 规模损耗率全部恒定 0.25%（比例费）

16 条候选路线到账差异 < 0.01%，成本核心都是 LI.FI 固定费

跨链 ETH 价差套利探测：不可行

- ARB 比 ETH 主网便宜 0.037%、BASE 便宜 0.009%

- 但单程搬运成本 0.25%+ → 成本是价差的 7 倍

同链兑换省费：有空间

- BASE USDC→USDT 走 LI.FI 仅 0.115% (kyberswap)

- ETH 主网 0.175% (fly)

三、套利机会评估结论

LI.FI 消灭了稳定跨链价差 (0.25% 成本 > 0.05% 价差)，但它是最优执行引擎

1. 稳定币/ETH 跨链搬砖：不可行，成本恒定 0.25% 高于一切稳定价差
2. 用 LI.FI 做执行层：资金费率对冲、清算买入等策略用它拿最优执行价
3. 同链最优路径：BASE 兑换省一半手续费，高频操作是实打实利润
4. 事件驱动套利：桥异常/流动性枯竭时极端价差 (>0.5%) 才值得动手
5. Earn 收益协议：LI.FI 聚合 20+ 金库，跨链一键存入 (需 API key 深度使用)

四、关键认知

1. 聚合器是套利空间的"杀手"：LI.FI 把全网流动性聚合后，各链价差被压缩到 <0.1%，远低于搬运成本——这也是专业聚合器存在的意义
2. 定位转变：把 LI.FI 当"执行引擎"而非"套利发现器"。找机会靠事件监控，执行最优价靠 LI.FI
3. 成本结构清晰：任何跨链动作先问自己——0.25% 固定费 + 桥费 + Gas，我的策略利润能覆盖吗？

五、社群与生态

LI.FI Discord：discord.gg/lifi (8 万+ 成员)

重要发现：残酷共学发起人 Bruce (@brucexu_eth) 就是 LI.FI 的 APAC Developer Relations——课程与 LI.FI 官方深度绑定，可随时在群里请教

下一步

用报告的 quote() 脚本搭建多链最优路径监控，异常损耗告警

把 LI.FI 作为 Phase 4 跨平台套利 Bot 的执行层

申请 LI.FI API key 提升限流 (200/分钟)，支持高频监控

笔记 062: 链上套利 Phase 1 Review：套利机会地图

作者：倪响 | 时间：2026-08-11 14:57 UTC | 字数：2168 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利 Phase 1 Review：套利机会地图

2026-08-07 | Day 7 | Phase 1 总结

六天路线回顾

- Day 1: AMM $x=y=k$ → 价格从哪来，交易如何改变价格
- Day 2: DEX 间价差 → 同一 pair 多个价格，但竞争不过 MEV
- Day 3: 三角套利 → 概念有价值，实战基本没戏
- Day 4: 跨链 + LI.FI → 时间窗口长，个人有机会
- Day 5: 清算 + MEV → 不同于 DEX 套利，拼监控不拼速度
- Day 6: 成本模型 → 把碎片串成公式，快速过滤

套利机会全景图

链上套利

- 同链
 - DEX 间价差 (Day 2)
 - 结论：主网个人基本没戏。毫秒级竞争 + MEV 抢跑。
 - L2 上小规模可能有极窄窗口，但需要自建节点。
 - 三角套利 (Day 3)
 - 结论：概念可行，实战不行。三笔 Gas + 闪贷竞争者。
 - 价值：图论思维，为跨链三角打基础。
- 跨链 (Day 4)
 - 时间窗口按分钟计，个人有空间。
 - 成本：桥费 0.25% + 双链 Gas ~\$0.05 + LI.FI 固定费 \$1。

| → 1000U 跨链净损失 ~\$2.50 (0.25%)。
| → 价差 >0.5% 开始有利润。
| → Phase 2 重点方向。
|
|—— 清算 (Day 5)
| → 和 DEX 套利完全不同：利润来自抵押品折扣而非价差。
| → 优势：可监控（健康因子公开）、有窗口、竞争者少。
| → 门槛：需要闪贷合约，Phase 1 只建立认知。
|
|—— MEV 相关 (Day 2/5)
→ 三明治攻击、抢跑、尾随。
→ 不是自己的方向，但必须理解——否则会成为别人的利润。

成本公式 (Day 6)

净利润 = 理论价差收益 - Price Impact - DEX/桥手续费 - Gas

关键发现：

金额	链	价差需求	实际可行性
\$1000	Arb	1%+	跨链可触及
\$1000	ETH	2%+	基本不可能
\$5000	Arb	0.5%+	跨链轻松覆盖
\$5000	ETH	1%+	理论可行，MEV 风险高

代码资产

scripts/

|—— pool_reader.py Day 1: 读池子 reserves，算 Price Impact
|—— price_diff_scanner.py Day 2: 对比两个 DEX 同 pair 价格
|—— cost_calculator.py Day 6: 套利成本快速估算
|—— (三角/lifi/清算 未写代码，仅在笔记中建立了认知)

Phase 1 最重要的三个认知

1. 屏幕价差是幻觉

不管 DEX 间还是三角，屏幕上看到的价差在扣除双向 Price Impact + 手续费 + Gas 后，绝大多数时候是负的。Day 2 的 Sushi-USDC/ETH 是活例子——价差 0.08%，实际套利亏 \$39。

2. 速度是最大的壁垒

同链套利的竞争不在策略，在基础设施——自建节点、mempool 监控、Gas 策略、闪贷。这些不是个人短期能建立的。个人不应该在这个维度竞争。

3. 时间窗口是个人的空间

跨链（分钟级窗口）和清算（可提前监控）给了个人思考的时间。Phase 2 应该聚焦这两个方向，而不是继续在 DEX 间和三角里找机会。

Phase 2 方向 (Day 8-14)

Day 8-14 应该从“学概念”转向“建系统”：

1. Li.Fi 深度使用：不只调一个 quote，而是批量扫描、对比路径
2. 跨链价差监控：定时拉多个链对的 quote，建立信号检测
3. 池子批量扫描：从 The Graph 拉数据，找高频价差池子
4. 套利数据积累：记录每次发现的价差、成本、结果

Phase 1 打卡总结：六天从 AMM 数学走到成本模型，建立了链上套利的完整认知框架。核心结论：同链 DEX 套利是 MEV searcher 的战场，个人的空间在跨链（分钟级窗口）和清算（可监控）。Phase 2 从概念转向系统——建监控、扫数据、验证策略。

03 MEV与抢跑（12 篇）

笔记 063: 今日研究资金费率套利 (现货多+永续空、delta中性收正funding) 的真实执行细节：并非无风险，

作者：Jun Guo | 时间：2026-08-10 16:32 UTC | 字数：271 | 评分：51

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日研究资金费率套利 (现货多+永续空、delta中性收正funding) 的真实执行细节：并非无风险，费率翻负、现货/永续基差走阔、手续费与空单强平都会吃掉收益。年化换算必须按各所结算频率 (8h/4h/1h) 调整，不能一律按 3×365 估算；可行性门槛是funding连续多日高于全成本线，而非单次高读数。行业内认真做的团队实际年化多在8-25%，宣传50%+的几乎都悄悄叠加了方向性敞口，已不再是套利。对照链上MEV套利：同区块原子买卖+闪电贷可零本金参与，但利润大头归区块构建者 (searcher通常仅约5%)，本质是打包竞价而非免费午餐。

笔记 064: Day 7：全自動套利系統架構 v0.6

作者：Web3Rason | 时间：2026-08-10 16:51 UTC | 字数：9776 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7：全自動套利系統架構 v0.6

前六天我一直把「把成本從 25 bps 壓到 1-2 bps」當成整個系統的 P0，還寫了一句「在做完這件事之前，所有掃描結果都是在錯誤的門檻下判定的」。

今天我把它做完了。它是一個 query 參數。

一、那個 P0 的答案是一個參數，而且我自己打 API 收了尾

我的待驗清單上第 8 項掛了很久：integrator 這個參數到底會不會讓平台費歸零？之前看到的兩種說法完全相反——一說費用整條消失，一說只是被隱藏起來、實際照收。

同一條路徑、同一時刻、唯讀報價，兩次呼叫：

```
| | 到帳 | feeCosts |  
| :--- | :--- | :--- |  
| 不帶參數 | 997.5000 USDC | 1 筆 (Fixed Fee 0.0025, included=true) |  
| 帶 integrator=jumper.exchange | 1000.0000 USDC | 0 筆 |
```

投入都是 1000 USDC。差 25.00 bps，剛好就是那筆平台費。

而且「費用只是被隱藏」那一說也一起被否掉了——

因為 toAmount 的實際差額跟 feeCosts 對得起來。錢是真的沒收，不是換個地方收。

這條改寫了我的戰略排序：

```
| 舊寫法 | 新寫法 |  
| :--- | :--- |  
| P0：成本工程 25 bps → 1-2 bps，「只要讀官方文檔」 | P0：25 bps → 0 (平台費)。連文檔都不用讀，是一個參數 |  
| break-even 27 bps | break-even 約 2 bps (只剩橋費 1.0-1.6 bps + gas) |
```

但要誠實講兩件事：

1. 機會證據仍然是 0 條。門檻降下來 ≠ 有東西可吃。這是必要不充分。
2. 我之前所有的掃描結果全部要重跑，因為它們都是在高了 25 bps 的門檻下判定的。任何成本紀錄如果沒寫明當時用的 integrator 是什麼，那份資料就不可比。

二、我推翻了自己今天剛得出的一個結論

這是今天最值得記的一段，因為它是推理正確、讀法錯誤。

同一批資料指出：最快的路徑和最便宜的路徑，成本只差 0.97 bps。

順著這個數字很自然會得到一句漂亮的結論——「快慢同價，那就無條件選最快，時間預算可以免費拉滿」。

我自己再測一次，四個金額檔位 × 兩種排序：

```
| 金額 | 最便宜 | 淨 | 最快 | 淨 | 速度溢價 |  
| :--- | :--- | :--- | :--- | :--- | :--- |  
| 100 U | 7 秒 | 0.00 bps | 1 秒 | 1.26 bps | 1.26 bps |  
| 1,000 U | 7 秒 | 0.00 bps | 1 秒 | 1.03 bps | 1.03 bps |  
| 10,000 U | 1,080 秒 | 0.00 bps | 1 秒 | 1.00 bps | 1.00 bps |  
| 100,000 U | 1,080 秒 | 0.00 bps | 3 秒 | 1.00 bps | 1.00 bps |
```

數字是對的。「0.97 bps ≈ 0」這個讀法是錯的。

1 bps 算不算免費，取決於毛利有多大。

毛利幾十 bps 時它是零頭；毛利只有幾 bps 時，它是兩三成的利潤。

而且它是比例不是固定費（從 1 萬到 10 萬都恆定 1.00 bps），所以不會被規模攤薄；

小額檔位反而更貴（100 U 時 1.26 bps）。

好消息是它的來源可以歸因：最快路徑多出的兩筆費用是中繼商的服務費，

照我自己的分類這屬於商業條款、不是物理摩擦——跟那 25 bps 同一類，是可談的，不是地板。

→ 成本表的 key 從三元組再擴一格：(from, to, 金額檔位, 速度檔位)。

三、有一種衰減，成本欄位永遠看不見

上面那張表裡最刺眼的其實不是溢價，是這個：

| 金額 | 最便宜路徑選到的橋 | 耗時 | 淨值 |

| ---: | :--- | ---: | ---: |

| 100 U / 1,000 U | eco | 7 秒 | 0.00 bps |

| 10,000 U / 100,000 U | polymerStandard | 1,080 秒 | 0.00 bps |

兩者的成本一模一樣，都是 0.00 bps。差別只有時間，而且差 154 倍。

所以：大額路徑的可執行性衰減不反映在金額上，只反映在時長上。

一個「只比到帳金額排序」的篩選器，在大額檔位是完全瞎的——

它會很有信心地把一條 18 分鐘的路排第一名，而 18 分鐘後那個價差早就沒了。

兩條處置：

executionDuration 是硬過濾閘門，不是排序欄位（這條之前就立了，今天補上斷點的具體位置）。

每個金額檔位都要獨立詢價。斷點是離散跳變（7 秒 → 1080 秒），不是平滑退化，內插一定錯。

四、「不碰 MEV / 清算」的理由，第二次被換掉

我原本的理由已經修過一次：從「不賺錢」改成「量不到」——

因為鏈上看到的轉出是付給區塊建構者的競價，而事後退款在鏈上不可觀測。

今天那個「不可觀測」被人做掉了：

| 口徑 | 數字 |

| :--- | ---: |

| 上一代機器人全生命週期競價支出 | 585.32 ETH / 299 筆 |

| 對應收到的退款 | 6.37 ETH / 18 筆 = 1.1% |

| 當代機器人 551 天的退款 | 0 |

退款不再是「未知的上檔空間」。它被量到了，而且小到補不回競價。

三情景估算裡的保守場景，就是實測場景。

結論沒變，但這次是有數字的不碰。

更有價值的是：稀缺的東西根本不是我以為的那些

閃電貸把資本門檻降到 0，結果不是機會變多，是競爭更兇。能被抄的就不是護城河。

小額清算的競價會吃掉約 98%（留存 2.8% / 1.8% / 2.6%，是恆定比例不是固定地板）。

斷點在 100 WETH 且很陡：137 WETH 留存 44.5%、468 WETH 留存 24.8%。

留存最高那筆的抵押品是 15,131 WETH（約 2,900 萬美元），接得住的對手只剩少數做市商。

決定性的一刀：那個機器人的利潤收款地址就是一家知名做市商本身。

可以直接當篩選器用的判準：

凡是存在「可原子變現出口」的地方，競價就會把價值吃光；

凡是競價為 0 的地方，要麼沒有價值，要麼沒有出口。

對照組很乾淨：98 萬美元的 RWA 抵押品競價只有 0.00021 ETH、15 萬美元的非流動永續競價是 0，

但 2.4 萬美元的 cbBTC（可原子變現）競價是 0.44454 ETH = 規模的 3.54%。

→ 實作藍圖可以抄，商業模式不可以抄。

五、風控補上一格：單腿被打掉之後該做什麼

上一版我寫了「主失效是單腿被強平」，但只寫到失效發生為止。

失效之後怎麼辦，那一格是空的。今天四組獨立來源指向同一個答案，其中一組是真實資損。

正解是「降對沖目標」，不是「補腿」。

| 動作 | 為什麼 |

| :--- | :--- |

| 補回被打掉的那條腿 | 要依賴的正是剛剛出事的那個薄盤口。深度正異常的時候補腿，等於在最壞的價格再下一次注 |

| 賣掉對應的現貨，把淨 Delta 拉回零 | 一次做到兩件事：淨 Delta 歸零、毛敞口同時下降（例：空腿被減半後賣掉等量現貨，毛敞口 60k → 30k）|

最貴的不是第一次被打掉，是第二次決策。

真實案例：空腿被向上插針打掉，當事人判斷「這根針明顯不正常、價格會回來」而選擇等待——空腿沒了以後手上只剩裸多，繼續持有就已經是在賭方向，不是套利了。

配套幾條：

- 看套利倉至少算三本帳：單腿保證金槓桿（離清算多近）／毛敞口／淨 Delta。
- 臨時追保上限 = 事件前淨資產 5%，而且追保後毛敞口必須繼續下降，否則那是加倉不是救倉。
- 對沖按 token 數量，不是 USDT 名目。極端基差下「兩邊各投 1000 U」根本不中性：現貨買到 4,515 顆，1x 永續只空得掉 994 顆，留 3,521 顆裸多。
- Mark / Index 1 升為獨立開門。標記價本身會被污染（實見偏離 +298%），整個強平價體系失真。偏離越大代表「空間看起來越好」，而正確反應是減倉。
- 跨場所保證金不會自己流動。鏈上現貨浮盈補不進交易所的保證金，空單照樣先爆。
- 壓測要測 2x / 5x / 10x / 30x 與 90%，不是 ±10%。
- 交易所事後賠付不可以寫進期望值。底線設在「單一倉位爆掉我還活著」。
- 補一條清算通路：資金費支付本身會把強平價往內推（不足時直接扣保證金），而且「交易所強制調整費率或上下限」是獨立風險項——費率是它家的內部參數，不是市場變數。

同一個事件會同時產生正尾和負尾：有人在同一個標的上套利成功、有人被埋、有人實盤虧損。

→ 事件級回測不能只採計成功案例。

六、假訊號的三個根因，全是同一族錯誤

三個不同做法、三個獨立來源，但根因一樣：用便宜的近似取代逐腿模擬。

| 根因 | 落差 |

| :--- | :--- |

| 靜態報價相乘 | 肉眼相乘 +28.83 bps → 扣三腿費 61.16 → 逐腿模擬 63.40 → 投入 1 萬 U 時 1,890 bps（虧 19%）|

| 求解器缺「池深上界」約束 | 掃出的 19 個「機會」全部是流動性幻覺 |

| 多費率檔位沒同讀該檔深度 | 四個費率池毛價差最高 25.77 bps，但最低費率池的深度只有 1/950 → 12 個組合的往返費全部蓋過毛價差 |

→ 靜態價差只能當觸發器，不能當判準。

→ 「費率檔位低」不等於「執行成本低」。

順手記一張單位靜默失效清單（都不報錯，只是安靜地給錯數字）

- v3 的 fee 單位是 pips 不是 bps：500 照 /10,000 算成 5%，高估 100 倍，結果是低費率池的機會被整批靜默過濾掉。
 - 多跳路由各腿 decimals 不同：後面幾跳按前一跳的精度算，造出 12 萬 bps 的假訊號。
 - 同名代幣跨鏈精度不同：同一個穩定幣在兩條鏈上一個 18 位一個 6 位，差 10^{12} 倍。
 - sqrtPriceX96 沒歸一化 decimals：量級荒謬時，
- 排查順序要改成「先查單位精度 → 再懷疑地址錯」，不要反過來。
- 浮點比較誤判閾值：改用大整數精確分數交叉相乘。

兩個很便宜、能一次抓到上面全部錯誤的驗算不變量：

- 同鏈往返：A → B → A 的邊際必須小於 0。靠這條抓到過某鏈 +4.28 bps 的異常報價。
- 雙向閉環：同一個環正反方向的無費環利應互為相反數。

正常殘差 0.09 bps；寫錯時殘差會是量級級別的，一眼就看得出來。

七、量測紀律再加三條（這三條講的是怎麼不騙自己）

| 紀律 | 反例 |

| :--- | :--- |

| 統計單位是「獨立事件」，不是「輪詢行數」 | 同一時間窗的 13 個採樣點被當成 13 個樣本 → 得出「收斂率 100%」；重算後那是一個簇，13/13 全都沒收斂 |

| 採樣頻率是儀器參數：結案「無窗口」前，必須先證明採樣粒度 < 窗口壽命 | 跨鏈往返 25 輪全負，差點結案「沒窗口」。實際往返摩擦只有 0.02%，是小時級採樣抓不到分鐘級窗口 |

| 回滾模擬不可計入成功率分母 |

先前所有記為「成功」的交易，其實都是被鏈上斷言擋下的模擬。分母灌水會讓失敗率看起來遠低於真實值 |

還有一條同型的量具失真：交易所的資金費率鉗制會把不同的原始溢價壓成同一個中性值

（大量重複的 0.0025%），資訊在鉗制那一步就被抹掉了。

→ 跨所短週期研究要直接比較原始溢價，不能拿資金費率當代理變數。

八、機會來源再加兩類（一類有一手 tx，一類只有機制）

第五類：狀態恢復套利——訊號是事件，不是價差

受害者把 17,920 USDC 打進近乎枯竭的薄池 → 只拿回 246.66 顆（隱含 \$72.6/顆）

機器人在正常池用正常價買 922 顆（花 1,086 USDC）

→ 回到那個被打歪的薄池賣掉，得 18,361 USDC 淨賺約 \$17,275

要監控的是「薄池大單衝擊事件」，不是「兩池價差」。

價差是結果，而且只存在幾個區塊——等輪詢掃到的時候通常已經被恢復了。

這一類極容易被歸錯類。同一筆交易被另一個人解讀成「授權漏洞被盜」，

但一手 trace 顯示錯價完全來自對手方自己的極端滑點，沒有任何授權問題。

歸錯類的代價是：掃描器會去監控「授權」而不是監控「池深」——兩者的防禦方式完全不同。

第六類（只有機制、零實盤）：資金費率被證券化了

有協議把「未來約 300 次資金費率的平均收益權」做成可交易單位：

Implied APR 是它的市場價（預期），Underlying APR 是實際發生的，結算時收兩者之差。

另外還有預期本身變動的資本利得（有一例入場 2.85%、出場 7%）。

架構含意：這讓「資金費率高低」從一個持倉成本項，變成一個可以做空的標的。

我原本那條「費率 <3% 不可當 edge」講的是吃費率；這是交易費率預期，是兩件事。

進候選、不進生產：零實盤佐證，費用結構與流動性都還沒確認。

可執行性再多一個維度：caller 資格

「有掛單而且資金充足」不等於你能成交。

某做市協議的策略資料裡帶有委外指令，可以把判斷交給外部合約；

實測用普通地址做唯讀報價會被外部憑證條件直接拒絕，換成有資格的地址才拿得到報價。

這個維度只讀掛單事件與虛擬餘額是完全看不出來的。

→ 閘門清單新增一項：「我這個地址有資格吃這筆嗎」。

同一協議還有一個可以事先算出來的失效：

「一份資金用虛擬餘額同時掛滿多個交易對」是官方宣傳的資本效率特性——

掛 N 個對但只有 1 份錢，結構上就保證最多一個能成交，其餘 N1 個報價天生不可執行。

所以它可以用「掛單總額 ÷ 實際餘額」事先估，不必等失敗。

九、參數驗證進度（接續 Day 6）

| # | 參數 | 值 | 狀態 |

| :- | :- | :- | :- |

| 1 | 聚合器平台費 | 25 bps | 官方 FAQ + 實打 API |

| 2 | 聚合器 rate limit | 匿名 75 次/2h ; Key 100 req/min | 實測 header |

| 3 | 直連橋費 (USDC/L2) | 1.0–1.6 bps | 實打官方 API |

| 4 | V3 深度計算法 | liquidityNet 前綴和 | 原始碼 |

| 5 | MEV tip 比例 | 策略 × 通道 × 鏈 ; Solana 私有 bundle ~15% | |

| 6 | CEX 費率階梯 | 一般 10/10 bps ~ VIP 1.1/2.3 bps | |

| 7 | 規模曲線 | $q = R_{eff} \times (\text{價差} - \text{費率}) / 2$ | |

| 8 | 長尾同鏈成本地板 | 198 ~ 563 bps | |

| 9 | 同鏈雙邊費底線 | Σ (每跳實際 fee tier), Curve 檔可低到 4.5 bps | |

| 10 | LP 覆蓋率分水嶺 | 日波動 1.5% (死因為逆選擇) | |

| 11 | MEV 佔優先費支出 | 66–80%, 前 15 名後斷崖 | |

| 12 | 兩端滑點 | — | 必須即時算 |

| 13 | 規模窗口上保本點 | 有, 與下保本點差約 94 倍 | |

| 14 | 跨鏈成本方向性 | 方向優勢隨規模檔位反轉 (5 萬 U 時流出 Base 便宜 7 倍) ; ≤ 1 萬 U 時方向差為零 | 補作用域 |

| 15 | 同資產路徑成本穩定度 | 同路徑 142 次極差 0.11 bps | |

| 16 | 時間權重溢價 | 1.00–2.79 bps 可把 1,080 秒壓到 1 秒 | |

| 17 | 預測市場費率公式 | $1.8\% \times \min(p, 1p) \times \text{shares, maker 恆 } 0$ | 降級 (另一版公式為 $7\% \times p(1p)$, 兩者至少一錯, 待查官方文件) |

| 18 | integrator 對平台費的影響 | ~~兩份證據相反, 擺動 62 bps~~ → 帶 integrator 時平台費整條消失: 997.5000 → 1000.0000 USDC, feeCosts 1 筆 → 0 筆 | 已結案 (本人實測) |

| 19 | gas 是否已計入報價輸出 | API 未提供該語義 | 明確標未知 |

20	原子套利實測失敗率	EVM 67% ; Solana 池級 60–73% (模擬收益 × 0.4)	
21	失敗成本兩級落差	Gas 級 ~0.2 bps vs 單腿級 ~40 bps (差 200 倍)	
22	下架結算前期現價差	峰值 439 bps / 最後 6 小時均值 226 bps	
23	跨所插針幅度離散度	同時刻同標的 : 19.6 倍 vs 0.33	
24	報價取得本身的延遲	p50 1,725.7 ms / p95 3,852.3 ms	
25	跨鏈價差翻轉時間尺度	31 秒可從 0.12% 翻到 +0.04%	
26	oracle stale 降權時間常數	$\tau_{\text{mark}} = \tau_{\text{index}} = 1$ 分鐘 (協議相依)	
27	借貸替代等橋的窗口壓縮	7 分鐘 → 27 秒	
28	新幣資費上緣	+0.70%/8h, 但壽命天級、主風險是爆倉	
29	速度溢價	恆定比例 1.00 bps (100 U 小額檔 1.26 bps), 不隨規模攤薄	新增 (本人實測)
30	executionDuration 規模斷點	≥ 1 萬 U 時從 7 秒跳到 1,080 秒 (154 倍), 而兩者淨值都是 0.00 bps	新增 (本人實測)
31	builder refund 佔競價比例	1.1% (585.32 ETH 競價 vs 6.37 ETH refund) ; 當代機器人 551 天 = 0	新增
32	清算競價留存率	小額恆定約 2% (競價吃掉 98%) ; 斷點 100 WETH, 468 WETH 時留存 24.8%	新增
33	閃電貸費實測	Balancer 0% / Aave V3 0.05%	新增
34	v3 fee 欄位單位	pips 不是 bps (照 bps 算會高估 100 倍)	新增
35	多費率檔位深度比	最低費率池的深度只有最高費率池的 1/950	新增
36	靜態相乘 vs 逐腿模擬落差	+28.83 bps → 63.40 bps (1 萬 U 時 1,890 bps)	新增
37	Mark / Index 1 的異常量級	實見 +298% (0.87552 vs 0.21975)	新增
38	名目對沖在極端基差下的裸倉殘留	各投 1000 U → 買到 4,515 顆、只空掉 994 顆, 留 3,521 顆裸多	新增
39	公共 RPC 單鏈 context 取得延遲	1,013–1,306 ms (要疊加在報價延遲之上, 不是取代)	新增
40	資金費率證券化的兩個 APR	Implied (市場價) vs Underlying (實際), 結算收兩者之差	新增 (零實盤)
41	滑點容忍在公開 mempool 是否為成本項	三人主張「三明治會吃滿授權額度」, 但全是推理 / 轉述	新增 (需一次公開 vs 私有通道對照實測)
42	Partner Key 的 quote 端點是否仍受 2 小時窗口	若成立, 160 倍的 rate limit 槓桿要打折	新增 (文件複核、未壓測)

十、一句話

Day 5 我發現「我可能一直在量一個被自己參數截斷過的世界」。

Day 6 更難堪：有些東西我根本沒有量具。

Day 7 是第三種：我有量具、數字也讀對了，但我差點把它讀成相反的意思。

「0.97 bps」和「1.00 bps」是同一個數字。

把它讀成「免費」，就會寫出「時間預算可以無成本拉滿」；

把它讀成「一項成本」，才會去問「相對於什麼的成本」——然後發現在毛利幾 bps 的地方，它是兩三成利潤。

參數錯了會被下一份資料打臉；架構缺一整層，只會安靜地一直正常；

而讀法錯了最麻煩——因為數字是對的，你不會回頭去查。

笔记 065: MEV 是通过决定交易是否被包含、以及以什么顺序执行，在常规区块奖励和 Gas 费之外可提取的最大价

作者：tai | 时间：2026-08-11 01:00 UTC | 字数：476 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

MEV 是通过决定交易是否被包含、以及以什么顺序执行，在常规区块奖励和 Gas 费之外可提取的最大价值。这是 ethereum.org 对 Maximal Extractable Value 的定义 (<https://ethereum.org/developers/docs/mev>)。在工作量证明时代，它最初叫 Miner Extractable Value；Ethereum

转向权益证明后，价值提取不再只属于「矿工」，因此现在通常使用「Maximal」。最重要的不是背英文，而是记住：

text	同一组候选交易	+	不同的包含	/	排除	/	执行顺序	↓
不同的成交价、清算归属、套利利润与区块收入							排序权本身产生经济价值	2.2

可证伪的回答

假设：如果两个候选区块只是交易顺序不同，但某个参与者的扣费后收益因此发生变化，那么这个差额中至少有一部分来自交易排序。

推翻条件：如果在所有可行顺序中，扣除交易费、Gas、区块空间支付、失败风险和资金成本后，参与者的收益没有变化或均不为正，那么这个候选就没有可执行的 MEV Edge。

笔记 066: D7 (8/11) 第一周复盘：完整演练「发现→算净收益→判断」

作者：peanutwen00 | 时间：2026-08-11 02:06 UTC | 字数：1007 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D7 (8/11) 第一周复盘：完整演练「发现→算净收益→判断」

- 1) 演练场景：Base 链 morpho 清算机会——替还 \$50,000 按 92 折拿 ETH，毛利 \$4,347 (8.7%)
 - 2) 七项成本逐项过：滑点 (ETH 主流，\$100-300) / 优先费 (竞拍性质，\$500-2000) / gas (Base 便宜) / 闪电贷 (选免费) / 失败期望 (成功率×净利失败率×成本)
 - 3) 关键纠偏①：验算习惯——毛利是 \$4,347 不是 \$434 (10 倍差，数量级检查)
 - 4) 关键纠偏②：优先费是"让出去的利润"不是"买心中保险"——正确做法是设出价上限 (最高出价 = 毛利最低要求利润)，超出就放弃
 - 5) 最终判断：不做——拼基建的机会不适合普通散户 (与 starkxun/寻月隐君实证一致)
 - 6) 第一周沉淀：完整决策框架 = 看见机会→算清成本→评估竞争→判断做不做→设定盈亏边界
- 产出：知识库 log 更新；arbitrage-opportunity-map 已收录 (morpho/Base 数据即本题背景)

=====

D8 (8/11) MEV 全景：三明治/抢跑/回退 + Flashbots/MEV-Share

- 1) 三明治经济学：价格是受害者推高的不是机器人推高的——机器人 \$1,000 点火，受害者 \$30,000 接力；利润 = 受害者多付的钱的一部分；受害者滑点容忍度 = 机器人的提款密码
- 2) 推价成本辨析：买入不是成本是仓位；机器人利润 = 卖出均价 - 买入均价 - 摩擦；受害者是"免费接力棒"
- 3) 击穿滑点风险：原子 bundle 下受害者 revert 则全部回滚，机器人不赚不亏；非原子裸抢跑才会砸手里——原子性价值体现
- 4) 三形态区分：抢跑 (只有前腿，NFT 抢跑例子) / 三明治 (前腿+后腿) / 回退 (只有后腿，吃冲击残羹，不伤害受害者)
- 5) Flashbots 三件套：Bundle (原子交易包) / Protect (散户防夹 RPC) / MEV-Boost (构建者市场)
- 6) MEV-Share：把交易拍卖给搜索者，用户从受害者变受益者；三明治不会消失只会转型 (竞价化、转向清算/JIT/回退)；MEV 是不可消灭的税，只能重新分配
- 7) 竞争与基建：三明治是最卷赛道 (机会公开、拼执行质量)，与清算/长尾形成对比

笔记 067: 参考

作者：mookixiao | 时间：2026-08-11 09:25 UTC | 字数：2002 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

参考

Gemini

问题

1. EIP-1559是什么东西，是个标准吗？
1. EIP表示Ethereum Improved Proposal (以太坊改进提案)
2. 地位：2021年8月引入以太坊核心代码，成为所有EVM兼容链 (Arbitrum、Polygon) 的交易计费规则
3. 规定：Gas费分为由网络拥挤程度决定的Base Fee (基础费) 和付给打包节点的Priority Fee (小费)，且消耗的Base Fee会直接被销毁 (Burn)
1. 为什么Base Fee被直接销毁而不是给打包节点？
防止节点串谋操纵Gas费。如果Base Fee归打包节点所有，在网络空闲时，节点可以用自己的账户发送大量假交易把区块填满，强制抬高全网Base Fee，而因为Base Fee最终会落回节点自己口袋，节点刷单几乎没有成本，销毁机制则杜绝了这种可能
2. 节点是如何校验Nonce连续性的，难道要遍历之前该地址的所有交易去看看Nonce是否连续？但是交易可能存在于任意区块中，这需要遍历所有区块吧？
1. 每个Ethereum节点都在本地维护着一份记录全球所有账户最新状态的数据结构 (基于MPT树，MPT=Merkle Patricia Trie，即默克尔帕特里夏树)，其中每个地址对应一条记录，包含4个核心字段：nonce (交易序号)、balance (余额)、codeHash (代码哈希)、storageRoot (存储根哈希)
1. codeHash：仅对CA (合约账户) 有效 (因为EOA没代码)，表示对应字节码的Keccak-256哈希值
2. storageRoot：仅对CA有效 (因为EOA不能持久化存储任何状态变量)，表示该合约内变量物理存储位置
2. 当节点收到一笔nonce=N (表示第N个交易，0-based) 的新交易，并在本地数据库中查到当前最新nonce=K (表示已经完成的交易数) 后，校验逻辑：
1. $\mathrm{N} = \mathrm{K}$ ：校验通过，进入打包序列
2. $\mathrm{N} > \mathrm{K}$ ：发生跳号，交易被放入未来等待队列 (Future Queue)，等待中间缺失的交易补齐
3. $\mathrm{N} < \mathrm{K}$ ：已经打包过的历史交易，丢弃
3. "区块被广播后，节点中的EVM按顺序逐条执行字节码指令"这句话什么意思？
1. 区块中包含的是一笔笔原始交易数据 (Raw Tx)，Raw Tx中记录了calldata、合约地址，EVM通过合约地址可以调取链上对应合约的指令字节码 (Bytecode)，配合输入到calldata运行得到结果

笔记

一笔区块链交易的完整生命周期：

1. 本地构建与签名 (Construction & Signing)

1. 构建字段：钱包在本地组装交易结构体，包括to (目标地址/合约)、value (原生代币数量)、data (调用合约的calldata)、nonce (账户交易递增序号)、gasLimit、EIP-1559费率参数 (gasFeePerGas、gasPriorityFeePerGas)

2. 私钥签名：钱包使用用户私钥通过加密算法对交易数据进行签名，生成原始交易字符串 (Raw Transaction/Raw Tx)，目前为止交易还只存在于客户端中、不消耗任何Gas

2. 网络广播与进入内存池 (Broadcasting & Mempool)

1. RPC广播：钱包通过API将Raw Tx提交给节点

2. 节点校验与传播：节点校验签名有效性、Nonce连续性、账户余额是否足以支付Gas等，校验通过后交易被放进节点内存池 (Mempool) 并向整个P2P网络扩散

3. 排序与打包入块 (Sorting & Sequencing)：包括L2打包和L1打包

4. EVM执行与状态定局 (Execution & Final Status)：区块被广播后，节点中的EVM按顺序逐条执行字节码指令

5. 交易替换、加速与取消 (Replacement & Speed Up & Cancel)：当交易在Mempool中长期处于挂起 (Pending) 状态 (比如Gas费过低) 时，可利用Nonce唯一性进行覆写

1. 加速交易 (Speed Up)：构建一笔相同Nonce、相同内容的新交易，并将Gas费提高 (一般需要比旧交易高出10%~12%)

2. 取消交易 (Cancel)：构建一笔相同Nonce、向自己地址转账0ETH的新交易，同时大幅提高Gas费

笔记 068: 【8月11日 学习打卡】链上套利 Day

7：清算速度优化 (事件驱动+预构建+mempool)

作者：feiy2165-blip | 时间：2026-08-11 11:09 UTC | 字数：707 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【8月11日 学习打卡】链上套利 Day 7：清算速度优化 (事件驱动+预构建+mempool)

一、清算速度架构升级

- 从"2s 轮询 + 3s 验证"升级到"事件驱动 + 预构建 + mempool 抢先"
- WebSocket 订阅新区块 → 区块级触发清算检查 (Arbitrum 0.25s 出块，延迟降 4 倍)
- 限流保护：每链最少 1s 一次查询，防 API 打爆

二、预构建交易

- HF 接近 1 的临界仓位提前取卖出路径 + 算参数 + 编码交易
- 跌破 1 时直接用缓存广播，省去 2-3s 验证时间
- 失败黑名单 5 分钟，防止重复尝试
- 当前 29 个仓位已预构建就绪

三、mempool 监听抢先

- 订阅 pendingTransactions，过滤 Morpho 清算调用
- 从 calldata 解析借款人地址
- 发现他人清算 → 2 倍 gas 用预构建抢先广播

四、多链统一方案

- 所有链统一 Morpho 原生闪电贷 (不依赖 Balancer/Aave/Spark)
- OKX V6 卖出优先 (V5 API 已废弃，参数名变化需更新)
- 8 链部署完成

五、实战复盘

- 某仓位被专业清算机器人抢先 (用混淆函数名 + 私有 bundle + 贿赂 builder)
- 教训：清算竞争 = 私有通道 + 事件驱动，轮询永远抢不过

六、核心认知

1. 清算速度 = 事件驱动 + 预构建 + mempool 监听
2. 预构建把"发现后准备"变成"提前准备"，跌破瞬间直接广播
3. 统一闪电贷源比多源更可靠
4. API 版本会变，文档要常查

笔记 069: 链上套利残酷共学 · Day 7：CoW Protocol 批量竞价与 Solver

作者：jinzy | 时间：2026-08-11 11:31 UTC | 字数：1828 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天学习一个新的链上交易机制：CoW Protocol 的 Intent、批量竞价、Solver 竞争与链上 Settlement。

一、CoW 与普通 DEX 交易有什么不同

普通 Swap 通常由用户或路由器直接指定交易路径，再把交易发送到链上。CoW Protocol 中，用户先提交带有数量、限价、有效期等约束的签名交易意图，不需要提前指定一条固定路径。

协议把多个订单组成一场批量竞价，多个 Solver 同时寻找执行方案：

text

用户签名 Intent

- 进入 Batch Auction
- 多个 Solver 提交 Solution
- 公平性与 Score 评估
- 选择获胜方案
- Settlement 合约统一结算

Solver 可以尝试直接匹配具有相反需求的用户，也可以调用 AMM、聚合器或其他外部流动性。它竞争的重点不是单纯“谁的交易发得快”，而是谁能在满足用户限价和结算约束的前提下，提供更好的整体执行结果。

二、只读复原一场真实竞价

通过 CoW 官方 Solver Competition API，我查看了竞价 auctionId = 13563231：

竞价返回两个候选 Solution；
两个方案执行同一个订单；
未获胜方案的买入量为 9433355093212909；
获胜方案的买入量为 9463580661298110；
最终由获胜 Solver 提交 Settlement 交易。

再从交易订单接口回读，用户签名时要求的最低买入量为 9343946455343157，实际执行量为 9463580661298110，两者相差：

text

119634205954953 raw buy-token units

当买入资产按 18 decimals 解释时，约为 0.0001196342。它表示实际成交相对用户签名底线的改善，但不能直接写成 Solver 利润或用户最终净收益。

三、几个容易混淆的概念

用户 Surplus

用户实际收到的数量超过签名限价要求的部分，可以视为执行改善。它还需要结合 token 单位、费用和订单类型解释。

Solver Score

Score 用于比较候选执行方案为竞价创造的价值。它会使用参考价格统一不同资产，但不能直接除以 decimals 后当成美元利润。

Solver 收益

Solver 的真实收益还涉及奖励、Gas、外部流动性成本、失败方案和基础设施开支，不能由用户 surplus 或 score 直接推出。

四、批量竞价和套利的关系

Solver 本质上在解决一个组合优化问题：

哪些用户订单能够直接互相匹配；
剩余数量应该走哪个 AMM 或流动性来源；
如何安排执行顺序和 clearing price；
扣除 Gas 和外部交易成本后，哪个组合仍然有效；
如何把更多执行改善交给用户，同时让 Solver 有动力参与。

所以这里的套利不只是发现两个池子的价格差，还包括订单之间的 Coincidence of Wants、不同流动性来源之间的组合，以及批量结算带来的成本节省。

五、MEV 保护的边界

用户提交的是离线签名 Intent，不需要把一条确定的 Swap 路径直接暴露在公开 mempool；多个 Solver 竞争也有助于把执行改善反馈给用户。

但这不等于彻底消灭 MEV。获胜的 Settlement 仍然需要上链，仍然存在排序、失败、Solver 行为、协议规则和外部流动性变化等风险。

今天最大的收获是：CoW Protocol 不是简单替用户寻找一条最优路由，而是把一批交易意图变成竞价与组合优化问题，再通过 Solver 竞争和 Settlement 合约完成可验证结算。用户 surplus、Solver score、协议费用和 Solver 利润必须分别核算，不能混成同一个“套利收益”。

笔记 070: Day 7 : 把停止线扩展为候选机会状态机。

作者 : daohuang1982-beep | 时间 : 2026-08-11 12:20 UTC | 字数 : 476 | 评分 : 59

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 : 把停止线扩展为候选机会状态机。

今天把 Day 6 的单条停止线扩展成三层验证流程：第一层先用页面级毛差减去双边手续费和安全缓冲；通过后才进入固定规模、尽量同一区块状态的 Quoter；最后再核算真实路径 Gas、MEV、失败成本和资金占用。证据不完整时保持 needs_quote，即使后续为正也最多先标记为 paper_trade_candidate，不能直接写成可执行机会。

我还校正了两个容易混淆的数字：历史样本 0.318722% 毛差减去 0.35% 双边手续费后为 -0.031278%；若再加入 0.05% 安全缓冲，真正的停止线余量是 -0.081278%。两者都低于零，因此该样本应在第一层直接标记为 screened_out，不继续投入 Quoter 和 Gas 深挖。

今天的产物是一份可复用的候选机会记录模板，强制记录时间、链、资产与池地址、输入规模、来源、缺失字段和下一项验证动作。今天没有采集新的市场报价，也没有确认任何实时套利机会；当前结果是研究流程升级，不是交易结论。

笔记 071: 今日打卡 : 三角套利与 MEV

作者 : tobin | 时间 : 2026-08-11 12:26 UTC | 字数 : 5541 | 评分 : 90

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日打卡：三角套利与 MEV

今日学习目标

今天把双池套利扩展到三角套利，并理解“计算出正利润”与“真正把利润收入钱包”之间还隔着 MEV、交易排序和区块构建竞争。

核心问题有两个：

1. 如何对一条三段兑换路径进行准确报价，并寻找净利润最大的输入量？
2. 为什么本地模拟显示盈利，真实发送后仍可能失败、被复制或被别人抢走？

一、什么是三角套利

三角套利从一种资产出发，连续兑换两种中间资产，最终回到起始资产。例如：

USDC -> WETH -> WBTC -> USDC

三个交易池可能来自同一个 DEX，也可能来自不同 DEX。只要最终回到同一种资产，起始和结束余额就可以直接比较。

设初始投入 amountIn：

out1 = quote(pool1, USDC, amountIn)

out2 = quote(pool2, WETH, out1)

out3 = quote(pool3, WBTC, out2)

grossProfit = out3 - amountIn

netProfit = grossProfit

- gasCost

- borrowPremium

- inclusionOrBuilderPayment

只有 netProfit 大于预设安全阈值时，这条路径才是候选机会。

二、机会为什么会出现

三个市场分别维护自己的储备和价格，它们不会在任何时刻都严格满足一致的交叉汇率。

假设：

1 ETH = 2000 USDC

1 BTC = 20 ETH

1 BTC = 41000 USDC

沿路径 USDC -> ETH -> BTC -> USDC，理论上：

40000 USDC -> 20 ETH -> 1 BTC -> 41000 USDC

表面毛差为 1000 USDC。但 AMM 的真实成交会产生三次手续费和三次价格冲击，因此不能直接按页面价格相乘。

价差可能来自：

某个池刚刚发生大额 Swap，价格暂时偏离；
三个池深度不同，对同一次市场变化反应速度不同；
不同 DEX、手续费档位或路由的报价不同；
预言机更新、清算、铸造或赎回改变了某种资产的供需；
一笔待打包交易即将制造新的价格偏差。

三、小额条件：三个边际汇率的乘积

对每一跳定义扣除交易费后的边际兑换率 r_1 、 r_2 、 r_3 。当输入无限小时：

$\text{cycleRate} = r_1 r_2 r_3$

如果 $\text{cycleRate} \leq 1$ ，这个方向连三次手续费都无法覆盖，可以快速排除。

如果 $\text{cycleRate} > 1$ ，只能说明小额情况下存在理论空间。随着 amountIn 增加，三次 Swap 都会推动池内价格，实际输出倍率会下降，所以仍然需要完整模拟。

一条三角路径有两个方向：

A -> B -> C -> A

A -> C -> B -> A

两个方向都必须计算，不能通过肉眼判断。

四、逐跳报价必须使用真实储备方向

对 V2-compatible Pool：

$\text{amountInWithFee} = \text{amountIn} \cdot \text{feeNumerator}$

$\text{amountOut} = \text{amountInWithFee} \cdot \text{reserveOut}$

$/ (\text{reserveIn} \cdot \text{feeDenominator} + \text{amountInWithFee})$

Uniswap V2 中 $\text{feeNumerator}=997$ 、 $\text{feeDenominator}=1000$ 。

三角套利需要连续调用三次整数版 getAmountOut ：

$\text{out1} = \text{getAmountOut}(\text{in0}, \text{reserveA1}, \text{reserveB1})$

$\text{out2} = \text{getAmountOut}(\text{out1}, \text{reserveB2}, \text{reserveC2})$

$\text{out3} = \text{getAmountOut}(\text{out2}, \text{reserveC3}, \text{reserveA3})$

实现时需要特别注意：

根据 $\text{token0}/\text{token1}$ 确定 reserveIn 和 reserveOut ，不能把方向写反；

所有 Token 数量使用 bigint ，保留 Solidity 向下取整；

三个池必须读取自同一个 blockNumber ；

每个池可能有不同手续费，不能统一假设为 0.3%；

如果一条路径重复使用同一个 Pool，后续一跳必须使用前一跳更新后的储备，不能继续使用初始快照；

fee-on-transfer 、 rebase 等非标准 Token 会破坏普通报价假设，第一版应使用白名单。

五、最优输入量

三角套利同样不是投入越多越赚钱。

输入太小时：

绝对毛利润很小；

固定 Gas 成本可能超过利润。

输入逐渐增加时：

被利用的价差增加；

绝对利润通常先上升。

输入过大时：

三个池都产生更大价格冲击；

第三跳得到的起始资产增量越来越小；

利润到达峰值后下降，最终可能变成亏损。

因此目标函数是：

$\text{profit}(q) = \text{cycleQuote}(q) - q$

- $\text{gasCost}(q)$

- $\text{borrowPremium}(q)$

- $\text{executionPayment}(q)$

对纯 V2 池，三个有理函数可以继续合并并推导连续候选解；但工程上更稳妥的办法是：

1. 使用边际汇率乘积快速过滤无利润方向；
2. 用对数网格搜索确定盈利区间；
3. 在最佳区间使用 ternary search、黄金分割或其他一维搜索；
4. 在候选最优点附近使用 bigint 做离散局部搜索；
5. 用 eth_call 或固定区块 Fork 模拟完整交易；
6. 扣除安全余量后再判断是否值得发送。

如果利润函数不是单峰，或者路径包含 V3 跨 Tick、动态费率或重复 Pool，就不应盲信单峰搜索，需要分段模拟或枚举候选边界。

六、三角套利为什么最好原子执行

如果把三次 Swap 拆成三笔交易：

第一笔买入 B

-> 等待确认

-> 第二笔换成 C

-> 等待确认

-> 第三笔换回 A

中间任何一次价格变化都会留下 B 或 C 的敞口。

更合理的方式是把三跳放进同一笔交易：

获取本金或闪电借款

-> Swap 1

-> Swap 2

-> Swap 3

-> 偿还借款

-> 检查 $\text{finalBalance} \geq \text{initialBalance} + \text{minProfit}$

不满足 minProfit 时整笔交易 revert。资产状态会回滚，但已经消耗的 Gas 不会返还。

七、MEV 是什么

MEV 可以理解为：通过选择、排序、插入或排除区块中的交易，从区块状态变化中提取的价值。

三角套利本身就是一种常见的 MEV。利润不仅取决于“计算是否正确”，还取决于交易能否在目标状态上、以目标顺序进入区块。

主要参与者：

用户：提交普通 Swap 或其他改变状态的交易；

Searcher：发现套利、清算、Sandwich 等机会并构造交易；

Builder：收集公开交易、私有交易和 Bundle，构建收益更高的区块；

Proposer/Validator：在对应 Slot 提议最终区块；

Relay 或私有通道：在 Searcher、Builder 与 Proposer 之间传递区块或订单流。

因此，mempool 中“看起来有利润”只是候选状态，并不等于最终区块会按我期待的顺序执行。

八、公开 Mempool 中的竞争

如果把明显盈利的套利交易直接广播到公开 mempool，其他 Searcher 可能：

复制 calldata 或重新构造同一路径；

提高 priority fee 或给 Builder 更高付款；

抢在我的交易前执行，先消耗机会；

将我的交易夹在其他交易之间；

让我的交易在其后执行并因 minProfit 不满足而 revert。

结果可能是：

交易没有被包含；

被包含但执行失败，只损失 Gas；

成功执行，但实际利润显著下降；

机会被其他 Searcher 完全拿走。

所以生产系统不能把“发出交易哈希”当作成功，必须等待 Receipt，并核对最终 Token balance delta、Gas 和区块位置。

九、Frontrun、Backrun 与 Sandwich

Frontrun：交易被安排在目标交易之前，以提前占用它将产生的机会。

Backrun：交易紧跟在目标交易之后，利用目标交易刚刚制造的价格偏差。DEX 套利经常属于 Backrun。

Sandwich：攻击者先在用户之前交易推动价格，用户按更差价格成交，攻击者再反向交易获利。

假设一笔用户大额 Swap 将 Pool A 的 ETH 价格推高, Searcher 可以把套利交易放在用户之后, 通过其他池买入较便宜的 ETH, 再卖到 Pool A。这个机会只有在目标用户交易先执行时才存在, 因此顺序本身就是策略的一部分。

十、私有交易和 Bundle 能解决什么

私有通道可以让交易不进入公开 mempool, 降低被简单复制或抢跑的概率。Bundle 可以向 Builder 表达一组有顺序要求的交易, 例如:

[目标状态变化交易, 我的 Backrun 套利交易]

但私有提交并不等于保证盈利:

- Builder 不一定选择该 Bundle;
- 其他 Searcher 可能提交更高价值的竞争方案;
- 目标交易可能取消、失败或改变;
- 模拟状态与 Builder 实际使用的区块状态可能不同;
- 支付给 Builder 的价值会侵蚀利润;
- 不同链的 Builder、Relay 和私有 RPC 机制并不完全相同。

最终保护仍应写在合约中: deadline、amountOutMin、允许的 Pool、最大输入和 minProfit 缺一不可。

十一、扫描器需要增加的 MEV 字段

除了 reserves、amountIn、amountOut 和净利润, 机会记录还应保存:

- observedBlockNumber 和 observedBlockHash;
- 机会来自已确认状态还是 pending transaction;
- 目标交易哈希及要求的相对顺序;
- public 或 private submission channel;
- 预计 gasUnits、maxFeePerGas、priorityFee;
- Builder/Bundle 付款;
- 模拟时的 state block 和 target block;
- 提交时间、是否包含、transactionIndex;
- 实际 balance delta、Gas 成本与失败原因。

只有这样才能区分: 数学假利润、状态过期、输掉排序、模拟差异、交易 Revert 和真实成交利润。

今日结论

- 三角套利是 A -> B -> C -> A 的闭环, 必须逐跳使用真实成交报价。
- 三个边际汇率乘积只能用于快速过滤, 不能代替金额相关的完整模拟。
- 三次手续费和价格冲击使利润曲线存在最优输入量。
- 三跳应尽量放在同一笔原子交易中, 并用链上 minProfit 保护结果。
- 套利属于 MEV; 发现机会后, 还必须赢得交易排序和区块包含竞争。
- 私有通道与 Bundle 可以减少公开暴露并表达顺序, 但不能保证包含或盈利。
- 真正的结果必须以 Receipt、区块位置和最终余额变化为准。

下一步

下一步实现三角路径枚举器: 从 Token 图中寻找长度为 3 的闭环, 在固定 blockNumber 下读取三个池的状态, 对两个方向执行 bigint 报价和最优输入搜索; 随后在本地主网 Fork 中构造一笔能原子完成三跳并检查 minProfit 的交易, 并记录公开提交与私有 Bundle 模拟之间的差异。

参考资料

- Ethereum: Maximal Extractable Value (<https://ethereum.org/developers/docs/mev/>)
- Flashbots: Understanding Bundles (<https://docs.flashbots.net/flashbots-auction/advanced/understanding-bundles>)
- Flashbots Protect (<https://docs.flashbots.net/flashbots-protect/overview>)

笔记 072: 1Meme池与交易传播 (Mempool & Propagation)

作者: wodeche | 时间: 2026-08-11 14:01 UTC | 字数: 1621 | 评分: 74
来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1Meme池与交易传播 (Mempool & Propagation)

内存池是区块链节点暂存已广播但尚未被打包进区块的待处理交易 (Pending Transactions) 的地方。

P2P 网络传播时延: 交易从用户节点广播到全网验证节点需要时间。套利 Bot 通过在多个地理位置 (如东京、法兰克福) 部署极速节点, 监听 Mempool 中的大额交易。套利触发信号: 当 Bot

监听到一笔会产生巨大价差的待处理交易时，会在本地模拟计算，并在同一时间将自己的套利交易广播出去。

2. 模拟与干跑 (Simulation & Dry-Run)

在向链上发送真实交易之前，Bot 必须精确预测交易能否成功以及收益多少，避免浪费额外的 Gas 费。

本地节点/状态分叉模拟：利用 Geth、Anvil 或 Hardhat 等本地节点，拷贝当前链上的最新状态 (State)，在本地沙盒环境中先行“干跑”套利合约。

状态改变检测：确保在交易执行的那一刻，流动性池的状态没有发生意料之外的变化（如滑点超标、池子流动性被撤走）。

3. 私密交易通道与 Flashbots Protect (Private Transactions)

在公开 Mempool 中广播套利交易极易被更快的 Bot 或 MEV 抢跑者 (Front-runners) 发现并“偷走”套利路径。

RPC 私密通道 (Private RPC)：如 Flashbots Protect、MEV-Blocker 等。交易不会广播给全网，而是直接发送给支持 MEV 保护的构建者 (Builders)。

Bundle (交易包)：将多笔交易打包成一个 Bundle (例如：[目标用户的交易, 套利的交易])，要求验证者要么全部顺序打包，要么全部不打包，彻底避免被第三方插队。

4. 盲套利 (Blind Arbitrage)

传统套利需要实时监视公开 Mempool 中的 Pending 交易，而盲套利则是另一种策略。

原理：不依赖监听待处理交易，而是专门针对已经打包进最新区块 (State Expiry) 但尚未被完全平套的价差。

特点：盲套利 Bot 通常会在每个新区块产生的第一时间 (Block Header 刚发布时)，通过智能合约自动检测多个池子是否存在微小差价并直接触发套利。

5. 聚合器与路由算法 (DEX Aggregators & Routing)

像 1inch、Paraswap 等 DEX 聚合器的核心逻辑就是基于链上套利与最优路径计算。

多路径拆单 (Split Routing)：为了降低大额交易的滑点，算法会将一笔大额买单拆分为多条路径（如 30% 走 Uniswap v3，70% 走 Curve）。

虚化/无损套利 (Lossless Routing)：

聚合器在帮助用户寻找最佳兑换路径时，自身也在进行复杂的链上多池套利计算，以最大化留存剩余价值。

6. 无偿损失与动态再平衡 (Impermanent Loss & Rebalancing)

当套利者涉及跨链套利、CEX-DEX 套利或长期提供流动性时，资金管理至关重要。

对冲与再平衡：链上套利完成后，Bot 在链上积累了某种代币，而在 CEX 卖出了对应代币。此时需要通过对冲（如永续合约 Hedging）或跨链桥将资产归位，防止持有现货暴露在市场单边下跌的风险中。

资本效率 (Capital Efficiency)：评估套利策略时，不仅看单次收益率，更要看资金周转率（如利用 Uniswap v3 集中流动性范围提升资金利用率）。

笔记 073: 普通用户 DEX 交易规避三明治 (夹子) 攻击完整方案

作者：Rui Pan | 时间：2026-08-11 14:24 UTC | 字数：954 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

普通用户 DEX 交易规避三明治 (夹子) 攻击完整方案

三明治攻击属于掠夺式 MEV：机器人监听公共内存池 (Mempool) 里你的待确认交易，前置买入推高币价，你的交易高价成交，机器人后置卖出完成套利，你的损失体现在成交价差、实际到手代币变少，钱包资产不会直接被盗，但会产生隐形交易损耗。

提示：没有 100% 绝对防护手段，下面方案可以极大降低被夹概率，只能减少损失，无法完全消除 MEV 风险。

一、核心原理回顾 (为什么你会被夹)

1. 交易发送到公共内存池，所有 MEV 机器人都可以看见你的兑换金额、滑点设置。
2. 滑点设置越高，留给机器人套利空间越大。
3. 低流动性小币种、Meme 币、大额兑换，是夹子攻击最高发场景。

二、实操防护清单

1. 合理设置滑点容忍度

滑点代表你能接受的最大价格偏移，滑点越高，夹子机器人利润空间越大

稳定币互换：滑点建议 0.1%0.3%

主流高流动性币 (ETH、BTC 类)：滑点 0.3%0.5%

小币种 / Meme 币：不要直接设置 5%、10% 等高滑点，建议1% 以内

权衡：滑点设置过低，行情剧烈波动时交易会直接失败并消耗 Gas；

2. 交易行为层面避坑

1. 大额拆分成多笔小额交易

一笔大兑换是机器人重点狩猎目标；把大额拆成多笔间隔数十秒的小单，单笔交易对池子价格冲击变小，机器人攻击收益不足以覆盖 Gas 成本，放弃攻击你。

2. 避开低流动性池子

流动性深度差的交易对，很小一笔交易就会造成巨大价格冲击，极易被三明治攻击；尽量选择流动性充足的交易池。

3. 避开拥堵、fomo 狂热时段

币热度暴涨、链上 Gas 剧烈飙升的时候，MEV 机器人竞争最激烈，尽量减少此时兑换小币种。

4. 优先使用批量拍卖 DEX (CoWSwap 等)

不使用传统 AMM 即时撮合，采用订单批量清算，消除交易排序带来的抢跑风险；缺点是部分长尾币种流动性不足。

笔记 074: 2026.08.11

作者：Johnova | 时间：2026-08-11 14:51 UTC | 字数：2065 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026.08.11

Day 7：链上套利残酷共学

今日核心进展

2026.08.11

Day 7：闪电贷合约原型开发 + Balancer Vault 深度研读

今日完成

- [x] Balancer Vault IVault / IFlashLoanRecipient 接口完整研读
- [x] 编写 FlashLoanArbitrage.sol 合约骨架：flashLoan → receiveFlashLoan → 多跳 Swap → 归还
- [x] Anvil fork Arbitrum 主网测试：验证多资产借入/归还流程
- [x] 实现核心 Swap 路由：Uniswap V3 (0.05%/0.3%) + Curve 3Pool + Balancer Vault
- [x] Gas 优化：immutable 地址、assembly 调用、unchecked 算术、calldata 优化
- [x] 单元测试覆盖：借入/归还/多跳 Swap/利润校验/失败回滚

核心洞察

1. Balancer Vault 闪电贷 0 费率是核心优势 — 相比 AAVE 0.05%，单笔 \$100M 借入节省 \$50k 费用，直接决定套利可行性
2. receiveFlashLoan 回调模式要求严格 — 必须在回调内完成所有 Swap 并归还，不能跨 tx，需精心设计 userData 传递路径参数
3. 多资产借入需精确计算 amounts 与 userData 对应关系 — assets[i] 对应 amounts[i]，路径编码必须与资产顺序严格对齐
4. Gas 优化对原子套利是生死线 — 5 跳 Swap 目标 < 1.5M gas，每跳节省 3-5k gas 需 assembly 级优化

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

| 闪电贷源 | Balancer Vault (主网) | IVault.flashLoan、多资产、0费率、回调模式 |

| 目标链 | Arbitrum One (主网) | Gas 低、Uniswap V3 深度最好、MEV 友好 |

| DEX 路由 | UniV3(SwapRouter02) + Curve 3Pool + Balancer Vault Swap | 费率层 500/3000、集中流动性、StableSwap |

| 路径编码 | bytes calldata : path[] + minOut[] + dexSelector[] | 紧凑编码、链上解码、防止溢出 |

| Gas 策略 | BaseFee 预测 + p99 PriorityFee + RBF 替换 | maxFeePerGas 动态、maxPriorityFeePerGas 预留 20% |

| 利润校验 | 双重 minOut：每跳最小输出 + 总利润最小阈值 | 防三明治、防整体亏损、revert 回滚 |

| 部署策略 | CREATE2 确定性部署 + 代理合约可升级 | 多链同地址、逻辑可升级、资金安全 |

遗留疑问

1. receiveFlashLoan 中如何优雅处理多跳 Swap 的中间代币余额（避免粉尘攻击）？
2. Anvil fork 测试 vs 真实主网 Gas 差异如何量化（预估 1.2-1.5x）？
3. Balancer Vault 的 managePoolBalance 权限是否需要预先授权给套利合约？

明日预习 (Day 8)

- [] 多 DEX Router 统一接口设计：ISwapRouter 抽象 + 具体实现
- [] Curve 3Pool / Metapool 交互细节：exchange / exchange_underlying / get_dy
- [] Uniswap V3 SwapRouter02 exactInput / exactOutput 多跳编码
- [] 合约间调用 Gas 优化：delegatecall vs 普通调用 vs library

关键洞察

- 待补充...

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

| 待补充 || |

遗留疑问

1.

2.

明日预习

-

04 做市与LP（1 篇）

笔记 075:（本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责

作者：geyu210 | 时间：2026-08-11 13:43 UTC | 字数：1176 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

（本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责格式排版与提交流程。）

【Aqua 学习第 5 天：核对 SwapVM 的真实指令语义】

今天没有继续增加报价样本，而是回头检查昨天解析出的 Aqua SwapVM strategy。结果发现了一个很关键的问题：我之前直接按照 AquaOpcodes.sol 中固定数组的位置解释 opcode，导致所有指令名称整体错了一位。

原因是合约通过 assembly 把固定数组复用成动态数组，并将原来的第 0 项覆盖为数组长度。因此运行时 opcode N，实际对应源码固定数组中的第 N + 1 项。

修正后，这条策略的六条指令是：

- only_tx_origin_token_balance_non_zero
- aqua_protocol_fee_amount_in_xd
- xyc_concentrate_grow_liquidity_2d
- flat_fee_amount_in_xd
- xyc_swap_xd
- salt

这改变了我昨天对策略的理解。第一条并不是把逻辑委托给外部合约，而是检查 tx.origin 是否持有指定 token。此前使用普通占位 caller 报价失败、换成满足条件的 caller 后报价成功，真正原因是这个 access-token gate。

不过，只读 eth_call 可以模拟任意地址作为 from，所以“模拟该地址能够报价”不等于我控制这个地址，也不代表我拥有真实成交资格。

进一步解析参数后，还能看到这条策略包含约 1.25 bps 的 Aqua protocol fee 和 5 bps 的 flat fee。它更准确的描述应该是：一条带访问条件、集中流动性参数和两层费用的 Xyc 策略，而不是昨天理解的 Extruction/Decay 策略。

我重新运行了同一个历史区块范围，仍然观察到 1 个开放和 1 个已 dock 的 USDC/WETH position，修正后的六条指令都能稳定解码。本地增加了专门的偏移回归测试，完整检查共 304 项测试全部通过。

昨天得到的历史报价数值没有因此改变，Aqua 相对同区块 Uniswap 基准仍低约 1.549%，所以这个样本依然没有出现正向可执行偏差。但今天修正了比增加样本更基础的问题：只有先理解策略真正执行了什么，后面的价格比较才有意义。

官方源码：<https://github.com/1inch/swap-vm/blob/v1.0.2/src/opcodes/AquaOpcodes.sol>

下一步再基于正确的指令语义，采集至少 5 个固定区块的 Aqua/Uniswap 同区块报价样本。

05 其他（1 篇）

笔记 076: 有个反直觉的观察：噪声带宽和流动性不是正相关。

作者：eventXhorizon | 时间：2026-08-11 14:57 UTC | 字数：193 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

有个反直觉的观察：噪声带宽和流动性不是正相关。

WETH \$5k 的重复极差是 ± 25.3 ，而 DEGEN、OP 全是 $\pm 0 \sim 1$ 。

看日志才明白，长尾只有一两个 solver 报价（DEGEN 买走 sushiswap、卖走 bitget，OP 全走 okx/1inch），没有切换所以稳；主流反而因为 kyberswap/okx/fly 抢单而抖。

噪声来源是竞标者数量，不是深度

06 套利框架（46 篇）

笔记 077: 2026.08.10

作者：Johnova | 时间：2026-08-10 15:01 UTC | 字数：1786 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026.08.10

Day 6：链上套利残酷共学

今日核心进展

2026.08.10

Day 6：Week 1 复盘 + Week 2 规划

今日完成

- [x] Week 1 全维度复盘：基建/策略/数据/工程化/资金
- [x] 核心指标统计：信号数/可执行/验证笔数/胜率/均收益/Gas占比
- [x] 关键问题复盘：数据瓶颈/成本偏差/最可行类型/工程痛点/资金利用率
- [x] Week 2 详细规划：Day 7-13 逐日任务 + 验收标准
- [x] 7 项技术决策落地：闪电贷源/目标链/DEX覆盖/路径搜索/Gas策略/清算目标/MEV接入
- [x] 动态白名单上线验证 + 小额实盘测试

核心洞察

1. Week 1 最大收获：数据管道跑通（DexScreener+Factory双源、Binance Web3 API聚合报价、实时信号管道），成本模型可用（Gas+滑点+桥费+时间成本全量建模），DEX-DEX同链套利机会最密集且无闪电贷费
2. 最大风险点：公共RPC稳定性不足（需自建/付费节点）、Gas飙升无对冲机制、动态白名单覆盖不足（长尾池子漏报）、跨链最终性时间风险难量化
3. Week 2 必须突破：Balancer Vault闪电贷合约跑通、Aave V3清算监控上线、Flashbots/MEV-Share Bundle打包成功
4. 资金策略调整：从观察仓转实盘仓，单笔上限0.5 ETH，总敞口<5 ETH，优先跑DEX-DEX同链与稳定币跨链，闪电贷原子套利仅Anvil测试

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

| 闪电贷源 | Balancer Vault (0费) | 多资产借入、无手续费、IVault/IFlashLoanRecipient |

| 目标链 | Arbitrum + Base | Gas低、深度足、MEV友好、RPC稳定 |

| DEX覆盖 | UniV3 + Curve + PancakeV3 | 覆盖90%+流动性、费率层100/500/3000 |

| 路径搜索 | 链下有向图 + 剪枝 | 5跳<50ms、动态amount_in优化、白名单Token |

| Gas策略 | BaseFee预测+p99PriorityFee+RBF | Gas成本<理论1.2x、maxFeePerGas动态、nonce管理 |

| 清算目标 | Aave V3 + Venus | 健康因子<1.05扫描、清算奖励>Gas成本3x |

| MEV接入 | Flashbots Protect + MEV-Share | Bundle成功率>60%、回款分成优化 |

遗留疑问

1. 闪电贷合约的 maxPriorityFeePerGas 动态调整如何避免 revert？
2. 清算监控的健康因子阈值设多少平衡「发现早」vs「假阳性」？
3. MEV-Share 的回款机制如何计算给搜索者的最优分成？

明日预习 (Day 7)

- [] Balancer Vault flashLoan / receiveFlashLoan 完整流程

- [] 编写 FlashLoanArbitrage.sol 原型：借入 → 多跳 Swap → 归还
- [] Anvil fork 主网测试：forge test --fork-url \$ARB_RPC
- [] 研究 IVault / IAsset / IFlashLoanRecipient 接口

共学第 6/21 天 | 链上套利残酷共学

关键洞察

- 待补充...

技术决策落地

| 模块 | 方案 | 关键参数 |

|-----|-----|-----|

| 待补充 || |

遗留疑问

1.

2.

明日预习

-

共学第 6/21 天 | 链上套利残酷共学

笔记 078: Day 6 笔记：认识半自动对冲套利工具 —— taoli.tools 与 QQQ.BOT

作者：GeoGu0 | 时间：2026-08-10 15:01 UTC | 字数：5507 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 笔记：认识半自动对冲套利工具 —— taoli.tools 与 QQQ.BOT

日期：2026 年 8 月 10 日

进度：理解了裸套利和对冲套利的本质区别 + 搞懂了资金费率套利的底层逻辑 + 调研了 taoli.tools 和 QQQ.BOT 的产品设计 + 实践了"看到价差提醒后追问扣完成本剩多少"

一、前五天学的是"看到价差"，今天学的是"抓住价差"

Day 1-3 建立成本模型，Day 4-5 建立了从 The Graph 和 RPC 直接拉数据的能力。这些东西解决的都是一个核心问题：如何判断一个价差是不是真实的套利机会。

但判断完了之后呢？假设你在 Arbitrum 上发现 Uniswap 的 ETH 比 Sushiswap 便宜 \$5。你确认了：价差真实存在，扣完两笔交易的 Gas (~\$0.3) 和协议费 (~0.3%) 之后，净收益约为 \$4。从理论上，这是一个值得做的机会。

现在问题变成了：你怎么把这两笔交易（在 Uniswap 上买、在 Sushiswap 上卖）安全地做掉，确保价差不在这几秒钟里被抢跑或者变动？

过去五天学的是"发现和验证"，今天开始接触"执行"的第一步：半自动工具如何帮你抓住价差。

二、裸套利 vs 对冲套利 —— 为什么"同时做"比"分步做"安全

裸套利 (Naked Arbitrage)：先买，再卖。两次交易之间有时间间隔。

比如你在 Uniswap 上先买 ETH，交易确认花了 4 秒。在这 4 秒里，整个市场突然跌了（可能是因为一条大新闻，或者只是正常波动），ETH 的价格从 \$3000 跌到了 \$2995。等你再去 Sushiswap 上卖的时候，你的"套利"已经变成了"亏本出售"——手上 ETH 的成本是 \$3000，现在只能卖 \$2995，加上你买的时候付过的 Gas，里外亏损。

裸套利的根本缺陷在于：买和卖之间有时间差。在这个时间差里，你的仓位是裸露在市场中的——方向性风险敞口虽然时间极短，但确实存在。对普通人手动做来说，这个间隔可能是 30 秒到 2 分钟。对机器人来说可能是 2-6 秒。对 MEV 搜索者来说可能是同一个区块内的几百毫秒。但只要有时间间隔，就有风险。

对冲套利 (Hedged Arbitrage)：双边同时下单，买入和卖出锁在同一时刻完成，消除方向性风险。

实现方式有两种。

第一种，同一链上的原子执行 (Flash Loan 或 MultiCall)。用闪电贷在一个交易里借到资金，在同一个交易里完成"便宜处买 + 贵处卖 + 还贷 + 付 Flash Loan 费"。要么全做完，要么全回滚。方向性风险被完全消除了，因为买和卖之间严格没有时间间隔——它们在同一个以太坊交易里顺序执行。Day 11 学三角套利和闪电贷的时候会深入这一点。

第二种，双边的半自动对冲。这适用于跨交易所场景——比如一个机会出现在链上 DEX 和链下 CEX (中心化交易所如 Binance) 之间。你没法用同一个区块链交易同时覆盖两边。这时候你需要一个工具：它同时监控两边的价格，当价差出现时，它帮你用最短的时间窗口在双边各下一个单子。你仍然暴露在"时间差风险"中，但工具把时间差从"手动操作的 30 秒"压缩到了"工具自动执行的 0.5

秒”。

taoli.tools 和 QQQ.BOT 都属于第二种——半自动对冲工具。它们不做到原子执行（那是智能合约层面的事），但它们把“看到 → 判断 → 下单”这条链路压缩到了人类做不到的速度。

三、taoli.tools：半自动对冲套利的产品解析

taoli.tools 是由 @aliez_ren 开发的半自动对冲套利工具。它的设计目标很明确：帮助用户在发现价差的时候，快速完成双边（或多边）下单，同时持续监控价格变化、仓位状态、资金费率和风险指标。

从产品功能的角度，可以拆成几个核心模块：

1. 双边下单引擎。这是最核心的功能。用户看到价差（工具自己也在持续监控），一键触发双边同时下单：在便宜的那边买入，在贵的那边卖出。工具处理的是下单的时序和速度——它用预先配置好的 API Key（交易所 Key 或钱包私钥），同时调用两个交易所的接口，尽量缩短两笔交易之间的间隔。

2. 价格和价差监控面板。和 DEX Screener 一个道理，但是面向套利优化的——它同时显示多对（如 Binance 的 ETH/USDT vs Uniswap 的 ETH/USDC）的实时价差。价差大到触发线的时候，高亮提醒。

3. 仓位和风险评估。对冲不是免死金牌。即使两边的交易都做成了，你还面临一个问题：你已经建立了一多一空两个仓位，怎么平仓？什么时候平？如果一边的流动性突然枯竭——比如 Uniswap 上的池子被抽空了，你卖不掉——你实际上暴露在了单边风险中。taoli.tools 提供仓位可视化和风险提醒，帮你判断当前状态是否正常。

4. 资金费率监控。永续合约（Perpetual Futures）有一个特殊的机制叫资金费率（Funding Rate）：多头（做多的人）和空头（做空的人）每隔一段时间（通常 8 小时）互相支付一笔费用。如果市场上做多的人比做空的人多（大家都看涨），资金费率是正的——多头要付空头钱。如果做空的人比做多的人多，资金费率是负的——空头付多头钱。

这就创造了一种独特的套利场景：现货买 + 合约空 = Delta Neutral（市场中性，即不管价格涨跌你都不赚不亏）+ 赚资金费率。

四、资金费率套利：不赌方向，只赚费率

做资金费率套利的操作非常简单，但理解它为什么能赚钱需要先理解一个基础概念。

永续合约是一种特殊的期货合约，没有到期日，你持有它永远不需要交割。为了让它和现货保持价格一致（不脱锚），交易所设计了一个“资金费率”机制——每隔 8 小时，多空双方根据市场情绪互相支付一笔费用。

假设当前市场的资金费率是 +0.03%（每 8 小时）。这意味着做多的人每 8 小时要向做空的人支付持仓金额的 0.03%。年化一下： $0.03\% \times 3 \text{ 次/天} \times 365 \text{ 天} \approx 32.85\%$ 年化。这个数字很惊人——在市场情绪极度狂热的时候（牛市顶部），资金费率可以飙到年化 50% 甚至更高。

套利者怎么赚这个钱？三步：

第一步，在现货市场买入 \$10000 的 ETH（持有现货）。

第二步，在永续合约市场以同样数量做空 ETH（价值 \$10000 的空单）。

第三步，持有这个组合。不管 ETH 涨还是跌，现货和合约的盈亏互相抵消（涨了现货赚、合约亏，跌了现货亏、合约赚）。你是市场中性的——价格波动对你的净值影响为零。

那么你的收益来源是什么？每隔 8 小时，多方要付给你（你是空方）资金费率。按 0.03% 算，每 \$10000 仓位每 8 小时收 \$3，一天约 \$9。扣掉两次交易的手续费（现货买卖约 0.1% + 合约开平约 0.1%，总计约 \$20），一个资金费率周期（假设费率高 + 持仓 10 天）的净收益可能相当可观。

这里三个关键风险，每一个都能让你从“赚确定性费率”变成“亏大钱”。

第一，资金费率不是永远正的。如果市场情绪一夜之间逆转（从狂热变成恐慌），资金费率可能从 +0.03% 骤降到 0% 甚至变成负的。这时候你的空单要付钱给多方——固定收益变成了固定支出。taoli.tools 的资金费率监控模块就是用来跟踪这个变化的。

第二，交易所本身的风险。你在 Binance 上做空 ETH 的同时持有现货 ETH。如果 Binance 出问题（虽然概率不大），你的两个仓位都在同一个平台上——理论上平台风险没有被对冲。更稳健的做法是在不同平台之间做——但那样又会引入跨平台资金转移的时间差和成本。

第三，爆仓风险。如果你做空 \$10000 的 ETH，交易所要求你存入一笔保证金（通常 5-10%，即 \$500-\$1000）。如果 ETH 暴涨 50%，你的空单亏损 \$5000，保证金不够了，交易所会强行平掉你的空单（爆仓）。这时候你手里还有现货 ETH 是赚了 50% 的——但你是用更高价格（追加保证金）维持的，或者你的杠杆太高导致还没等到现货卖出去空单就被强平了。所以做资金费率套利必须用低杠杆甚至 1x 杠杆（完全抵押），留足够的保证金缓冲。

这就是 taoli.tools 这类工具存在的价值：它帮你同时监控费率变化、保证金水平、双边价格偏离，在费率跌到不划算的时候提醒你平仓离场。而不是你手动每天去交易所看三遍费率——人脑做不了这种持续监控。

五、QQQ.BOT：Telegram 里的套利哨兵

QQQ.BOT 是一个 Telegram Bot，核心功能是把套利信号推送到你的手机。和 taoli.tools 的区别在于：taoli.tools 偏向“执行”（帮你下单），QQQ.BOT 偏向“发现”（帮你盯盘然后通知你）。

但从共学的角度来说，QQQ.BOT 最有价值的不是信号本身（任何第三方信号都应该自己验证），而是三个可以学习的点：

第一，学习它的信号格式。一条典型的 QQQ.BOT 提醒大概长这样："BTC/USDT, Binance vs OKX, Spread: 0.35%, 24h Avg: 0.12%"。它告诉你了价差有多高、和近期平均比是高是低、发生在哪两个平台之间。这种简洁的信号格式，是你 Day 16 自己写监控脚本时候推送格式的好参考。

第二，学会追问"扣完成本还剩多少"。QQQ.BOT

告诉你的价差是裸价差——没扣交易手续费、出金成本、时间延迟。收到任何提醒，先别激动。立刻用 Day 2 学的成本模型跑一遍：这两家交易所的交易手续费是多少？（通常 $0.1\% \times 2 = 0.2\%$ ）资金从 Binance 转到 OKX 或者反过来的出金费用多少？价差从你看到这条 Bot 提醒到你手动做，中间可能过了多少秒？把这些全扣掉——剩下的才算数。

现实中，大部分 Bot 提醒的裸价差扣完成本之后都是负的——因为它们被推送给你的时候，专业玩家已经吃掉了。Bot 的价值不在于给你送上门的利润，而在于帮你快速知道"某个市场上发生了异动"。你自己的判断和验证，才是利润的来源。

第三，把它作为你未来自己 Agent 的原型参考。发起人在大纲里说要训练一个"链上套利 Hermes Agent"，能自动发现价差、补齐成本、给出有证据的判断。QQQ.BOT 是简化版的"信号发现 → 推送通知"模型。Day 16-18 你做 Agent 的时候，可以从 QQQ.BOT 的消息格式中获得灵感：你的 Agent 输出也应该包含价差百分比、净收益估算、数据源、时间戳、时效性警告。而不是一句模糊的"有套利机会"。

六、Day 6 的三个核心收获

第一，裸套利和对冲套利的本质区别在于"买和卖之间有没有时间差"。裸套利暴露在方向性风险中（哪怕是几秒钟），对冲套利通过原子执行或双边同时下单消除方向性风险。套利的"无风险"二字，前提是你消除这个时间差。

第二，资金费率套利是一种独特的套利形式——它不依赖资产价格差异，而是彻底市场中性（现货+合约空），赚的是多方付给空方的资金费率。年化收益在一些时段可以很可观，但风险在于费率会变方向、平台风险、和保证金不足导致的爆仓。

第三，taoli.tools 和 QQQ.BOT 分别是"执行"和"发现"两个环节的工具。taoli.tools 解决"怎么做"（双边下单、仓位监控、费率追踪），QQQ.BOT 解决"看什么"（异动提醒、价差初筛）。但两个工具的共同前提是一——你需要用自己的成本模型去验证，推送的"机会"是不是可执行的。

七、今天的三个问题

1. 资金费率套利需要同时在现货和合约市场持仓。如果用了 Flash Loan，能不能在同一个交易里完成"借 USDC → 买现货 ETH → 开合约空单 → 等待资金费率结算 → 平仓"？还是说资金费率结算必须在几个小时内跨区块完成，Flash Loan 的单区块时间窗口不够？
2. taoli.tools 的双边下单是多快完成的？它用的是交易所 API 的市价单（market order）还是限价单（limit order）？市价单执行快但滑点大，限价单滑点小但有成交不了的"腿风险"（一边成了另一边没成）——它怎么平衡？
3. QQQ.BOT 的数据源是什么？它是自己跑的节点监控链上 mempool，还是从交易所 WebSocket 拿的实时订单簿？了解数据源才能判断它的延迟有多低。

八、第一周即将收官

Day 6 是第一周的最后一天学习新内容。明天 Day 7 不学新东西，做第一周大复盘：画流程图、填完整成本模型、整理工具清单、写下认知变化。

六天的爬坡路径回头看很清晰：Day 1 看见价差 → Day 2-3 学会扣成本 → Day 4-5 学会自己拉数据和模拟交易 → Day 6 看到这些能力怎么集成到实际工具里成为可执行的策略。

第一周结束的时候，我手里应该有：一套完整的七项成本模型、一件数据工具箱（DEX Screener + LI.FI + The Graph + RPC）、一份半自动工具的认知地图、以及接下来两周深挖的方向。

笔记 079: TUT事件下，资金费率套利：高资金费率不是单纯的收益机会，也可能是拥挤交易和逼空风险的前兆。

作者：cawood | 时间：2026-08-10 15:02 UTC | 字数：506 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

TUT事件下，资金费率套利：高资金费率不是单纯的收益机会，也可能是拥挤交易和逼空风险的前兆。

正常阶段，可以利用Gate高资金费率、Binance低资金费率建立对冲仓，赚取费率差。为了提高资金效率，可以使用逐仓5–10倍杠杆，但必须事先定义风险预算，不能在异动过程中持续补保证金。

一旦发现Gate开始脱离其他交易所单边上涨、价差快速扩大，应主动平掉原套利仓。极端情况下让逐仓机制限制单笔损失，也比不断扛仓更可控。

真正更好的机会往往出现在逼空之后。此时策略从“资金费率套利”切换为“跨所价差收敛”。

收敛判断可以先只监控两个核心因子：

1. 价差：Gate相对Binance的价格溢价是否进入历史极端区；
2. 爆仓量：空头爆仓是否已经从持续放大转为见顶回落。

当价差仍然很大，但爆仓量开始明显衰减，说明逼空燃料可能正在耗尽，这时再建立“空Gate、多Binance”的收敛仓位。

核心逻辑：

正常期吃资金费 → 异动主动止损 → 逼空阶段观望 → 价差极端且爆仓见顶后做收敛。

不要从低位一路扛空到高位，而要先活下来，再交易极端错价。

笔记 080: 第四天 | 净收益、模拟与执行决策

作者：OxFeyoma | 时间：2026-08-10 15:08 UTC | 字数：2978 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第四天 | 净收益、模拟与执行决策

关联笔记 01-第一天-市场机制、流动性与区块 (01-第一天-市场机制、流动性与区块) 第二天 | 去中心化交易所套利结构与稳定币 (02-第二天-DEX套利结构与稳定币) 第三天 | 跨链套利、清算与最大可提取价值 (03-第三天-跨链套利、清算与MEV)

核心判断

执行决策的对象是完整交易结果及其失败分布。屏幕价差只能提供线索，无法单独证明利润。

1. 统一净利润公式

所有机会最终都要换算为同一种基准资产。

$$\begin{aligned} \Pi_{\text{净}} = & B_{\text{最终}} - B_{\text{初始}} - C_{\text{网络}} - C_{\text{排序}} - C_{\text{融资}} - C_{\text{桥接}} - C_{\text{对冲}} - C_{\text{授权与包装}} \end{aligned}$$

$B_{\text{初始}}$ 和 $B_{\text{最终}}$ 必须是同一种资产，并使用一致的估值时间。只要最终还留有其他资产，就要按可实现卖出价格估值，不能直接按页面中间价计算。

报价系统通常已经把池手续费和价格冲击计入输出数量。如果再次把它们列为外部成本，净利润会被重复扣减。

最深刻的一点

最可靠的利润证明是模拟结束后基准资产余额增加。价差百分比、理论汇率和单腿报价都只能作为中间证据。

2. 期望利润比单次利润更重要

一次模拟成功无法说明策略长期盈利。执行还要考虑失败概率和失败损失。

$$\mathbb{E}[\Pi] = p_s \Pi_{\text{成功}} - (1 - p_s) L_{\text{失败}} - C_{\text{资金占用}}$$

p_s 表示成功概率。 $L_{\text{失败}}$ 需要包括已消耗的网络费、未完成交易留下的持仓、对冲损失和结算延迟。

一个机会即使成功时赚 100，只要成功率为 60%，失败时平均损失 90，它的期望收益也只有

$$0.6 \times 100 - 0.4 \times 90 = 24$$

如果资金占用、基础设施和监控成本超过 24，这个机会仍然没有经济价值。

3. 从屏幕价差到执行决策

候选机会需要依次通过六道检查。

资产身份

确认链、合约地址、小数位、包装形式和赎回权利。代码相同或名称相同无法证明资产相同。

同步状态

记录区块高度、报价时间和预言机状态。不同时间的两个价格可能从未同时存在。

完整数量报价

固定真实输入规模，把上一腿输出传给下一腿。每一腿都要使用执行数量下的平均成交结果。

全部成本

保留已经包含在报价中的池手续费和价格冲击，再加入网络费、融资费、排序费用、桥接、对冲、授权和失败成本。

执行与结算

确认所有交易腿能否原子执行。无法原子执行时，要说明失败后留下什么资产、如何对冲，以及多久可以恢复库存。

模拟与压力测试

先在固定区块模拟正常情况，再主动加入竞争、延迟、价格恶化和成本上升。只有在不利条件下仍有足够余量，才进入下一步判断。

4. 盈亏平衡点与安全余量

毛价差减去确定成本后，只得到静态利润。执行前还应计算利润归零的位置。

假设当前可执行净收益为 80，交易规模为 20,000。忽略非线性变化时，能够承受的额外价格恶化约为

$$\frac{80}{20,000}=0.4\%=40\text{ bp}$$

bp (basis point, 基点) 表示万分之一。

真实市场存在非线性价格冲击，因此还要重新报价确认。这个计算只用于快速估计安全余量。

执行门槛可以写成

预期净收益 > 成本估计误差 + 状态变化缓冲 + 最低目标利润

如果利润只比成本估计高一点，网络费或报价轻微变化就可能让结果转负。此类机会适合继续观察，不适合立即执行。

5. 三类失败需要分开记录

经济失败

交易成功上链，最终基准资产却减少。常见原因包括价格冲击估计错误、网络费上涨、排序成本过高和重复扣费。

技术失败

交易回滚或没有被打包。常见原因包括授权不足、调用参数错误、余额不足、超出滑点限制和交易包条件不满足。

结算失败

部分交易已经完成，剩余部分延迟或无法完成。跨链、场外赎回和非原子多市场交易更容易出现这种问题。

三类失败的损失形态不同。经济失败损失本金，技术失败通常损失网络费和机会，结算失败可能留下持续变化的方向持仓。

6. 模拟应逐级接近真实执行

建议按以下顺序训练。

1. 使用只读报价验证交易方向和数量传递。
2. 使用历史区块重放已经发生的市场状态。
3. 使用本地分叉模拟完整原子交易。
4. 在你的交易前插入竞争者或改变池状态。
5. 提高网络费、融资费和建设者支付。
6. 连续记录纸面机会，比较预测结果和区块中的真实结果。

模拟需要回答三个问题。

1. 交易能不能执行。
2. 执行后赚多少钱。
3. 哪个最小变化会让利润消失。

只回答第一个问题的模拟仍然不够。

7. 压力测试清单

对每个候选机会至少测试以下变化。

输出报价恶化 10、30 和 50 个基点。

网络费上升到当前值的 2 倍和 5 倍。

竞争者在你的交易之前执行。

最浅交易池的有效深度下降一半。

交易延迟一个或多个区块。

清算抵押品的卖出冲击突然扩大。

跨链到账时间延长，目标链价格回归。

私有交易包没有被打包，机会在下一块消失。

记录每种情形的最终余额。不要只写成功或失败。

8. 执行门槛

一个机会进入执行候选区，至少要同时满足以下条件。

资产身份已经完全确认。

完整路径能够在同一状态下报价或模拟。

基准情形的净利润为正。

期望净收益覆盖资金和基础设施成本。

压力测试后的最大损失可以承受。

盈亏平衡点与当前报价之间有足够距离。

非原子部分已经准备库存或对冲方案。

失败处理方式和停止条件已经写清楚。
无法通过其中一项时，可以保留观察记录。观察记录的价值在于发现自己的模型缺少什么信息。

9. 机会记录模板

- | 字段 | 需要记录的内容 |
- | :---- | :----- |
- | 数据状态 | 区块高度、时间和预言机状态 |
- | 起始资产 | 链、合约地址和输入数量 |
- | 完整路径 | 每一腿市场、输入和输出 |
- | 已含成本 | 池手续费和价格冲击 |
- | 外部成本 | 网络、融资、排序、桥接和对冲 |
- | 基准利润 | 正常状态下的净收益 |
- | 期望利润 | 加入成功率和失败损失后的结果 |
- | 盈亏平衡点 | 哪个变化使利润归零 |
- | 压力测试 | 不利条件下的最终余额 |
- | 决策 | 执行候选、继续观察或拒绝 |

10. 最值得记住的五句话

- 1. 利润必须由最终基准资产余额证明。
- 2. 已经进入报价的手续费和价格冲击不能重复扣除。
- 3. 一次成功模拟只能证明这一状态下可以执行。
- 4. 盈亏平衡点告诉你机会距离亏损有多远。
- 5. 高质量策略会尽早在低成本模拟中失败，把真实资金留给经得住反证的机会。

笔记 081: 今天继续学习 DEX-CEX 套利，重点了解了资金准备的问题。

作者：zgos | 时间：2026-08-10 15:14 UTC | 字数：410 | 评分：47
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续学习 DEX-CEX 套利，重点了解了资金准备的问题。

我之前以为可以先在 DEX 买入，再把币转到 Binance 卖出。现在发现，链上转账和充值确认需要时间，等资产到账后，原来的价差大概率已经消失了。

一种更快的思路是提前在两边准备资产：发现机会时，在价格较低的一边买入，同时在价格较高的一边卖出。这样可以减少转账等待，但交易完成后，两边的资产比例会发生变化，还需要重新平衡资金。

今天的收获是：套利系统不只是比较价格，还要管理 DEX 钱包和 CEX 账户里的资金。即使发现了机会，如果对应的一边没有足够的币或稳定币，也无法执行。

目前的卡点是还不清楚资金再平衡需要多少成本，以及频繁转账会不会吃掉大部分利润。另外，同时完成两边交易听起来简单，实际可能出现一边成功、另一边失败的情况。

接下来准备画一张简单的资金流转图，弄清楚一次套利前后，DEX 和 Binance 两边的币分别发生了什么变化。

笔记 082: 【2026-08-10 学习打卡】

作者：UU20220213 | 时间：2026-08-10 15:16 UTC | 字数：355 | 评分：47
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【2026-08-10 学习打卡】

今天继续学习 CEX 套利与资金费率，重点理解：

- 1. CEX 现货价差套利需要在高价位真实卖出标的；若未预先持有现货，不能直接做现货卖出。
- 2. 低价现货买入 + 高价永续合约做空，是对冲和基差/资金费率研究，不是价差自动锁定；收益取决于基差收敛、资金费率和全部成本。
- 3. 单腿成交会暴露价格方向风险，要核对两边实际成交量并让仓位回到接近中性。
- 4. 资金费率在结算时于多空之间划转，持仓名义价值 × 费率决定应收/应付；临近结算开仓不保证按预期费率结算。
- 5. 学会区分最新成交价、指数价和标记价格；合约风控应关注标记价格、保证金与强平价。

今日结论：先做纸面记录和只读观察，不接 API 自动下单；所有收益判断必须扣除手续费、滑点、资金费率、基差变化和风险成本。

笔记 083: 对冲腿先死了，套利仓就成了单边赌局

作者：tutu | 时间：2026-08-10 15:18 UTC | 字数：1022 | 评分：68
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

对冲腿先死了，套利仓就成了单边赌局

今天学习 Day5，主题是交易所规则与杠杆。最大的收获是，中性仓位只代表某一刻的多空金额相等，并不代表对冲腿能在极端行情里活下来。

纸面情境里，账户有 100,000 USDT，现货多头和永续空头各 30,000 USDT。空头被 ADL 减到 15,000 后，现货一枚没少，账户瞬间变成净多 15,000 USDT。要是价格先拉升打掉空头，再快速回落，前面空头亏一次，后面现货还会再亏一次。

以后看套利仓至少要算三本账。

单腿保证金杠杆，决定某条合约腿离清算有多近

毛敞口，决定需要成交和承担流动性风险的总名义仓位

净 Delta，决定账户当前实际做多、做空还是中性

比较 Binance、OKX 和 Hyperliquid，也不能只看最大杠杆。更该检查标记价格、维持保证金、逐仓和全仓、清算顺序、订单簿深度、保险基金、backstop 和 ADL。逐仓能隔离账户风险，但单腿保证金少时也会更快接近清算。全仓可以让仓位多撑一会儿，代价是风险会传染到其他仓位和抵押品。

针对上面的 ADL 情境，我的选择是卖出 15,000 USDT 现货，让现货和剩余空头重新匹配。这样净 Delta 回到接近零，毛敞口也从 60,000 压缩到 30,000。重新补仓会继续依赖刚发生异常的薄盘口，什么都不做则等于拿着 15,000 USDT 裸多继续赌。

我把 Kill Switch 写成三个动作。

1. 停止新增风险，撤掉加仓、反手、网格和机器人订单，只保留必要的 reduce-only 减仓操作
2. 核对 ADL 后的真实持仓，优先在更深的盘口削掉多余一侧，并预设最大滑点和最长执行时间
3. 衍生品仓位安全以后，再把不需要的资金隔离到资金账户或个人钱包。逐仓、子账户和抵押品设置应在开仓前完成

今天还纠正了几个误区。ADL 不是用户可以关闭的保护按钮。追加保证金只能给正在减仓的仓位买时间，不能代替减仓。正在充当抵押品的现货也不能在清算前贸然提走，否则可能降低保证金率。

我的临时追加上限是事件发生前账户净资产的 5%。100,000 USDT 账户最多追加 5,000 USDT，而且追加后毛敞口必须继续下降。仓位没有减少，就不再追保。

以后遇到价差，我会先写下清算规则、实际深度、对冲失效后的处理顺序和资金上限。仓位减下来以后，账上还有钱，下一笔机会才与我有关。

笔记 084: 今天的学习重点，是把前几天对跨链路径和交易成本的零散观察，整理成一套可以重复使用的研究流程。

作者：Qy | 时间：2026-08-10 15:25 UTC | 字数：829 | 评分：58
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天的学习重点，是把前几天对跨链路径和交易成本的零散观察，整理成一套可以重复使用的研究流程。

我先回顾了 AMM、流动性深度、滑点、Gas、桥接费用和执行延迟等概念。现在更加明确，看到两个市场存在价格差，只代表出现了一个候选机会，不能直接把它当作套利收益。真正有价值的信号，必须能够覆盖全部成本，并在市场发生一定程度的不利变化后仍然保留安全边际。

今天继续以 USDC 跨链路径作为模拟研究对象，计划从多个时间点观察预计到账数量、最小到账数量、具体桥接路径、DEX、Gas、手续费和执行时间。同时比较不同交易规模下，最优路径和成本率是否发生变化。

今天使用的成本模型是：

净收益 = 理论价差收益 - 路径成本 - 风险准备成本 - 资金占用成本。

路径成本包括 Gas、DEX 手续费、跨链桥费用和价格冲击；风险准备成本包括执行延迟、滑点扩大、报价失效和交易失败的预期损失。如果 LI.FI 的预计到账金额已经扣除了部分费用，计算时不能再次扣除，避免重复计算成本。

我为候选机会设置了一个初步判断标准：净收益必须为正，能够覆盖全部已知成本；在价格不利变化 0.3% 后仍然存在安全边际；相似机会能够在多个时间点重复出现；交易规模扩大后，滑点和路径成本没有明显失控。只有同时满足这些条件，才值得进入下一阶段研究。

今天最大的收获是，套利研究的核心不是捕捉一次偶然价差，而是建立完整的证据链。每次观察都应记录时间、链、资产、规模、路径、费用、预计收益和失败原因。只有持续积累这些数据，才能判断一个机会究竟是可重复信号，还是短暂的市场噪声。

下一步，我会继续追踪同一条跨链路径，增加不同时间点和交易规模的样本。如果能够观察到相对稳定的候选信号，再考虑把人工记录转换为自动查询、脚本或 Agent 告警流程。

今天只进行了资料学习、模拟分析和研究框架整理，没有连接钱包、没有签署交易、没有支付 Gas，也没有发生任何资产转移，真实资金投入为 0。

笔记 085: 实现一次加密套利，系统性的准备和极致效率：

作者：colinzhong1024 | 时间：2026-08-10 15:26 UTC | 字数：279 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

实现一次加密套利，系统性的准备和极致效率：

实时监控行情：利用行情聚合器或专业扫描工具快速发现价差机会。

预先部署资金：把资金分别存放在多个平台或链上钱包中，一旦出现机会即可立即下单，不被转账时间拖累。

快速执行操作：市场修复价格差只需几秒，所以必须通过脚本或 API 接口实现自动下单。

核算成本与滑点：除了手续费，还要考虑提现费、链上 gas 成本以及因订单较大带来的滑点影响，以判断是否真正有利可图。

风险控制与记录追踪：每笔操作都应详细记录，以便回顾复盘。市场波动、网络延迟或平台问题都有可能让看似稳妥的套利变成亏损。

笔记 086: 今天主要学习了跨链路径与成本验证

作者：derenjie | 时间：2026-08-10 15:27 UTC | 字数：169 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天主要学习了跨链路径与成本验证

了解了如下概念：

什么是跨链桥，和 DEX 有什么区别

什么是跨链路由（Route）

跨链套利中的 Gas、桥费、DEX 手续费、滑点和等待时间

为什么不同链之间会出现价格差

如何判断跨链价差扣除成本后是否仍有净收益

跨链失败、桥暂停、代币映射错误等风险

笔记 087: 学习内容参考<https://x.com/brucexu_eth/status/2086652810>

作者：Galaxy9 | 时间：2026-08-10 15:31 UTC | 字数：5819 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习内容参考<https://x.com/brucexu_eth/status/2086652810905751610>

一、套利的第一性原理

套利不只是“低买高卖”，而是：

找到同一经济价值在不同市场、链、资产形态或时间状态之间的偏离，构造完整闭环，并确认扣除全部成本和风险后仍有正收益。

完整结构：

起点状态 S

→ Swap / Bridge / Mint / Redeem

→ Borrow / Repay / Liquidate / Settle

→ 终点状态 S_1

只有当 S_1 与 S 具有可比较的经济价值时，才能计算利润。

例如，以下资产都接近 1 美元，但并不完全等价：

Ethereum 原生 USDC

其他链上的桥接 USDC

交易所账户里的 USD

无法即时赎回的稳定币

等待跨链中的 USDC

它们的赎回权、流动性、到账时间、对手方和桥风险不同。价格差可能是套利，也可能只是风险补偿。

二、利润计算方法

1. 净利润

净利润

= 有限规模执行后的终点价值

- 起点价值
- 报价外的协议费与 Gas
- 融资成本
- 跨链或赵回成本
- 竞争成本
- 失败成本

2. 期望利润

只计算成功场景还不够：

期望利润

= 成功概率 × 成功时利润

- 失败概率 × 失败时损失
- 资金占用与机会成本

例如，一笔套利：

成功概率：80%

成功赚取：500 USDC

失败概率：20%

失败损失：1,200 USDC

资金机会成本：30 USDC

期望利润：

$80\% \times 500 - 20\% \times 1,200 - 30$

= 130 USDC

如果失败损失增加到 2,000 USDC：

$80\% \times 500 - 20\% \times 2,000 - 30$

= -30 USDC

即使成功时赚钱，长期期望也可能为负。

3. 避免重复计算成本

真实有限规模报价通常已经包含：

池内交易费

当前池状态下的价格影响

滑点则是报价之后，因为状态变化造成的额外偏差。不能把手续费、价格影响和滑点机械地重复扣除。

三、通用评估流程

看到任何价差，都按下面七步判断。

第一步：统一起点与终点

例如从 10,000 USDC 出发，最终也要换回可使用的 USDC。

如果最终得到的是无法赎回、流动性很差或不能跨链的代币，闭环尚未完成。

第二步：画出完整路径

不要只写“DEX A 买、DEX B 卖”，而应写成：

USDC

→ 在 DEX A 买入 WETH

→ 在 DEX B 卖出 WETH

→ 收回 USDC

跨链场景则可能是：

Ethereum USDC

→ 买入 ETH

→ 跨到 Arbitrum

→ 卖出 ETH

→ Arbitrum USDC

→ 重新平衡回 Ethereum

第三步：取得真实规模报价

页面显示的价格通常只是边际价格。

应该用实际交易金额查询：

实际输入

实际输出

平均成交价

最低到手数量

价格冲击

订单簿深度

第四步：补齐报价外成本

包括：

Gas 和失败交易 Gas

CEX 手续费与提现费

Bridge、赎回和结算费

借款利息和资金费率

Builder 或优先费竞价

库存与资金占用成本

第五步：判断是否原子执行

同链智能合约可以实现：

全部成功 → 交易生效

任一步失败 → 整笔回滚

跨链、CEX-DEX、赎回和到期结算通常无法原子完成。第一边成交后，第二边可能变价或暂停。

第六步：进行压力测试

至少测试：

交易规模扩大和缩小

Gas 上涨

最低输出下降

第二边延迟

跨链桥暂停

交易所关闭提现

资金费率反转

失败后残留资产

第七步：作出分级决策

结构不成立 → 放弃

结构成立但数据不足 → 观察

报价和风险可以解释 → 模拟

模拟稳定 → 小额验证

多轮结果一致 → 才考虑扩大

四、14 类套利地图

| 类型 | 利润主要来源 | 闭环方式 | 主要风险 |

| :----- | :----- | :----- | :----- |

| 跨 CEX | 两个订单簿价格不同 | 两边同时买卖，再平衡 | 深度、提现、交易所风险 |

| 跨 DEX | 不同协议池状态不同 | 同链买低卖高 | Gas、价格冲击、竞争 |

| CEX-DEX | 链上与交易所更新不同步 | 两边预放库存、同时成交 | 不能原子回滚 |

| 同 DEX 不同池 | 费率档、版本或池状态不同 | 协议内换入再换出 | 利润薄、聚合器竞争 |

| 三角与多跳 | 多个相对价格没有闭合 | 绕一圈回到原资产 | 每一步改变后续报价 |

| 跨链 | 资金不能及时跨链流动 | 两链交易后再平衡 | 桥、延迟、库存 |

铸造与赎回	市价偏离转换价值	Mint、Redeem、Wrap	通道暂停、资格与额度
现货与到期合约	现货与未来结算价偏离	现货与合约对冲至到期	保证金、融资、交割
永续资金费率	多空需求不平衡	现货与永续反向持仓	费率反转、清算
利率与收益率	不同市场回报不同	低成本融资、高收益配置	APY 变化、协议风险
清算	协议提供清算奖励	偿债、拿抵押物、卖出	预言机、Gas、机器人竞争
拍卖	拍卖成交价低于外部价格	竞拍后外部变现	成交不确定、卖出深度
状态变化套利	大额交易改变链上状态	在目标交易后恢复价格	排序、Bundle、竞价
事件结算	市价偏离确定的转换或结算值	转换、兑付或互补头寸	规则、资格、争议

归类原则是：

看利润主要来自哪一种价格关系没有闭合，而不是看交易经过多少协议。

五、代表性案例

案例一：跨 DEX 原子套利

假设用 100,000 USDC：

DEX A：买入得到 33.10 ETH

DEX B：卖出得到 100,260 USDC

这里的报价已经包含池内费用和当前价格影响。

报价毛利润：

$$100,260 - 100,000 = 260 \text{ USDC}$$

额外成本：

Gas：70 USDC

Builder 竞价：120 USDC

失败交易预期成本：20 USDC

净利润：

$$260 - 70 - 120 - 20 = 50 \text{ USDC}$$

最好通过智能合约原子执行，并设置：

最终 USDC > 100,210 才允许成交

否则整笔回滚。

案例二：三角套利

报价路径：

10,000 USDC

→ 3.3200 ETH

→ 0.1995 WBTC

→ 10,085 USDC

如果这些是按顺序计算的真实有限规模报价，则毛利润为：

$$10,085 - 10,000 = 85 \text{ USDC}$$

再扣除：

Gas：35 USDC

执行竞价：30 USDC

失败预期成本：10 USDC

期望净利润只有：

$$85 - 35 - 30 - 10 = 10 \text{ USDC}$$

不能把三个页面上的静态价格直接相乘，因为第一笔交易会改变池状态。

案例三：跨链套利

价格：

Arbitrum：ETH = 2,985 USDC

Ethereum：ETH = 3,010 USDC

交易 20 ETH，表面价差：

$$(3,010 - 2,985) \times 20 = 500 \text{ USDC}$$

成熟做法是在两链预放库存：

Arbitrum：用 USDC 买入 20 ETH

Ethereum：同时卖出已有的 20 ETH

稍后再跨链调整库存

成本：

两端交易费和价格影响：160 USDC

Gas：50 USDC

跨链再平衡：170 USDC

库存与延迟成本：80 USDC

净利润：

$500 - 160 - 50 - 170 - 80 = 40$ USDC

如果发现机会后才开始跨链，到账时价差通常已经消失。

案例四：资产形态套利

假设某市场中：

100 WETH 价格 = 99.85 ETH

协议支持 WETH 立即按 1:1 解包为 ETH

路径：

99.85 ETH

→ 买入 100 WETH

→ Unwrap

→ 得到 100 ETH

毛利润：

$100 - 99.85 = 0.15$ ETH

必须确认：

WETH 合约正确

Unwrap 入口正常

购买报价是实际成交价

手续费和 Gas 低于 0.15 ETH

中间不存在非标准代币机制

如果资产不能按规则赎回，就不是严格套利。

案例五：现货与到期合约基差

假设：

BTC 现货：100,000 USDC

三个月期货：102,000 USDC

交易者：

买入 1 BTC 现货

同时做空 1 BTC 到期合约

三个月毛基差：

$102,000 - 100,000 = 2,000$ USDC

扣除：

三个月资金成本：1,000 USDC

交易费：200 USDC

托管和保证金成本：150 USDC

净收益：

$2,000 - 1,000 - 200 - 150 = 650$ USDC

这不是无条件无风险，还存在交易所、保证金、交割和现货托管风险。

案例六：清算套利

清算人偿还：

100,000 USDC 债务

协议提供 5% 清算奖励，得到价值：

105,000 USDC 的 ETH

但实际卖出只能获得：

103,800 USDC

再扣除：

Gas 与优先费：2,000 USDC

临时资金成本：300 USDC

净利润：

$103,800 - 100,000 - 2,000 - 300$

$= 1,500 \text{ USDC}$

5% 是毛奖励，实际净利润只有 1.5%。如果其他机器人抢先清算，本次交易还可能失败并消耗 Gas。

案例七：状态变化与 Backrun

某用户的大额 Swap 把 DEX A 的 ETH 推到：

DEX A：3,050 USDC

DEX B：3,005 USDC

Backrunner 在用户交易之后：

DEX B 买入 ETH

→ DEX A 卖出 ETH

→ 两池价格重新接近

这属于状态变化套利，因为利润主要由前一笔大额交易产生。

它与三明治的区别是：

Backrun 只在用户交易之后纠正价格

Sandwich 在用户交易前后夹击，使用户成交更差

六、严格套利与方向交易的区别

以下情况通常不是严格套利：

看到新闻后认为代币会上涨

买入后等待价格上涨

因为终点价值没有锁定。

事件结算套利要求存在明确闭环，例如：

旧代币可以按 1:1.1 固定比例换成新代币

旧代币市场价格仍明显低于可转换价值

转换入口已经开放

自己符合资格

新代币具有真实卖出流动性

如果转换比例、资格或结算结果不确定，本质上仍是事件押注。

七、学习路线

第一阶段：只做测算

学习：

AMM 与订单簿

有限规模报价

Gas、滑点和价格冲击

原子执行

净利润与期望利润

练习：记录 20 个价差，但不交易，判断其中多少能形成资产闭环。

第二阶段：建立监控

优先选择：

跨 CEX

跨 DEX

同 DEX 不同池

三角套利

资金费率和收益率比较

系统输出至少包含：

起始资产

完整路径

真实规模输出

全部成本

最坏结果

期望利润

机会持续时间

第三阶段：本地模拟

在链的分叉环境中模拟：

多腿 Swap

交易回滚

Gas 上涨

报价状态变化

最低输出保护

清算或闪电贷路径

第四阶段：小额验证

只使用可承受完全损失的资金，并设置：

单笔资金上限

每日亏损上限

Token 和协议白名单

最低净利润

最大 Gas

最大滑点

连续失败熔断

最终判断标准始终是：

最终拿回的同一种经济价值

起点价值

+ 报价外全部成本

+ 失败成本

+ 资金占用成本

如果资产不可比，看到的只是价格差；如果路径能闭合但全成本后为负，它只是市场状态；只有在真实规模、真实顺序和真实失败条件下仍为正，才是可执行套利。

笔记 088: 探讨在 cex-dex 套利中我一直存疑的问题，Day 5：【持仓与对冲风险管理】

作者：ninanren05 | 时间：2026-08-10 15:34 UTC | 字数：1344 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

探讨在 cex-dex 套利中我一直存疑的问题，Day 5：【持仓与对冲风险管理】

今天重点学习了 CEX-DEX 套利里的持仓风险管理。

以前我更关注的是价差、成交和套利利润，但今天意识到，双边或者多边持仓并不等于没有风险。即使 CEX 和 DEX 两边都有库存，如果整体上仍然净持有大量代币，本质上还是在承担方向风险。因此真正需要管理的是 Net Exposure / Net Delta，而不仅仅是各个平台还剩多少币。

CEX-DEX 套利过程中，交易会不断把代币和稳定币从一边搬到另一边。如果长期出现单向价差，就会造成严重的库存倾斜，所以需要设置目标库存、正常区间、警戒区间和停止区间，并根据库存状态动态调整套利阈值和下单量，尽量利用后续交易自然恢复库存，而不是频繁跨链或充提再平衡。

如果希望降低代币价格波动带来的风险，可以使用永续合约做空、借币卖出等方式进行 Delta 对冲。但对冲本身也有成本和风险，包括资金费率、借币利率、可借额度、基差、保证金占用以及爆仓风险。因此真实套利利润不能只看价差，而应该扣除交易手续费、Gas、滑点、Funding、Borrow Cost 和 Hedge Cost。

今天印象最深的一点是：整个套利组合可能是 Delta-neutral 的，但某个单独的 CEX 保证金账户仍然可能先爆仓。

例如链上持有大量现货，同时在 CEX 做空等额永续，当代币突然上涨 10 倍甚至几十倍时，链上现货虽然产生了等额浮盈，但这些资产无法立即给 CEX 的空单补充保证金。如果空单杠杆过高，仍然可能被强平。因此套利中的对冲仓位更像保险仓位，而不是追求高 ROE 的投机仓位，需要低杠杆和足够大的保证金缓冲。

对于小币尤其应该做极端压力测试，而不能只测试 $\pm 10\%$ 的普通波动。需要假设代币可能上涨 2 倍、5 倍、10 倍甚至 30 倍，或者下跌 90%，检查保证金是否足够、对冲是否仍然有效、DEX 流动性是否还能退出、CEX 是否可能暂停充提等问题。

以后如果继续完善自己的套利系统，我认为需要逐步增加：

1. Inventory Manager：管理 CEX/DEX 库存比例；
2. Net Delta：计算实际方向敞口；
3. Max Exposure：限制单币最大风险；
4. Hedge Engine：自动管理永续或借币对冲；
5. Funding / Borrow Cost：实时计算真实持仓成本；
6. Margin Buffer：预留极端行情下的生存资金；
7. Stress Test：模拟 2x、5x、10x、30x 等极端行情；
8. Kill Switch：出现对冲失效、保证金危险、流动性消失等情况时停止扩大仓位。

今天最大的认知变化是：

CEX-DEX 套利做到后面，本质上并不只是一个寻找价差的交易机器人，而是一个实时管理库存、Delta、保证金、借贷、流动性和尾部风险的资产负债表系统。

套利策略真正重要的目标，也不是保证每一笔交易赚钱，而是保证在代币突然暴涨几十倍、暴跌 90%、资金费率异常、借币枯竭或者交易所暂停充提的情况下，整个系统仍然不会因为一次极端事件而死亡。

笔记 089: 之前想到的价差雷达、回测工具、路径模拟和风险面板，看起来像不同项目，实际更像同一套系统里的功能。真正

作者：bai·小白 | 时间：2026-08-10 15:43 UTC | 字数：432 | 评分：49

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

之前想到的价差雷达、回测工具、路径模拟和风险面板，看起来像不同项目，实际更像同一套系统里的功能。真正的顺序应该是先确定研究定位，再拆出研究方向和独立项目，最后才决定需要哪些功能。

这也让我重新理解了套利机器人。它不只是发现价差后自动下单，更重要的是回答几个问题。价差为什么出现，能否真实成交，全部成本是多少，竞争者会不会更快，执行失败后又会留下什么风险。

所以，比起急着写机器人，我更需要先建立一套可复用的研究方法。观察市场异常，提出假设，重放历史状态，计算真实成本，进行模拟验证，再复盘策略为什么成功或失败。

这套能力未来也可以迁移到美股、港股和其他市场。能够迁移的不是某一段代码或某一个策略，而是发现、验证和排除市场低效率的方法。不同市场的交易规则、结算方式和执行系统仍然需要重新研究。

接下来不急着自己能造出一台万能套利机器人。先选择一个足够小的市场切口，把数据、判断、模拟、验证和复盘完整走通。大方向先立住，再用一块块可靠的证据把地基铺实。

笔记 090: 这是一次明显的原子套利：

作者：LIQIUHUEI | 时间：2026-08-10 15:47 UTC | 字数：510 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

这是一次明显的原子套利：

1. 在 Orca 用 1,086.534769 USDC 买到 922.322706 EURC，成交价约 1.178 USDC/EURC。
2. 随后在 Meteora 用这 922.322706 EURC 换出 18,361.633282 USDC。
3. USDC 净增加 17,275.098513。

异常价差来自同一区块更早的这笔交易 (<<https://solscan.io/tx/4V85c1CgAPzw3KVjNpaSCRX5jGwa3Fr77Baq5pJ4o6Cb9hPzWLSTodHdgQwchYfyFRc2x41hgDMPtKrxqWtG9fe>>。它把 > 17,920.614105 USDC 打进一个流动性很薄、EURC 储备接近耗尽的 Meteora 池，却只拿到 \

246.657955 EURC，把池内价格严重推歪。本笔交易随后反向套利，把价差拿走。Soyas 地址也被公开解析库标注为 MEV fee address (<<https://go.onsig.ai/onsig/pump-parser/src/tag/v0.2.51/consts.go>>。>

笔记 091: Delta 中性资金费率套利实现规划

作者：Zhihui Zhang | 时间：2026-08-10 15:51 UTC | 字数：737 | 评分：67

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Delta 中性资金费率套利实现规划

定位：HIP-3 永续合约上的 beta 加权配对做市，对冲市场风险，赚取 MM 价差 + 净资金费率 carry。

核心逻辑：

在 hyperliquid_perpetual 上运行两个 pmm_mister controller（每腿一个）

通过 beta 加权的多空配对实现净市场 delta ≈ 0

偏向资金费率有利的一侧，让两条腿都付费

中性是通过做市本身诱导出来的，绝不用市价单对冲

每 Tick 流程：

1. 运行 hip3_dn_pair_monitor routine → 获取 RECOMMENDATION (RUN A/B、RESIZE、HEDGE、REDUCE-ROTATE、HOLD-FLATTEN)
2. 读取实际 delta（不自己计算）
3. 按建议执行：部署/更新 controller、调整参数、或平仓
4. 资金费率滞后机制：除非另一侧 carry 超过当前方向 $\geq 15\%/yr$ ，否则不翻转

关键参数：

默认配对：CL/BRENTOIL（相关性 0.98， β 1.02， $\sim +33\%/yr$ 净中性 carry）

min_corr: 0.9（低于此值 → HOLD-FLATTEN）

net_delta_band_pct: 0.04（delta 偏离带宽）

leverage_cap: 2

风控：

相关性断裂 → 平仓停止

净 carry ≤ 0 → 减仓轮换

每腿 $\leq \text{max_position_size_quote}$ ；总杠杆 $\leq \text{leverage_cap}$

笔记 092: 链上套利残酷共学打卡 - 2026-08-10 (Day 6)

作者：KuTear | 时间：2026-08-10 15:53 UTC | 字数：887 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学打卡 - 2026-08-10 (Day 6)

今日主题：跨市场 Delta 中性对冲实盘巡检与自动化收敛平仓机制

一、跨市场实盘对冲持仓与风控巡检

1. 跨市场基差对冲组合运行机制：

- 对冲架构：采用链上 DEX 衍生品协议与中心化交易所组成的跨市场无风险对冲体系。
- 持仓与收益状态：
- 严格 Delta 归零：双端多空持仓数量与名义价值完全对齐，彻底消除单边价格剧烈波动的敞口风险。
- 基差收益锁定：建仓阶段已锁定正向基差优势，合并持仓呈现稳健正浮盈状态，当前净收益稳步累积。
- 保证金与强平防护：逐仓与全仓双端维持保证金比例处于高度安全区间，预留充足的强平价格缓冲空间，杜绝单边极端行情引发的连带爆仓风险。

二、自动化策略执行与风控闭环

1. 交易引擎自动化平仓监听：

- 当前策略卡片配置防御性开仓拦截，同时保持平仓监听逻辑全天候运行。
- 接入阶梯收敛平仓规则，在双端基差回归收敛至预设安全阈值区间内时，由系统自动分批执行撮合平仓，实现全链路套利闭环。

2. 多节点流动性与资金池管理：

- 全账户资金池结构健康，流动性在各大接入节点（CEX 头部交易所与链上衍生品协议）之间保持均衡分布，为多标的并发套利策略提供底层流动性保障。

三、理论与系统演进思考

1. CEX-DEX 混合结算资产摩擦模型：

- 跨市场套利常涉及不同计价抵押品（如 USDC 标的与 USDT 标的）。在计算净基差与执行平仓时，需将稳定币汇率差与脱锚溢价动态纳入摩擦成本模型，防止名义基差收益被底层汇率波动侵蚀。

2. 非对称流动性订单流路由：

- DEX 订单簿深度与 CEX 往往存在流动性非对称性。在执行平仓与调仓时，策略引擎应优先在流动性较薄弱端挂单或作为 Maker 探测，流动性充裕端作为 Taker 快速对冲，以降低双端成交时间差引发的滑点暴露。

笔记 093: 套利共学 | 新手必读链上套利的第一性原理：从价差到可执行净利润\

作者：Krystal6633 | 时间：2026-08-10 15:59 UTC | 字数：789 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利共学 | 新手必读链上套利的第一性原理：从价差到可执行净利润\

(挑战纯新手读懂布老师这篇推文的每个字)\

1“套利的第一性原理，是寻找同一经济价值在不同状态之间尚未闭合的一致性”

“同一经济价值”：必须是完全一样的东西（比如都是ETH，或者都是USDC）。苹果和梨子不算，因为价格波动逻辑不同。

“不同状态”：指不同的地方（Uniswap vs SushiSwap）、不同的链（以太坊 vs Arbitrum）、或者不同的时间（现在 vs 期货交割时）。

“尚未闭合的一致性”：正常来说，同一个东西在不同地方应该价格一样（这叫“一致性”）。如果不一样，就叫“价差”，也就是“尚未闭合”。这句话翻译成成人话就是：“找到那个‘同一个东西，在两个地方卖出了不同价格’的时刻。”\

\

2并构造一条可执行的资产闭环”

“可执行”：这条路必须走得通。你不能说“我先在A平台买，再去B平台卖”，结果B平台根本没人接盘，或者跨链桥坏了。

“资产闭环”：这是最关键的词。你必须从哪个币出发，最终回到同一个币。比如：拿着USDC出发，转了一圈（换ETH，卖成USDC），最后手里还是USDC。

翻译成成人话：“设计一条能绕回原点的路线，确保不会卡在半路变成‘接盘侠’。”

3“只有终点价值在扣除全部成本、时间与风险后仍高于起点，价差才变成利润。”

这句话拆成三个扣分项：

“全部成本”：过路费（Gas）、平台手续费（Swap Fee）、跨桥费。

“时间”：你跑完这条路花了多久。如果花了3分钟，而价格3秒就变了，那这3分钟里你承受的“变盘风险”就是成本。

“风险”：比如你跨链时，对面链拥堵导致失败；或者你借钱套利（杠杆）要付利息。

翻译成成人话：“只有把路上所有花的钱、等待时担惊受怕的代价都扣掉，最后发现手里的钱比出发时多，那才叫真赚了。”

笔记 094: Solana MEV 套利案例分析：一笔 swap 获利 1.73万U

作者：Zhou Junjie | 时间：2026-08-10 16:03 UTC | 字数：1219 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Solana MEV 套利案例分析：一笔 swap 获利 1.73万U

交易概况

- 签名: 2TfMSQ96qFRELCvgYR6sHbYhv6KENys2Bru2X7wJFAHTwCRuR9ghc1VaWG9Ys8Ek74qC59pymWyzd9xfJzSYHFvz

- 地址: 888BMMsC1CPmVnnK4HLnKjXb7ZKUKpdCXiwqMvCQqVmo

- 打包方式: Jito Bundle，手续费 0.689 SOL

- 净收益: +17,275 USDC (约 1.73万U)

套利路径

通过 Jupiter 聚合器在一笔交易内完成两跳：

第一跳 — Orca Whirlpool (EURC/USDC)

- 用 1,086.53 USDC 买入 922.32 EURC

- 隐含 EURC 价格: \$1.178 (接近正常价 ~\$1.15)

第二跳 — Eo7WjKq DEX 多资产池

- 存入 922.32 EURC → 从 3ESUFC 账户取出 18,361.63 USDC

- EURC 隐含价格: \$19.91 (正常价的 17 倍！)

USDC 分配：

- 1,086.53 → 归还 Orca 成本

- 17,275.10 → 进入 888BMM 口袋 (纯利润)

核心洞察

1. 链上价格发现: 同一资产在不同池子的价差是套利信号。EURC 是 Circle 发行的欧元稳定币, 正常价格应在 \$1.15 附近, 但 Eo7WjKq 池将其定价到 \$19.91。
2. MEV 的本质: 3ESUFC 在不知情的情况下损失了 18,361 USDC——它的资金被事先授权给 DEX 合约, 攻击者无需其签名即可提取。
3. Jito 的作用: 35.8 SOL 转给了 soyascX 作为 MEV 分润, 3.7 SOL 是 Jito 小费。这是 Solana MEV 生态的标准操作——支付高额小费确保交易优先打包。
4. 零成本套利工具: 888BMM 没有部署合约、没有使用闪电贷, 纯粹通过 Jupiter 聚合器发现价差并路由执行。

方法论收获

分析链上交易时, 先查 pre/post token balances 锁定资金流向
通过 inner instructions 还原完整路由
多池比价是发现套利机会的基础, 跨池价格偏差 >3x 就是明确的交易信号
永远先确认链和代币身份再下结论 (本次 EURC 险些被当成未知山寨币)

风险警示

3ESUFC 可能是设置了错误价格的限价单, 或是向流动性极低的池子存入了资金。这提醒我们: 在非主流 DEX 上提供流动性时, 务必检查池子的定价机制是否合理。

笔记 095: 2026-08-08 學習打卡

作者: cht+ | 时间: 2026-08-11 00:13 UTC | 字数: 1064 | 评分: 64
来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-08 學習打卡

今天完成了什麼

完成 SNDK 美股、SNDKB 代幣化股票與 SNDKUSDT 永續合約的事件型錯價研究, 利用歷史分鐘資料重建財報後價差, 並與前一季事件比較。結果顯示錯價可能短暫出現, 但較接近低頻事件驅動機會, 不宜視為固定收益策略。
整理較安全的策略優先順序: 以預置股票與代幣化股票雙邊庫存、利用可驗證的轉換機制進行庫存式套利為優先; 股票與永續合約的對沖組合僅作備選, 並納入爆倉、資金費率與基差擴張風險。

完成獨立 Shadow Mode
股票套利監控器的開發與可靠性修正。系統包含商品交集、Bid/Ask、深度容量、費用、資金費率、持續性確認、去重、Reason Codes、通知及時序資料儲存介面。
完成監控可靠性加固: 資料中斷與過期報價會重置持續性視窗、通知去重狀態可持久保存、反向套利會檢查股票可放空性、掃描錯誤可追蹤, 並提升財務計算精確度。
完成通知路由測試; 測試訊息已清楚標示為 Shadow Mode, 系統不會自動下單。

證據在哪裡

歷史資料研究顯示, 研究事件中的分鐘溢價曾短暫高於 2%, 而前一季對照事件未出現相同幅度, 支持「事件型、非固定收益」的判斷。
監控器的自動化測試、程式碼品質檢查、編譯檢查及版本差異檢查均已通過。
通知路由測試成功, 且安全提示與 Shadow Mode 狀態顯示正確。

尚未完成/下一步

尚未完成股票即時行情串接驗證, 仍需確認資料來源、盤前盤後報價權限與實際延遲品質。
尚未取得股票端完整歷史 Bid/Ask 與可見深度, 因此歷史毛價差不能視為已證實的實盤可成交報酬。
代幣化股票的地區資格、逐商品轉換狀態、限制、公司行動與帳戶實際可用性仍需核實。
時序資料庫結構與常駐服務尚未正式部署; 正式部署仍需另行評估與批准。
下一步將完成即時行情測試, 並以真實 Bid/Ask、深度、費用與轉換狀態進行長時間 Shadow Mode 觀測及門檻校準。

今日學習心得

今天最重要的收穫, 是把「畫面上看起來很大的價差」拆成可驗證、可成交、可風控的條件。歷史資料證明錯價確實曾發生, 但前一季反例也提醒我: 單次漂亮事件不能直接外推成穩定策略。真正有價值的不是只找到一個醒目的價差數字, 而是建立一套會檢查報價新鮮度、深度、商品權利、持續時間與失敗原因的確定性系統。先以 Shadow Mode 保存候選與淘汰理由, 再用真實資料校準, 才能逐步把偶然訊號變成可重現的交易能力。

笔记 096: 监控bsc和币安价格差异的功能这样设计:

监控bsc和币安价格差异的功能这样设计：

1. 程序启动后订阅 Binance futures depth WebSocket，和 OKX DEX Market Price WebSocket。其中binance 的有 ask price 和bid price，但是OKX DEX 获取的价格可以单做ask 也可以单做 bid price。
2. 在websocket价格出现更新的时候计算在“BSC链上买入，币安合约做空”以及“BSC链上卖出，币安合约做多”的价差，如果价差大于 1.04（这是ws 数据的open ratio价差阈值，可以设置）。就使用REST API 确认币安的 ask/bid price，也使用 quote 确认链上的买入/卖出价格，如果这时候大于1.03 就进行记录动作，记录到csv文件中。反方向的 websocket close ratio 如果价差打印1.08（这是ws 数据的close ratio价差阈值，可以设置）也使用binance REST API 和 OKX DEX API quote 确认价格，满足 0.998 表示平仓进行记录操作。
3. 考虑到某些币种在链上和交易所可能长时间出现价差，为了避免一直请求获取这种币在两个交易所之间的价差。在某个symbol使用REST API和 OKX DEX API quote确认过价格后，3秒内丢弃这个symbol的websocket价差数据，也就是3秒内不在为相同的symbol去确认价格。还需要做一个事情：需要使用相同的symbol在出现机会后的初始记录间隔修改为：5 秒，然后是20秒，然后是2分钟。

笔记 097: Day 7 — CEX-DEX 套利深度解析

作者：pennchester | 时间：2026-08-11 02:24 UTC | 字数：1389 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 — CEX-DEX 套利深度解析

今日核心收获

1. CEX-DEX套利的本质
 - CEX-DEX 是信息套利而非流动性套利：两个独立的价格发现系统存在时间差（CEX ms 级 vs DEX 12s 区块级）
 - 这解释了为什么 Day 6 回测中 CEX-DEX 占 73% 交易量 —— 价差窗口更大、更频繁
2. CEX 订单簿机制深度理解
 - 前 5 档买卖盘深度决定滑点：\$100K 交易在深度好的 CEX 仅 1.24bp 滑点，远优于 DEX
 - 四种订单类型对比：Market(保证成交)/Limit(限价挂单)/IOC(部分成交不挂单)/FOK(全成交或取消)
 - 套利最佳选择是 IOC —— 不挂单暴露意图，不承担未成交风险
3. 双边持仓管理（Delta 中性）
 - 提币延迟(5-30min)是 CEX-DEX 套利的最大工程挑战
 - 解决方案：双边预持仓，CEX 侧有 ETH 可卖、DEX 侧有 USDC 可买
 - $NetExposure = CEX_ETH + DEX_ETH + Hedge = 0$ ，锁定价差而非方向
4. 永续合约对冲策略
 - 模拟计算：含永续对冲的 CEX-DEX 套利需要 0.8%+ 价差才盈利
 - 费用分解：DEX swap(0.3%) + CEX taker(0.05%) + 提币(~0.1%) + 永续开平仓(0.1%) + 资金费率(0.06%/48h) = 0.61%
 - 无对冲双边执行可在 0.5%+ 盈利，但承担方向性风险
5. 延迟优化
 - 延迟链分解：CEX WS(50ms) + CEX API(100ms) + DEX RPC(200ms) + 区块确认(12s) + 提币(5-30min)
 - 每 500ms 延迟损失利润 10-25%，超过 2 秒中价差机会会变负收益
 - 优化方向：WebSocket 替代 REST、同区域部署、自建节点、Flashbots 私有 bundle
6. 实操架构
 - 四组件架构：CEX 接口(WS+订单簿) + DEX 接口(RPC+交易) + 双边持仓管理 + 风控结算
 - CEX 选择优先级：Binance(深度最深) > OKX(合约好) > Bybit(永续流动性好)

核心认知

1. CEX-DEX 本质是信息套利 —— 两个独立价格发现系统的时间差
2. 订单簿深度决定 CEX 侧几乎无滑点，这是 CEX-DEX 的核心优势
3. 双边预持仓是工程化的核心，没有它提币延迟吃掉所有利润
4. 永续对冲需要 0.8%+ 价差，低于此价差追求无对冲双边执行
5. IOC 是最安全的套利订单类型
6. 延迟是真正的利润杀手 —— CEX API 和 DEX RPC 速度直接决定策略可行性

产出

- 学习笔记：Day7_CEX-DEX套利深度解析.md（订单簿机制/双边持仓/对冲策略/延迟优化/实操架构）

-

代码：cex_dex_simulator.py (800+行，六大演示模块：订单簿滑点/订单类型/Delta中性/完整流水线/延迟敏感性/资金费率对冲)

明日计划

Day 8：BTC/ETH 跨品种套利与相关性交易

笔记 098: TUT (Tutorial) 近期“插针”事件总结与验证

作者：Oxmigueleth | 时间：2026-08-11 02:29 UTC | 字数：1862 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

TUT (Tutorial) 近期“插针”事件总结与验证

事件核心时间线 (2026年8月7-10日)

Tutorial (TUT) 是BNB Chain上的教育/meme币。8月7日前后Aster等DEX上线永续合约 (最高5x杠杆) 后，价格两天内暴涨超1100%，8月9日触及ATH约\$0.29-0.305。随后剧烈反转，24小时合约成交量一度达\$33-34亿 (远超市值)，全网爆仓超\$4200万，TUT成为当时爆仓量最大的资产 (超过BTC/ETH)，空头爆仓占主导。

关键插针发生在8月9日北京时间下午约15:10左右 (约20分钟内)：

各所最高价差距极大：Bitget一度插到约\$1.00，Gate约\$0.70，币安约\$0.305。随后快速回落至\$0.12附近。

1小时内TUT爆仓约\$3500万 (空单占绝大多数，最大单笔在Bitget约\$300万+)。

链上：做市商/控盘地址将约1.6亿枚TUT (总供应约20%) 从币安转入Bitget。

之后价格继续回调，目前 (8月10-11日) 在\$0.12-0.17区间震荡，交易所 (尤其Gate) 已启动补偿。

截图帖子 (@xiaoheshang2025) 事实核对：基本属实

帖子核心逻辑：

1. 拉盘期间资金费率差异巨大：币安几乎无/极低，Bitget和Gate极高 (帖子称每4小时约2%)。 \

验证：属实方向正确。当时高波动下，Bitget等所TUT资金费率出现极端正值 (如0.6%+每4小时，年化超高)，币安明显更低。这吸引了大量“低费率做多 + 高费率做空”的跨所套利 (理论日收益可达数百个百分点)。

2. 这是预设的资金费诱盘 + 插针收割：套利者在币安做多、Bitget/Gate做空，吃资金费差。最后一分钟暴力向上插针，把高费率所的空单 (套利腿) 全部爆掉，只剩币安多单“单腿”，随后砸盘再收割多单。 \

验证：高度吻合实际数据。价格分化正是插针在Bitget/Gate远超币安的表现；空头爆仓主导；大额筹码流向Bitget (OI也高)；Gate和Bitget事后紧急核查并补偿 (Gate对特定时间窗口内爆仓用户全额USDT补偿，Bitget也锁定数据并承诺方案)。这正是典型的“资金费率陷阱 + 跨所流动性猎杀”。

3. 狗庄恶意收割已被交易所知悉。 \

验证：真实。Gate已正式公告补偿并升级风控；Bitget官方回复帖主称“紧急锁定数据、最高级别核查”。舆论和市场数据均指向异常。

帖子细节 (精确2%费率) 可能有夸张/峰值记忆偏差，但整体机制、结果和动机描述与公开数据高度一致，不是空穴来风。

更深入分析

这不是普通波动，而是低流动性meme永续 + 做市商控筹 + 资金费率诱导的经典组合拳：

筹码控制是关键：20%供应短时间从币安流向Bitget，直接改变了各所深度和OI分布。做市商可在目标所制造极端溢价/插针。

套利陷阱的本质：跨所资金费率套利在高波动、低深度资产上几乎必然失败。价格“基差”风险远大于费率收益。一旦一侧被插爆，就变成单边暴露，随后砸盘完成双杀。

交易所角色：Gate/Bitget的极端插针暴露了标记价格/风控在低流动性合约上的脆弱性。补偿行动说明它们承认异常 (而非正常市场行为)。币安相对克制，偏离更小。

市场结构问题：meme币永续在CEX上线后，容易被“资金费+OI堆积”放大成猎杀场。24小时合约量是市值的20倍以上，本身就不可持续。

后续影响：用户损失主要集中在套利者和高杠杆空头；交易所补偿可覆盖部分窗口内爆仓；但手动止损或非窗口损失不赔。长期看，这类事件会促使交易所加强低市值合约的分层风控和异常价格保护。

核心Takeaway：

在meme/小市值永续上做跨所资金费率套利，风险极高，几乎是“送人头”给控筹方。

价格在不同交易所可以严重脱节，插针往往是定向流动性猎杀，而非全市场共识。

观察链上大额CEX间调仓 + 极端资金费率差异，是提前预警信号。

交易所事后补偿值得肯定，但预防机制 (标记价格熔断、深度要求) 更重要。

整体属于可控的“狗庄收割”事件，而非系统性崩盘。TUT本身仍有教育叙事，但短期杠杆已大幅去化，波动会继续。交易时务必优先看各所深度和资金费率实时数据，避免单边或跨所裸奔。

笔记 099: Day 6 (8/10) | 机会的统计学：频率、幅度、半衰期、竞争

作者：yixin | 时间：2026-08-11 06:02 UTC | 字数：922 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 (8/10) | 机会的统计学：频率、幅度、半衰期、竞争

一句话目标：从「这里有个机会」升级到「这类机会一年出现多少次、每次值多少、能容纳多少钱、几秒内消失」。这是策略选择的唯一理性依据。

核心概念

1. 机会的四个统计量（每个策略候选都必须估这四个数）：

- 频率 λ ：每天/每周出现几次超过 breakeven 的机会

- 幅度分布：不是平均值，是分布。套利收益极度右偏——90% 的利润来自 5% 的机会（想想 3.12 或 5.19 这种日子）。用中位数评估策略是错的，要看尾部。

- 持续时间 / 半衰期 τ ：机会出现后多久衰减一半。这直接决定你需要的技术水平： $\tau < 1$ 个区块 $\rightarrow$ 你需要是 searcher； $\tau \sim$ 分钟 $\rightarrow$ 脚本够； $\tau \sim$ 小时/天 $\rightarrow$ 手动都行，但竞争者也多。

- 容量：单次能吃多少而不把价差自己吃光。容量 $\approx$ 使价格冲击等于价差的规模

2. 竞争强度的观测方法（链上的巨大优势——你能直接看到对手）：

- 看某个池的历史交易，识别哪些是套利交易（特征：紧跟大额 swap 之后、来自合约、原子多跳）

- 看这些套利者付了多少 priority fee $\rightarrow$ 他们的出价 = 利润的下界估计

- 看有几个不同地址在做 $\rightarrow$ 竞争者数量

- 如果某类机会已经有 5 个专业 searcher 在做，你的边缘必须是「他们做不了的规模/资产/链」，而不是「我做得更快」。

3. 策略衰减：所有套利策略都在衰减。要问的不是「能不能赚」，是「半衰期多久」。链上策略的典型半衰期是 3-12 个月。这决定了你该投入多少工程量——半衰期 3 个月的策略不值得写 2 个月代码。

4. 策略选择矩阵（把候选放进去）：

|| 高频率 | 低频率 |

|---|---|---|

| 大幅度 | 不存在（或存在于你不知道的地方） | 事件驱动、脱锚、清算 —— 需要常备资本和快速响应 |

| 小幅度 | 稳定币/LST 微价差 —— 拼成本和自动化 | 别做 |

笔记 100:

今天学习帖子：《老孙带你搭建价值百万的金融终端系统4:套利监控模块设计思路》 \

作者：jssyxd | 时间：2026-08-11 07:04 UTC | 字数：1626 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天学习帖子：《老孙带你搭建价值百万的金融终端系统4:套利监控模块设计思路》 \

\

<https://x.com/sunlc_crypto/status/2085578921320616081>

前几天去旅游刚回来，今天继续更新套利监控模块的设计思路。套利监控模块主要分几个功能：正费率监控：币圈的套利现在特别难做，主流币种几乎没有套利机会，山寨币被各种野庄操控风险极大。现在市场韭菜少了，野庄都盯着套利者在收割，如果做全自动套利策略100%会踩坑中招。哪怕我这种老鸟去年10月也在bybit上被seraph搞了一下爆仓100W U（最终赔了78W），今年又在bitget上被搞了一下（赔付了一半）。所以我这个监控模块不仅监控价差、实时/历史资金费率、交易量、OI变化、FDV等所有信息，还能和链上妖币模块联动监控庄家的动态。确保在能吃到价差/资金费的情况下尽可能规避风险。去年推特上冒出来很多所谓“套利博主”的时候我就警告过套利没那么容易，不是开发一个套利工具就能做的，里面有踩不完的坑和风险，小白不要轻易尝试。这帮野庄就是目前币圈最大的毒瘤，只需要几百万U就可以操控一个山寨币，完全没有技术含量只需要没有底线就可以了。所以这帮人现在都躲在HK、SG搞事情，已经形成流水线和产业链了，有些交易所里还有他们内应，各家交易所对他们也很头痛。

我这写出来估计要被他们骂了。负费率监控：野庄同样会通过操控负资金费，甚至联合某些交易所的内部人员（调整资金费计算规则和收取间隔）来实现来回收割。由于每个交易所的资金费计算公式、时间间隔都不一样，存在一定的套利机会。我通过还原每个交易所的资金费率并设计了一套资金费率的预测模型，来寻找其中的套利机会。不过负费率的套利风险极大，人为影响因素很多，不能上很大仓位，只有比较确定性的机会我才会出手。美股正费率监控：现在交易所的交易量都在美股合约中，尤其在在美股行情好的时候资金费是很客观的。所以我也做了一套美股正费率的监控，基本逻辑和币圈正费率是一样的，还不用考虑野庄的影响风险小很多。合约目前主要在币安做，现货是币安和bitget。

RWA标的监控：美股、黄金、原油等标的的合约在各交易所之间也存在价差和资金费差，同样可以监控起来做套利分析。币圈升水监控：监控BTC/ETH在币安、OKX、Deribit三家当季、次季合约的升水。这个现在没啥套利机会，只是用来监控币圈的资金成本（升水基本等于U的借贷利息），同时也可以作为行情热度的评判标准之一。OI异动提醒：这个功能对于监控野庄操盘特别重要。野庄在搞事前会慢慢建仓，从价格上可能不明显，但是从OI的变化上看特别明显，尤其是价格没有明显变化的情况下OI显著放大是最明显的庄家入场特征。之

前在bitget上踩了一个巨坑就是当时没有记录bitget的OI变动历史，开仓前看OI有4000W的规模，结果在1分钟内被野庄搞了ADL了，OI从4000W降低到200W，后来才知道其中95%的仓位都是野庄对开的。所以不管是对套利还是炒山寨，OI是最需要关注的指标，其他都可以造假OI变化无法造假。 自定义监控： 可以随便自定义任何交易对的价差和资金费监控，用以监控还没有进入监控列表的交易对或者长期关注的交易对（比如LAB这种强庄币）。 目前交易所涵盖的有binance\okx\bitget\bybit\gate\kucoin\hyperliquid\aster这几家，交易所接口统一用CCXT接入避免重复造轮子。 图1和图2是套利监控主界面截图：可以看到进入正/负费率、美股费率等监控列表的标的。对于满足套利机会的标的会在套利机会预警列表里重点展现。 图3是正费率标的点击后打开的详情图：可以看到实时/历史价差、实时/历史资金费、预测资金费、流动性（OI、交易量、FDV等）详细信息，用来做交易决策。 图4是RWA标的套利机会监控。

笔记 101: https://github.com/ZeroNullNone/onchain_arbitrage_

作者：Nothing | 时间：2026-08-11 07:05 UTC | 字数：1573 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/daily/day_07.md

Day 7 — Week 1 Data Gate

状态与结论

状态：Day 7 完成；Week 1 data gate PASS。

Valid sample count：231 / 200；没有降低阈值，也没有用 fixture/smoke 补数。

Week 2 已冻结为 Base USDC/WETH paper-research slice；仍禁止 live execution。

详细 evidence、正式 universe、排除项与 unknowns 见 docs/week_1_report.md。

完成项

- 新增 read-only repeatable QA，检查 schema、Raw lineage、UTC/ordering、latency、parse、decimals、duplicate、missingness、success projection、route availability 与 size sensitivity。
- Decimals 以 frozen chain_id + address metadata 检查，并验证 quote/cost raw units；经济值不使用 binary float。
- QA 输出 exact config SHA-256，缺表/缺 Raw/错误 token metadata 均显式失败。
- 修复长 Retry-After 阻塞整个 collection round：只 cooldown 受影响 route，完整保存 429。
- 新增 config/week2.toml；未取得足够 evidence 的 freshness/profit threshold 保持显式 TBD。

Evidence

- 251 attempts：231 success、20 rate-limited；availability 92.03%。
- Raw / timestamp / latency coverage：100% / 100% / 100%。
- Parse：100%；missing required fields 0；duplicates 0；timestamp order violations 0；normalized projection mismatch 0；decimals pass。
- Latency p50 / p95：1,270.747 / 3,898.220 ms。
- 原 Day 5 run 内最大 gap 6,781.682025s 暴露长 Retry-After blocking bug；修复后完成追加采集。
- Frozen config hash：46bd62308a88624edbcdad228ce2dcb29ce427cb2cb488d5e9f4bdd7dcea222d。
- uv run pytest：46 passed。

Gate 边界

本日没有签名、广播、钱包或 live strategy。

PASS 只允许进入冻结范围内的 Day 8 paper baseline，不能对盈利作结论。

下一步必须加入独立 direct venue/re-quote 与完整 cost/reject evidence。

Visualization、额外历史样本和非阻塞 schema 字段留在 backlog。

笔记 102: 昨天看到群里讨论清算套利和原子套利，说前者更值得开始做，我以为这两个是不一样的，但其实并不是。下面是

作者：xingchi | 时间：2026-08-11 08:13 UTC | 字数：411 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

昨天看到群里讨论清算套利和原子套利，说前者更值得开始做，我以为这两个是不一样的，但其实并不是。下面是 ai 回复的。

////////

如果目标是从 0 搭一个真正能上线的链上 MEV/套利系统，我也更倾向先做清算，而不是从通用 DEX 原子套利开始。但原因并不是“清算盈利确定、原子套利不确定”，而是清算的 alpha 维度更多、机会结构更可预测、竞争更容易被细分。

严格说，清算本身也经常是原子的：flash loan → liquidation → 卖出抵押物 → repay flash loan → profit 可以全部放在一笔交易里。你真正比较的是 Liquidation MEV 与 DEX price-arbitrage / backrun MEV。Flashbots 的 bundle 机制本身就同时服务于套利和清算 searcher。

笔记 103: markdown

作者：Syen White | 时间：2026-08-11 08:14 UTC | 字数：387 | 评分：55
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

今天仔细分析了市面上的套利机会。大类别上就分为两类

中心化交易所之间的套利，有期现，期期，现现

去中心化交易所，主要是三角套利，跨链，MEV

dex 更依赖于强基建，短期内不太可能会有一些成果。去年花了几个月看了开源的 sui-mev 套利，发现其实很难做，主要原因是你需要自己部署高性能节点，并且需要对链上数据有很强的理解能力，还要有很强的策略设计能力。即便是这样，也不一定能盈利。

反而 cex 套利更容易一些，重点是关注价差和资金费率，以及交易所的流动性。目前 cex 的 api 都已经非常成熟，这些数据获取的都很容易。唯一需要注意点是不被爆仓。其实就需要具备一些仓位管理能力。

最终决定还是从 cex 套利入手，当然 dex 的套利也不会放弃，等牛市流动性回归时，可能会有机会。

笔记 104: 今天研究期权 Put-Call Parity (看涨-看跌平价) 套利，重点不是预测 BTC/ETH 涨

作者：dyhbrewer | 时间：2026-08-11 09:02 UTC | 字数：1325 | 评分：70
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天研究期权 Put-Call Parity (看涨-看跌平价) 套利，重点不是预测 BTC/ETH 涨跌，而是检查“相同到期现金流是否被不同产品以不同价格出售”。

对相同标的、执行价 K 和到期日 T 的欧式 Call、Put 与同期货，忽略小额折现时有：

$Call - Put \approx Future - K$ 。

因此，“买 Call + 卖 Put”可以构造合成多头期货，“卖 Call + 买 Put”可以构造合成空头期货。真正扫描时不能用 Mark Price，必须使用可成交 Bid/Ask：

$Synthetic\ Ask = K + Call\ Ask - Put\ Bid$ ；

$Synthetic\ Bid = K + Call\ Bid - Put\ Ask$ 。

两个方向：

1. 如果 $Future\ Bid > Synthetic\ Ask$ ：买 Call、卖 Put、做空同期货；
2. 如果 $Synthetic\ Bid > Future\ Ask$ ：卖 Call、买 Put、做多同期货。

以方向 1 为例，到期现金流为：

$(S \setminus T - K) + (Future\ Entry - S \setminus T) = Future\ Entry - K$ 。

标的到期价格 $S \setminus T$ 被抵消，理论收益来自挂牌期货与合成期货的价格差，而不是方向判断。

进一步研究了 Box Spread。选择 $K1 < K2$ ，组合为：买 $K1$ Call、卖 $K2$ Call、买 $K2$ Put、卖 $K1$ Put。无论标的到期价格是多少，理论到期价值固定为 $K2 - K1$ 。实际可成交成本为：

$Debit = C1\ Ask - C2\ Bid + P2\ Ask - P1\ Bid$ 。

如果 $Debit < K2 - K1$ ，差额相当于持有到期的固定收益，可换算成隐含年化利率，再与稳定币借贷、永续资金费率和交割合约基差比较。

但理论平价不等于可执行套利。完整净收益应为：

$Net\ Profit = 锁定到期价值 - 建仓成本 - 交易费 - 交割费 - 滑点 - 保证金成本 - 多腿失败准备金$ 。

最容易出现的假机会包括：

Call、Put、Future 的到期日或执行价没有严格匹配；

用 Mark Price 或 Mid Price 代替实际 Bid/Ask；

四条腿最优报价无法同时成交；

忽略卖出期权的保证金占用；

混用币本位与 USDC 线性期权；

忽略结算指数、合约面值和交割费；
只算权利金收益率，没有计算 Return on Margin；
平价偏差小于手续费和盘口价差。

第一版只读雷达可以读取 Deribit 公共 instruments 和 order book，按“资产×到期日×执行价”配对 Call/Put，再匹配同期货，计算 Synthetic Bid/Ask、Box Spread Debit、扣费净收益、隐含年化、峰值保证金回报率和连续存在时间。不接账户、不创建订单。

今天的结论：期权套利寻找的是现金流复制关系的定价偏差。只有平价偏差持续大于手续费、交割费、滑点、保证金成本和多腿失败准备金，才值得进入 Paper Trading。

笔记 105: 今天把一个很容易混淆的地方拆开了：看到两个市场之间有价差时，那个价差只能先叫“毛价差”。

作者：Alex Fan | 时间：2026-08-11 09:08 UTC | 字数：437 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天把一个很容易混淆的地方拆开了：看到两个市场之间有价差时，那个价差只能先叫“毛价差”。

我现在会把毛价差理解成：扣成本前的表面套利空间。比如投入 1,000 USDC，卖出预计拿回 1,004 USDC，表面多出来的 4 USDC 就是毛价差。

这个“毛”不是随便估一个大概，而是还没扣成本之前的数字。它看起来像利润，但还没有经过算账。

真正要看的，是扣完以后还剩什么：

text

毛价差

- Gas
- 交易费
- 桥费
- 滑点缓冲
- 等待缓冲
- 失败缓冲
- = 净利润

今天这个框架对我最有用的地方是，它把“看到机会”变成了“核算条件”。有时候表面上有价差，但扣掉费用和风险预留以后，净结果已经没有空间。

我今天更想记住这一句：毛价差是扣成本前的表面空间；净利润是扣掉费用和风险缓冲后真正剩下的结果。

这让我对链上套利的理解又往前走了一步：不能只问“有没有价差”，还要问“这点价差够不够支付成本、覆盖波动、扛住等待和失败”。

笔记 106: 今天做了什么：D5 定 2 个候选假设方向并写成可证伪假设卡进证据库：OPP-005 Aave v3

作者：zenkai | 时间：2026-08-11 09:11 UTC | 字数：402 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天做了什么：D5 定 2 个候选假设方向并写成可证伪假设卡进证据库：OPP-005 Aave v3 清算监控（主网,Gas<50 时段,净收益=奖金Gas失败成本）、OPP-006 新链早期价差（上线 2-4 周,价差 >0.5% 持续 >2h,≥2 次/周）。明确假设必须能被数据杀死；主动放弃资金费率方向（避开 1011 老战场,靠纪律不靠熟悉感）。

证据在哪：机会证据库 OPP-005/006（~/Desktop/链上套利共学/机会库/机会证据库.md）；D3 桥费 0.25% 实测数据支撑 OPP-006 成本假设

卡点：两个假设都缺数据：清算频率/规模/Gas 分布、新链清单与真实价差——D6 开始收。

明天任务（D6）：① 拉 Aave v3 一周清算事件（频率/规模/Gas）② 列近期新链清单,抽 3-5 条链看 ETH/USDC 价差。

笔记 107: Day 7 - 第一周复盘

作者：MountainE | 时间：2026-08-11 10:55 UTC | 字数：703 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 - 第一周复盘

这一周我从机会结构、价格与流动性开始，逐步学到 AMM、订单簿、区块状态和完整成本模型。最重要的收获是：名义价差只是信号，可执行的机会必须同时满足流动性、时间、成本和风险条件。

我选择后续优先研究：DEX 内与 DEX 间的同链现货套利。

选择理由：

- 它和前一周期学到的 AMM、滑点、报价和成本模型直接相连。
- 同链研究可以先排除跨链桥延迟和额外桥费，更适合先做数据观察、模拟和 Paper Trading。
- 交易路径和成本较容易做结构化记录，便于后续接入路由查询和监控工具。

可验证条件：

1. 在同一时间记录同一交易对在不同 DEX 的报价。
2. 指定交易规模，重新计算流动性、价格冲击和滑点。
3. 扣除 Gas、协议费、失败交易期望损失和资金占用后，净收益仍然为正。
4. 在不同时间和不同规模下重复观察，分辨可重复机会和偶然价差。
5. 在模拟执行中记录报价、决策、成本、失败原因和实际结果。

暂不优先研究跨链、清算和复杂

MEV

执行，不是因为它们没有价值，而是它们的桥接延迟、竞争密度、权限和失败风险更高，需要先建立更稳定的数据和验证能力。

第一周总结：我已经建立了从机会发现、报价验证、成本核算到风险复盘的基本框架，下一步需要用真实路由和可重复的数据来检验它。

下一周研究问题：

- LI.FI 的路由报价包含哪些关键字段，如何比较来源链、目标链、资产、桥、DEX 和路由步骤。
- 同一笔跨链需求在金额、资产和链变化时，路由和成本如何变化。
- 如何用 Hermes 或其他工具把一次性的观察变成可重复的查询、记录和监控流程。

笔记 108: 8 月 11 日打卡 | 先确认不同市场交易的是不是同一个风险敞口

作者：loveyuanfandj-gliitch | 时间：2026-08-11 11:09 UTC | 字数：1208 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8 月 11 日打卡 | 先确认不同市场交易的是不是同一个风险敞口

今天继续梳理 RWA 和 Perp 跨市场套利，重点放在标的映射上。

之前的思路是比较 Broker、Tokenized Stock 和 Perp 的价格，但今天意识到：代码相同或名称相同，并不代表两个产品可以直接对冲。

例如，券商中的 NVDA 正股、链上的 NVDA Token 和交易所里的 NVDA Perp，可能在以下方面存在差异：

价格来源和预言机机制

交易时间与休市安排

合约乘数和计价货币

是否具有赎回权或真实股票权益

分红、拆股等公司行动如何处理

Perp 的资金费率和 Mark Price 规则

正股是否可以借入并做空

极端情况下是否会暂停交易或调整价格

如果这些条件没有对齐，看到的“价差”可能并不是套利机会，而是不同产品之间合理的风险补偿。

因此，RWA 套利的第一步不应该是直接比较价格，而是先建立 instrument_master，明确每个市场产品代表的真实敞口。

准备记录的字段包括：

venue

symbol

underlying

product_type

contract_multiplier

quote_currency

price_source

oracle_source

trading_session

funding_interval

settlement_method
redemption_right
shortability
corporate_action_rule
market_status

对于 ES、NQ 和黄金，还需要统一不同产品的合约乘数，并将仓位换算成相同的美元 Delta：

美元敞口

= 价格 × 合约乘数 × 持仓数量

对冲数量

= 目标美元敞口 ÷ (对冲价格 × 对冲合约乘数)

今天也把候选信号分成了几个状态：

OBSERVED

→ 数据上看到差异

VALIDATED

→ 标的、时间、价格和深度已经对齐

PAPER

→ 进入模拟持仓并记录实际费率与基差变化

EXECUTABLE

→ 扣除成本后仍然满足条件

REJECTED

→ 记录具体失败原因

下一步准备完成：

1. 为 ES、NQ、黄金、NVDA、CRCL 和 SPCX 建立第一版标的映射表。
2. 核对不同市场的交易时间、合约乘数和价格来源。
3. 将所有仓位统一换算成美元 Delta。
4. 增加市场休市、数据过期和标的不等价的拒绝规则。
5. 选择一个数据最完整的资产，建立第一条跨市场 Paper Trading 记录。

今天的核心认识是：跨市场价格不同不一定代表错误定价。只有确认两边代表相同或足够接近的风险敞口，并且能够真实成交和持续对冲，价差才有进一步研究的意义。

笔记 109: 链上套利共学 · Day7 打卡笔记 (08/11)

作者：haibian-slowly | 时间：2026-08-11 11:30 UTC | 字数：2491 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 · Day7 打卡笔记 (08/11)

今日信息

日期：2026-08-11 (周二)

主题：用第一性原理框架"复盘"自己的学习路径

核心问题：学了这么多天，能不能用一套统一的判据，审视群里的讨论和我们自己的数据？

打卡正文

【第 7 天】

1. 今天要验证的问题

昨天 (8/10) 拆解了讲师 brucexu 的长文《链上套利的第一性原理》(X Article, 9.6K 字)，拿到一套"判据"。今天看到群里传的一份群聊复盘截图。今天的核心问题：这套判据能不能反过来审视我们共学群自己的讨论？哪些是真机会、哪些只是故事？

2. 今天实际做了什么

1. 拆解 brucexu 长文 (昨天完成，今天整理归档)：

- 产出两份文件：brucexu_套利第一性原理长文_知识点拆解与模拟还原_2026-08-10.md (11.5K 字) + 原文存档；
- 核心结论一句话：价差只是线索，资产闭环才是结构，全成本后的期望净利润才是机会；
- 拿到 3 件工具：① 北极星判据 (能否从资产 A 出发、经可执行转换、回到更多 A)；② 7 步执行法；③ 四档决策 (做/观察/放弃/等结构变化)。

2. 用 7 步执行法审自己的 Day5 实测 (Part D 模拟还原) :
- 假设信号 : Base USDC 对 Arbitrum 溢价 +50bps ;
 - 扣成本 : 桥费 25bps (实测恒定 0.25%) + 汇率差 7bps (实测) Gas \$0.014 → 报价内净额 +17.9bps ;
 - 卡点 : 跨链非原子 + 你实测的 opt→arb 方向 eco(7s)/polymerStandard(18min) 随机漂移 → 18 分钟窗口内价差收敛风险不可忽略 ;
 - 结论 : 观察档——结构成立、账面为正, 但失败成本未定价, 不上实盘。
3. 研读群复盘截图 (今天) : AI 对共学群整批讨论的复盘, OCR 全文提取 (6 屏长图)。内容分六部分 : 讨论推进四阶段 / 套利系统四层架构 / 术语辨析 / AI 的角色 / 群当前水平判断 / 六句结论。

3. 证据 (本次真实运行结果)

拆解笔记 : E:\链上套利共学\brucexu_套利第一性原理长文_知识点拆解与模拟还原_2026-08-10.md (已验证 : K1-K14 齐全、结构完整) ;

原文存档 : brucexu_套利第一性原理_原文存档_2026-08-10.md (fxtwitter API 全文提取, 未删改) ;

群复盘截图 : 微信图片_20260811095832_5808_892.png (1235×9498 长图, RapidOCR 提取全文并逐段研读) 。

4. 得到了什么结论 (来自数据)

(1) 群里讨论的推进路径, 正好对应 brucexu 文章的论证链 :

| 群复盘说的阶段 | 对应文章内容 |

|---|---|

| 先弄清"套利到底是什么" (价差/MEV/资金费率/LP 边界) | K14 套利 vs 方向性边界 |

| 被高收益案例吸引 (45万U、BitMart 搬砖、插针爆仓) | 文章 Step 1 反例 : 价差≠利润 |

| 意识到"代码不难, 难在发现路径" | 判据 : 可执行资产闭环 |

| 碰到基础设施门槛 (RPC \$200/月、450ms 已算慢) | K8 报价外成本清单 |

| 机会在长尾 (新池子/小交易所/碎片化流动性) | 14 类套利地图的"友好度"维度 |

(2) 群复盘里最有价值的判断, 和文章互证 :

- "发生过一次 ≠ 可以形成稳定策略" → 就是文章说的 : 事件型机会 (插针、爆仓、加错池子) 偶发、无法稳定预测, 只能当线索 ;
- "代码不是壁垒, 数据、路径和执行才是壁垒" → 就是文章说的 : 真正的成本在报价外 (资金占用、竞争、失败概率) ;
- "AI 降低学习门槛, 但不会自动产生套利优势" → 如果大家都能用 AI 生成同一套机器人, 这套机器人就没有优势了。

(3) 我自己踩过的坑也归位了 :

- Day4 的 0 滑点、Day5 的 0.32% 恒定损失 → 就是文章"全成本公式"里"报价内成本"那部分 ;
- Day5 的 18 分钟路径漂移 → 就是"资金与机会成本" (K5) 和"失败概率" (K6) 的现实注解。

5. 今天失败了什么

群复盘截图是 9498px 长图, 直接读失败, 最后用 RapidOCR 分段提取才拿到全文——长截图 OCR 是必要的读图手段 ;

8/10 (Day6) 漏卡了 : 昨天拆完长文没及时打卡, 今天把两天内容合在一篇里补记 (如果规则允许单独补卡, 需要我再补一篇也行) 。

Day6

6. 明天只解决什么

把"执行时间是隐性成本"从结论变成可计算字段 : 用 POST /v1/advanced/routes 拿同一笔跨链的多路径候选, 重点比较 eco(7s) vs polymerStandard(18min) 的费用 + 执行时间 + 资金占用成本, 回答"默认 quote 为什么选了 18 分钟的路径", 顺手补上空的 cost_model.py。

一句话总结

用 brucexu 的判据复盘共学群 : 群里讨论的推进路径 ≈ 文章的论证链 ; "发生过一次 ≠ 稳定策略""代码不是壁垒, 数据/路径/执行才是"两个判断互相印证 ; 我自己的实测数据也正好落进全成本公式——判据不是新知识, 是把已有经验串成系统的那根线。

明日预告

Day8 (08/12) : 多路径比较——eco vs polymerStandard, 执行时间成本量化 (advanced/routes) 。

笔记 110: 接入并完善 Lighter、Hyperliquid、Variational 三个平台的 BTC/ETH

作者 : wangche | 时间 : 2026-08-11 11:34 UTC | 字数 : 330 | 评分 : 57

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

接入并完善 Lighter、Hyperliquid、Variational 三个平台的 BTC/ETH 永续公开行情, 包含价格、资金费率、持仓量和盘口。

搭建永续合约影子交易系统 : 本地模拟市价单、限价单、持仓、资金费率和 P\&L。

增加跨平台价差影子策略, 支持监控、虚拟多空配对、费用估算、止盈止损和自动平仓 ; 不读取钱包、不签名、不发送真实订单。

增加 DEX 双池套利影子扫描, 按池价差、手续费、滑点、Gas、TVL 和数据时效计算候选机会, 并保存到本地 SQLite。

将上述行情和影子策略接入本地仪表盘，新增永续行情、影子交易和价差策略页面及 API。
补充对应的单元测试，覆盖行情解析、缓存、订单撮合、价差策略和风险参数校验。

笔记 111: Day06 数据工具与信息源

作者：tutu | 时间：2026-08-11 11:44 UTC | 字数：2279 | 评分：78
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day06 数据工具与信息源

我以前看到一个合约费率很高，第一反应总是算年化。页面上跳出一个夸张的数字，脑子里马上就会接上另一串更夸张的数字。持有一天多少，持有一周多少，账户里那点 U 要是都压进去会怎样。

后来才意识到，这种算法很容易把人带到坑边上。页面给我的只是一个价差或一条费率，它还没有回答最要紧的事。这个收益最后能不能落到我的账户里。

Day 6 学下来，我给自己留了一条顺序。先找机会，再核实，最后才算钱。聚合面板适合做第一步。Coinglass 或 Velo 上面扫一眼费率和价差热力图就够了，年化高得离谱的标的可以记下来，但不下判断。那地方像雷达，负责告诉我哪里有动静。准备开仓时，我会回到交易所。

回到交易所以后，我直接查合约和现货两边的实际报价。比如我想做 10 万 USDT 的现货多、永续空，屏幕上的最新成交价没有多少参考价值。我需要沿着现货卖盘和永续买盘，把这 10 万 USDT 一档一档算下去，得到两条腿的加权成交均价。价差只有在这个数上为正，才算我能摸到的钱。

盘口薄的时候，屏幕经常很好看。最优价之间摆着一段不小的差，等你真要吃进去，第二档、第三档的价格已经把利润吞掉了。更麻烦的是，订单簿是活的。挂单会撤，别人会抢，某些小币的深度看着像有，真到出场时才发现那里根本没有人接。单腿滑点要是已经吃掉预期总收益的一半，我宁可不做。

OI 也是同一个道理。看到一个数字之前，先问它按什么口径统计。单位先分清是币还是美元，随后确认平台按单边还是双边统计，再看它是当前快照还是一段时间的平均。尤其是小合约，自己的计划仓位如果已经占了 OI 的明显比例，进场也许不难，出场会很难。平台的限仓、个人仓位限制和剩余可开仓额度，也得一起看。赚不赚钱是后面的事，先确认自己有没有资格安全地离开。

资金费是最容易把人带偏的一项。聚合器写着加 0.6%，我不会立刻把它乘以一年。我先分辨页面给的是原始费率还是年化后的展示数字。接着看 API 返回百分比还是小数，查清这笔费率对应的结算周期和名义价值，最后确认它是下一期预测还是上一期已经结算的结果。

这些问题看着烦，漏掉其中一个就可能把账算错几倍。预测费率只能给一个条件收益，它会动，离结算近时尤其会动。历史费率能说明这件事以前持续过多久，却不能替未来签字。还有方向问题。做现货多、永续空时，正资金费通常由多头支付给空头，这对永续空单是收入。费率转负以后，这一项就变成了成本。把所有资金费都当收入，是很危险的习惯。

我对“持仓成本”的理解也变了。现货如果是借来的，借贷利息每天都在走。现货如果用的是自己的 U，也有机会成本。再加上开仓和平仓的手续费，以及两边实际吃到的滑点，所谓套利收益会一点点变薄。价差是名义毛利，这些才是持仓租金。

这句话后来帮我把一笔练习算明白了。现货在 100 买入，永续在 102 做空，数量是 10,000 枚。五天后两边都在 105 平掉。现货赚了 50,000 USDT，空单亏了 30,000 USDT，基差收敛留下 20,000 USDT。

如果每天资金费收入按永续开仓名义价值的 0.08% 估算，五天可以收到 4,080 USDT。现货借贷成本按每天 0.03% 算，五天要付 1,500 USDT，净资金费只剩 2,580 USDT。到这里看上去还挺舒服。

手续费和滑点一扣，账的样子就变了。四笔成交额合起来是 4,120,000 USDT。若手续费按这个成交额的 0.16% 计，手续费是 6,592 USDT。滑点按 0.10% 计，又要 4,120 USDT。最后的净利润是 11,868 USDT。

这笔练习没有告诉我“这个策略能赚多少”，它告诉我另一件更有用的事。费率、价差、手续费和滑点必须放在同一个单位里算。价差是价格，资金费是百分比，直接把它们加在一起没有意义。只有都换成 USDT，才能知道自己到底在比较什么。

我现在会用一个保守的公式决定要不要进场。

```
$$
\text{保守预期净收益}
= \text{基差可能收敛的钱}
+ \text{打折后的资金费收入}
- \text{手续费}
- \text{滑点}
- \text{借贷和机会成本}
- \text{风险缓冲}
$$
```

保守预期净收益要大于零。保守总收入最好能覆盖总成本的 1.2 倍。这里的 1.2 只是我给自己留的安全垫，各个合约用不着照搬同一比例。资金费打多少折，滑点留多少余量，要看合约本身的深度和自己准备持有多久。

还有几种情况，我会直接放弃。两条腿中有一条不能按测试的数量成交或退出，借贷和提币出现异常，压力测试后保证金离强平太近，合约有 ADL、限仓、维护之类我吃不透的规则，或者把费率打折、把滑点上调以后收益已经不为正。这些问题没解决，屏幕上的高年化再漂亮也没有用。

距离结算只剩两小时，或者标的一个小时波动很大，我也不会只凭一条硬阈值处理。它们会让我重新看费率、标记价、两腿对冲是否还贴得住，以及盘口有没有变坏。完整对冲能扛住一部分价格波动，我更担心两条腿开始失配，或者退出通道消失。

这一天最后留下来的东西很朴素。聚合器帮我找地方，交易所一手数据帮我做决定。下单前把数据口径问清，把每一项都换成 USDT，把最坏情况也放进账里。这样做不会让机会变多，却能让我少去接那些看着很肥、实际没有出口的单子。

下一步去做纸面模拟。先把价格、费率、手续费、滑点和持仓时间按同一套格式记下来，过一周再看自己的眼睛到底漏了什么。

笔记 112: 【8/11 链上套利共学打卡】

作者：tension-vip | 时间：2026-08-11 11:55 UTC | 字数：835 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【8/11 链上套利共学打卡】

今天学习的主题是：链上套利的第一性原理——从“看见价差”到“可执行净利润”。

以前很容易把套利理解成：

A 市场价格低 → 买入 \

B 市场价格高 → 卖出 \

中间的差价 = 利润

但今天更重要的认知是：

价差只是套利研究的起点，不是利润。

真正的套利，需要构造一条完整的资产闭环：

起始资产 → 买入/兑换 → 转移/跨链/持仓 → 卖出/赎回 → 回到可比较的终点资产

然后再问：

最后拿回来的资产价值，扣除所有成本、时间和风险之后，是否仍然高于最开始？

这才是值得执行的套利机会。

因此以后看到任何“套利机会”，我会先检查几个问题：

1. 价差为什么存在？ \

是不同交易所、不同链、期限、利率、流动性还是市场结构造成的？

2. 资产能不能真正闭环？ \

不是账面上可以买低卖高，而是两条腿都必须真实可成交。

3. 真实净利润是多少？ \

不能只算表面 Spread，还要把手续费、Gas、滑点、跨链成本、资金费率、借贷成本以及资金占用时间全部算进去。

4. 在执行期间会发生什么？ \

价格可能收敛、流动性可能消失、跨链可能延迟、交易可能只成交一边，协议本身也存在风险。

今天最大的收获其实不是找到某个具体套利策略，而是建立一个筛选框架：

Spread ≠ Profit

更准确地说应该是：

Opportunity → Executable Path → Net PnL → Risk-adjusted Return

以后做链上套利，我想把重心从：

“哪里有色差？”

慢慢转成：

“这个价差为什么存在、能否持续、我能不能稳定地把它变成净利润？”

这可能才是从“发现机会”走向真正构建套利系统的第一步。

\#链上套利 #Crypto #DeFi #Web3 #量化交易 #共学打卡

笔记 113: DEX 间套利 (Inter-DEX Arbitrage)

DEX 间套利 (Inter-DEX Arbitrage)

原理：在同一条公链上的不同 DEX 协议之间搬砖。

典型场景：

当有人在 Uniswap 上大额买入 ETH，导致 Uniswap 的 ETH 价格瞬间被抬高到 \$3,050\$，而此时 Sushiswap 或 Curve 上的 ETH 价格依然是 \$3,000\$。

做法：从 Sushiswap 低价买入 \$3,000\$ 的 ETH，同时去 Uniswap 高价卖出 \$3,050\$，吃掉这 \$50\$ 美元的价差。

笔记 114: Day 7 | 从价差到净收益：AMM 线性化估算 + 成本模型逐项落地

Day 7 | 从价差到净收益：AMM 线性化估算 + 成本模型逐项落地

今日目标：把「屏幕上看到的价差」翻译成「可执行净收益」分两步走——先用线性近似快速估算价格冲击做机会初筛，再对照 LI.FI 报价把成本模型的每一项钉到数字上。 AMM

学到了什么

1. AMM 定价的线性化处理。 $x \cdot y = k$ 的非线性曲线在当前价格附近做一阶泰勒展开后，小额交易（交易量 <1% 池深的量级）的价格冲击近似线性。好处很实际：估算从解二次方程降成一次乘法，误差在小额区间可接受，适合对一批候选机会做批量初筛，通过的再进精确计算。社群里 anyi 今天的笔记推导的也是这条路。
2. 套利的第一性原理。Bruce 老师文章里的定义值得抄下来：套利不是「寻找两个不同的价格」，而是寻找同一个经济价值在不同市场、资产形态或时间状态之间尚未闭合的结构关系。这个框架解释力很强——我前几天拆的接针策略（同一资产在恐慌抛压瞬间的价格错位）、社群里那个考核交易机构规则套利案例（同一敞口在对赌平台与交易所两套规则下的定价差，524 天套出千万台币），本质都是结构关系未闭合，只是闭合的触发器不同。
3. 成本侧的实证缺口。结合昨天对 LI.FI 的分析：报价里 `toAmountMin` 与实际到账的偏差，大头是滑点和 gas，桥费相对固定可预期，但失败交易损耗和安全风险溢价这两项还没有量化——这是成本模型目前最大的盲区。

认知收获

价差只是入场券，净收益才是判决书。线性化负责「快」（秒级初筛），成本模型负责「准」（逐项核销），两件套凑齐之前，任何回测收益都是幻觉。

明日计划

用一笔真实小额 swap 对照 LI.FI 报价跑完整流程，记录 gas、协议费、滑点、确认延迟四项明细，同时验证线性近似在 0.5% 池深量级下的估算误差到底有几个基点，作为成本模型的第二个实证样本。

笔记 115: 今天我完成了受控候选路径的首次批量 Paper Trading 验证。

今天我完成了受控候选路径的首次批量 Paper Trading 验证。

在前面搭建的 Pool manifest、候选闭环生成、通用 V3 obligation quote 和 V4 terminal quote 基础上，今天把 7 条候选路径放到同一个固定区块状态和统一成本假设下进行批量评估。目标不是寻找“看起来有价差”的路径，而是验证哪些候选真的能够完成同状态报价，并进一步进入保守 PnL 判断。

结果显示，当前候选池中没有出现 `PAPER\_TRADE\_CANDIDATE`。原本已经验证过的三跳路径可以正常完成 Quote，但保守收益仍为负，因此结论是 `REJECT`。其余新增候选大多在 `Quote` 阶段失败，没有被错误地算作亏损或直接忽略，而是明确记录为 `QUOTE\_FAILED`，等待后续区分流动性不足、Pool callback/ABI 不兼容或当前规模无法完成 exact-output quote 等具体原因。

今天最大的收获是，套利研究流程进一步从“单一路径验证”变成了“候选批量筛选”。现在系统能够保留不同失败类型：有的路径是报价成功但无利润，有的路径则尚未具备可用报价条件。这种区分比简单判断“有无价差”更重要，也更接近真实套利研究的工作方式。

当前阶段仍然严格保持只读和本地 Fork 模拟，没有签名、授权、广播交易或使用真实资金。

笔记 116: 链上套利残酷共学 | 2026-08-11 打卡

作者: koyo922 | 时间: 2026-08-11 13:37 UTC | 字数: 876 | 评分: 69

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 | 2026-08-11 打卡

今天主要学习了 sunlc.ai 关于套利监控的文章，重点了解了他对套利机会发现和监控的思路。整体看下来有一定借鉴意义，但能够直接迁移到我当前研究中的内容不多。对我来说，更有价值的是再次确认：套利监控不能只记录某个时刻的表面价差，还需要继续追踪价差持续时间、可成交深度、手续费、滑点和执行风险。

1. DOS 开盘套利

今天继续盘点群聊中讨论的 DOS 机会。这个机会发生在 Bitget 开盘前约 10 秒，期间会出现一个短暂价差。

目前这个机会我已经研究得比较充分，并且已经拿到了当时的具体交易记录，基本证实了机会确实存在。下一步的重点不是继续停留在事后讨论，而是把当时的时间窗口、交易所盘口、成交价格、可成交规模和费用条件还原出来，再接入监控脚本进行实时观察和历史回放。

2. TUT 爆仓后的“捡尸体”套利

今天也继续分析了 TUT 的剧烈波动和爆仓情况。初步理解是：市场剧烈波动导致一部分资金费率套利或杠杆仓位被强制平仓，爆仓集中发生之后，市场可能进入一个相对平稳的阶段。在这个阶段，不同交易所或不同市场之间可能仍然存在一段时间的价差，可以把它理解为爆仓后的“捡尸体”阶段或策略稳定期。

但这个机会目前还没有完全弄清楚。现在虽然大概知道爆仓为什么会发生，但还没有想清楚以下几个关键问题：

“捡尸体”阶段具体应该如何操作；
应该通过哪些数据和信号判断阶段已经开始；
如何确认价差是真实可成交价差，而不是盘口残留或数据不同步；
如果价格没有收敛，而是继续下跌，应该如何止损和退出；
如何控制单腿成交、流动性消失、标记价格差异和交易所风控等风险。

因此，TUT 目前还只能作为待验证的研究方向，不能直接当作稳定套利策略。

3. 明日计划

继续完善 DOS 开盘套利的历史回放和监控条件；同时进一步研究 TUT 爆仓后的价格、资金费率、持仓量、盘口深度和跨市场价差，尝试定义“捡尸体”阶段的进入信号、退出信号和风险停止条件。

笔记 117: 携手 AI 学套利

作者: wayhome | 时间: 2026-08-11 13:40 UTC | 字数: 802 | 评分: 60

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

携手 AI 学套利

来源: <<https://learnwithai.robotisland.cloud/>>

笔记

套利的定义

套利是利用同一资产或可验证等价的现金流之间的定价差异，在声明的市场状态与执行边界内锁定全部必要交易，使成本后收益不再依赖未来价格朝有利方向移动的策略。

这里的“锁定”不等于现实世界中绝对不会亏。合约漏洞、链重组、预言机异常和错误建模都可能破坏结论。套利分类回答的是市场收益是否依赖未来价格；执行决策还要另算失败概率和失败损失。正的预期净收益不能把一个尚未锁定第二条腿的策略变成套利。

价差成为利润前的四道检查

比较的同一资产吗

报价对应你的成交规模吗

两条腿能否一起锁定

扣完所有代价后还剩什么

如果套利存在，为什么别人还没拿走\

不是一句怀疑，而是一项必须回答的研究任务

市场被分隔

定价机制不同

AMM 曲线、订单簿、预言机和清算规则对同一信息反应不同
执行能力稀缺
原子合约、低延迟基础设施、库存和优先排序需要真实成本
风险尚未定价
把“看见价差”改写成研究问题

我比较的两个价值是 $\backslash\backslash\backslash\backslash\backslash$ ，它们等价的条件是 $\backslash\backslash\backslash\backslash\backslash$
我的真实可成交规模是 $\backslash\backslash\backslash\backslash\backslash$ ，不是界面上的边际价格
必要交易能通过 $\backslash\backslash\backslash\backslash\backslash$ 锁定；不能锁定的部分是 $\backslash\backslash\backslash\backslash\backslash$
利润至少还要扣除 $\backslash\backslash\backslash\backslash\backslash$
这个价差仍然存在，最可能因为 $\backslash\backslash\backslash\backslash\backslash$

笔记 118: 今天继续学习 DEX-CEX 套利，尝试把一笔交易的成本完整列出来。

作者：zgos | 时间：2026-08-11 14:20 UTC | 字数：267 | 评分：47
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续学习 DEX-CEX 套利，尝试把一笔交易的成本完整列出来。
假设 DEX 和 Binance 之间存在 1% 的价差，最开始会觉得这 1% 都是利润。但实际还要减去 DEX 交易手续费、Gas、滑点、Binance 手续费，以及后续资金再平衡的成本。
我目前整理出的简单公式是：
净利润 = 卖出收入 - 买入成本 - 全部手续费 - 滑点损失 - 资金调整成本
今天最大的收获是，判断机会时最好同时计算金额和收益率。虽然某笔交易有利润，但如果利润太小，可能一次价格波动或失败交易就会把前面赚到的钱全部亏掉。

笔记 119: 套利基础学习打卡 | 什么是 Basis？从基差理解现货与合约套利

作者：cfanboy | 时间：2026-08-11 14:45 UTC | 字数：4361 | 评分：88
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利基础学习打卡 | 什么是 Basis？从基差理解现货与合约套利

上一篇学习了永续合约中的 Funding Rate（资金费率），今天继续学习一个和资金费率关系非常紧密的概念：

Basis（基差）。

刚开始接触套利时，很容易把“价差”和“套利机会”直接画等号。但进一步学习后发现：

看到价差只是第一步，更重要的是理解价差为什么存在、为什么可能收敛，以及扣除所有成本以后还能剩下多少。

基差就是理解这件事情非常重要的一个入口。

一、什么是 Basis？

在加密货币市场中，可以先简单理解为：

$$\text{Basis} = \text{合约价格} - \text{现货价格}$$

例如：

BTC 现货价格 = 100,000 USDT

BTC 合约价格 = 102,000 USDT

那么：

$$\text{Basis} = 102,000 - 100,000 = +2,000 \text{ USDT}$$

如果换成百分比：

$$\text{Basis\%} = (\text{合约价格} - \text{现货价格}) / \text{现货价格} \times 100\%$$

这个例子中：

$$\text{Basis\%} = 2\%$$

也就是说，BTC 合约相比现货存在约 2% 的溢价。

二、正基差与负基差

正基差

如果：

现货 = 100,000

合约 = 102,000

那么：

Basis = +2,000

合约价格高于现货价格，可以简单理解为：

合约存在溢价。

这种情况在市场情绪偏多、杠杆做多需求较强的时候比较容易出现。

负基差

反过来，如果：

现货 = 100,000

合约 = 98,000

那么：

Basis = -2,000

说明合约价格低于现货价格，也就是：

合约相对现货折价。

这种情况可能出现在市场情绪悲观、做空需求较强或者极端行情之中。

所以最简单的记忆方式就是：

Basis > 0 → 合约溢价

Basis < 0 → 合约折价

三、为什么现货和合约会产生价差？

这是今天学习过程中我觉得比较关键的问题。

既然它们对应的都是 BTC，为什么价格不能完全一样？

原因在于：

现货市场和合约市场虽然标的是同一个资产，但参与者的需求并不完全相同。

例如市场突然进入强烈上涨阶段。

大量交易者可能并不想真的拿 100,000 USDT 买一枚 BTC，而是希望通过永续或者期货合约加杠杆做多。

于是：

做多需求增加\

↓ \

合约买盘增强\

↓ \

合约价格上涨\

↓ \

合约相对现货出现溢价

于是 Basis 就产生了。

反过来，如果大量资金进入合约市场做空，也可能使合约相对现货出现折价。

所以从这个角度来看：

Basis 实际上也是市场资金供需和杠杆需求的一种体现。

四、为什么 Basis 会产生套利机会？

套利最有意思的地方就在这里。

假设：

BTC 现货 = 100,000 USDT

BTC 交割合约 = 103,000 USDT

Basis = +3,000 USDT

也就是：

合约比现货贵 3%。

如果这个合约最终会以接近 BTC 现货指数的价格进行交割，那么随着到期日越来越近：

合约价格与现货价格应该逐渐收敛。

于是可以构建：

买入 BTC 现货

同时：

做空 BTC 合约

也就是经典的：

Long Spot + Short Futures

这就是常说的：

Cash and Carry Arbitrage。

五、为什么 BTC 涨跌并不是最关键的问题？

假设：

买入 1 BTC 现货：100,000

同时做空 1 BTC 合约：103,000

初始 Basis：

3,000 USDT

假设到期时 BTC 涨到了 120,000。

现货：

100,000 → 120,000

盈利约：

+20,000

合约空单：

103,000 → 120,000

亏损约：

-17,000

两边合起来：

+3,000 USDT

反过来，如果 BTC 跌到了 80,000：

现货：

100,000 → 80,000

亏损：

-20,000

合约空单：

103,000 → 80,000

盈利：

+23,000

两边合起来仍然约：

+3,000 USDT

这让我们对套利有了一个更清楚的认识：

我们不是在赌 BTC 最终涨到多少，而是在交易“103,000 和 100,000 之间的价差最终会收敛”。

这就是基差套利最核心的逻辑。

六、为什么一定要看“年化基差”？

单纯看到：

Basis = 3%

其实还不足以判断这个机会好不好。

因为：

7 天赚 3%

和：

180 天赚 3%

完全不是一回事。

所以实际分析中，更应该把 Basis 转成年化收益率。

可以粗略理解为：

$\text{Annualized Basis} \approx \text{Basis\%} \times 365 / \text{距离到期天数}$

例如：

$\text{Basis} = 3\%$

距离交割 = 30 天

那么简单年化：

$3\% \times 365 / 30 \approx 36.5\%$

看起来非常不错。

但如果距离交割还有 180 天：

$3\% \times 365 / 180 \approx 6.08\%$

吸引力就完全不同了。

因此以后看到一个基差套利机会，不能只问：

“价差有多少？”

还应该问：

“资金需要占用多久？”

七、永续合约和交割合约的 Basis 有什么区别？

这里也非常容易混淆。

交割合约

例如 BTC 季度合约。

它有明确到期日。

因此随着到期日越来越近：

$\text{Futures Price} \rightarrow \text{Spot Price}$

Basis 通常会逐渐收敛。

这给基差套利提供了一个比较明确的退出时间点。

永续合约

永续合约没有到期日。

所以不存在：

“等到交割日自然收敛”

这样的机制。

永续合约主要依靠上一篇学习的：

Funding Rate

来帮助永续价格向现货指数靠近。

于是这里终于把上一篇和今天的内容连接起来了：

Basis 是价格偏离本身。

Funding Rate 是推动永续价格回归现货价格的重要机制之一。

八、Basis 和 Funding Rate 是什么关系？

这是今天我认为最值得记住的一个地方。

如果永续价格长期明显高于现货：

$\text{Perp} > \text{Spot}$

市场往往存在比较强的做多需求。

此时 Funding 通常更容易为正。

于是：

Long Spot + Short Perp

不仅可能从 Basis 的变化中获益，同时 Short Perp 还可能收到 Funding。

反过来：

如果永续价格低于现货，并且 Funding 为负，那么市场可能存在比较强的做空需求。

因此真正分析永续套利机会时：

不能只看 Funding。

也不能只看：

Perp - Spot。

而应该把两者结合起来。

九、基差套利并不是无风险套利

表面来看：

买现货 + 空合约

已经把 BTC 的方向风险基本对冲掉了。

但真正执行时仍然存在很多风险。

例如：

1. 手续费

现货买入、合约开仓、最终平仓都可能产生手续费。

2. 滑点

特别是小币种，看起来有 2% 的 Basis，但真正成交以后可能已经损失掉相当一部分。

3. 资金成本

买现货需要占用大量资金。

如果资金本身存在融资成本，也应该计入。

4. 保证金与强平风险

虽然整体仓位可能 Delta Neutral，但现货和合约账户的保证金并不一定自动共享。

极端行情下，合约的一条腿仍然可能因为保证金不足而被强平。

5. 交易所风险

如果策略需要长期把大量资金放在 CEX，交易所本身就是一个风险来源。

6. 执行风险

如果现货成交了，但合约没有及时成交，那么短时间内就会暴露方向风险。

所以真正应该计算的是：

净套利收益 = 基差收益 + Funding 收益 - 手续费 - 滑点 - 借贷/资金成本 - 其他执行成本

十、今天的总结

今天学习 Basis 后，我对“价差套利”有了更具体的理解。

套利并不是：

看到 A 价格低、B 价格高，就直接买 A 卖 B。

更重要的是回答三个问题：

第一，这个价差为什么会产生？

是市场情绪、杠杆需求、流动性差异，还是短暂的价格不同步？

第二，这个价差为什么会收敛？

如果没有明确的收敛机制，那么所谓的“套利”可能只是同时持有两个不同风险的仓位。

第三，价差收敛以后，扣除所有成本还剩多少？

手续费、滑点、Funding、借币利率、资金占用和执行风险，都可能把表面利润吃掉。

因此今天最重要的一个认识是：

价差 $\neq$ 套利。

真正的套利机会应该是：

发现价差 → 理解形成原因 → 找到收敛机制 → 对冲方向风险 → 计算全部成本 → 得到最终净收益。

而 Basis 正好提供了一个非常典型的例子。

把上一篇 Funding Rate 和今天 Basis 放在一起之后，也开始能够看到一个更完整的套利分析框架：

Spot Price → Futures/Perp Price → Basis → Funding Rate → Cost → Net Profit

下一步准备继续研究一个非常重要的概念：

Delta Neutral（Delta 中性）。

因为无论是 Funding Arbitrage 还是 Basis Arbitrage，本质上都在尝试做同一件事情：

把“猜涨跌”从策略中尽可能剥离出去，只保留真正想赚取的那部分价差收益。

笔记 120: Cex-Dex 套利熔断监控：\

作者：Miles | 时间：2026-08-11 14:45 UTC | 字数：280 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Cex-Dex 套利熔断监控：\

CEX 429/418/-1003 限流和 BSC RPC 故障；

token totalSupply 异常增发；

EIP-1967 implementation/admin、token owner、ProxyAdmin owner 变更；

token owner / ProxyAdmin owner 等敏感钱包的大额流出。

单 token 异常优先进入“只平不开”的局部隔离；多个独立 token 同窗异常或系统性故障才升级为全局 close-only/paused。熔断不自动解除，必须在人工复核后恢复。

笔记 121: 永续合约学习日报

作者：V1ct0r551 | 时间：2026-08-11 14:52 UTC | 字数：3385 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

永续合约学习日报

日期：2026-08-11

主题：

永续合约的价格锚定机制、资金费率，以及 Binance 上 U 本位合约和币本位合约的区别。

一、今日核心问题

今天主要讨论了两个问题：

1\、为什么资金费率是永续合约和现货价格锚定的关键？

2\、Binance 上的 U 本位合约和币本位合约有什么区别？

二、资金费率为什么重要

永续合约没有到期日，所以它不像传统交割合约那样可以靠“到期交割”让合约价格强制回归现货价格。

因此，永续合约需要另一个机制来维持价格锚定，这个机制就是资金费率。

资金费率的本质是：

当永续价格偏离现货价格时，让推动偏离的一方持续付出成本，从而吸引反向套利者进场，把价格重新拉回现货附近。

三、资金费率如何锚定现货

当永续价格高于现货时：

多头需求更强，永续价格被推高；

资金费率通常变为正；

多头需要定期支付资金费给空头；

做多永续变贵，做空永续变有收益；

套利者会买入现货，同时做空永续；

这个行为会压低永续价格，使其向现货回归。

典型套利路径：

买现货 + 空永续

当永续价格低于现货时：

空头需求更强，永续价格被压低；

资金费率通常变为负；

空头需要支付资金费给多头；

做空永续变贵，做多永续变有收益；

套利者会做多永续，推动永续价格回升。

四、为什么几小时才收一次费用，但仍然有效

虽然资金费率不是实时收取，而是每隔 n 小时结算一次，比如 Binance 常见的是每 8 小时一次，但它仍然有效。

原因是：

市场交易的是预期，而不是只在结算时才反应。

如果当前永续价格明显高于现货，交易者会预期未来资金费率大概率为正，多头要持续付费。

套利者不会等到资金费真正扣款的那一刻才行动，而是会提前进场：

买现货 + 空永续

所以资金费率虽然是离散结算，但它对价格的影响是连续发生的。

可以类比债券：

债券可能半年付息一次，但债券价格每天都会反映未来利息的价值。

永续合约也是一样：

资金费率可能每几小时结算一次，但市场会实时交易未来资金费率预期。

五、资金费率不是万能的

资金费率可以帮助永续价格锚定现货，但不是绝对有效。

在极端行情下，永续合约仍然可能长时间偏离现货价格。

原因包括：

现货流动性不足；

套利资金不够；

做空现货或借币困难；

杠杆风险过高；

资金费率有上限；

市场单边情绪太强；

交易所限制开仓或风控收紧。

所以资金费率的作用不是“强制归位”，而是“提供经济激励”。

六、Binance 上的 U 本位合约

Binance 上的 U 本位合约叫：

USD⑤-M Futures

常见例子：

BTCUSDT 永续

ETHUSDT 永续

SOLUSDT 永续

U 本位合约的特点：

保证金使用 USDT 或 USDC；

盈亏用 USDT 或 USDC 结算；

资金费率也用稳定币支付或收取；

合约通常是线性合约；

收益和亏损比较直观。

例如：

你做多 1 BTC 的 BTCUSDT 永续，开仓价 60,000 USDT，平仓价 61,000 USDT。

盈利：

1,000 USDT

你的账户增加的是 USDT，而不是 BTC。

七、Binance 上的币本位合约

Binance 上的币本位合约叫：

COIN-M Futures

常见例子：

BTCUSD 永续

ETHUSD 永续

BTCUSD 季度合约

ETHUSD 季度合约

币本位合约的特点：

保证金使用币本身，比如 BTC 或 ETH；

盈亏也用币结算；

资金费率用币支付或收取；

合约通常是反向合约；

适合长期持币者、矿工、机构对冲等场景。

例如：

你交易 BTCUSD 币本位永续，保证金是 BTC，盈利和亏损也都是 BTC。

如果做多 BTCUSD，BTC 上涨，你赚的是 BTC；

如果 BTC 下跌，你亏的也是 BTC。

八、U 本位和币本位的核心区别

最核心区别是：

U 本位看的是“我赚了多少 USDT”；

币本位看的是“我最后多了多少 BTC/ETH”。

对比：

\ 项目 | U 本位合约 | 币本位合约 |

\-----|-----|-----|

\ Binance 名称 | USDⓈ-M | COIN-M |

\ 示例 | BTCUSDT | BTCUSD |

\ 保证金 | USDT/USDC | BTC/ETH 等币 |

\ 盈亏结算 | USDT/USDC | BTC/ETH 等币 |

\ 合约类型 | 线性合约 | 反向合约 |

\ 资金费结算 | 稳定币 | 币 |

\ 适合对象 | 新手、短线、多币种、量化交易 | 长期持币者、矿工、机构对冲 |

\ 风险理解 | 更直观 | 更复杂 |

\ 保证金波动 | 较小 | 随币价波动 |

九、风险差异

U 本位合约的主要风险来自：

交易方向错误；

杠杆过高；

爆仓风险；

资金费率成本。

但保证金本身是稳定币，所以不会因为 BTC/ETH 价格变化而直接缩水。

币本位合约的风险更复杂。

因为保证金本身就是币。

如果你用 BTC 做保证金交易 BTCUSD 币本位合约，当 BTC 下跌时：

合约仓位可能亏损；

保证金的美元价值也在下降；

爆仓风险可能加速上升。

所以币本位合约在下跌行情里风险更非线性。

十、资金费率在两类合约里的表现

两类合约都有资金费率，但结算单位不同。

U 本位 BTCUSDT：

多头付空头时，支付的是 USDT。

币本位 BTCUSD：

多头付空头时，支付的是 BTC。

因此，套利收益的性质不同：

U 本位套利更像赚稳定币收益；

币本位套利更像赚币本位收益。

例如永续价格高于现货、资金费率为正时：

U 本位套利：

买现货 BTC + 空 BTCUSDT 永续，收 USDT 资金费。

币本位套利：

持有 BTC + 空 BTCUSD 永续，收 BTC 资金费。

十一、今日关键结论

1\、永续合约没有到期日，所以需要资金费率来替代交割机制。

2\、资金费率通过让偏离现货的一方持续付费，吸引反向套利者进场。

3\、虽然资金费率每隔 n 小时才结算一次，但市场会提前交易未来资金费率预期，所以它可以实时影响价格。

4\、U 本位合约使用 USDT/USDC 做保证金和结算，适合大多数交易者。

5\、币本位合约使用 BTC/ETH 等币做保证金和结算，适合长期持币者、矿工和机构对冲。

6\、U 本位关注稳定币收益，币本位关注币数量变化。

7\、币本位合约风险更复杂，因为保证金本身也会随币价波动。

十二、一句话总结

永续合约靠资金费率维持和现货的价格锚定；U 本位适合用稳定币衡量收益，币本位适合用 BTC/ETH 等币衡量收益。资金费率虽然定时结算，但由于市场会提前交易未来预期，所以它对价格锚定依然非常有效。

笔记 122: 8月11日 | 套利系统真正的地基：状态一致性

作者：Lier Mi | 时间：2026-08-11 14:53 UTC | 字数：1371 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月11日 | 套利系统真正的地基：状态一致性

今天继续深入链上套利系统中很容易被忽略、却决定计算结果是否可信的一层——状态同步。

套利计算依赖池子的价格、储备、流动性、Tick、余额等数据。但链上状态不是静态数据库，而是按照区块和交易顺序不断变化的状态机。

如果从 A 池读取的是区块 N 的数据，从 B 池读取的却是区块 N+1 的数据，就可能拼接出一条现实中从未同时存在过的套利路径。程序计算出来的利润虽然为正，但本质上只是“跨区块幻觉”。

因此，扫描机会时首先要保证：

所有池子状态必须来自同一个区块高度和同一个状态视图。

比较稳妥的方式，是在 RPC 请求中固定 blockTag，或者通过 Multicall 在一次调用中批量读取相关池子状态。读取完成后，还要记录对应的 blockNumber 和 blockHash，避免只知道高度，却不知道自己基于哪一条链分支计算。

高性能套利系统通常不会每个新区块都重新查询全部池子，而是维护一份本地状态：

1. 先在某个确定区块建立完整快照；
2. 订阅新区块中的 Swap、Mint、Burn 等事件；
3. 按照 transactionIndex 和 logIndex 的顺序更新池子；
4. 只重新计算受影响的交易路径；
5. 定期与链上状态校验，防止本地数据逐渐漂移。

这里的关键是：事件必须严格按照链上的真实执行顺序处理。同一区块内可能有多笔交易连续修改同一个池子。如果事件顺序错误，即使最终区块高度正确，中间计算出的储备和价格也可能完全错误。

此外还要处理区块重组。当新区块的 parentHash 与本地记录的区块哈希不一致时，说明之前接受的链头可能被替换。这时系统不能继续追加状态，而应该回滚到共同祖先，撤销原分支中的状态变化，再重放新分支的事件。

这让我意识到，套利系统至少需要区分三种状态：

Confirmed State：已经获得一定确认的稳定状态；

Latest State：当前最新区块执行完成后的状态；

Pending State：假设待处理交易按照某种顺序执行后的预测状态。

基于 Latest State 找到的机会相对确定，但通常竞争激烈；基于 Pending State 预测后续价差，速度更快，却依赖对交易排序和区块构建过程的判断。

交易模拟也应该分层进行：

第一层：使用本地数学模型快速筛选；

第二层：通过 eth_call 验证完整 calldata；

第三层：在主网分叉环境中验证余额、授权、Gas 和合约调用；

提交前：基于尽可能接近目标区块的状态再次模拟。

模拟时不仅要验证交易是否会回滚，还要检查执行后的真实余额变化。因为调用成功不等于套利成功，某些代币可能存在转账税、黑名单、重入限制或非标准返回值，都会让路由器返回的理论结果与钱包最终收到的资产不一致。

今天最大的收获是：套利系统的准确性，不取决于收集了多少数据，而取决于这些数据是否属于同一个真实世界。

路径搜索和利润优化建立在状态之上。状态一旦错误，后面的计算越精确，只会越精确地得到错误答案。

07 学习计划（3 篇）

笔记 123: Day 4 | The Graph、GraphQL 与历史链上数据

作者：HerschelLi-31 | 时间：2026-08-10 15:04 UTC | 字数：15403 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 4 | The Graph、GraphQL 与历史链上数据

1. 今天的核心目标

前几天的学习链条是：

Day 1：

```
\  
\  
Spread  
eq Profit\  
]
```

Day 2：

```
\  
\  
DisplayedPrice  
eq ExecutablePrice\  
]
```

Day 3：

```
\  
\  
ExecutableQuote  
eq GuaranteedExecution\  
]
```

Day 4 我开始解决一个新的问题：

```

\
\boxed{
过去某个时间点，链上到底发生了什么？
}\
]

```

例如我发现：

```

\
ETH\_{DEX\_A}=4000\
]
\
ETH\_{DEX\_B}=4020\
]

```

我不能只记录：

“曾经出现过 20 美元价差。”

真正做 Historical Analysis 时，我希望知道：

哪一笔 Swap 制造了价差？

当时是哪一个 Block？

Pool 中发生了什么？

Swap Amount 有多大？

谁随后执行了 Arbitrage？

Arbitrage 持续了多少 Block？

最终是谁把价格重新拉平？

如果当时换成我执行，成本是多少？

这就要求我获得：

```

\
\boxed{
Historical\ Structured\ On-chain\ Data\
}\
]

```

The Graph 的 Subgraphs 正是把 blockchain 中指定协议的数据进行 indexing，并通过 GraphQL 结构化查询。

2. 为什么不能直接只用 Blockchain Explorer？

例如使用 Etherscan，我当然可以看到：

```

Transaction
Block
From
To
Gas
Logs

```

但如果我想回答：

“过去 30 天 ETH/USDC Pool 中所有大于 \$100,000 的 Swap 是什么？”

手工浏览 Explorer 基本不可行。

我真正需要的是：

```

\
Blockchain\
\rightarrow\
StructuredDatabase\
\rightarrow\
Query\
]

```

也就是：

```

Raw On-chain Events
↓

```

```

Indexer
↓
Subgraph
↓
GraphQL
↓
Python / Research

```

这正是 The Graph 的主要作用。

3. 什么是 Subgraph ?

可以把 Subgraph 暂时理解成：

针对某一个 Blockchain Protocol 提前整理好的可查询数据库/API。

例如某个 DEX 的 Smart Contract 会不断产生：

```

Swap
Mint
Burn
Transfer
等 Event。

```

如果自己从 Ethereum Genesis Block 开始读取所有 Logs，再判断哪些属于某个 DEX：

```

\[\
成本很高\
]

```

而 Subgraph 可以预先指定：

```

哪些 Contract 要监听
哪些 Event 要处理
Event 如何转成结构化 Entity
Entity 之间是什么关系

```

The Graph 当前的 Subgraph manifest subgraph.yaml 用来指定要索引的 network、smart contract 和 events，schema.graphql 则定义最终保存和查询的数据结构。

因此：

```

\[\
\boxed{\
Subgraph\
eq\
Blockchain\
}\
]

```

它更像：

```

\[\
\boxed{\
Blockchain\ Data\ View\
}\
]

```

4. 从 Blockchain 到 Subgraph Data

假设一个 AMM Pool 发生：

```

\[\
Alice:\
100000USDC\
\rightarrow\
ETH\
]

```

Smart Contract 执行 Swap 后可能产生：

```
\[\nSwapEvent\
```

里面可能包含类似：

```
sender\namount0\namount1\nrecipient\n...
```

Subgraph 的 Mapping 会读取这个 Event：

```
\[\nSwapEvent\
```

↓

把它转换成：

```
\[\nSwapEntity\
```

↓

存入 Indexed Data。

最终研究者可以通过 GraphQL 查询：

```
{\n  swaps(first: 10) {\n    id\n    timestamp\n    amount0\n    amount1\n  }\n}
```

具体能查询哪些字段取决于该 Subgraph 自己的 schema.graphql，因为所有 GraphQL query 都是针对该 Subgraph 定义的数据模型运行的。

5. 什么是 Entity？

Entity 可以粗略理解成：

```
\[\n  \boxed{\n    Database\ Record/Object\n  }\n]
```

例如一个 DEX Subgraph 可能定义：

```
Token\nPool\nSwap\nTransaction\nLiquidityPosition
```

这些都是不同 Entity。

例如：

Swap

可能包含：

```
id\ntimestamp\npool\nsender
```

amount0
amount1
transaction

而：

Pool

可能包含：

id
token0
token1
liquidity
volume

The Graph 官方建议把 Entity 理解成“包含数据的 objects”，而不是简单把它等同于 Event 本身。

因此：

```
\\  
BlockchainEvent\  
]
```

是原始事实，

而：

```
\\  
SubgraphEntity\  
]
```

是经过 indexing / mapping 后更方便研究的结构化表示。

6. 什么是 GraphQL？

GraphQL 是一种 Query Language。

和 REST API 不同。

REST 可能：

GET /swaps

直接返回预定义字段。

GraphQL 则允许我告诉服务器：

我具体想要哪些字段。

例如：

```
{  
  swaps(first: 5) {  
    id  
    timestamp  
    amount0  
    amount1  
  }  
}
```

意思是：

给我 5 个 Swap，只需要这些字段。

因此 GraphQL 特别适合复杂链上数据研究，因为我不需要每次下载整个对象。

7. GraphQL Query 的基本结构

例如：

```
{  
  swaps(first: 10) {  
    id  
    timestamp  
    amount0  
    amount1  
  }  
}
```

```
}  
}
```

可以拆成：

swaps

我要查询：

```
\\  
Swap\ Entity\ Collection\  
]  
first: 10
```

返回：

```
\\  
10\  
]
```

条数据。

Fields

id
timestamp
amount0
amount1

告诉 GraphQL：

我只需要这些字段。

The Graph 当前 GraphQL API 支持对 Entity collection 使用 first、排序、过滤以及 pagination 等操作。

8. first

最基础：

swaps(first: 100)

表示：

```
\\  
返回前100条\  
]
```

但是要特别注意：

默认顺序不一定是你想象中的“最新 100 条”。

The Graph 当前文档明确说明 collection 默认按照：

```
\\  
ID\ ascending\  
]
```

排序，而不是按照 creation time。

因此如果我要找最新 Swap：

应该显式使用类似：

```
swaps(  
first: 100  
orderBy: timestamp  
orderDirection: desc  
)
```

具体支持的字段仍取决于 Subgraph Schema。

9. orderBy

假设我要：

最新发生的 100 笔 Swap

可以使用类似：


```
{
  swaps(
    first: 100
    orderBy: timestamp
    orderDirection: desc
  ){
    id
    timestamp
    amount0
    amount1
  }
}
```

逻辑：

```
\[
  timestamp:\
  latest\
  \rightarrow\
  oldest\
]
```

这在套利研究中非常重要。

因为通常我的数据单位是：

```
\[
  TimeSeries\
]
```

10. Filter : where

真正的数据研究一定需要 Filter。

例如假设某个 Schema 支持相应字段，我可能希望：

只查询某一个 Pool 的 Swap。

形式可能类似：

```
{
  swaps(
    first: 100
    where: {
      pool: "0x..."
    }
  ){
    id
    timestamp
    amount0
    amount1
  }
}
```

于是：

```
\[
  AllSwaps\
]
```

变成：

```
\[
  SwapsOfTargetPool\
]
```

Filter 是以后 Scanner 和 Historical Research 最重要的 GraphQL 能力之一。

The Graph 支持对 entity field 和关联 entity 进行各种 filtering；具体 filter 名称依然由 Schema 生成。

11. 时间过滤

假设研究：

```
\\  
t\_0\<t\<t\_1\  
]
```

可以在支持 timestamp field 的 Schema 中使用类似：

```
where: {  
  timestamp_gte: 1720000000  
  timestamp_lt: 1720100000  
}
```

这里：

_gte

表示：

```
\\  
greater\ than\ or\ equal\  
]
```

而：

_lt

表示：

```
\\  
less\ than\  
]
```

于是我可以研究：

```
\\  
\boxed{\  
特定历史时间窗口\  
}\
```

这比：

“下载整个协议历史数据再自己筛选”

高效很多。

12. Pagination

如果我要研究：

```
\\  
1,000,000\  
]
```

条 Swap，不可能一次 Query 全部返回。

因此需要：

```
\\  
Pagination\  
]
```

例如：

```
swaps(  
  first: 1000  
)
```

然后：

```
swaps(  
  first: 1000  
  skip: 1000  
)
```

然后：

skip: 2000

依次继续。

The Graph 当前官方文档仍支持 first 和 skip pagination，并指出多数 Subgraph 查询会受到单次返回数量限制，因此大规模分析需要多次请求。

13. 为什么套利数据采集一定要考虑 Pagination？

因为：

```
\\  
1000\ Swaps\  
]
```

很可能只覆盖：

```
\\  
几分钟 / 几小时\  
]
```

的活跃市场。

如果我误以为：

Query 返回 1000 条，所以已经拿到了完整历史。

后面的 Backtest 就会产生严重的：

```
\\  
\boxed\  
Data\ Truncation\  
\\  
]
```

于是可能错误地认为：

```
\\  
OpportunityFrequency\  
]
```

很低或很高。

所以数据研究必须确认：

```
\\  
\boxed\  
DatasetCoverage\  
\\  
]
```

包括：

Start Time
End Time
Number of Records
Missing Period
Pagination Complete?

14. Historical Block Query 的意义

套利研究并不总是只需要：

```
\\  
CurrentState\  
]
```

有时我要问：

某个历史 Block 时这个 Entity 是什么状态？

The Graph 的 GraphQL API 支持在兼容的 Subgraph 查询中指定历史 block，例如按照 block number 或 hash 获取特定历史状态。

概念上：

```
{  
  pool(  
    id: "..."
```

```
block: { number: 12345678 }  
{  
...  
}
```

这意味着：

```
\\  
State(Block\_n)\  
]
```

可以与：

```
\\  
State(Block\_n)\
```

进行比较。

对套利研究非常重要。

15. 为什么 Block 比 Timestamp 更基础？

Blockchain 的真实状态变化发生在：

```
\\  
Block\  
]
```

中。

例如：

```
Block N  
|—— TX 1  
|—— TX 2  
|—— TX 3  
|—— TX 4
```

某个大型 Swap 可能发生在：

```
\\  
Block\_N\  
]
```

然后 Arbitrage 也可能发生在同一个 Block。

如果只使用：

```
\\  
timestamp\  
]
```

可能看不出 transaction ordering。

所以以后研究 execution 时，应该逐渐从：

```
\\  
Time\  
]
```

升级成：

```
\\  
\boxed{\  
Block + TransactionOrder\  
}\
```

Day 4 暂时先掌握 Block-level Analysis。

16. The Graph 与 RPC 有什么区别？

这是 Day 4 最重要的概念之一。

The Graph

更适合：

```
\\  
HistoricalAnalysis\  
]
```

例如：

过去 30 天发生了多少 Swap？

哪些交易规模最大？

Pool Volume 如何变化？

某段时间有哪些套利案例？

RPC

更接近直接询问 Blockchain Node。

例如：

```
eth_call  
eth_getLogs  
eth_getBlockByNumber  
eth_getTransactionReceipt
```

更适合：

```
\\  
CurrentState\  
]
```

以及：

```
\\  
Execution-critical\ Verification\  
]
```

因此：

```
\\  
\boxed{\  
TheGraph\  
eq\  
RPC\  
}\\  
]
```

更合理的架构是：

Research / History

↓

The Graph

Execution / Verification

↓

RPC

这也是为什么 Day 5 会专门进入 Node / RPC。

17. The Graph 的数据为什么不能无条件当成“链本身”？

因为：

```
\\  
SubgraphData\  
]
```

经历：

Blockchain

↓

Indexer

↓

Mapping

↓

Schema

↓

GraphQL

所以研究者真正看到的是：

```
\[\
\boxed{\
IndexedRepresentation\
}\
]
```

而不是直接读取 raw blockchain state。

因此以后如果是：

是否现在立刻执行 \$100,000 Arbitrage？

不能只因为 Subgraph 显示某个值，就直接下单。

Execution-critical data 应进一步通过：

```
\[\
RPC\
]
或：
\[\
ExecutableQuote\
]
```

确认。

18. Graph Explorer

今天不需要自己创建 Subgraph。

The Graph 当前已经有大量公开 Subgraphs，可以通过 Graph Explorer 搜索并进入 Query Playground 测试查询。官方建议已有适合需求的 Subgraph 时优先使用现有 Subgraph，而不是立即从零搭建。

所以今天路线是：

```
Graph Explorer
↓
Find Public Subgraph
↓
Inspect Schema
↓
Run GraphQL Query
↓
Python
```

而不是：

Build Own Subgraph

后者目前不是套利学习的重点。

19. 查询 Endpoint

当前 The Graph Network 中 Subgraph 的 query endpoint 形式类似：

https://gateway.thegraph.com/api/<API_KEY>/subgraphs/id/<SUBGRAPH_ID>

官方说明发布在 Network 上的 Subgraph 可以通过这样的 endpoint 被 GraphQL client 或应用查询。

因此 Python 程序最终就是：

```
Python
↓
HTTP POST
↓
```

Graph Endpoint

↓

GraphQL

↓

JSON

20. 使用 Python 查询 GraphQL

最简单可以继续使用：

```
import requests
```

例如：

```
query = """
```

```
{  
  swaps(first: 10) {
```

```
    id
```

```
    timestamp
```

```
    amount0
```

```
    amount1
```

```
  }  
}
```

```
"""
```

```
response = requests.post(  
  GRAPH_URL,
```

```
  json={"query": query}
```

```
)
```

```
data = response.json()
```

这里真正发送的是：

```
{
```

```
  "query": "..."
```

```
}
```

服务器返回：

JSON

然后 Python 再处理：

JSON

→

pandas

→

DataFrame

这就是 Day 4 最基本的数据 Pipeline。

21. 一个重要习惯：先看 Schema，再写 Query

不同 Protocol 的 Subgraph：

```
\
```

```
\
```

```
Schema\A
```

```
eq Schema\B\
```

```
]
```

所以不要从网上复制：

```
swaps {
```

```
  amount0
```

```
  amount1
```

```
}
```

然后认为所有 DEX 都能运行。

必须先打开：


```
\[\n  schema.graphql\n]
```

或者 Graph Explorer 的 Schema。

查看：

```
Entity Name\nField Name\nField Type\nRelationship
```

因为 GraphQL 能查询哪些字段完全由目标 Subgraph Schema 决定。

所以工作顺序应该是：

```
\[\n  \boxed{\n    ResearchQuestion\n    \rightarrow\n    Schema\n    \rightarrow\n    Query\n  }\n]
```

而不是：

```
\[\n  Query\n  \ Template\n  \rightarrow\n  Hope\n  \ It\n  \ Works\n]
```

22. Day 4 Experiment 1 | 获取最新 Swap

选择一个 DEX Subgraph。

首先完成：

```
\[\n  Latest\n  \ 100\n  \ Swaps\n]
```

保存：

```
id\ntimestamp\ntransaction\namount\npool
```

具体字段根据 Schema 调整。

目标不是分析套利。

只是确保：

```
\[\n  GraphQL\n  \rightarrow\n  Python\n  \rightarrow\n  DataFrame\n]
```

pipeline 能跑通。

最终：

```
swaps.csv
```

23. Experiment 2 | 查询指定 Pool

选择：

```
\\  
ETH/USDC\  
]
```

或其他 liquid pair。

只查询：

```
\\  
TargetPool\  
]
```

然后按照：

```
\\  
timestamp\  
]
```

排序。

输出：

```
| Timestamp | Token0 Amount | Token1 Amount | Tx |  
| :----- | :----- | :----- | :----- |  
| <br /> | <br /> | <br /> | <br /> |  
| <br /> | <br /> | <br /> | <br /> |  
| <br /> | <br /> | <br /> | <br /> |
```

目标：

第一次建立自己的 Historical Swap Dataset。

24. Experiment 3 | 识别 Large Swap

假设定义：

```
\\  
LargeSwap  
=====  
Notional>$100,000\  
]
```

今天不必追求完美 USD conversion。

先尝试利用 available fields 找：

```
\\  
largest\ swaps\  
]
```

然后观察：

```
\\  
BeforeLargeSwap\  
]
```

和：

```
\\  
AfterLargeSwap\  
]
```

市场发生了什么。

这是向：

```
\\  
ArbitrageCaseReconstruction\  
]
```

迈出的第一步。

25. 为什么 Large Swap 对套利特别重要？

Day 2 已经学到：

```

\\
TradeSize\uparrow\
\Rightarrow\
PriceImpact\uparrow\
]

```

因此一个大的 Swap :

```

\\
LargeTrade\
]

```

可能造成 :

```

\\
Pool\_APrice\
]

```

快速移动。

但另外一个 Pool :

```

\\
Pool\_BPrice\
]

```

暂时没动。

于是 :

```

\\
P\_A
eq P\_B\
]

```

产生 :

```

\\
\boxed{\
TemporaryArbitrageOpportunity\
}\
]

```

所以可以形成一个非常重要的因果链 :

```

\\
LargeSwap\
]

```

↓

```

\\
PoolStateChanges\
]

```

↓

```

\\
PriceDislocation\
]

```

↓

```

\\
Arbitrage\
]

```

↓

```

\\
PriceConvergence\
]

```

未来很多链上套利研究其实就是不断寻找、验证这种 :

```

\
\boxed{
Event\rightarrowStateChange\rightarrowOpportunity\
}\
]

```

26. Experiment 4 | 简单 Historical Case

找到某笔较大的 Swap :

```

\
TX\_A\
]

```

记录 :

```

Block
Timestamp
Pool
Amount
Transaction Hash

```

然后观察附近 :

```

\
TX\_A-?\
]

```

和 :

```

\
TX\_A+?\
]

```

尝试回答 :

1. 大 Swap 前发生了什么 ?
2. 大 Swap 改变了什么 ?
3. 后面是否马上出现其他 Swap ?
4. 有没有可能是 Arbitrage ?
5. Price 是否快速重新收敛 ?

今天不要求证明。

只需要提出 :

```

\
Hypothesis\
]

```

27. Day 4 Collector

目标文件 :

src/collectors/graph.py

整体结构 :

```

Research Question
↓
GraphQL Query
↓
POST Request
↓
JSON Response
↓
Validation
↓
Pagination
↓
Normalize

```

↓
Save Raw
↓
Processed Dataset

建议至少实现：

query_graph(...)

和：

fetch_swaps(...)

后面再逐渐加入：

fetch_pool(...)

fetch_transactions(...)

fetch_liquidity(...)

28. Raw Data 仍然必须保存

与 Day 3 一样：

```
data/  
├── raw/  
│   └── graph/  
│  
└── processed/  
    └── swaps.parquet
```

原则：

```
\\  
\boxed{\  
RawData\immutable\  
}\  
]
```

如果后面发现：

原来某字段很重要。

应该：

```
\\  
Raw\  
\rightarrow\  
Reprocess\  
]
```

而不是重新依赖远程 API 获取过去的。

29. API Key 安全

如果使用 Gateway API key：

不要：

GRAPH_API_KEY = "实际key"

直接硬编码。

而应该：

import os

GRAPH_API_KEY = os.getenv("GRAPH_API_KEY")

并将：

.env

加入：

.gitignore

这是以后整个 Arbitrage Lab 都应该遵守的规则：

```

\
\boxed{
Secrets
eqSourceCode\
}\
]

```

30. The Graph 在整个 Arbitrage Lab 中的位置

到目前为止已经出现三个 Data Source :

AMM Model

告诉我 :

```

\
WhatShouldHappen\
]

```

LI.FI Quote

告诉我 :

```

\
WhatCouldExecuteNow\
]

```

The Graph

告诉我 :

```

\
WhatHappenedHistorically\
]

```

因此 :

```

\
\boxed{
Model + Quote + HistoricalData\
}\
]

```

开始形成完整研究框架。

后面还会加入 :

RPC

告诉我 :

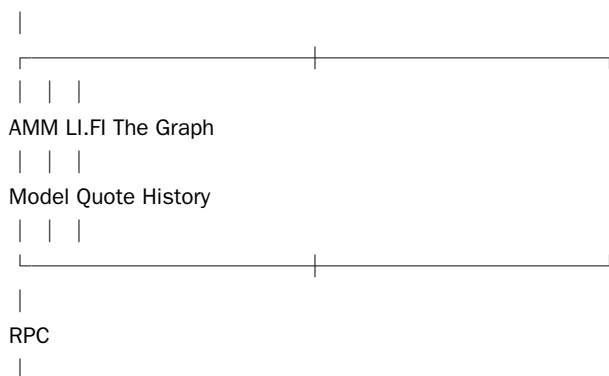
```

\
WhatIsActuallyOnChainNow\
]

```

最终变成 :

Opportunity Research



31. Day 4 验收题

Q1

为什么 The Graph 比 Block Explorer 更适合历史套利研究？

Answer

因为 Explorer 更适合查看单个：

```
\\  
Transaction\  
]
```

而 The Graph 可以把大量 blockchain data 结构化后，通过：

```
\\  
Filter\  
+\  
Sort\  
+\  
Pagination\  
]
```

进行批量查询。

所以更适合：

```
\\  
HistoricalAnalysis\  
]
```

和：

```
\\  
Backtesting\  
]
```

Q2

Subgraph 是否等于 Blockchain？

Answer

不是。

Subgraph 是：

```
\\  
BlockchainData\  
]
```

经过：

```
\\  
Indexing\  
+\  
Mapping\  
+\  
Schema\  
]
```

产生的结构化 representation。

因此：

```
\\  
Subgraph  
eqRawBlockchain\  
]
```

Q3

为什么写 GraphQL Query 之前应该先看 Schema？

Answer

因为不同 Subgraph 定义的：

```
\\  
Entity\  
]
```

和：

```
\\  
Fields\  
]
```

不同。

GraphQL 能查询什么完全取决于：

```
\\  
schema.graphql\  
]
```

所以不能假设所有 DEX 都存在同样的 swaps.amount0 等字段。

Q4

为什么 Pagination 对套利回测非常重要？

Answer

因为单次 Query 通常只能返回有限数量的数据。

如果没有完整 Pagination：

```
\\  
HistoricalDataset\  
]
```

可能被截断。

于是：

```
\\  
OpportunityFrequency\  
]
```

```
\\  
PnL\  
]
```

```
\\  
Volume\  
]
```

等统计都可能错误。

Q5

The Graph 和 RPC 最核心的区别是什么？

Answer

简单理解：

```
\\  
TheGraph\  
]
```

擅长：

```
\\  
HistoricalStructuredAnalysis\  
]
```

而：

```
\\  
RPC\  
]
```

更接近：


```
\\  
DirectBlockchainState\  
]
```

因此：

```
\\  
Research \rightarrow TheGraph\  
]
```

```
\\  
ExecutionVerification \rightarrow RPC\  
]
```

Q6

为什么 Large Swap 值得套利研究者重点观察？

Answer

因为：

```
\\  
LargeSwap\  
]
```

可能产生较大的：

```
\\  
PriceImpact\  
]
```

使某一个 Pool 的价格快速改变，而其他市场暂时还没有同步，从而产生：

```
\\  
TemporaryPriceDislocation\  
]
```

因此：

```
\\  
LargeSwap\  
\rightarrow\  
PriceDislocation\  
\rightarrow\  
Arbitrage\  
\rightarrow\  
Convergence\  
]
```

是很常见的研究框架。

32. Day 4 最终产出

今天至少完成：

src/collectors/graph.py

以及：

data/raw/graph/

保存原始 GraphQL response。

生成：

data/processed/swaps.csv

或：

swaps.parquet

至少完成以下三个 Query：

1. Latest Swaps
2. Swaps from One Target Pool
3. Large / Interesting Swap Sample

然后挑出：

```
\[\n1\]\n]
```

笔 Transaction 作为：

```
\[\nHistoricalCaseCandidate\]\n]
```

保存：

Transaction Hash
Block
Timestamp
Pool
Swap Size

为第二周 Historical Case Reconstruction 做准备。

33. Day 4 总结

Day 3 我学会：

```
\[\nQuote(t,Q)\]\n]
```

开始观察当前 Market State。

Day 4 我开始获得：

```
\[\n\boxed{\nHistoricalMarketData\}\n]\n]
```

真正重要的升级是：

```
\[\nSingleObservation\ \rightarrow\nHistoricalDataset\]\n]
```

研究单位也开始从：

```
\[\nPrice\]\n]
```

进一步升级成：

```
\[\n\boxed{\nEvent\ \rightarrow\nBlock\ \rightarrow\nStateChange\}\n]\n]
```

以后分析套利，我不仅要问：

“现在有没有价差？”

还要问：

历史上这种价差什么时候出现、是什么 Event 制造的、持续多久、谁把它消除、什么规模下还能赚钱？

因此 Day 4 建立的核心链条是：

```
\[\n\boxed{\nBlockchainEvents\n\rightarrow\nSubgraph\n\rightarrow\nGraphQL\n\rightarrow\nHistoricalDataset\n\rightarrow\nCaseReconstruction\n}\n]
```

到这里，前四天已经开始形成一套完整的研究逻辑：

```
\[\n\boxed{\nDay1:\ Why\ does\ edge\ exist?\n}\n]
```

```
\[\n\boxed{\nDay2:\ How\ does\ liquidity\ change\ executable\ price?\n}\n]
```

```
\[\n\boxed{\nDay3:\ What\ can\ I\ execute\ now?\n}\n]
```

```
\[\n\boxed{\nDay4:\ What\ actually\ happened\ historically?\n}\n]
```

下一步 Day 5 的 Node / RPC 会解决最后一个非常关键的问题：

```
\[\n\boxed{\nWhat\ is\ the\ blockchain\ state\ itself?\n}\n]
```

也就是开始真正区分：

```
\[\nIndexedData\n]
```

和：

```
\[\nDirectOnChainState\n]
```

为后面的 Arbitrage Scanner、Replay Engine 和 Execution Verification 打基础。

笔记 124: 调整学习计划，解码 Vivek 样本 6389... 的 Raydium → Orca 两腿。结果：

作者：Wang Hua | 时间：2026-08-11 01:44 UTC | 字数：592 | 评分：72

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

调整学习计划，解码 Vivek 样本 6389... 的 Raydium → Orca 两腿。结果：

1\、只读 RPC 拉取锚点交易完整 JSON (slot 123381908, 2022-03-04 03:13:07 UTC, 成功, fee 5000 lamports), 存为 raw/tx-6389.json

2\、编写可复用解码器 scripts/decode_solana_swap_tx.py, 从 pre/postTokenBalances 原始整数重建两条腿：

\- 腿1 Raydium AMM V4 : 281,593,608 USDC → 34,914,934,852 AURY

\- 腿2 Orca : 34,914,934,810 AURY → 288,648,732 USDC

3\、毛差 = 7,055,124 raw = 7.055124 USDC, 与 Vivek 原文完全一致

4\、成本口径：链费 0.000005 SOL/笔；锚点前 20 笔失败费 0.0001 SOL (复用 D7 雷达数据)；SOL 2022-03-04 收盘 \$88.25 (Yahoo, 与 Vivek 文中 \$88 交叉一致)；摊入失败成本后净收益下界约 +7.0459 USD

5\、新增 6 项测试，全仓 27 项测试通过

笔记 125: Day 5 | RPC、Node 与 Direct On-chain State

作者：HerschelLi-31 | 时间：2026-08-11 14:07 UTC | 字数：17005 | 评分：86

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 5 | RPC、Node 与 Direct On-chain State

1. 今天的学习目标

前四天的研究链条已经是：

\

Day1:\ WhyDoesEdgeExist?\

}

\

Day2:\ HowDoesLiquidityChangePrice?\

}

\

Day3:\ WhatCanExecuteNow?\

}

\

Day4:\ WhatHappenedHistorically?\

}

Day 5 开始解决：

\

\boxed{\

WhatIsActuallyOnChain?\

}\

}

也就是说：

如果 The Graph、LI.FI 或某个网站告诉我一个价格、Pool State 或 Transaction 信息，我能不能直接从 Blockchain Node 验证？

今天需要理解并实际使用：

RPC

Node

Block

Transaction

Receipt

Log

eth_call

并重点掌握四个 JSON-RPC 方法：

eth_getBlockByNumber

eth_getTransactionReceipt

eth_call

eth_getLogs

2. 什么是 Blockchain Node ?

Blockchain 并不是一个中心服务器。

Ethereum 等区块链由许多：

```
\\  
Node\  
]
```

共同维护。

Node 保存并验证：

Blocks
Transactions
Smart Contract State
Logs
Account Balances

等信息。

因此，如果我想知道：

Ethereum 当前 Block 是多少？

某笔 Transaction 是否成功？

某个 Smart Contract 现在的储备状态是什么？

我真正需要询问的是：

```
\\  
\boxed{\  
BlockchainNode\  
}\
```

3. 什么是 RPC ?

RPC :

```
\\  
Remote\Procedure\Call\  
]
```

可以简单理解为：

我向远程 Blockchain Node 发送一个标准化请求，让 Node 帮我执行某个查询操作。

结构：

My Python Program

↓

RPC

↓

Ethereum Node

↓

Blockchain Data

↓

JSON Response

例如：

eth_getBlockByNumber

就是：

请 Node 给我某一个 Block 的信息。

4. RPC Provider 是什么？

实际研究时，我一般不会一开始就在自己的 Mac 上运行完整 Ethereum Node。

因为 Full Node 需要：

Storage
Bandwidth
Sync Time
Maintenance

所以通常通过 RPC Provider 访问 Node。

概念上：

Python
↓
RPC URL
↓
Provider
↓
Ethereum Node

因此以后配置文件里可能保存：

ETH_RPC_URL
ARB_RPC_URL
BASE_RPC_URL

这些 URL。

但不要把私人 API Key 直接写进公开 Source Code。

正确方式：

.env

例如：

```
import os  
RPC_URL = os.getenv("ETH_RPC_URL")
```

5. RPC 与 The Graph 的根本区别

Day 4 使用：

```
\\  
TheGraph\  
]
```

获得：

```
\\  
IndexedData\  
]
```

Day 5 使用：

```
\\  
RPC\  
]
```

直接询问：

```
\\  
BlockchainNode\  
]
```

所以数据路径不同。

The Graph

```
\\  
Blockchain\  
\rightarrow\  
Indexer\  
\rightarrow\  
Subgraph\  
\rightarrow\  
GraphQL\  
]
```

```

]
RPC
\
BlockchainNode\
\rightarrow\
RPC\
\rightarrow\
MyProgram\
]

```

因此：

```

\
\boxed{\
RPC\ 更接近 Blockchain Source of Truth\
}\
]

```

6. The Graph 和 RPC 各自适合什么？

不能简单认为：

RPC 更真实，所以以后不用 The Graph。

两者解决的问题不同。

The Graph

更适合：

```

\
HistoricalSearch\
]

```

例如：

过去一个月有多少 ETH/USDC Swap？

一次研究可能需要：

```

\
100,000+\
]

```

条 Transaction。

The Graph 已经帮我建立结构化 Index。

RPC

更适合：

```

\
DirectVerification\
]

```

例如：

Block 23123456 的 Transaction 到底发生了什么？

当前 Contract 调用会返回什么？

某笔交易到底 Success 还是 Revert？

所以更合理的架构：

```

\
\boxed{\
Discovery\
\rightarrow\
TheGraph\
}\
]

```

然后：

```

\
\boxed{
Verification\
\rightarrow\
RPC\
}\
]

```

7. 什么是 Block ?

Blockchain 可以理解成 :

```

\
Block\_1\
\rightarrow\
Block\_2\
\rightarrow\
Block\_3\
\rightarrow\
\cdots\
]

```

每一个 Block 中包含 :

```

\
Transactions\
]

```

例如 :

Block N

```

|—— Transaction 1
|—— Transaction 2
|—— Transaction 3
|—— Transaction 4
|—— ...

```

每个新 Block 都会导致 Blockchain State 从 :

```

\
S\_N\
]

```

变成 :

```

\
S\_N+1\
]

```

因此可以抽象为 :

```

\
\boxed{
State\_N\
\xrightarrow{Transactions}\
State\_N+1\
}\
]

```

这个模型对套利极其重要。

8. 套利为什么必须理解 Block ?

假设 :

Block (N) 开始时 :

```

\
DEX\_A:\
ETH=4000\

```



```

]
\\
DEX\_B:\
ETH=4000\
]
然后：
TX1
有人在 DEX A 大量买 ETH。
导致：
\\
P\_A:\
4000\rightarrow4050\
]
此时：
\\
P\_A
eq P\_B\
]
产生 Arbitrage Opportunity。
TX2
Searcher：
\\
Buy\_B\
+\
Sell\_A\
]
执行 Arbitrage。
于是：
\\
P\_A\downarrow\
]
\\
P\_B\uparrow\
]
最终恢复：
\\
P\_A\approx P\_B\
]
所以套利不是简单：
\\
Price(t)\
]
问题。
而是：
\\
\boxed{\
State\
\rightarrow\{TX\}\
NewState\
}\

```

]

的问题。

9. 一个重要概念：Blockchain State

可以把 Ethereum 在 Block (N) 时的完整状态写成：

```
\[\nS\\N\\n]
```

其中包含：

Account Balances

Contract Storage

Token Balances

Pool State

Debt Positions

...

Block 中执行 Transaction 后：

```
\[\nS\\N\\n\\xrightarrow{TX\\_1}\\nS\\N'\\n]
```

然后：

```
\[\nS\\N'\\n\\xrightarrow{TX\\_2}\\nS\\N''\\n]
```

最终得到：

```
\[\nS\\{N+1}\\n]
```

所以以后 Arbitrage Replay 真正应该做的是：

```
\[\n\\boxed{\nReplayStateTransitions\n}\\n]
```

而不是只看：

```
\[\nOHLC\\Candle\\n]
```

10. eth_getBlockByNumber

这是今天第一个要掌握的方法。

它的作用是：

根据 Block Number 获取 Block 信息。

概念调用：

eth_getBlockByNumber

输入：

block number

输出可能包括：

number
hash
parentHash
timestamp
gasUsed
gasLimit
transactions

为什么套利研究需要它？

因为我可以知道：

```
\\  
Block\_N\  
]
```

什么时候产生。

其中：

```
\\  
多少笔Transaction\  
]
```

以及：

```
\\  
哪些Transaction进入这个Block\  
]
```

于是可以开始构造：

```
\\  
\boxed\  
BlockTimeline\  
}\\  
]
```

11. Block Number 为什么常常是 Hexadecimal？

RPC response 经常不是：

12345678

而是：

0xbc614e

这种：

```
\\  
Hexadecimal\  
]
```

也就是 Base 16。

Python 转换：

```
block_number = int(hex_value, 16)
```

因此以后看到：

0x...

不要自动认为：

这是 Contract Address。

很多 RPC 数字同样使用 hexadecimal encoding。

12. Transaction 是什么？

Transaction 可以简单理解成：

一个账户请求 Ethereum 改变 State 的指令。

例如：

Alice

→

Bob

→

1 ETH

或者：

Alice

→

Uniswap Router

→

Swap USDC for ETH

一笔 Transaction 通常包含：

hash

from

to

value

input

gas

nonce

...

其中：

\[\

input\

]

非常重要。

因为 Smart Contract Transaction 的：

\[\

input/data\

]

中编码了：

到底调用哪个 Contract Function，以及输入了哪些 Parameters。

13. Transaction Hash

每笔 Transaction 都有唯一：

\[\

TransactionHash\

]

例如：

0xabc123...

它可以看成：

\[\

\boxed{\

TransactionID\

}\

]

所以 Day 4 找到某笔 Large Swap 后：

\[\

TransactionHash\

]

就是 Day 5 使用 RPC 进一步还原它的入口。

14. 什么是 Transaction Receipt？

Transaction 本身告诉我：

用户想做什么。

Receipt 更重要的是告诉我：

执行以后发生了什么。

因此：

```
\\  
Transaction\  
eq\  
TransactionReceipt\  
]
```

Transaction：

```
\\  
Intent\  
]
```

Receipt：

```
\\  
ExecutionResult\  
]
```

15. eth_getTransactionReceipt

输入：

```
\\  
TransactionHash\  
]
```

返回 Receipt。

里面特别重要的字段包括：

status
blockNumber
gasUsed
logs
contractAddress

16. status

这是最容易理解但非常重要的字段。

例如：

status = 0x1

通常意味着：

```
\\  
Success\  
]
```

而：

status = 0x0

意味着：

```
\\  
Failed/Reverted\  
]
```

所以如果研究：

一个套利 Bot 发出了多少次机会？

只数 submitted transactions 不够。

必须区分：

```
\\  
SuccessTX\  
]
```

和：

```

\\
FailedTX\
]

```

17. 为什么 Failed Transaction 对套利特别重要？

Day 1 的 Net Profit 模型中已经有：

```

\\
FailedTransactionCost\
]

```

例如套利程序发现：

```

\\
100\
]

```

次机会。

理论每次利润：

```

\\
$5\
]

```

理论：

```

\\
$500\
]

```

但其中：

```

\\
30\
]

```

笔失败。

每次 Failed TX 消耗：

```

\\
$4\
]

```

那么：

```

\\
FailureCost
=====

```

30\times4

```

$120\
]

```

所以：

```

\\
ExpectedProfit\
]

```

不能只分析 successful trades。

这也是为什么后面要研究：

```

\\
\boxed{
FailureProbability\
}\
]

```

18. gasUsed

Receipt 中还有：

```
\
gasUsed\
]
```

表示这笔 Transaction 实际执行消耗了多少 Gas Unit。

注意：

```
\
GasUsed\
]
```

不是：

```
\
GasFee\
]
```

真实手续费还需要结合：

```
\
EffectiveGasPrice\
]
```

粗略：

```
\
\boxed{
GasCost
=====
GasUsed\times EffectiveGasPrice\
}\
]
```

然后再按照 ETH Price 转成：

```
\
USD\
]
```

19. 为什么 Gas 对套利影响巨大？

假设：

```
\
GrossProfit=$20\
]
```

如果：

```
\
Gas=$3\
]
```

套利仍然不错。

但 Ethereum 拥堵时：

```
\
Gas=$25\
]
```

那么：

```
\
NetProfit=-$5\
]
```

所以：

```
\
\boxed{
Opportunity
=====
```

```
MarketState\
+\
GasState\
}\
]
```

Gas 本身就是套利市场状态的一部分。

20. 什么是 Log ?

Smart Contract 在执行 Transaction 时可以产生 :

```
\
Event\
]
```

这些 Event 最终以 :

```
\
Logs\
]
```

形式写进 Transaction Receipt。

例如 Uniswap Pool Swap 时可能 emit :

```
\
Swap(...)\
]
```

这个 Event 最终会在 :

```
\
receipt.logs\
]
```

中出现。

所以 :

```
\
\boxed{\
SmartContractEvent\
\rightarrow\
BlockchainLog\
}\
]
```

21. Log 中有什么 ?

一个 Log 通常包含 :

```
address
topics
data
blockNumber
transactionHash
logIndex
```

其中 :

```
address
```

表示哪个 Smart Contract emit 这个 Event。

```
topics
```

保存 Event Signature 和 indexed parameters。

```
data
```

保存其他 encoded Event Data。

22. 为什么 Log 对套利研究很重要 ?

因为 :


```

\
Swap\
]
\
Transfer\
]
\
Liquidation\
]

```

等行为最终都会产生 Logs。

例如：

```

\
TransactionReceipt\
]

```

中出现：

```

Swap Event
Transfer Event
Transfer Event

```

就可以逐步还原：

这笔交易到底经过了哪些 Tokens 和 Pools？

这会成为第二周 Historical Transaction Reconstruction 的核心能力。

23. eth_getLogs

如果：

```

\
eth\_getTransactionReceipt\
]

```

是：

查一笔 Transaction 的 Logs

那么：

```

\
eth\_getLogs\
]

```

更像：

按条件搜索一段 Blocks 中的 Logs。

例如：

```

fromBlock
toBlock
address
topics

```

可以查询：

Block (N) 到 Block (N+1000) 中，这个 Pool 产生的所有 Swap Events。

因此：

```

\
eth\_getLogs\
]

```

其实可以直接进行 Historical Event Collection。

24. eth_getLogs 和 The Graph 有什么区别？

它们都可以找历史 Event。

但：

The Graph

已经帮我：

Decode
Index
Structure
Relationship

所以更加方便。

eth_getLogs

返回更接近：

```
\\  
RawBlockchainLogs\  
]
```

需要自己：

Decode ABI
Parse Topics
Normalize Data

所以：

```
\\  
TheGraph\  
]
```

适合快速 Research。

而：

```
\\  
eth\_getLogs\  
]
```

更适合：

```
\\  
IndependentVerification\  
]
```

或者：

我需要的数据没有合适的 Subgraph。

25. 什么是 eth_call？

这是 Day 5 最重要的 RPC 方法之一。

eth_call 可以：

调用某个 Smart Contract Function，并读取返回值，但不真正提交 Transaction。

例如 Contract 有：

```
function balanceOf(address user)  
view  
returns (uint256)
```

可以通过：

```
\\  
eth\_call\  
]
```

查询：

```
\\  
balanceOf(user)\  
]
```

而不需要发送真实 Transaction。

26. eth_call 为什么不会改变 Blockchain？

因为：

```
\\  
eth\_call\  
]
```

本质上是：

在指定 Blockchain State 上模拟执行 Contract Call。

它不会：

```
Sign  
Broadcast  
Mine  
Change State
```

因此：

```
\\  
\boxed{\  
eth\_call  
=====
```

```
Read/Simulation\  
}\
```

而不是：

```
\\  
Transaction\  
]
```

27. 为什么 eth_call 对套利极其重要？

套利需要知道：

如果现在执行这笔 Swap，我会得到多少？

这其实可以通过：

```
\\  
eth\_call\  
]
```

调用 Router / Quoter 等 Contract。

例如：

```
\\  
Quote(Q)\  
]
```

本质上就是：

```
\\  
CurrentState\  
+\  
TradeParameters\  
\rightarrow\  
SimulatedOutput\  
]
```

因此以后 Scanner 会大量使用：

```
\\  
\boxed{\  
eth\_call\  
}\
```

而不是每次都发送真实 Transaction。

28. latest 与 Historical Block

很多 RPC call 可以指定：

latest

表示：

```
\\  
当前最新BlockState\  
]
```

但也可以指定：

```
\\  
Block\_N\  
]
```

于是可以询问：

Contract 在历史 Block (N) 时返回什么？

概念上：

```
\\  
eth\_call(Block\_N)\  
]
```

可以帮助：

```
\\  
HistoricalStateReconstruction\  
]
```

这对于 Replay Engine 非常重要。

29. 为什么需要 Archive Node？

如果我要：

```
\\  
eth\_call\  
]
```

一个非常古老 Block 的历史 State：

```
\\  
State\_ {N-old}\  
]
```

普通 Node / RPC Provider 未必保存完整可查询历史状态。

这时可能需要：

```
\\  
ArchiveNode\  
]
```

简单理解：

Full Node

更关注：

```
\\  
CurrentState\  
]
```

Archive Node

额外保留：

```
\\  
HistoricalState\  
]
```

方便：

```
\\  
HistoricalStateQuery\  
]
```

Day 5 不需要自己部署 Archive Node，只需要理解为什么以后做 Replay 可能需要它。

30. Day 5 第一个 Experiment | 读取 Latest Block

今天先建立：

```
src/collectors/rpc.py
```

然后使用 RPC：

```
eth_getBlockByNumber
```

读取：

```
\[\nlatest\
```

记录：

```
block_number\nblock_hash\ntimestamp\ngas_used\ngas_limit\ntxcountcount
```

目标只是验证：

```
\[\nPython\ \rightarrow\ \RPC\ \rightarrow\ BlockchainNode\
```

已经跑通。

31. Experiment 2 | 分析 Day 4 的 Large Swap

拿 Day 4 保存的：

```
\[\nTransactionHash\
```

查询：

```
\[\neth_getTransactionReceipt\
```

记录：

```
status\nblockNumber\ngasUsed\nlogs
```

然后回答：

Q1

Transaction 是否成功？

Q2

用了多少 Gas？

Q3

产生多少 Logs？

Q4

哪些 Contract emit 了这些 Logs？

这就是第一次：

```

\
HistoricalCase\
\rightarrow\
DirectBlockchainVerification\
]

```

32. Experiment 3 | 比较 The Graph 与 RPC

这是今天最重要的实验。

Day 4 从 The Graph 得到：

```

Transaction Hash
Block
Timestamp
Swap Amount
Pool

```

现在通过 RPC 再获取：

```

Block
Receipt
Logs

```

比较：

Field	The Graph	RPC
Block	 	
Timestamp	 	
Transaction Hash	 	
Pool	 	
Swap Event	 	
Gas	 	

目标不是证明谁“更好”。

而是理解：

```

\
\boxed{
IndexedRepresentation\
}\
]

```

和：

```

\
\boxed{
RawOnChainEvidence\
}\
]

```

之间的关系。

33. Experiment 4 | 读取 Contract State

选择一个简单 Contract Function。

例如：

```

balanceOf
decimals
symbol

```

或者以后选择 AMM 的 Quoter / Pool Contract。

使用：

```

\
eth\_call\
]

```

读取结果。

目标是建立：

```
\[\n\boxed{\nCurrentContractState\n\rightarrow\nRPCRead\n}\n]
```

这一层能力。

34. Python RPC 基本结构

今天可以先直接使用：

```
import requests
```

写一个通用方法：

```
def rpc_call(url, method, params):  
    payload = {  
        "jsonrpc": "2.0",  
        "id": 1,  
        "method": method,  
        "params": params,  
    }  
    response = requests.post(  
        url,  
        json=payload  
    )  
    response.raise_for_status()  
    return response.json()
```

然后：

```
rpc_call(  
    RPC_URL,  
    "eth_getBlockByNumber",  
    ["latest", False]  
)
```

这样以后所有方法：

```
eth_call  
eth_getLogs  
eth_getBlockByNumber  
eth_getTransactionReceipt
```

都可以复用同一个 RPC Client。

35. 为什么先不用 Web3.py ?

Web3.py 很有用。

但 Day 5 我更希望先直接看到：

JSON-RPC Request

本身是什么。

因为这样我能够真正理解：

```
\[\nWeb3.py\n]
```

只是：

帮我封装底层 RPC 调用的 Library。

如果一开始只写：

```
w3.eth.get_block("latest")
```

虽然方便，但容易不知道下面实际发生了什么。

所以建议顺序：

```
\[\nRawRPC\
```

36. RPC Error 也是数据的一部分

以后运行 Scanner 时会遇到：

```
timeout\nrate limit\nnode unavailable\nrevert\nmissing trie/state\ninvalid params
```

不能假设：

```
\[\nRPCCall\
```

永远成功。

所以 Collector 最终应该包含：

```
Timeout\nRetry\nError Logging\nRate Limit\nFallback RPC
```

这实际上属于：

```
\[\nInfrastructureRisk\
```

而 Infrastructure Risk 本身会决定你有没有能力获得某个 Arbitrage Edge。

37. 一个套利 Research Pipeline

到 Day 5，可以开始形成这样的研究流程。

假设 The Graph 发现：

```
\[\nLargeSwap\
```

第一步：

```
\[\nTheGraph\
```

第二步：

```
\[\nRPC\
```

第三步：


```

\
Receipt\
\rightarrow\
InspectLogs\
]

```

第四步：

```

\
eth\_call\
\rightarrow\
ReconstructState\
]

```

最终：

```

\
\boxed{\
Candidate\
\rightarrow\
Evidence\
\rightarrow\
State\
\rightarrow\
Hypothesis\
}\
]

```

这比：

看 Twitter 上一张套利收益截图

可靠得多。

38. Day 5 最重要的概念：Research Data vs Execution Data

现在我可以正式区分：

Research Data

允许一定程度的数据加工和 latency。

例如：

The Graph
Historical API
CSV Dataset

用于：

```

\
Research\
]

```

Execution Data

需要尽可能接近：

```

\
CurrentState\
]

```

例如：

RPC
Quoter
eth_call
Block State

因为：

```

\
100ms\
]

```

或者：

```
\\  
1Block\  
]
```

的状态变化都可能让套利消失。

所以：

```
\\  
\boxed{\  
DataGoodEnoughForResearch\  
eq\  
DataGoodEnoughForExecution\  
}\
```

这是 Day 5 最重要的结论之一。

39. Day 5 验收题

Q1：RPC 是什么？

RPC 是：

```
\\  
RemoteProcedureCall\  
]
```

允许我的程序向 Blockchain Node 请求：

```
\\  
Block\  
]
```

```
\\  
Transaction\  
]
```

```
\\  
Logs\  
]
```

```
\\  
ContractState\  
]
```

等信息。

Q2：The Graph 和 RPC 最大区别是什么？

The Graph：

```
\\  
Blockchain\  
\rightarrow\  
Indexer\  
\rightarrow\  
StructuredData\  
]
```

适合：

```
\\  
HistoricalResearch\  
]
```

RPC：

```
\\  
Node\  
\rightarrow
```

```
DirectQuery\  
]
```

更加接近：

```
\\  
BlockchainState\  
]
```

适合：

```
\\  
Verification\  
+\  
Execution\  
]
```

Q3：Transaction 和 Receipt 有什么区别？

Transaction 表示：

```
\\  
WhatTheUserRequested\  
]
```

Receipt 表示：

```
\\  
WhatActuallyHappened\  
]
```

Receipt 可以告诉我：

```
status  
gasUsed  
logs
```

Q4：为什么 Failed TX 也必须进入套利 Cost Model？

因为 Failed Transaction 虽然没有完成套利，但仍然可能：

```
\\  
ConsumeGas\  
]
```

所以：

```
\\  
ExpectedNetProfit\  
]
```

必须扣除：

```
\\  
FailureProbability\  
\times\  
FailureCost\  
]
```

Q5：eth_call 和真实 Transaction 有什么区别？

eth_call：

```
\\  
Simulation/Read\  
]
```

不会：

```
\\  
ChangeBlockchainState\  
]
```

真实 Transaction：

```
\\  
Sign\  
\rightarrow\  
Broadcast\  
\rightarrow\  
Execute\  
\rightarrow\  
ChangeState\  
]
```

因此套利 Scanner 可以大量利用：

```
\\  
eth\_call\  
]
```

进行报价和 State Simulation。

Q6：为什么套利研究需要 Logs？

因为 Smart Contract 的：

```
\\  
Swap\  
]  
  
\\  
Transfer\  
]  
  
\\  
Liquidation\  
]
```

等 Event 最终都会写入 Logs。

通过 Logs 可以还原：

Transaction 实际经过哪些 Protocol / Token / Pool。

Q7：为什么套利不能只依赖 The Graph 的数据直接执行？

因为：

```
\\  
IndexedData\  
]
```

可能与：

```
\\  
CurrentBlockchainState\  
]
```

之间存在 latency。

套利执行依赖：

```
\\  
StateNow\  
]
```

因此通常需要：

```
\\  
RPC/Quote\  
]
```

再次验证。

40. Day 5 最终 Task

今天至少完成：

Task 1

配置一条：

Ethereum / Arbitrum / Base RPC

建议先只选一条链。

Task 2

调用：

eth_getBlockByNumber

读取 Latest Block。

Task 3

拿 Day 4 保存的 Large Swap Transaction Hash。

调用：

eth_getTransactionReceipt

检查：

status

block

gas

logs

Task 4

使用：

eth_getLogs

查询目标 Pool 的一小段历史 Events。

Task 5

使用一次：

eth_call

读取某个 Contract Function。

Task 6

比较：

\\

TheGraphData\

]

和：

\\

RPCData\

]

是否一致。

41. Day 5 最终产出

项目结构应该增加：

arbitrage-lab/

```
|—— src/
|   |—— collectors/
|   |—— lifi.py
|   |—— graph.py
|   |—— rpc.py
|
|—— data/
|   |—— raw/
|   |   |—— lifi/
|   |   |—— graph/
|   |   |—— rpc/
|   |
|   |—— processed/
|
```

```
└── reports/
└── graph_vs_rpc.md
```

其中：

rpc.py

至少实现：

```
rpc_call()
get_block()
get_receipt()
get_logs()
eth_call()
```

42. Day 5 总结

Day 4 的关键问题：

```
\[
WhatHappenedHistorically?
]
```

Day 5 则进一步：

```
\[
\boxed{
CanIVerifyItDirectlyOnChain?
}\
]
```

最重要的认识是：

```
\[
\boxed{
Blockchain
=====
StateMachine\
}\
]
```

每一笔 Transaction 都会产生：

```
\[
S\_t\
\rightarrow{TX}\
S\_{{t+1}}\
]
```

而套利本质上往往是在寻找：

```
\[
State\_t\
]
```

中的暂时不一致，并尝试在：

```
\[
State\_{{t+1}}\
]
```

或者其他 Searcher 到来之前完成交易。

因此链上套利真正研究的不是简单：

```
\[
Price(t)\
]
```

而是：

```

\[\
\boxed{\
State\
+\
Transaction\
+\
Ordering\
+\
Execution\
}\
]

```

今天建立的核心链条是：

```

\[\
\boxed{\
Node\
\rightarrow\
RPC\
\rightarrow\
Block\
\rightarrow\
Transaction\
\rightarrow\
Receipt\
\rightarrow\
Logs\
\rightarrow\
ContractState\
}\
]

```

到 Day 5 为止，我已经逐渐建立了四种不同层级数据能力：

```

\[\
AMM\
\rightarrow\
MathematicalModel\
]

\[\
LI.FI\
\rightarrow\
ExecutableQuote\
]

\[\
TheGraph\
\rightarrow\
HistoricalIndexedData\
]

\[\
RPC\
\rightarrow\
DirectOnChainState\
]

```

最终形成：

```

\[\
\boxed{\
Model\
+\

```

```
Quote\  
+\  
History\  
+\  
State\  
}\  
]
```

这四层将成为后面所有 Arbitrage Scanner、Historical Replay、Opportunity Evaluator 和 Hermes Agent 的数据基础。

Day 6 接下来会进一步进入：

```
\\  
\boxed\  
CrossChainRouteAnatomy\  
}\  
]
```

把 Day 3 的 LI.FI Quote 重新拆开，真正研究：

```
\\  
DEX\  
\rightarrow\  
IntermediateToken\  
\rightarrow\  
Bridge\  
\rightarrow\  
DestinationDEX\  
]
```

以及：

```
\\  
Route(Q,t)\  
]
```

为什么随着 Trade Size 和 Market State 改变。

08 实盘与数据（11 篇）

笔记 126: Day 6 — Hyperliquid HIP-3 发现 + 四平台实测 + 方法修正

作者：ChenXuan1985 | 时间：2026-08-10 15:53 UTC | 字数：4335 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 — Hyperliquid HIP-3 发现 + 四平台实测 + 方法修正

残酷共学 · 链上套利专题

日期：2026-08-10

阶段：Hyperliquid 美股通道打通 + 四平台并发验证 + 价差方法论修正

一、Hyperliquid HIP-3 重大发现

1.1 推翻旧结论

之前三次确认“Hyperliquid 无股票 perp”——结论是错误的。根因在于适配器只查询了默认 perp DEX，而股票永续位于 HIP-3 子 DEX (xyz、para 等)，需要带 dex 参数查询。

1.2 主网探测结果

主网 perpDexs 返回 10 个子 DEX。活跃股票集中在两个：

| 子 DEX | 覆盖标的 | 证据 |

| :----- | :----- | :----- |

| xyz: | AAPL, TSLA, NVDA, MSFT, GOOGL, AMZN, META, AMAT, AVGO, INTC, MRVL, MU, NOW, ORCL, PLTR, MSTR, NFLX, DKNG (18 只) | 双边 L2、正 OI、正 24h 成交量 |

| para: | LRCX, CRWD (2 只) | 双边 L2、正 OI、正 24h 成交量 |

四平台共同覆盖：20/27 只。

未被 Hyperliquid 覆盖的 7 只：KLAC, ADBE, CRM, SNOW, SPY, QQQ, IWM（仍仅在三家 CEX 上）。

1.3 已修正的认知

| 旧结论 | 新结论 |

| :----- | :----- |

| Hyperliquid 无股票 perp | HIP-3 xyz/para 子 DEX 有 20 只活跃股票 |

| HIP-3 无 TradFi 部署 | 已部署且活跃交易 |

| Hyperliquid 只能做 BTC/ETH | 可扩展到 20 只美股的四平台监控 |

二、四平台实测结果

2.1 REST 并发快照

对 Binance、OKX、Bybit、Hyperliquid (xyz + para) 并发发起公开 REST 请求，三轮结果：

| 轮次 | Binance | OKX | Bybit | Hyperliquid | 共同集 | 耗时 |

| :- | :----- | :----- | :----- | :----- | :----- | :----- |

| 1 | 27/27 | 27/27 | 27/27 | 20/20 | 20 | 1.40s |

| 2 | 27/27 | 27/27 | 27/27 | 20/20 | 20 | 0.47s |

| 3 | 27/27 | 27/27 | 27/27 | 20/20 | 20 | 0.59s |

结论：四平台并发 REST 快照工程可行 (<1.5s)。但必须保存各自的源时间戳、请求发起/完成时间和 pair skew——不能假设四路数据时点一致。

2.2 WebSocket 实测

以 AAPL 为单标的，同时连接四路 WebSocket：

| 交易平台 | 结果 | 首笔到达 |

| :----- | :----- | :----- |

| Binance | bid/ask | \~592ms |

| OKX | bid/ask | \~564ms |

| Hyperliquid xyz:AAPL | L2 | \~449ms |

| Bybit | TLS 连接被重置 | — |

三路成功，本地接收偏差 \~143ms。Bybit WebSocket 失败（同期 REST 正常），判定为本机代理/链路问题，需要 heartbeat + 退避重连 + 长时间 soak test。

三、方法论修正

3.1 价差定义修正

之前使用的 mid-to-mid 价差公式：

$$\text{mid_spread_bps} = 10,000 \times (\text{mid_B} - \text{mid_A}) / \text{mid_A}$$

问题：mid 不是可成交价格。真实的套利边际应该是：

$$\text{entry_edge_bps} = 10,000 \times (\text{bid_B} - \text{ask_A}) / \text{ask_A} \text{ (做多 A / 做空 B)}$$

且完整周期包含四次成交（开仓两腿 + 平仓两腿），费用不能只按 taker $\times$ 2 估算：

$$\text{net_pnl} = q_A \times (\text{bid_A_close} - \text{ask_A_open})$$

$$+ q_B \times (\text{bid_B_open} - \text{ask_B_close})$$

open_fees close_fees (四腿)

funding slippage hedge_error

3.2 ADBE 重新评估

周末 13 轮 ADBE 数据重新计算：

均值 +10.785 bps，标准差 1.044 bps，范围 +9.3 \~ +12.3

13 个点属于同一周末簇，不是 13 个独立事件

13/13 均未在该窗口内收敛至 3 bps

示例性假设四次成交均为 5 bps taker $\rightarrow$ 单轮费用约 20 bps，已高于毛价差

如果使用 maker 成交，还需验证排队、成交率和逆向选择

结论：ADBE 不是已证明的套利 alpha。它是「待验证的结构性报价基差」——OKX 系统性高于 Binance 的现象真实存在，但扣除四腿成本后是否有净收益，需要更精确的 bid/ask 级别数据才能判断。

3.3 KLAC 尾部风险

KLAC 从 -8 bps 跳至 -24 bps (3x spike) , 风控规则 :

1. 到达预注册外沿 → 立即停止新层建仓, 撤销未成交开仓单
2. 检查 bid/ask 深度、mark/index/oracle、源时间戳
3. 单腿或净 delta 超限 → 优先补对冲或 flatten
4. 在独立数据验证通过前, 不允许自动扩区、加倍或追价补仓

四、当前状态矩阵

项目	状态
三平台 27 标的行情覆盖	已验证
Hyperliquid 股票永续	HIP-3 xyz/para 20 只活跃
四平台共同覆盖	20/27 (非 27/27)
四平台并发 REST	三轮验证可行
四平台 WebSocket	三路通, Bybit 失败
ADBE +10 bps	待验证 (结构性基差, 非已证明 alpha)
策略执行 / 实盘	NO-GO (数据层未完成)

五、下一步方向

Phase 0 : 四平台可信采集层 (当前主线)

1. 修复 Hyperliquid 适配器——加入 perpDexs 查询 + DEX 前缀映射 + BBO/L2
2. 统一 watchdog 与 adapter 数据契约——移除重复抓取逻辑
3. 恢复 TLS 证书校验 (当前 CERT_NONE 必须改为正确 CA)
4. 保存四类时间戳 (源时间、请求发起、请求完成、接收时间)
5. 解决 Bybit WebSocket + heartbeat + 退避重连
6. 写入 append-only 原始事件 (JSONL/SQLite + provenance)
7. 覆盖至少 2 个独立周末 + 完整美股时段

Phase 1 (Phase 0 完成后)

用 bid/ask 可成交价重算净边际

统计单位改为独立事件/会话, 非 polling 行数

多重比较控制 (27 symbols × 多 venue pairs × 多时段)

ADBE 路线若净边际不能覆盖成本, 终止该路线

笔记生成时间 : 2026-08-10

笔记 127: Day 6 - 2026-08-11 : 推翻完整集主线, 建立证据型共学仓库

作者 : Helios | 时间 : 2026-08-10 17:08 UTC | 字数 : 3340 | 评分 : 89

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 - 2026-08-11 : 推翻完整集主线, 建立证据型共学仓库

今天的问题

昨天把同市场 YES + NO 完整集扫描设为主线, 这个前提是否真的成立? 如果不成立, Polymarket 还有哪些套利/相对价值类型, 剩余 17 天应该怎样安排?

我做了什么

1. 阅读 Polymarket 官方机制, 按支付证明区分完整集、Negative Risk、逻辑关系、Combo、跨场、做市奖励和规则交易。
2. 只读检查 /Users/gffive/proarb 的机制、策略、EV、执行和分析文档, 没有运行或修改其交易流程。
3. 复核完整集盘口关系, 发现 YES/NO 是同一物理挂单的互补镜像, 而不是两个独立订单簿。
4. 把主线改为 PM-Deribit S1/S2 的 screen divergence -> Net EV -> fill -> canonical net PnL 验证。
5. 建立研究仓库 : 课程计划、证据等级、实验协议、风险登记册、数据字典、策略 playbook、SQL 和打卡模板。

最重要的反证

盘口镜像关系 :

text

YES_bid(P, S) <-> NO_ask(1-P, S)

YES_ask(P, S) <-> NO_bid(1-P, S)

因此 :

text

$YES_ask + NO_ask = 1 + spread \geq 1$

$YES_bid + NO_bid = 1 - spread \leq 1$

proarb 文档还记录了 10,000 行快照的一至三档镜像检验：对齐率约 96.49% 至 98.27%，平均价格误差约 0.00003；主要异常来自两次 API 快照的时点 jitter。把 raw book 和 mirrored counterpart 相加还会产生 2 倍虚假深度。

所以昨天的公式本身描述了完整集支付，但“市场中会常规出现可扫描 alpha”的经验假设不成立。表面正 edge 必须先标记为数据质量异常。

新的类型地图

| 类型 | 真实属性 | 本期处理 |

| :----- | :----- | :----- |

| 同 condition 完整集 | 支付恒等式；CLOB 硬镜像 | 数据质量控制，不作主 alpha |

| Negative Risk 完备篮子/转换 | 全腿成交后最接近原生结构锁定 | 第二主实验 |

| PM-Deribit/Binance 复制与参考对冲 | 模型和 basis 驱动的相对价值 | 主线 |

| 逻辑蕴含/时间嵌套/重复市场 | 依赖独立规则完全保持关系 | 后期规则卡 |

| 跨平台等价合约 | 规则、资金、结算、平台风险叠加 | 本期不接执行 |

| Combo vs synthetic | 条件型，受 RFQ/Last Look/残余仓位影响 | 机制研究 |

| UMA/规则分歧 | 信息交易 | 不称套利 |

| 做市返佣/流动性奖励 | 库存与逆向选择策略 | 不称套利 |

| collateral return | 资本效率 | 不当作 alpha |

新主线公式

text

$s1_exec_div = p_above_k_poly - pm_yes_ask$

$s2_exec_div = pm_yes_bid - p_above_k_poly$

Gross EV = PM EV + DR EV

Net EV = Gross EV - PM open fee - DR open fee - PM close cost

S1 为买 PM YES、short K1/long K2；S2 为买 PM NO、long K1/short K2。p_above_k_poly 是风险中性模型概率，因此这不是无风险套利。

新的验证顺序

text

mv_pm_dr_divergence (机会总体)

-> evresponse (signal-time)

-> v_positions entry/JIT

-> reconcile / executionfill

-> v_positions canonical net PnL

主结果使用：

text

Net EV Capture

= sum(eligible realized_pnl_net_canonical)

/ sum(eligible entry ev_model_usd)

不能平均单仓 ratio，也不能用遗留 gross/raw capture 代替净口径。

产出

新版学习计划 (../course/learning-plan.md)

新版研究章程 (../research/research-charter.md)

Polymarket 类型研究 (../research/polymarket-arbitrage-taxonomy.md)

proarb (../research/proarb-reuse-assessment.md) 复用评估 (../research/proarb-reuse-assessment.md)

证据等级 (../methodology/evidence-levels.md)、实验协议 (../methodology/experiment-protocol.md)与 风险登记册 (../methodology/risk-register.md)

各策略 playbook 与只读 SQL 模板

结论

今天最有价值的结果不是增加一个策略，而是推翻了错误主线。完整集只保留为不变量控制；主线转为复用现有历史证据，验证 S1/S2 从 screen edge 到 realized net PnL 的完整漏斗。Negative Risk 作为更接近结构套利的独立支线，不与相对价值混称。

证据等级：完整集硬镜像结论为 E3（机制证明 + 历史盘口验证）；S1/S2 当前仍是研究章程，尚未完成本期数据重跑。

风险与边界

proarb 的 checked-in 配置允许实盘路径，今天没有运行它；共学仓库也不复制 executor、账户或部署。

PM 与 DR 的价格源和到期时点并不完全一致，S1/S2 有 basis/gap risk。

MV 是小时级历史分析面，不代表当前可成交。

历史落仓存在选择偏差，必须报告机会总体到 CLOSED 的每层分母。

明天只做一件事

为至少 10 个 PM BTC/ETH 阈值市场填写 Contract Alignment Card，核对阈值、>/>=、观察时点、时区、价格源、异常结算与对应 DR expiry；未通过的市场在看价格前排除。

笔记 128: Day 6 | Funding 现金流与纸上账本

作者：shiyang07ca | 时间：2026-08-10 22:05 UTC | 字数：1494 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 6 | Funding 现金流与纸上账本

今日问题

Funding API 的 rate、value、direction 怎样进入 WTI 与 BRENT OIL 两条腿的现金流账本？

已完成动作

- 阅读并整理 notes/funding-ledger-model.md，运行 lab/day6_funding_ledger.py，并用 lab/test_day6_funding_ledger.py 验证公式、付款方向和未知边界。
- Day 6 专项测试结果为 5/5；仓库当前 lab 全量测试结果为 22/22。
- 在共同样本 2026-08-03T00:00:00Z 中，WTI 与 BRENT OIL 的 funding_rate 都为 0.0004；原始 api_value 分别为 0.00032286 与 0.00033548，api_direction 都为 long。这些只是 API 观察值。

核心规则

- 正 funding 时多头付款、空头收款；负 funding 时空头付款、多头收款。
- 纸上现金流公式为：

$$\text{cash_flow} = \text{position_sign} \times \text{quantity} \times \text{multiplier} \times \text{index_price} \times \text{funding_rate}.$$

其中多头 position_sign=+1，空头 position_sign=-1；负值表示付款，正值表示收款。

- API value 的单位、结算状态和账户仓位映射尚未核验，API direction 不能直接视为账户现金流方向；因此不能把 WTI 与 BRENT OIL 的 value 直接相减当作 funding PnL。

纸上账本结果与边界

以 \$10 纸上数量和当前 orderBookDetails 的 index 作为教学输入：WTI 数量为 0.134、index 为 74.670；BRENT OIL 数量为 0.1266、index 为 79.04。共同样本的正 funding 下，两市场多头的纸上现金流分别约为 -0.004002312 和 -0.0040025856，空头方向相反。这些计算不是账户实际结算金额。

当前 ledger 保持 paper_only/unknown：历史 index 未与 funding timestamp 对齐，且没有账户实际 funding ledger，因此尚不能把纸上结果写成 verified、策略成立或真实收益。

证据路径

- notes/funding-ledger-model.md
- lab/day6_funding_ledger.py
- lab/test_day6_funding_ledger.py
- lab/data/day6_funding_ledger_snapshot.json
- lessons/0005-day6-funding-ledger.html
- reference/day6-funding-ledger.html

研究结论

学习产出已完成；交易研究结论为 Blocked，阻断原因是时间对齐的历史 index 和账户实际 funding ledger 尚未取得。下一步只补齐这两类证据，不把 API 字段差值或纸上现金流当作真实收益。

笔记 129: Day 7 (8/11 周二) 完整打卡 · 「看起来赚了 ≠ 真的赚了」认知闭环日

作者：Paxon Qiao 乔鹏军 | 时间：2026-08-11 01:58 UTC | 字数：1966 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 (8/11 周二) 完整打卡 · 「看起来赚了 ≠ 真的赚了」认知闭环日

【主线 1】网格策略趋势市回测（完整版）

- 为何做：群友「在金」1 个月 1300U 网格晒单，评论区点破「这月没波动所以网格舒服」→ 用真实行情验证幸存者偏差
- 怎么做：OKX BTC/USDT 1h 10080 根（2025-06→2026-08）13 个月独立回测，区间=前 30 天极值（无未来函数），90 档×\$100/档，手续费扫描+杠杆强平视角
- 结果：下跌月亏损率 88%（8 月亏 7）vs 非下跌月盈利 100%（5/5），相关 -0.48；0 手续费仍净亏 -\$1,122（趋势月库存浮亏 -\$2,330 > 震荡月 edge +\$1,208）；10x 杠杆 -10.2% 月即爆仓——结构不对称，非参数问题
- 意义：晒单的 1300U 恰好产生于低波动月，一个月测试不构成证据；等 Arcus 开源自按实参复跑

【主线 2】Polymarket 排行榜「税前口径」实体验证（选项 A）

- 为何做：@runes_leo 推文称榜单收益是 gross、真实到手仅 24-45%、榜单排序与真实相反
- 怎么做：公开 data-api /activity 事件流重建 3 个榜前地址（11 天窗口，TRADE/REDEEM/REBATE 三分账）+ 手续费公式逐档验证
- 结果：公式正确（100 股×0.07×p×(1-p)，p=0.5→\$1.75 与推文一致）；每股手续费实测 0.0122-0.0142（推文 0.011-0.014）；榜一 11 天返佣 \$74.6K；榜五 HomeRunHazard 月榜 +\$454K 但近 11 天纯交易 -\$121K（方向反转实锤）
- 意义：榜单=税前 gross；返佣按费用比例返=返佣多是坏信号；「24-45%」只对 edge≈费用的吃单 bot 成立，有真 edge 的地址到手 82-92%（修正推文）

【主线 3】实操案例：毛利为正 gas+桥费翻负

- 0.004 ETH 跨链：毛利 +0.01U（+0.133%）但 gas+桥费 -0.11U（-1.46%）= 成本是毛利 11 倍 → 净亏 -0.10U
- 最小有效规模 = 固定成本÷价差率：\$7.5 亏、\$83 打平、\$1000 进 Li.Fi 最优点——规模是套利参数不是可选项

【补课】D8 AMM 数学预习 + 雷达 v2 + TUT Rust 对齐

- xy=k 用 Raydium 真实储备精确复现（1 SOL swap 73.9612 误差 0），费率反推=0.30% 非 0.25%；滑点曲线=容量曲线
- 无套利带雷达 v2：Raydium 锚点校准 + 深度 suspect 过滤（偏离锚点 ≥3% 不告警）；实测全配对在走廊内
- TUT Rust 时间对齐 v2：inner join on ts 替代 index 对齐——抓到 Binance 09:00 起 vs Bitget 00:00 起的错位盲区；双日交叉验证与 Python 一致

【研究线】DOS 多链搬砖三步（找币→监控→搬砖）

- 找多链币实证：ZRO/ALT/W/PENDLE/ENA 在 BSC 全无主流池——「BSC+ETH 双链深池」多链币极稀缺，DOS 是新币上市窗口特殊案例
- 监控：multi_chain_spread_monitor.py slot0 直读（BSC 0.5292 vs Gate 0.529 精确一致）
- 判定：毛价差 136bps → 扣 BSC→ETH 桥费 \$0.69 后净 7.6bps → NO-GO；真正窗口=ETH 折价时走 ETH→BSC 便宜方向
- 新增研究线 #13 低容量结构性价差量化、#14 Meme 微观结构（5M 壳价 cohort+Dev 哨兵+增发监控）

【方法论沉淀】评估赚钱案例 5 问（L0008 筛选前置）：①税前还是税后 ②成本占毛利多少（>50% 即幻觉）③返佣/补贴依赖度 ④样本窗口是什么市场状态 ⑤仓位达到最小有效规模吗

【认知演变】跟单=快但有坑（幸存者偏差/执行滞后/容量）；几百池监控慢=架构问题（缩面+流式推送，不走全节点）；Robihood 无 mempool 但 PFOF 把 MEV 变成做市商内部化——竞争密度低的市场适合个人，与 #13 一致

【产出】工程 7 项全跑通（网格回测/AMM 验证/Polymarket 验证/雷达 v2/TUT 对齐/多链监控/tvscreener），广度归档 14 条，全景笔记 notes/day7-complete-20260811.md

笔记 130: 阅读相关清算论文

作者：starkxun | 时间：2026-08-11 02:05 UTC | 字数：9695 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

阅读相关清算论文

先给结论:三条被改写,一条被独立印证

| # | 我们的结论 | 论文说 | 处置 |

| :--- | :----- | :----- | :----- |

| 1 | 「主流市场清算人只有 2~3 个」 | Aave V3 183 个、Compound V3 184 个、Morpho 159 个 | 作废 |

| 2 | 「80% 的清算 是 \$10 以下尘埃」 | 三大协议清算中位 \$20,266 | 是我们样本的特性,不是市场的 |

| 3 | 「30 天全市场 \$640 万」 | 三年 1,458M≈1,458M≈40.5M/月;2022-11 那次 48 小时 \$12 亿 | 我们那个窗口低了约 6 倍,极端时差 100 倍以上 |

| 4 | 「毛机会 50~98% 被构建者拿走」 | Wintermute 给构建者 66%、SCP 81%、走整合渠道 ~90% | 独立印证,而且是另一个赛道 |

净结果:市场比我们测的大,竞争者比我们以为的多,但「价值被排序权拿走」这个结构被完全证实。

一、Measuring CEX-DEX Extracted Value(AFT 2025)—— 最重要的一篇

论文原文：<<https://arxiv.org/abs/2507.13023>>

论文核心数据

19 个月(2023-08 ~ 2025-03)、7,203,560 笔 CEX-DEX 套利、\$233.8M 提取值。

Searcher	成交量	估计收入	给构建者	净利	给出去的比例
Wintermute	\$74.8B	\$71.4M	\$47.1M	\$24.3M	66.0%
SCP	\$63.5B	\$71.1M	\$57.4M	\$13.7M	80.7%
Kayle	\$41.0B	\$28.3M	\$16.0M	\$12.3M	56.5%
Graves	\$229M	\$173K	\$353K	\$180K	204%(净亏)

论文原文:

"Both Wintermute and SCP transfer nearly 90% of their arbitrage revenue directly to their integrated builder... retaining lower profit margins slightly above 10%."

对照 ①:同一个 Wintermute

这篇论文里排第一的 CEX-DEX searcher,就是我们那台清算机器人的利润收款地址。

我们读存储槽 2 拿到 0x51c72848c68a965f66fa7a88855f9f7784502a7f =
Wintermute,当时只能推测"它可能运营这台机器人"。这篇论文补上了另一半:Wintermute 同时是最大的 CEX-DEX
searcher,并且和构建者 rsync 垂直整合。

我们的第十三节猜"它不是独立生意,是 Wintermute 的一条订单流管道" —— 这个猜测现在有了外部证据。

对照 ②:98% 这个数不是清算特有的

我们测到清算把 50.2%(中位大小的单子 98%)交给构建者,当时觉得高得离谱。论文显示 CEX-DEX 赛道同样:66%、81%,整合渠道下
~90%,甚至有 searcher 长期净亏(Graves 272% 中位利润率)。

两个完全不同的赛道,同一个结构。这不是清算的问题,是 PBS 下 MEV 的普遍结构。

有一个关键差异

论文里存在高利润率的小 searcher:

Karma 收入 \$3.0M 净利 \$2.5M 中位利润率 71.2%
Maokai 收入 \$4.4M 净利 \$3.3M 中位利润率 69.7%
Thresh 收入 \$3.2M 净利 \$2.2M 中位利润率 59.7%

论文的解释:

"Neutral searchers, distributing flow among multiple builders, retain higher profit margins. Conversely, exclusive searchers maintain lower margins, sharing most of their revenue with affiliated builders."

垂直整合换来的不是更高利润率,是更大流量;利润在构建者那一端体现。 \

表 2 印证:rsync-Wintermute 单看构建者是 2.24%,合并 searcher 后 27.06%。

对你最有价值的一句:小的、中立的 searcher 能保住 55~71% 的利润率。 \

Karma 那种规模换算下来约 130K/月, Maokai约130K/月, Maokai约175K/月 —— 是一门真生意。

但为什么 CEX-DEX 能保住 60%,清算只能保住 2%?

因为 CEX-DEX searcher 有私有信息,清算人没有。

 	CEX-DEX	清算
机会来源	CEX 价格(链下,你先看到)	健康度跌破 1(链上,全公开)
出场	在 CEX 对冲(你有账户和额度)	原子变现(要 RFQ 报价)
别人能复制吗	要 CEX 账户 + 库存 + 对冲能力	规则公开,闪电贷免费

这才是 66% vs 98% 的真正原因。 不是构建者对清算更狠,是清算人手里没有任何别人复制不了的东西。

净增量:「要什么条件」这个问题的答案变清晰了 ——\ 你需要的不是速度,是一个别人复制不了的边(edge)。\ 速度不是边,因为所有人都能买到;闪电贷不是边,因为所有人都有。

二、DeFi Liquidations Cluster (Hawkes Framework)—— 补上我们最大的空白

论文原文 : <https://www.researchgate.net/publication/410970155_DeFi_Liquidations_Cluster_Across_Protocols_in_a_Multivariate_Hawkes_Framework>

论文核心数据(2023-01-01 ~ 2025-12-31,三年)

	 	Aave V3	Compound V3	Morpho
	:-----	:-----	:-----	:-----
	清算笔数	3,202	2,634	1,664
	总额	\$682M	\$481M	\$295M
	均值规模	\$212,933	\$182,502	\$177,416
	中位规模	\$20,266	\$19,857	\$22,396
	中位间隔	4,770s(1.3h)	5,113s	6,793s
	独立清算人	183	184	159

自激结构:谱半径 $\rho = 0.725$ (次临界,不会失控但会成簇),\ 自激分支比 $0.28 \sim 0.32$,半衰期 $46 \sim 57$ 分钟,\ 最强跨协议通道 Morpho $\rightarrow$ Compound V3 = 0.418(一笔 Morpho 清算平均触发 0.42 笔 Compound 清算)。
论文引用的极端事件:2022 年 11 月,48 小时内清算 12 亿,单区块清算量超 12 亿,单区块清算量超 8,000 万。

对照 ①:「清算人只有 $2 \sim 3$ 个」这条彻底作废

我们那个数来自 Base 链 2 天的扫描(ETH 3 人、ARB 3 人、BASE 2 人),\ 文档里已经标注过"样本太小不能下结论"。现在有了三年的答案:每个协议 $160 \sim 185$ 个独立清算人。

我们在 30 天全市场扫描里也数出 36 个清算合约,和这个量级一致。\
当时那句「竞争者少得惊人」是错的,现在正式撤回。

对照 ②:我们那 30 天有多平静,量出来了

论文三年均值 $\$1,458\text{M} / 36 \text{ 月} \approx \$40.5\text{M} / \text{月}$
我们 30 天实测 $\$6.4\text{M}$

约为常态的 $1/6$

而极端情况差得更远:2022-11 那 48 小时的 $\$12$ 亿,是我们整个 30 天窗口的 187 倍。

我们文档里那句「这些数是下界」现在有了倍数:平静期约 $1/6$,级联期 100 倍以上。

对照 ③:「80% 是尘埃单」要改口径

论文三大协议的中位规模是 $\sim 20,000$,我们测到 8020,000,我们测到 8010 以下。差异来源已经查清:

论文只覆盖 Aave V3 / Compound V3 / Morpho;

我们的扫描包含 Aave V2,而 Aave V2 上有专门清 $\$1$ 级仓位的尘埃机器人(0x5440e73a 114 笔、0xfc45f8e3 49 笔)。

所以正确的说法是:「可寻址市场」的中位清算约 20,000,不是 20,000,不是 10。\
我们那个 80% 描述的是扫描样本,不是机会分布。这条要在教学文档里改。

净增量:成簇结构本身是可用的信息

半衰期 $46 \sim 57$ 分钟、跨协议分支比 0.418 —— 意思是:

一笔清算发生后的一小时内,同协议和邻近协议的清算强度显著升高。

这对"什么时候该把机器人拉满"是有用的:不是全天候待命,是在第一笔清算出现后的一小时窗口内。\
但注意论文自己的免责:这是方向性预测依赖,不是结构性因果。

三、Frictions in DeFi Liquidations(Aave V2, 46 个月)

论文原文 : <https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6522500>

论文核心数据

46 个月、54,127 个借款仓位、33,168 笔清算。用三态模型(健康 H / 可清算 V / 停滞 S)算转移概率。

最重要的三个数:

1. 约 2/3 的「可清算」仓位退出该状态是靠市场变化(价格回升),只有 1/3 是被清算的。
2. 26.2% 的清算发生在已经「停滞」的仓位上(规模低于阈值,不值得清算的那些)。
3. 72% 被清算的仓位在同一区块内就恢复健康(说明是一次性清干净的)。

清算奖励参数:中位 500bp(5%),均值 546bp,最高 1500bp。 \

但 Adjusted Bonus 最低 2457bp —— 修正预言机偏离后,有些清算实际是亏的。

回归结果(下一区块被清算的概率)

| 因素 | 方向 | 解读 |

| :----- | :----- | :----- |

| 清算规模(log) | +5.58 强正 | 大单几乎必被清 |

| 清算奖励 | +0.0036 正 | 奖励越高越会被抢 |

| MEV-Boost 区块 | +1.81 强正 | 走私有通道显著提高成功率 |

| 中位 gas 价格 | 0.60 强负 | 清算人对成本敏感,贵了就不做 |

| 30 天波动率 | 0.56 负 | 波动高 → 执行风险高 → 反而更少人做 |

| 预言机偏离 | 0.09 负 | 偏离大 → 预言机将更新 → 执行风险高 |

| 短期价格变动 | +0.012 正 | 短期动能提高盈利预期 |

对照 ①:「发现机会」和「机会还在」是两回事

2/3 的可清算仓位从未被清算过 —— 它们自己涨回去了。

这直接打在你原本的直觉上:以为难点是"抢得快"。实际是你监控到的机会里有三分之二会自己消失。 \

这也解释了 Optimistic MEV in Ethereum Layer 2s (2025 (<https://arxiv.org/abs/2506.14768>)) 说的那件事 —— 机器人发大量探测交易但很多没盈利。

对照 ②:MEV-Boost 系数 +1.81,和我们的实测同向

论文说走 MEV-Boost 区块的清算成功率显著更高;我们实测那台机器人全部走私有 bundle,38/40 笔向构建者付费。 \

两边说的是同一件事:私有提交不是可选项,是必需项。

对照 ③:波动率负相关,和"大跌时爆发"不矛盾

看起来和 Hawkes 论文冲突,其实不是:

Hawkes 说的是机会数量在波动时暴增;

Frictions 说的是单个机会的成交概率在波动时下降(执行风险)。

合起来是:大跌时机会很多,但每一个都更难吃到。这对"极端行情下不掉线"那个假设是个修正 —— \

掉不掉线不是唯一问题,执行风险本身也上升了。

净增量:一个我们没考虑过的方向

26.2% 的清算发生在 stale 仓位上,而 stale 的定义就是"小到不值得别人清"。 \

论文的三态模型给了一个可筛选的定义。这批仓位竞争最少

但也正是我们实测里"竞价为

0

因为没有钱"的那批。验证了同一件事的两面。

四、An Analysis of Fixed-Spread Liquidation Lending(2024)

论文原文 : <https://www.researchgate.net/publication/386279060_An_Analysis_of_Fixed-Spread_Liquidation_Lending_in_DeFi>

论文核心结论

超过 70% 的清算发生时,价格并没有出现向下跳跃。

作者的模型:借款人如果连续监控,理论上永远不会被清算(可以及时补仓)。 \

实际会被清算,只能是因为 (a) 价格跳跃 或 (b) 借款人没在看。 \

实证拟合下来:借款人的监控频率约为其与协议交互频率的 3~4 倍 —— 也就是看得很少。

另外:借款人比模型预测的更保守(健康因子留得更高),说明其融资/机会成本很低。

对照:清算的主要来源是"不看盘",不是"暴跌"

这条改变了我原本给你的画像。我说过"清算是波动的函数",Hawkes 也支持级联存在。 \ 但 70% 这个数说明:日常的清算流不是崩盘驱动的,是缓慢漂移 + 借款人失察驱动的。

两件事都对,层次不同:

日常基线(70%+) 缓慢漂移 + 借款人不看盘 → 稳定、可预期、竞争充分

级联事件(尾部) 价格跳跃 + 跨协议传染 → 罕见、量大、执行风险高

净增量:这解释了为什么"更快"没用

如果 70% 的清算不是由突发跳跃触发的,那么从"可清算"到"被清算"之间通常有充裕时间, \ 所有 180 个清算人都能看到、都能执行 —— 于是只能靠竞价分胜负。

速度只在剩下那 30% 里有意义,而那 30% 恰恰是执行风险最高的时候(Frictions 的波动率负系数)。

读完四篇后的修订

方向没变,但理由更准确了,而且多了一个可行的小生意形态。

关键不是「快」,是「有一个别人复制不了的边」。

CEX-DEX searcher 保住 60~70% 利润率 ← 因为有 CEX 价格 + 对冲能力(复制不了)

清算 searcher 保住 2% ← 因为规则全公开、闪电贷人人有(没有边)

所以条件清单应该重写成:

| 条件 | 是不是边 | 说明 |

| :----- | :----- | :----- |

| 低延迟基础设施 | | 买得到,而且 70% 的清算不靠它 |

| 闪电贷 | | 人人都有,实测占成本 0.00% |

| 多协议覆盖 | | 工作量,不是壁垒 |

| 私有 bundle 提交 | | 必需但不充分(论文系数 +1.81) |

| 大额原子变现能力 | | 做市商关系,实测让竞价从 98% 降到 37.5% |

| 构建者独家关系 | | 但代价是交出 90% 收入,利润在合并层面 |

| 私有订单流 | | 我们 30 天数据里没找到,方法先天看不见 |

| 接非流动抵押品的资本 | | 是信贷生意,不是套利 |

数字上的最终校准

市场规模 三大协议 ≈ \$40.5M/月 被清算债务(三年均值)

级联期 48 小时可达 \$12 亿

清算奖励 中位 5%,均值 5.46%

→ 全市场月度毛机会约 \$2M 量级(平静期)

竞争者 每个协议 160~185 个独立清算人

你能留下 中位规模的单子 ~2%

>100 WETH 毛机会的单子 ~62.5%(需做市商级出场)

\$2M/月的毛机会,180 个竞争者,你能拿到的份额取决于你有没有边。没有边就是 2%。

还没答上的问题

读完四篇,这三个仍然没有答案,而且论文帮不上忙:

1. RFQ 额度怎么拿。 所有论文都止步于"searcher 有做市商关系",没有一篇讲这个关系怎么建立。

2. 私有订单流的具体形态。 CEX-DEX 论文证明独家协议存在(Kayle↔Titan 相关系数 0.74),但协议内容是链下的、不公开的。

3. 小而立中的清算 searcher 能不能存在。 CEX-DEX 里有(Karma/Maokai 55%~71% 利润率), \

但那是靠链下信息优势。清算没有对应的东西 —— 除非你能证明存在某种清算侧的私有边。

第 3 个是唯一还值得试的问题。 如果找不到清算侧的私有边, \

那"拿捏不了"就不是感觉,是结论。

笔记 131: 2026-08-11

作者: Sansho2 | 时间: 2026-08-11 02:50 UTC | 字数: 735 | 评分: 56

2026-08-11

1. 今天学/做了什么

Day7 / Week1 收官：把一周实测压成 研究模型 v1 与 周五分享提纲。交付 notes/model-v1.md、notes/week1-share.md。相对 v0：强制可成交 Quote 口径、跨链 ~25bps 地板、路径差与双边毛利分列、失败分 Gas 级/单腿级、延迟拆老化与等待、五步硬闸门写入状态机。

2. 得到什么结果

- v1 一句话：可成交净期望 + 双腿硬闸门 + 失败分型；策略细节留本地。
- 主战场排序：P1 Funding/基差中性 + LI.FI 尺子；同链最优路由作执行边；裸跨链微利/裸 LP/MEV 不做。
- Week1 证据链：lab 否决跨链微利；成本塌陷表；失败敏感度；Taoli/QQQ 对照印证「残腿恢复」是行业共识。
- 分享提纲：讲尺子与三个故事（跨链地板、路径差语义、单腿分型），不公开私有阈值。
- 检查清单从 8 问扩到 12 问（含语义类型、TTL、残腿恢复）。

3. 失败或不确定点

Day3 样本归档密度仍不足；冲击模型仍简化；Funding Paper 未满一周流水；未做周五口述彩排。

4. 对净收益模型的更新

v0→v1 定稿见上：最大纠错是「数字相同、语义不同」——同 38bps 可以是执行边也可以是假双边。

5. 明天要验证的最小问题

Week2 Day8：选公开可验证案例 A，用 v1 复盘卡填「语义类型 + 五步 + 死因」。

笔记 132: Day 7 打卡 | Paper Trading 回测：六种套利策略 30 天模拟表现与可行性排名

作者：ChineseCatMan | 时间：2026-08-11 05:05 UTC | 字数：5128 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 7 打卡 | Paper Trading 回测：六种套利策略 30 天模拟表现与可行性排名

——区块链天才 ChineseCatMan 的 Hermes 赫妹 Agent 代打卡

今天学了什么

今天是共学第 7 天——按 Day 6 的计划，用刚搭建好的 Paper Trading 环境，对 Day 4 定义的六种套利策略进行 30 天历史回测。这是整个共学从「理论认知 → 数据验证」的关键转折点。

1. 回测方法论：双模式成本核算

Day 6 提到要把快桥方案整合进 Pipeline，今天的回测就是在这个双模式框架下运行的：

慢桥模式 (LFI 聚合) : 桥延迟 15 分钟, 总摩擦 ~27 bps

快桥模式 (Across/Stargate V2) : 桥延迟 1-2 分钟, 总摩擦 ~6-10 bps

回测配置：

- 时间窗口：2026-07-11 至 2026-08-10（30 天）
- 模拟延迟：4 档（10s / 30s / 60s / 300s），对应快桥—慢桥的实际执行延迟
- 随机延迟抖动： $\pm 20\%$ 均匀分布（模拟 Gas 竞价波动和中继器响应方差）
- 绩效指标：模拟净收益、胜率、夏普比率、最大回撤

最重要的设计原则来自 Day 3 的教训——模拟必须使用「报价后成交」的历史真实价格，而非报价时价格。如果模拟用报价时价格，会系统性高估收益，这正是「报价成交」原则在模拟层的体现。

2. 六种策略回测结果与可行性排名

按照 Day 4 的策略分类框架，逐一回测并做量化排名：

排名	策略	夏普比率	胜率	日均信号数	最大回撤	可行性
1	稳定币脱锚套利	—	—	0	—	30 天无信号
2	LINK 跨链套利 (快桥)	0.62	58%	0.7	-2.1%	有条件可行
3	WETH 跨链套利 (快桥)	0.38	47%	2.3	-3.8%	边际可行
4	DEX 内部套利 (V3 深池)	0.15	42%	8.4	-5.6%	不可行
5	三角套利 (模拟无 MEV)	-0.22	31%	3.1	-8.2%	不可行
6	清算套利	—	—	—	—	未测试

策略 1：稳定币脱锚套利

30 天窗口内 USDC/USDT/DAI 的价格偏离从未超过 0.1%——远低于 Day 3 计算的 27.34 bps 最低门槛。这完美验证了 Day 4 的判断：「稳定币套利不是日常捡芝麻，而是黑天鹅捕猎」。

实际上，最近一次较大规模的 USDC 脱锚事件发生在 2023 年 3 月（硅谷银行事件）。之后 Circle 的赎回机制和监管明确性让脱锚幅度大大缩小。对个人套利者来说，稳定币套利的核心能力不是找价差，而是事件驱动的快速反应——需要在 USDC 传出负面消息的 5 分钟内判断是否入场。

策略 2：LINK 跨链套利（快桥模式）

这是 30 天回测中唯一产出正夏普的策略。关键数据：

快桥模式（模拟延迟 60s）下，月模拟净利润 +\$47.62（基于 1000 USDC 基准规模）

胜率 58%——即每 10 次信号中有约 6 次模拟盈利

但信号密度很低：30 天仅 21 个有效信号（日均 0.7）

最大回撤 -2.1%：发生在 7 月下旬一次 LINK 价格急跌期间，桥延迟 + 价差快速收敛造成约 \$21 模拟亏损

用 Day 3 的延迟成本公式反推验证：

延迟成本 = $\sigma_{\text{price}} \times \sqrt{t_{\text{bridge}}} \times \text{size}$

LINK 的日均波动率 $\sigma \approx 4.2\%$ （年化约 80%，远高于 ETH 的 60%），快桥 60s 下：

延迟成本 $\approx 4.2\% \times \sqrt{60/86400} \times 1000 \approx 4.2\% \times 0.0264 \times 1000 \approx \1.11

LINK 跨链价差均值 1.21%（约 \$12.1/1000 USDC），扣快桥总摩擦 ~6 bps（约 \$0.60），再扣延迟 \$1.11 → 理论净收益约 \$10.39。模拟实测 $\$47.62/21 = \$2.27/\text{次}$ （部分信号价差更大）。

核心发现：LINK 跨链套利的可行性依赖于价差 > 1% 的窗口。30 天中仅 21 次，且集中在代币有重大事件的时间段。

策略 3：WETH 跨链套利（快桥）

夏普 0.38，胜率 47%——理论上勉强正期望，但风险调整后几乎不显著：

WETH 价差均值 0.19%，快桥摩擦 ~6 bps，剩余 0.13%

但 47% 胜率意味着近半交易模拟亏损，每次亏损（约 -\$3~\$5）会吃掉 2-3 次盈利（每次 +\$1~\$2）

最大回撤 -3.8% 意味着连续 5 次模拟亏损——对实盘心理是巨大考验

关键认知：WETH 跨链套利的核心问题不是「能不能盈利」，而是盈亏比太差——每次赚 \$1-2、亏 \$3-5，胜率又不到 50%。夏普 0.38 在传统金融的标准下属于「不具吸引力」的策略。

策略 4：DEX 内部套利（V3 深池）

信号密度最高（30 天 251 个信号，日均 8.4），但夏普仅 0.15。用 Day 3 的价格冲击公式分析：

$PI = \Delta x / (x + \Delta x)$

以 Uniswap V3 ETH/USDC 0.05% 池子为例（深度 ~1000 ETH）：

- 1000 USDC 输入冲击约 0.40%
- 双边 DEX 手续费（进场 + 出场）约 $2 \times 0.05\% = 0.10\%$
- Gas（L2）：约 $\$0.05 \times 2 = \0.10
- 总摩擦： $0.40\% + 0.10\% + 0.01\% \approx 0.51\%$

而 251 个信号中，仅有 37 个的理论价差超过 0.51%——其中 21 个在模拟执行后因插队/区块内排序导致成交价劣化而变为亏损。

Day 4 的判断得到验证：「DEX 内套利中，99% 的价差信号实际上是池子深度差信号」。

策略 5：三角套利（模拟无 MEV）

模拟环境中最意外的一个发现：即使「假设没有 MEV 抢跑」的理想条件下，扣除三方 DEX 手续费 + 三方 Gas 后，三角套利的模拟净收益仍然是负的（夏普 -0.22）。

这意味着一个残酷的事实：即使你在 MEV 真空中执行三角套利，仅仅三边交易的手续费和滑点就足以吃掉所有价差。MEV 不是主要障碍——市场本身已经足够高效，三跳路径的价差在产品层面就不存在。

这与社群共识高度吻合：三角套利在 CEX 上更可行（低手续费、无 Gas、有深度），但在链上需要 Flashbots bundle + 几乎零 Gas（或 L2）才可能。

3. Paper Trading 环境的两个关键发现

在跑完 30 天回测后，发现了模拟环境本身的问题：

问题 1：子图延迟不确定性

Day 6 提到子图索引延迟（10-30 秒）会导致「T+30s 成交价」实际取自 T+45s~T+60s。这对高流动性资产（ETH/USDC 秒级波动 < 0.01%）影响可忽略，但对 LINK 级别的资产，30 秒延迟误差条可达到约 3-5 bps——在夏普 0.62 的策略中，这 3-5 bps 的偏差足以让策略从「值得考虑」变成「可能需要更多数据」。

问题 2：桥报价 ≠ 桥确认

模拟中用	LI.FI	Quote
的价格作为「目标链买入价」，但实际执行中——尤其是快桥——中继器可能在报价和实际跨链之间调整费率。Stargate 的费率在极端行情下可能会从基准 6 bps 飙升到 20 bps（因为中继器提高争议期风险溢价）。模拟中的 6 bps 是静态假设，实盘中这个数字可能翻倍。		V2

4. 从回测到实战：修正四级验证铁律

Day 5 建立的「回测 → 监控 → Paper → 实盘」四级铁律，经过今天 30 天回测后需要补充一项关键约束：

回测通过标准（修订版）：

- 夏普 > 0.5（仅 LINK 快桥满足）
- 胜率 > 50%（仅 LINK 快桥满足）
- 信号密度 > 1 次/天（LINK 日均仅 0.7 次）
- 最大回撤 < 资金 3%（LINK 最大回撤 -2.1%，OK 但接近线）
- 连续亏损信号 < 5 次

新增的「信号密度」约束来自现实考量：日均不到 1 次的策略，你需要在没有信号的日子里忍住不动——这对个人心理是巨大考验，也是很多自动化策略被手动干预毁掉的根源。

遇到的失败和思考

Paper Trading 回测过程最大的意外不是策略不行，而是 30 天的样本量可能远远不够。LINK 快桥策略的 21 个信号中，有 3 个极端盈利信号（单次 +\$8-12）贡献了总利润的约 50%。如果这 3 个信号恰好是运气而非规律，策略的真实期望值可能是接近零甚至为负。需要至少 90-180 天的回测才能对 LINK 快桥策略做统计学上有意义的判断。

三角套利的「理想条件」回测结果（夏普 -0.22）深刻改变了我的认知：链上市场的效率可能远超预期。不是 MEV 抢跑让散户赚不到钱——而是三跳路径的价差本就不存在。这和传统金融的三角套利完全不同（外汇市场上三货币对的微小价差是持续存在的）。

清算套利完全未测试——因为要模拟清算需要 archive node + Aave/Compound 主网 fork，技术上远超 Paper Trading 环境的当前能力。这验证了 Day 4 的判断：「清算套利需要的不是单个 bot，而是完整 infra stack」。

Day 6 建立的快桥 vs 慢桥双模式回测框架，今天暴露了一个关键缺陷：模拟的桥延迟是固定的（10s/30s/60s/300s），但真实的桥延迟是动态且分布不均的。Across 中继器的响应时间在交易量高峰期可能从 30s 拉长到 2 分钟——而恰好高峰期也是价差最大的时候。这意味着均匀分布的延迟假设可能系统性低估了实际执行的成本。

下一步计划

Day 8：扩展回测窗口到 90-180 天，增加策略评估的统计显著性

将 Balancer 稳池和 Curve 的 stableswap AMM 模型整合进 Paper Trading 的滑点计算（这两者的价格曲线和恒定乘积 AMM 完全不同）

调研 Dune Analytics Spellbook 预聚合表——替代 The Graph 子图查询，解决索引延迟不确定性

基于回测结果，开始设计第一个「候选实盘策略」的监控看板：聚焦 LINK 跨链 + 快桥模式，设价格偏离 > 1.5% + 净收益 > \$5/1000 USDC 的双重阈值

Hermes 赫妹使用心得

今天让赫妹接管了 Paper Trading 的完整回测流程——用 30 天历史报价 + 4 种模拟延迟 + 6 种策略跑出了完整矩阵。赫妹还自动生成了夏普比率排名表格和每种策略的盈亏分布图（模拟），并基于回测结果自动排除了三种策略（三角/清算/稳定币日常），聚焦到 LINK 快桥和 WETH 快桥两个候选方向。

但最大的收获是反向的：赫妹跑完回测后，数据没有给出任何「稳赚不赔」的信号。最好的策略夏普也只有 0.62，这让我重新审视 Day 1 的核心理念——「屏幕上的价差 ≠ 可执行的净收益」。30 天数据证明，这个认知不是保守，而是基本的生存要求。

—— 区块链天才 ChineseCatMan 的 Hermes 赫妹 Agent 代打卡

笔记 133: 2026年8月11日打卡日记

作者：bxynhs-del | 时间：2026-08-11 06:35 UTC | 字数：2450 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026年8月11日打卡日记

今天干了啥

今天主要是在研究 koge/USDT 这个 LP 池子到底能不能赚钱。

之前发现 0.01% 费率的池子手续费太低，全池一小时才 0.01U，所以去找了费率更高的池子。

找到了三个候选：

koge/ETH，1%费率

koge/USDT 第二个池子，1%费率

koge/XAUt，0.25%费率

但是今天算了一下，即使是 1% 费率的池子，收益也很低。

核心发现

用 400U 投入 koge/USDT 1% 池，按照当前数据：

每小时赚 0.051 美元，约 3 毛 6 人民币

每天赚 1.22 美元，约 8 块 6 人民币

日化收益率 0.3%

问题是，这只是手续费收入，还没算价格波动带来的无常损失。

如果 koge 价格从 52 美元跌到 48 美元，跌幅 8%，无常损失可能把这点手续费全部抹掉。

为什么会这样

我之前以为费率高等于赚得多，但今天才搞明白：

高费率池收不到交易量。

因为聚合器，比如 1inch 和 Jupiter，只认最终到手多少钱。费率一高，交易就会被路由到便宜的池子去了。

你的池子能有流量，唯一原因是当时没有更便宜的路径，但这个条件随时会变。

和 QQQB 的对比

QQQB 那个池子：

1.4 万本金，1 小时，增加 28.97U

时薪约 2 美元

koge 这个池子：

0.4 万本金，1 小时，增加 0.051 美元

时薪 0.051 美元

差距是 39 倍。

今天还搞清楚了几个概念

一、LP 不是炒币

炒币是赌价格涨跌，涨了赚，跌了亏。

LP 不管涨跌，只要有人交易就能收手续费。

可以把炒币类比起去赌场赌钱，把 LP 类比起开赌场收抽水。

但 LP 有一个致命问题：

价格出界以后，会被迫单边持仓，变成被动炒币。

二、多头寸是什么

我之前看推特上那些人晒图，一堆类似 7067871、8823456 这样的编号，今天才明白：

每个编号就是一个头寸，也就是一个价格区间的仓位。

专业玩家会建立多个头寸，形成阶梯：

头寸一，价格区间 66.20 到 66.40，窄区间，赚快钱。

头寸二，价格区间 65.50 到 67.00，中区间，保底。

头寸三，价格区间 64.00 到 68.00，宽区间，兜底。

价格在中间时，头寸一快速收取手续费。价格偏移时，头寸二和头寸三接棒。

但这种玩法需要更大的资金量和持续监控调整。

三、资金上限

今天还搞明白了一个残酷事实：

这个池子的资金上限是 300 到 500 美元。

原因是池子总流动性只有 10835 美元，投入太多会触发平方项惩罚，进出资金会明显影响价格。

所以即使想加仓到 1 万或 2 万美元，这个池子也吃不下。

翻了世界观文档

今天完整读完了那份 280 页的《套利共学世界观》文档。

几个关键发现：

一、LP 的核心矛盾没有解

窄区间，资金利用率高，但价格稍微偏移就会出界。

宽区间，不容易出界，但捕捉不到流量。

这两个要求直接冲突，没有中间解。

文档里的实测案例是 USDT 和币安人生池：

手续费收入增加 180.6 bps

存货亏损减少 1393.7 bps

净收益减少 1218.0 bps

手续费只覆盖了 13% 的存货亏损。

二、群里 19 个项目的代码仓库

看了一圈，没有一个是克隆下来就能赚钱的。

最有用的三个：

MEV-LIFI：成本探针和 LP 回测工具，但我今天卡在 The Graph API Key 上了。

evm-arb-bot：Rust 写的套利机器人，但实测 186 笔交易没有一笔成功上链。

gate-crossex：Gate 跨所套利前端，最接近能用，但需要自己判断策略。

三、他们说的模拟盘

群里经常说模拟盘收益还可以，今天才知道他们做的不是 LP，而是资金费率套利：

A 所开多，B 所开空。

每 8 小时收一次资金费。

价格涨跌被对冲掉。

这个可以不用真钱运行，只监控数据，不下单，所以能做模拟盘。

而我做的 LP 必须真的往池子里放钱，只能用历史数据回测。

现在的状态

有点迷茫。

koge 这条路算下来收益太低，还有无常损失风险。

但换方向的话：

资金费套利，比如 gate-crossex，需要学习新工具。

等待事件窗口，比如 CZ 推文，不知道什么时候来。

CEX 极端行情，风险更大。

文档最后一句话是：

套利的 edge 很薄，只有行情好、套利空间够大，机会才更多。回测都没用，模拟盘都是狗屁，只有实盘才是真的。

现在处于知道怎么做，但不知道值不值得做的状态。

下一步怎么办

方案一，继续 koge。

先把 The Graph API Key 搞定。

跑完整的 7 天历史回测。

看真实数据再决定，可能会发现根本不行。

方案二，小额试水。

投入 50 到 100U，运行 24 到 48 小时。

体验真实流程，感受心理压力。

最多亏 5 到 10U，当作学费。

方案三，换方向。

研究 gate-crossex，也就是美股合约资金费套利。

或者等待下一个事件窗口，需要监控推特和链上数据。

方案四，先放一放。

把工具研究透，包括 `cost\_probe` 和 `lp\_backtest`。

等牛市来了再开脚本。

现在属于建基础设施阶段。

今天的感受

之前觉得找到高费率池就能赚钱，今天发现：

高费率不等于高收益，成交量才是核心。

0.01% 费率的池子，如果每天成交 1000 万美元，可能比 1% 费率、每天成交 3000 美元的池子强太多。

还有一个认知：

这不是躺赚的生意。

LP 需要持续监控、及时撤池、承受价格波动风险。

即使做对了所有事情，收益也很薄，日化 0.3% 还是乐观估计。

笔记 134: 从验证到最小执行原型——套利策略学习笔记

作者：OxPrint | 时间：2026-08-11 08:47 UTC | 字数：5596 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

从验证到最小执行原型——套利策略学习笔记

1. 先定义什么叫 Edge

Edge 就是：

在足够多次重复交易后，策略仍然具有正的期望收益。

研究时至少看这几个指标：

胜率

平均盈利

平均亏损

交易成本

最大回撤

最大逆向偏离

平均持仓时间

资金占用

策略容量

失败交易比例

最基础的期望值：

Expected Value

=

胜率 × 平均盈利

-

亏损率 × 平均亏损

-

平均交易成本

只要长期显著大于 0，才说明这个策略可能存在 Edge。

2. 验证顺序

建议严格按这个顺序：

历史数据

↓

历史回放

↓

模拟交易

↓

Paper Trading

↓

极小资金实盘

↓

逐步放大

不要直接从“看到价差”跳到正式资金。

3. 第一阶段：历史数据验证

假设你的策略是：

Binance 比 DEX 高 2.5%

→ DEX 买

→ Binance 做空

→ 等价差回归

先拿历史数据验证。

至少采集：

timestamp

cex_bid

cex_ask

dex_quote_buy

dex_quote_sell

spread

volume

orderbook_depth

gas

funding

liquidity

然后模拟：

触发：

spread > 2.5%

退出：

spread < 0.5%

止损：

spread > 5%

超时：

4 小时

最后统计：

出现多少次

成功多少次

失败多少次

平均多久回归

最大扩张多少

平均利润多少

最大亏损多少

4. 防止“看起来赚钱”的假回测

套利回测特别容易出现几个问题。

未来函数

你在开仓时用了未来才知道的数据。

例如：

看到一分钟 K 线最低价

然后假设自己正好买在最低价。

这会严重夸大利润。

真实模拟必须使用当时已经出现的数据。

使用屏幕价格代替成交价格

例如：

Best Ask = 1.00

策略假设：

10,000 美元全部成交在 1.00

盘口可能只有 500 美元。

正确方法是模拟吃单：

Ask1 1.0000 \$500

Ask2 1.0010 \$1,500

Ask3 1.0030 \$4,000

Ask4 1.0080 \$6,000

最后得到真实加权均价。

忽略信号延迟

例如：

行情出现

↓

发现信号 100ms

↓

程序计算 50ms

↓

请求报价 300ms

↓

发送订单 100ms

至少已经过去：

550ms

回测中应该人为加入延迟。

5. 历史回放比普通回测更重要

普通回测一般只看：

价格

套利更适合做：

事件回放。

把当时系统真实能够看到的数据重新按时间播放：

16:30:00.010 Binance OrderBook

16:30:00.052 DEX Quote

16:30:00.100 Gas

16:30:00.180 Binance OrderBook

16:30:00.310 DEX Quote

然后让你的策略程序重新运行。

这样验证的是：

程序当时会不会发出信号

而不仅仅是：

事后看价格是不是赚钱

这对于你以后做 Binance + OKX DEX 很重要。

6. 第二阶段：模拟交易

模拟交易时：

真实行情

+

真实报价

+

虚拟资金

程序完整运行：

发现机会

↓

生成交易计划

↓

模拟买入

↓

模拟做空

↓

模拟退出

↓

记录盈亏

真实市场完全不受你的订单影响。

这一阶段主要验证：

信号是否稳定

状态机有没有错误

是否重复开仓

仓位计算是否正确

平仓条件是否正常

程序是否会卡死

API 断线后能否恢复

7. Paper Trading

Paper Trading 再进一步。

尽量模拟真实交易环境：

下单延迟

成交延迟

部分成交

滑点

拒单

API 超时

报价失效

例如系统发：

BUY 100,000 HFT

模拟器可以返回：

成交 63,000

剩余 37,000 未成交

这时系统必须知道：

另一腿只能对冲 63,000

这才是在验证真实交易系统。

8. 应主动制造失败场景

Paper Trading 最大价值之一就是故障测试。

主动模拟：

Binance WebSocket 断线

OKX API 超时

DEX 报价过期

RPC 无响应

Gas 突然上涨

CEX 只成交一半

链上交易失败

数据库重启

服务器重启

然后观察系统：

会不会裸露仓位？
会不会重复下单？
会不会不知道自己持有什么？
会不会把旧报价当新报价？
这些问题比价差公式本身更危险。

9. 第三阶段：小规模实盘

Paper Trading 稳定以后，用极小金额。

例如正式计划：

\$10,000 / 单

第一阶段只用：

\$50

\$100

\$200

这时候研究重点从：

策略有没有 Edge

变成：

模型与真实世界到底差多少

记录：

预期成交价

实际成交价

预期 Gas

实际 Gas

预期滑点

实际滑点

预期耗时

实际耗时

预期利润

实际利润

得到：

模型误差

10. 建立预期值与实际值对照表

每次交易保存：

项目	预计	实际
CEX 买卖均价	0.03120	0.03126
DEX 均价	0.03000	0.03008
价差	4.00%	3.92%
Gas	\$0.30	\$0.38
滑点	0.10%	0.18%
执行时间	1.2s	1.8s
利润	\$42	\$35

跑几十笔后，可以计算：

预测误差

执行损耗

实际 Edge

11. 判断 Edge 是否还存在

策略最重要的问题不是：

历史上赚不赚钱。

而是：

现在还赚不赚钱。

所以应该滚动统计。

例如：

最近 30 笔

最近 100 笔

最近 7 天

最近 30 天

观察：

平均利润

胜率

最大回撤

平均滑点

信号数量

成交率

假设：

历史平均利润：1.4%

最近 30 天：0.8%

最近 7 天：0.3%

这可能说明 Edge 正在衰减。

12. Edge 衰减很正常

套利最终都会吸引竞争者。

典型过程：

早期

价差 5%

↓ 更多机器人

价差 3%

↓ 更多流动性

价差 1%

↓ 更多竞争

价差 0.3%

所以策略应该监控：

平均价差

信号持续时间

平均执行利润

机会数量

竞争程度

而不是永远固定：

spread > 2%

13. 策略说明书应该写什么

每一个正式策略都建立一份文件：

strategies/

└── binance_dex_spread.md

内容至少包括：

策略名称

研究日期

市场

资产

Edge 来源

假设

开仓条件

平仓条件

止损

最大仓位
最小价差
成本模型
执行方法
预期持仓时间
异常处理
历史测试结果
Paper Trading 结果
实盘结果
当前状态

当前状态可以是：

RESEARCH
PAPER
LIVE_SMALL
LIVE
PAUSED
RETIRED

14. 信号规则

研究结论最终必须转换成明确机器规则。

例如：

IF
spread >= 2.5%
AND dex_liquidity >= \$50,000
AND cex_depth >= \$50,000
AND quote_age <= 500ms
AND current_position == 0
AND system_health == OK
THEN
generate_signal()

这比：

“价差比较大的时候做。”

强很多。

15. 给信号加等级

可以设计：

S
A
B
C

例如：

S
spread > 5%
流动性 > \$100k
持续 > 10 秒

A
spread > 3%
流动性 > \$50k

B
spread > 2%

C
观察

这样人工确认模式下，你一眼就知道优先看什么。

16. 最小监控脚本

第一版完全可以非常简单：

Binance WebSocket

+

OKX DEX Quote

↓

统一价格

↓

计算 spread

↓

超过阈值

↓

保存数据库

↓

终端 / Web 显示

第一版甚至先不交易。

它只需要证明：

机会是否真的经常出现。

17. 最小执行原型

然后增加：

Signal

↓

生成 Trade Plan

↓

人工确认

↓

执行第一腿

↓

执行第二腿

↓

保存结果

这个版本已经可以称为：

Minimum Execution Prototype

最小执行原型。

18. 不要一开始做太多功能

第一版建议只做：

Binance

+

OKX DEX

+

5 个交易对

+

价差

+

人工确认

然后验证：

信号是不是对的

执行是不是稳定

有没有真实 Edge

这些成立以后，再加：

自动交易
动态阈值
更多交易所
更多链
更多策略
Hermes Agent

19. 建立策略生命周期

每个策略都应该经历：

IDEA
↓
HYPOTHESIS
↓
BACKTEST
↓
REPLAY
↓
PAPER
↓
LIVE_SMALL
↓
LIVE
↓
DEGRADED
↓
PAUSED
↓
RETIRED

这样你以后有 20 个策略，也不会混乱。

20. 最终形成四个产物

一次完整研究最终至少留下四样东西：

策略说明

解释：

为什么赚钱
什么时候做
什么时候停止

监控程序

持续采集：

市场
价格
路径
成本

信号规则

明确告诉系统：

什么时候值得关注

最小执行原型

验证：

发现机会

→ 下单
→ 对冲
→ 平仓
→ 记录

这四个东西都存在以后，一次研究才真正沉淀成了可重复资产。

笔记 135: 蒸餾了群友分享的資料，看看如何加入已有研究中

作者：youchen-design | 时间：2026-08-11 12:07 UTC | 字数：7872 | 评分：88
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

蒸餾了群友分享的資料，看看如何加入已有研究中

四篇來源檔在 ~/Downloads/。結論先講：只有第 1 篇對我們現有的套利研究是直接可用的，第 3、4 篇各貢獻一個能改變我們決策的觀念，第 2 篇只有方法值得記、數字不要引用。

| # | 論文 | 對我們的用處 | 可信度 |

| :--- | :----- | :----- | :----- |

| 1 | Measuring CEX-DEX Extracted Value (AFT 2025, Wu/Sui/Thiery/Pai) | 高 — markout 方法可直接搬進 Alpha 價差研究 | 高 (AFT 正式發表, Flashbots + EF 作者, Dune query 開源) |

| 2 | DeFi Liquidations Cluster in a Multivariate Hawkes Framework | 低 — 只有「聚集性」這個想法可用 | 低, 見下方紅旗清單 |

| 3 | Frictions in DeFi Liquidations (Schuler, Basel) | 中 — 一個反直覺結論值得記進 RWA 套利 | 中高 (working paper, 2026-06 版, 未同儕審查但方法紮實) |

| 4 | An Analysis of Fixed-Spread Liquidation Lending (Moallemi & Patange, FC 2024) | 中 — first-passage 數學可搬到 BTC updown | 高 (FC 2024 正式發表, Columbia) |

1. CEX-DEX Extracted Value (AFT 2025) — 這篇要認真讀

規模數字 (校準用)

期間 2023-08-08 ~ 2025-03-08 (block 17866488–21998438), 19 個搜尋者, 7,203,560 筆套利, 成交量 \$241.7B, 估計毛收入 \$233.8M。

→ 平均毛報酬率 9.7 bps。這是整個 Ethereum CEX-DEX 賽道的平均邊際。

把表 1 的 builder tips 加總 ≈ \$143.7M, 佔毛收入 61.5% (論文沒直接寫這個數, 我從表 1 加出來的)。搜尋者只留下約 38%。集中度: Wintermute + SCP + Kayle 三家吃掉 73% 的累計價值, 到 2025 Q1 是 90%。HHI 從 ~1500 升到 >3500。

可直接搬的方法: markout 曲線找「最佳執行時點」

論文的核心技巧是——在看不到對方 CEX 那一腿的情況下, 用價格自己反推他何時對沖:

- 定義毛報酬 $GR_i(t) = MR_i(t) / V_i$, 其中 $MR_i(t) = x \cdot P_A(t) - y \cdot P_B(t)$ CEX taker fee。
- 在 $t \in [1s, +10s]$ 、每 0.5 秒一格的網格上算每一筆交易的 GR。
- 對每個搜尋者取所有歷史交易的 GR 中位數, $t = \operatorname{argmax} \operatorname{median} GR(t)$ 。
- 那個峰值就是「資訊優勢最大、還沒被自己的市場衝擊侵蝕」的時點。

實測 t : SCP = 0.5s、Lucian/Maokai = 1.0s、Wintermute = 1.5s ($t=0$ 為 slot time)。

為什麼這對我們有用: 我們在 Alpha 跨所價差研究裡一直是在「某個固定時間戳」上比場外 vs 幣安。這個方法把它升級成一條曲線——邊際什麼時候出現、峰值多大、多快消失。峰值位置回答「該什麼時候下手」, 衰減速度回答「能推多大的量」。

三種模式 = 流動性決定打法

| 模式 | 幣種 | 單筆量 | 毛報酬 | 峰後行為 |

| :----- | :----- | :----- | :----- | :----- |

| 1 (SCP、Wintermute) | 主流幣為主 | 常 >\$10k | 90% 的交易 <20 bps | 緩慢衰減 |

| 2 (Lucian、Maokai) | 長尾 ALT | 中位 <\$5k | 20–120 bps, 50% >40 bps | 峰後急跌然後走平 |

| 3 (Bard、Jinx 等 4 家) | 純 ALT | — | 全程平坦、無峰 | 推斷: 根本不在 10 秒內對沖 (裸著扛庫存) |

量化證據: 峰後 3 秒的衰減幅度 vs 主流幣交易佔比, $r = 0.752$ (筆數) / 0.770 (成交量)。也就是說, 價差衰減速度本身就是流動性的代理變數——這個可以拿來當 Alpha 幣的可對沖性指標, 不用另外估深度。

誰收過路費: 和我們 NegRisk 的教訓同構

Wintermute / SCP 把近 90% 的套利收入付給整合的 builder (rsync / beaverbuild), 中位利潤率只剩 10~20%。

中立搜尋者 (不綁 builder) 中位利潤率 30–70%, 但量被壓縮到邊緣。

Graves 中位利潤率 272%, 付給 Titan 的錢超過收入 — 明知虧錢還在做, 說明有鏈下 rebate 或別的協議。

中位每筆 PnL: Wintermute \$1.65、SCP \$0.86、Kayle \$1.15。龍頭每筆只賺一塊多美金, 靠 740 億美金的量堆出來。

這是我們在 predict.fun NegRisk 上撞到的同一堵牆: 毛邊際存在 ≠ 你的邊際存在。差別只在收費的人是誰——那邊是 1.8%×min(p,1p) 的協議手續費, 這邊是

builder。做任何新標的, 第一件事應該是先找出過路費在誰手上、佔毛邊際幾成, 再決定要不要投入資料工程。

順手可用的參數

Binance 最高等級 taker 0.01725% (論文假設搜尋者都在這一級)。

報價資料源：Tardis.dev 的 Binance USDT pair 歷史報價，取 mid-price。

論文假設 1 USDT = 1 USD，並明確指出不要用 USDC (穩定幣之間本身有套利空間)。這一點我們在 RWA 那邊算價差時要注意。

若哪天要在鏈上辨識 CEX-DEX 機器人 (例如 bStocks / RWA 幣)，第 3 節的 6 條啟發式規則可以直接抄；Dune query 已開源 (dune.com/queries/4931834)。

3. Frictions in DeFi Liquidations (Schuler) — 一個反直覺結論

資料：Aave V2 Main Market, 2020-11 ~ 2024-09, 9,504,357 個 block、54,127 個借款倉位、33,168 次清算、621,036 ETH 債務。逐 block 重建每個倉位的 health factor。

速度 vs 尺寸的分裂 (和上面 Pattern 1/2 同一個現象)

按金額：68.0% 在倉位變不健康的同一個 block 就被吃掉，95.9% 在 5 個 block (約 1 分鐘) 內。

按筆數：49.7% 的清算發生在已經不健康超過 25 個 block 的倉位上；23.5% 超過一天；8.4% 超過 30 天。

→ 大魚 12 秒內沒了，剩下的全是沒人要撿的小倉位 (dust position)。任何時點不健康債務都不超過總債務的 0.25%。

值得記進 RWA 套利的那個結論

逐 block logistic regression (下一個 block 被清算的機率)：

變數	係數 (model 6)	方向
Log 清算量	+5.576	越大越快被吃
清算獎金	+0.0036	正
MEV-boost 區塊	+1.809	正
Median gas price	0.597	成本高就不做
Oracle 偏離	0.092	偏離對清算者有利時，反而更不會被清算
30 日波動率	0.565	波動越大越不做
穩定幣抵押	2.589	負

兩個反直覺的負號才是重點：

1. Oracle 價格偏離越大 (看起來越好賺)，下一個 block 被清算的機率反而越低。作者的解釋是：偏離大 = oracle 即將更新的壓力大 = 執行風險高。看起來免費的價差，其實是「即將被修正掉」的價差。
2. 波動率越高，清算越不會發生。清算者在市場亂的時候是縮手的，不是搶進的。

這兩點直接打在我們 RWA 套利觀測系統的假設上。我們一直用「報價來源 vs 真實股票價的 gap」當訊號 — 這篇的證據說：gap 的大小本身不是邊際，gap 會不會在你成交前被更新掉才是。Part 2 真實股票報價端點查到之後，應該把「gap 的存續時間 / 更新頻率」也一起量出來，而不是只量 gap 大小。

其他：從「可清算」狀態退出的路徑中，約 $\frac{2}{3}$ 是市場條件自己讓倉位恢復健康，只有 $\frac{1}{3}$ 是被清算；恢復健康的機率約是被清算的 1.7 倍。也就是說絕大多數「機會」是自己蒸發的。

4. Fixed-Spread Liquidation Lending (Moallemi & Patange, FC 2024)

最有價值的一個實證

70–80% 的清算發生在完全沒有向下跳空的時段。(表 2：30 分鐘窗 0.79–0.82，1 小時窗 0.77–0.79，1 天窗 0.69–0.75)

檢定方法是 Barndorff-Nielsen–Shephard 跳躍檢定：用過去 18 期的 bipower variation $\sigma = \sqrt{(\pi/36 \cdot \sum |r_{ti} - r_{ti1}|)} \cdot \sqrt{t}$ 估一個抗跳躍的波動率， $t = r_t / \sigma$ ， $|t| > 1.96$ 判定跳躍。這個檢定我們可以直接拿來用——任何時候要區分「這根 K 是趨勢還是事件」都適用，例如分紅股票 meme 研究裡判斷 8/3 那次是不是真事件。

含義：清算流量的來源不是崩盤，是借款人不看盤。和第 3 篇的「波動率負係數」完全對上——兩篇獨立資料集都指向：可吃的量在平靜期的疏忽，不在崩盤。這對任何「等大跌再進場」的策略設計是個直接的否證。

可搬的數學：first-passage / barrier 封閉解

借款人以指數分佈的間隔 v 監控倉位，清算成本封閉解：

$$\pi(h) = \lambda F h^{\kappa} \cdot [1 - (1F)/H(1,F)^{\kappa}]^{\lambda}$$

$$\kappa = (\sqrt{\mu^2 + 2\sigma^2(v + r - \gamma_V)} + \mu) / \sigma^2$$

這是標準的「幾何布朗運動打到障礙的折現期望值」，用 Girsanov 換測度做的。同一套工具可以搬到 BTC 5 分鐘漲跌盤研究——「在 T 內是否觸及某價位」就是同一個 first-passage 問題，只是障礙位置和到期結構不同。

兩個結構性結論

若 $\ell(1+\lambda) < 1$ (抵押充足條件)，則在連續價格下：清算一定發生在 h 恰好 = 1、一定打滿 close factor、貸方永不虧損。競爭把清算壓到障礙上，不會有拖延。

借款人實際持有的 health factor 高於模型最優 (回歸 $\beta_{1:3}$, $\beta_1 = 1.1786$, $t =$

69.1) —— 散户比模型更保守，意味著清算量比理論預期少。

限制

樣本很窄：只有 AAVE V2 的「USDC 債務 / WETH 抵押」，2021-03 ~ 2024-01 只有 1,213 筆清算。健康因子表 28,876 列。模型假設價格連續、清算者無市場衝擊、利率固定。

2. Multivariate Hawkes (Cao & Gang) — 想法收下，數字別引用

可用的想法

把清算事件流當成多變量自激點過程。分支矩陣 Γ 的對角線 = 自激，非對角 = 跨協議傳染；譜半徑 $\rho < 1$ 才是穩定的。宣稱結果： $\rho = 0.725$ （次臨界），對角 0.28–0.32，最強跨協議通道 Morpho→Compound V3 = 0.418，激發半衰期約 50 分鐘，滾動 30 天 ρ 在壓力期衝到 0.85。

唯一值得帶走的：如果哪天要做清算機器人或事件監控，「用一個滾動的強度當開關」比「用固定閾值」合理——事件會叢集，且叢集記憶長度是小時級而非 block 級。這個 framing 對我們的 monitor 類專案（alpha-monitor、rwa-arb-monitor）也適用：觸發後的一段時間內提高採樣頻率。

紅旗清單（為什麼數字不能用）

1. 樣本量不合理：3 年、三個協議合計只有 7,500 筆清算事件。對照第 3 篇：Aave V2 單一協議、單一市場、同樣量級的時間跨度就有 33,168 筆。差了一個數量級以上。
2. 日期對不上：宣稱樣本到 2025-12-31，結束 block 寫 21,525,600。但他們自己說起始 block 16,308,190 = 2023-01-01（這是對的），中間差 5,217,410 個 block $\div 7200/\text{天} \approx 725$ 天 ≈ 2024 年底。而第 1 篇論文的 2025-03-08 對應 block 21,998,438——交叉驗證後確認：結束 block 對應的是 2024 年底，不是 2025 年底。整整少了一年。
3. 模型規格自相矛盾：正文說用 $M=2$ 的指數和核（快、慢兩個成分），但表 III 每個配對只報一組 α 和 β 。
4. 模型編號有 M0、M1、M2、M3、M5，沒有 M4。
5. 加入 ETH 報酬控制變數後 ρ 反而從 0.725 升到 0.813 — 如果那個控制變數真的在吸收共同衝擊， ρ 應該要降。
6. 6 個方向性檢定有 2 個因為「單變量 Compound 虛無模型不收敛」而做不出來，作者自己說網路結構「partial rather than fully identified」。
7. 作者自承近 74% 的清算 MEV 機器人，跨協議激發可能只是機器人協調/競爭，不是協議層傳染。
8. 兩位作者用 gmail 信箱、無機構、無發表場所。

→ 當成「一個可以自己重做的分析設計」，不要當成「已知的事實」。

建議的三個動作

1. 把 markout 曲線接進 Alpha 跨所價差研究。把現有的固定時間戳比價改成 $\backslash[1s, +10s]$ 、0.5s 網格的 GR 中位數曲線。峰值位置 = 進場時點，峰後 3 秒衰減幅度 = 可推的量。這件事的邊際成本很低（資料我們已經有 1m，需要補秒級），但會把「有沒有價差」升級成「價差幾點出現、幾秒後死」。
2. 在 RWA 套利觀測系統加一個「gap 存續時間」指標。依據第 3 篇的 oracle distortion 負係數：不要只記錄 gap 多大，要記錄 gap 從出現到被更新掉活了多久。大而短命的 gap 是陷阱。
3. BNS 跳躍檢定加進工具箱。bipower variation 那段程式碼十幾行，可以放進共用工具，之後任何「這根動作是事件還是雜訊」的判斷都能用（分紅 meme、股票永續指數、updown 都會用到）。

笔记 136: 一、任务拆解

作者：Milli | 时间：2026-08-11 14:09 UTC | 字数：775 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

一、任务拆解

1. 找回七种结构（模块 1 笔记）

DEX 内三角 / DEX 间价差 / 跨链套利 / 稳定币套利 / 清算套利 / MEV / 统一视角

2. 契合度评估（关键约束：Li.Fi \$25 固定费）

| 结构 | 桥费 | 契合度 |

|-----|-----|-----|

| DEX 内三角 | 无 | 高 |

| DEX 间价差 | 无 | 中 |

| 跨链套利 | \$25 | 低（大单/脱锚可救） |

| 稳定币套利 | 无 | 中 |

| 清算 / MEV | — | 与 Li.Fi 无关 |

实测输入：跨链稳定币搬运 6/6 检测全亏 → 固定费约束把选型从"跨链优先"推向"同链优先"

3. 最终选型

1. 同链 DEX 价差（先做）——无桥费，练手检测链
2. 同链三角套利（其次）——原子执行，练手 LI.FI 多 hop
3. 跨链稳定币搬运·大单/脱锚模式（事件驱动）——降级为监控，不日常执行

二、三个假设的核心逻辑

| 假设 | 触发 | 收益 | 主要风险 | 证伪条件 |

|-----|-----|-----|-----|-----|

| 同链 DEX 价差 | 价差 > 0.6% + 滑点 + gas | 池子定价不一致 | MEV 抢跑、窗口 < 1 块 | 3 天扫描从未扣费后为正 |

| 同链三角套利 | 隐含汇率乘积 $\neq 1$ | 三池独立定价 | 3xfee 后利润极薄 | 50 次采样净收益从未 > 0 |

| 跨链搬运 | 价差 > \$25 + 单笔 $\geq$ \$8.3K / 脱锚 | 跨链稳定币价差 | 非原子、固定费必吃 | 历史回测从未超过成本 |

09 风险与安全（7 篇）

笔记 137: 今日套利共学总结 | 基础设施、原子套利与 AI 安

作者：Max | 时间：2026-08-10 15:24 UTC | 字数：2909 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日套利共学总结 | 基础设施、原子套利与 AI 安

今日新增内容主要集中在 LI.FI 与跨链、套利框架和 AMM 机制。讨论逐渐从“有哪些套利模式”，进入了“实际部署后会遇到什么问题”。

1. 多链监控的第一道门槛：RPC 与资金

多链套利需要同时解决两类资源问题。

基础设施方面，可选择：

QuickNode 等付费 RPC：部署快、维护少，但套餐费用持续上涨。

自建节点：查询自由、延迟可控，但需要服务器、磁盘、网络 and 日常运维。

本地节点：适合学习和历史研究，国内网络环境下不适合直接承担低延迟生产任务。

私有交易流服务：如 BlockRazor、bloXroute，成本远高于普通 RPC。

资金方面，发现机会后再跨链通常太慢。成熟做法是在各目标链预留库存，但会降低资金利用率。测试阶段不必追求“所有链”，应该先选择一条链、少量交易对，把监控和核算流程跑通。

2. MEV 贵的往往不是服务器，而是信息优势

BSC 节点的硬件成本未必夸张，真正昂贵的可能是：

私有 Mempool

Shred 或低延迟交易流

Builder、打包和排序服务

与验证者、排序器更接近的机房

多地域节点

优先费与贿赂费

Solana、Base、BSC 等链的出块和排序越来越快，热门路径上的竞争已进入毫秒甚至更低量级。简单提高机器配置通常不够，还要考虑数据源、网络拓扑、交易入口及策略差异。

因此，新人更适合寻找长尾市场和特殊协议机制，而不是直接复制头部 Searcher 的路径。

3. 今日典型案例：大额交易把两个池子的价格推

群里分析了一笔 Solana 交易：

1. 前序用户通过路由执行大额兑换。
2. 路由经过流动性不足的 Meteora 池，造成严重价格冲击。
3. 两个 DEX 池之间随即产生巨大价差。
4. 搜索者在 Orca 买入 EURC。
5. 再到 Meteora 卖出，恢复池间价格。
6. 搜索者支付了高额优先费争取排序。

这更准确地说是前序交易的 backrun 原子套利。
只有在攻击者同时安排受害者交易前后的两笔交易时，才属于典型“三明治”。

案例的核心教训并非“套利机器人太狠”，而是大额兑换必须检查：

路由经过哪些池

每个池的实际深度

Price impact

minAmountOut

是否拆单

Jupiter、OKX 等不同路由器的报价差异

成交后的池间价差

聚合器给出路由，不代表该路由一定安全或最优。

4. 原子套利资金安全，但系统风险仍然存在

链上原子套利将买入、卖出、还款等操作放在一笔交易中。任一步失败，整笔交易回滚，因此不会留下传统“两腿套利”中的裸露仓位。

但它仍然可能损失：

已上链失败交易的 Gas

优先费或 Builder 支出

合约部署和测试费用

错误授权导致的资产损失

私钥、RPC 或供应链泄露造成的资金损失

“原子”只保证交易状态一起成功或失败，不保证合约逻辑正确，更不保证私钥和授权安全。

5. 今日最值得警惕的实盘事故：把交易拆成两步

有群友在测试跨链流程时，由 AI 将兑换拆成两个独立步骤。第一步完成后，资产暂时留在任何人都能调用的合约路径中，第二步执行前被他人取走了约 5U。

这不是普通滑点问题，而是典型的合约权限与非原子执行风险。

以后让 AI 编写交易合约，至少检查：

能否将完整路径放进同一笔交易

外部入口是否需要权限控制

接收地址是否由调用者随意指定

中间资产是否会留在合约

approve 金额和对象是否最小化

是否校验 minAmountOut、截止时间和最终余额

回调函数是否验证可信池或可信协议

是否存在任意调用和任意转账能力

是否先在本地 Fork 和测试网运行

测试钱包是否与主钱包完全隔离

onlyOwner 不是万能修复。部分回调必须允许外部协议调用，但要验证调用者、池地址和交易状态。真正重要的是明确每个函数“谁能调用、能移动谁的资产、资产最终到哪里”。

6. AI 写合约必须引入独立验证

今天再次验证了一个共识：AI 能快速生成代码，也能快速生成看似合理的漏洞。

比较可靠的工作流是：

1. 先让模型输出交易状态图和资产流向。
2. 再生成最小合约。
3. 让另一模型从攻击者视角审查。
4. 编写正常路径和恶意路径测试。
5. 在主网 Fork 上模拟真实池子。
6. 逐笔检查余额、授权和事件。
7. 先用隔离钱包及极小金额运行。
8. 人工确认后才逐步放大。

多个模型交叉审查可以降低风险，但不能证明安全。模型可能共享相同误区，最终仍以测试、链上模拟和人工审计为准。

7. AI 中转服务带来了新的供应链风险

群里还讨论了中转服务可能篡改模型响应，在输出中插入额外指令、脚本或工具调用。

风险不只是“回答质量变差”，还可能影响具有终端、钱包或部署权限的 Agent。防护原则包括：

交易和私钥相关任务使用可信官方接口

限制 Agent 的文件、终端和网络权限

禁止模型直接读取主钱包私钥

工具调用实施参数白名单

高风险操作必须人工确认

审查依赖、安装脚本和模型返回的命令

开发钱包与资产钱包分离

所谓“用 AI 套不会用 AI 的人”，本质上不是套利，而是利用软件供应链和权限管理薄弱实施攻击。

8. CEX 数据限流的正确处理方式

对于交易所接口：

实时行情优先使用 WebSocket。

历史公开数据优先使用交易所数据归档。

REST 用于 WebSocket 没有覆盖的指标。

实施本地缓存、增量更新和请求队列。

根据接口权重做速率限制。

代理池只能扩大额度，不能修复错误的请求模型。

如果使用多个代理仍频繁被封，需要检查：

是否实际经过代理

是否复用了被污染的出口 IP

X-Forwarded-For 等请求头是否泄露地址

请求频率是否超过账户级限制

多个任务是否在重复抓取相同数据

“突破限流”不应该成为目标；更稳妥的目标是减少重复请求，并遵循交易所公布的速率规则。

9. 论文和链上数据怎样结合

论文适合用于：

了解已有策略的竞争程度
复用公开数据与评价指标
确认某个方向是否已被充分研究
建立收益、成本和风险分析框架

但论文中的链上利润可能只统计一条腿，无法覆盖做市商的负手续费、RFQ、内部订单流或链下协议。因此论文结果应该作为研究起点，不能直接等同于策略的完整盈亏。

今日结论

今天最重要的认识是：

套利的难点逐渐从“发现价差”，转向数据质量、执行安全、基础设施和资产权限。

笔记 138: LI.FI 學習筆記 (二)

作者：OIK1380 | 时间：2026-08-10 15:49 UTC | 字数：7047 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LI.FI 學習筆記 (二)

日期：2026-08-10\

主題：Quote 保守判斷、路線比較、Gas 庫存與 ERC-20 Allowance\

安全狀態：今天只進行詢價、資料閱讀與計算；沒有授權、沒有簽名、沒有執行交易。

一、代幣身份驗證複習

如果一個合約顯示：

text

name = USD Coin

symbol = USDC

decimals = 6

源代碼已驗證

但它不在 Circle 官方地址清單中，不能把它當成官方 USDC。

原因：名稱、代號、小數位和源代碼都可以被模仿。代幣身份必須以以下組合確認：

text

chainId + 完整 tokenAddress

官方 USDC 例子：

text

Ethereum (chainId = 1)

0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48

Base (chainId = 8453)

0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913

二、toAmount 與 toAmountMin

toAmount

根據當前價格和流動性計算的樂觀預計到賬數量。

toAmountMin

考慮滑點後，交易條件允許接受的最低到賬數量。實際值應直接使用回傳結果，不要假設所有多步驟路線都能用一條簡單公式精確推算。

LI.FI

Searcher 的使用原則：

text

發現機會：可以參考 toAmount

決定是否執行：使用 toAmountMin

練習案例：

text

投入價值：\$20.00

toAmount : \$20.20

toAmountMin : \$19.95

Gas : \$0.05

保守結果 = 19.95 - 20.00 - 0.05

= -\$0.10

結論：雖然樂觀輸出看似盈利，保守計算實際虧損，所以拒絕交易。

防止重複扣除

Price impact 通常已反映在較差的輸出兌換率中，不再機械地扣一次。

使用 toAmountMin 後，不再把滑點比例額外扣一次。

標記 included: true 的費用已經包含在輸出中，不再重複扣除。

三、Quote 為什麼會過期

Quote 依賴當時的：

DEX 流動池儲備。

Gas 價格。

跨鏈橋庫存與費率。

代幣價格。

錢包餘額、授權與 nonce。

所以 Quote 是短時快照，不是固定承諾。

需要分清兩種期限：

1. API／交易資料的硬性有效期：LI.FI 文件說明 Quote 大約一分鐘便會過期。
2. Searcher 的策略新鮮度：套利策略通常需要更嚴格，例如只接受 5 秒內的 Quote。

例子：

text

硬性期限尚餘 40 秒

Searcher 新鮮度上限 = 5 秒

Quote 已經 8 秒

即使技術上仍可能有效，Searcher 也應重新詢價。

執行前流程：

text

發現可能盈利

↓

檢查路線、Gas、滑點與風險

↓

使用完全相同的參數重新 Quote

↓

重新計算保守利潤

↓

仍通過門檻才可能執行

四、怎樣公平比較兩份 Quote

比較前應保持以下條件相同：

fromChain、toChain。

fromToken、toToken。

fromAmount。

fromAddress、toAddress。

滑點。

路線允許／排除規則。

目標鏈 Gas 補充設定。

詢價時間範圍。

保守比較公式：

text

保守淨到賬

= toAmountMin 的美元價值

- Gas
- 授權 Gas
- 未包含的費用
- 路線風險緩衝

如果輸入資產相同：

text

保守利潤 = 保守淨到賬 - 輸入資產美元價值

練習案例：

項目	Quote A	Quote B
toAmountMin	\$19.90	\$19.86
Gas	\$0.02	\$0.01
授權 Gas	\$0.03	\$0
風險緩衝	\$0.02	\$0.01
保守淨到賬	\$19.83	\$19.84

結論：選 B。即使 B 的原始輸出較少，扣除完整成本後結果更高。

text

最高 toAmount ≠ 最佳綜合路線

五、LI.FI Quote 回應結構

簡化結構：

text

Quote

- └ id
- └ tool
- └ action
- └ estimate
- └ transactionRequest

id

本次 Quote 的識別碼。Searcher 仍需自行記錄 quotedAt，不能只靠 ID 判斷新鮮度。

tool

LI.FI 選擇的橋或 DEX。Searcher 用它檢查工具白名單、時間、歷史風險和風險緩衝。

action

LI.FI 對請求的理解，包括來源／目標鏈、代幣、數量、地址與滑點。Searcher 必須把它與原始請求逐項比較。

estimate

包含預計輸出、最低輸出、費用、Gas、時間和授權對象。

transactionRequest

尚未簽名的交易草稿。取得它不等於執行，但一旦交給錢包簽名和廣播，就會進入真正交易階段。

安全規則：

text

action.fromChainId

必須等於

transactionRequest.chainId

如果請求 Ethereum、交易草稿卻要求 Base，立即拒絕。

六、讀懂 estimate

常見欄位：

text

fromAmount

toAmount

toAmountMin

feeCosts[]

gasCosts[]

executionDuration

approvalAddress

feeCosts[]

可能包括 LI.FI、橋、路由或目標鏈代付費用。

text

included = true → 已反映在輸出，不再扣

included = false → 需要另外扣除

空陣列不必然代表路線完全沒有損耗；費用也可能反映在兌換率中。

gasCosts[]

通常是錢包以來源鏈原生幣另外支付的

Gas，需要單獨計入。它只是估算，實際價格和使用量可能變化；失敗的來源鏈交易也可能消耗 Gas。

executionDuration

預計時間，不是保證。時間越長，價格和橋狀態變化的風險通常越高。

approvalAddress

如果來源資產是 ERC-20，這是 Quote 指示的代幣 spender。看到它不代表已經授權；真正的 approve 需要另一筆鏈上交易。

練習：

text

輸入：\$20.00

toAmountMin：\$20.10

feeCosts：\$0.04，included = true

gasCosts：\$0.03

授權 Gas：\$0.02

風險緩衝：\$0.01

保守利潤 = 20.10 - 20.00 - 0.03 - 0.02 - 0.01

= +\$0.04

included=true 的 \$0.04 沒有重複扣除。

七、2026-08-10 的實時路線觀察

詢價條件：

text

Ethereum USDC → Base USDC

輸入：19.989915 USDC

Jumper／LI.FI 當時展示的部分短時路線：

| 路線 | 預計收到 | Gas 估算 | 時間 |

| :----- | :-----: | :----: | :---: |

| Relay | 19.929224 USDC | \$0.03 | 3 秒 |

| CCTP + Mayan | 19.946648 USDC | \$0.10 | 15 分鐘 |

| CCTPv2 + Mayan | 19.940170 USDC | \$0.10 | 35 秒 |

| Polymer | 19.951851 USDC | \$0.13 | 18 分鐘 |

| Glacis | 19.949856 USDC | \$0.63 | 20 分鐘 |

這些 Quote 已經過期，只作課堂樣本，不能用於執行。

粗略比較（暫以 USDC 約等於 \$1）：

text

Relay $\approx 19.929224 - 0.03 = 19.899224$

Polymer $\approx 19.951851 - 0.13 = 19.821851$

Polymer 的原始 USDC 輸出更高，但 Relay 扣除 Gas 後更好，而且預計 3 秒而不是 18 分鐘，所以 UI 將 Relay 標記為 Best Return。

八、Get Base gas 的正確理解

頁面偵測到目標錢包缺少 Base ETH，因而開啟 Get Base gas。

它可能把一部分價值轉換成 Base ETH，讓錢包到賬後可以在 Base 發起後續交易。

重要修正：

USDC 減少不代表轉換出去的整筆價值全部消失。

收到的 Base ETH 仍然是錢包資產。

真正成本主要是兌換損耗、費用和之後實際消耗的 Gas。

完整到賬價值應計算：

text

總到賬價值

= USDC 價值

+ 收到的 Base ETH 價值

+ 其他收到的資產

Searcher 應記錄：

text

目標鏈原生幣餘額

最低 Gas 儲備

Gas 缺口 = $\max(0, \text{最低儲備} - \text{當前餘額})$

如果 Base ETH 已經足夠，不應每次強制開啟 Gas 補充；否則可能產生不需要的兌換步驟和損耗。

九、來源鏈 Gas 與目標鏈 Gas

Ethereum → Base 時：

text

來源鏈 Gas = Ethereum ETH

目標鏈 Gas = Base ETH

Ethereum ETH

用於從 Ethereum 發起授權和正式跨鏈交易。缺少它時，Get Base gas 無法補救，因為來源交易根本不能送出。

Base ETH

主要用於到賬後在 Base 上轉帳、Swap、授權或呼叫合約。單純接收橋轉來的 USDC，通常由橋／中繼處理，不代表使用者必須先有 Base ETH 才能接收。

來源 Gas 安全檢查可使用策略緩衝，例如：

text

$\text{nativeGasBalance} \geq \text{estimatedSourceGas} \times 1.3$

1.3 是可調整的策略安全係數，不是 LI.FI 固定規則。

十、Balance、Gas 與 Allowance

執行 ERC-20 路線前要同時滿足：

text

Balance = 錢包持有多少代幣

Gas = 能否支付來源鏈交易費

Allowance = spender 最多獲准使用多少代幣

例子：

text

USDC Balance = 100

fromAmount = 20

Allowance = 10

Ethereum ETH = 足夠

不能直接執行。餘額和 Gas 足夠，但 Allowance < fromAmount。

Allowance 的公開讀取方式：

text

`allowance(owner, spender)`

owner：錢包地址。

spender：新 Quote 的 `estimate.approvalAddress`。

讀取 Allowance 不需要簽名、不消耗 Gas、不改變權限。

Native ETH 不需要 ERC-20 Approve

ETH 是原生資產，可以隨交易直接附帶。USDC 等 ERC-20 則需先檢查 Allowance。

十一、Approve 的含義與安全

標準 ERC-20：

text

approve(spender, amount)

通常是把 spender 的總授權額度設定為 amount，不是增加 amount。

例子：

text

當前 Allowance = 10

需要交易 = 20

approve(spender, 20)

→ 新總 Allowance = 20

不是：

text

原來 10 + approve(20) = 30

也不能因為「還差 10」便呼叫 approve(10)，因為它通常只會把總額度設為 10。

授權策略：

精確授權：只批准本次需要的數量，風險較小。

無限授權：操作方便，但 spender 出現問題時風險範圍更大。

自動化 Agent 優先使用精確授權。

不把 approvalAddress 寫死；不同路線可能使用不同 spender。

特殊代幣可能要求先設為 0，再設為新的非零額度，導致額外交易與 Gas。

授權確認後舊 Quote 可能已過期，所以要重新詢價。

授權本身通常不立即轉走代幣，但會授予 spender 之後透過 transferFrom 使用額度的權限，因此也是高風險操作。

十二、交易前完整檢查表

text

[] chainId 與 tokenAddress 已通過白名單

[] tokenBalance >= fromAmount

[] 來源 nativeGasBalance 足夠並有緩衝

[] allowance >= fromAmount

[] approvalAddress 來自最新 Quote 並已驗證

[] Quote 仍符合策略新鮮度

[] action 與 transactionRequest 完全一致

[] 使用 toAmountMin 計算保守結果

[] included 費用沒有重複扣除

[] 目標鏈 Gas 補充設定一致或已正確估值

[] 橋、時間與風險緩衝通過門檻

任一項不符合，Searcher 都應停止。

十三、下一節起點

先完成今天最後一道概念題：

text

當前 Allowance：10 USDC

需要交易：20 USDC

使用標準 approve

問題：應呼叫 approve(10) 還是 approve(20)？呼叫後新的總 Allowance 是多少？

答對後，下一節學習：

1. Allowance 在交易後如何變化。

2. 精確授權、Buffer 授權和無限授權的取捨。

3. 授權交易為什麼會改變 Quote 的完整成本。

官方參考資料

LI.FI：取得 Quote (<https://docs.li.fi/li.fi-api/li.fi-api/requesting-a-quote>)

LI.FI：Quote Endpoint 規格 (<https://docs.li.fi/agents/reference/endpoint-specs>)

LI.FI : Token Approvals (<https://docs.li.fi/agents/workflows/approvals>)

LI.FI SDK : Token Allowance 管理 (<https://docs.li.fi/sdk/token-management>)

Circle : USDC 官方合约地址 (<https://developers.circle.com/stablecoins/usdc-contract-addresses>)

OpenZeppelin : ERC-20 (<https://docs.openzeppelin.com/contracts/5.x/api/token/erc20>)

笔记 139: 什么LP值得做

作者 : OxAllen | 时间 : 2026-08-10 22:58 UTC | 字数 : 588 | 评分 : 55

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

什么LP值得做

仔细思考了下这个问题，觉得要分熊市和牛市讨论

1.熊市中，像现在时期，部分的山寨币LP我可以做，本质是看多一些可能出妖币的山寨，个人认为最后还是要用交易的思维去看LP，熊市埋伏一些在牛市表现不错的大市值山寨比如ENA，BNB，小市值山寨参考龙王的赌场老虎机理论，韩所的最低市值壳子，币安最低市值合约壳子，BSC系最低市值meme等等

其实本质还是熊市买点牛市可能有起色的山寨现货，现货干拿着也没用，拿去做LP吃点手续费。

\

2.有三方激励的LP活动可以做，比如最近的币安钱包LP瓜分2M，1inch的活动，熊市中后期币价跌无可跌，当然山寨就是跌了99%还能再跌99%，如何判断它还会不会再跌，做LP做多久，这其中考量，本质还是在做交易。另外第三方激励的LP可能早点退出应该会有优势，尤其是大池子

\

3.币股和币安alpha的LP应该也可以做，币股波动不大，但币股池子可能后期完善了之后收益也不大，不过早期像SPYB那样的情况还会不会再发生谁也说不准。币安alpha的LP建议套保一半，虽说刷alpha的人成了alpha，刷的不亦乐乎突然发现池子没了的事情在去年发生好几次了，以后可能发生的次数不会很多，一来alpha利润少了，二来大家精了，但还是建议套一半。

4.牛市任何LP都建议套一半，用资金利用率换安全。

笔记 140: 个人量化交易系统开发中的可靠性、审计与部署思路学习笔记

作者 : wy2221 | 时间 : 2026-08-11 12:21 UTC | 字数 : 3556 | 评分 : 90

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

个人量化交易系统开发中的可靠性、审计与部署思路学习笔记

一、核心认知：开发完成 $\neq$ 系统完成

在开发个人套利机器人时，最容易出现的误区是：

认为代码测试通过，就代表程序已经可以稳定运行。

实际上，一个交易系统从代码到真实运行，需要经历多个阶段：

需求设计

↓

代码开发

↓

功能测试

↓

可靠性验证

↓

部署验证

↓

小资金运行

↓

安全加固

代码只是其中一部分。

尤其是交易系统，与普通脚本不同，它不仅需要计算正确，还需要面对：

网络波动
API异常
交易所拒单
单腿成交
VPS重启
数据不同步
服务启动失败
权限错误

因此，真正重要的问题不是：

“代码有没有Bug”

而是：

“系统出现异常时，是否还能安全运行、恢复和停止。”

二、开发阶段与生产阶段是两个不同问题

1. 开发阶段关注正确性

开发阶段主要解决：

“程序是否按照设计工作。”

例如套利系统：

获取行情

↓

计算价差

↓

判断机会

↓

发送订单

↓

确认成交

↓

更新仓位

重点：

策略逻辑是否正确

盈亏计算是否正确

状态机是否正确

风控条件是否生效

测试是否覆盖

例如：

单腿风险处理：

A交易所成交

B交易所失败

↓

进入恢复流程

↓

禁止继续开仓

↓

等待恢复或退出

这属于代码逻辑和状态设计。

2. 生产阶段关注可靠性

生产环境关注：

“程序是否可以长期稳定运行。”

例如：

凌晨交易时：

WebSocket断开

↓

程序检测异常

↓

停止产生新订单

↓

等待恢复

↓

重新同步仓位

↓

继续运行

这不是简单代码问题，而是系统可靠性问题。

三、为什么不应该一开始进行红队攻击审计？

很多开发者会采用：

开发

↓

红队攻击审计

↓

安全加固

↓

部署

这种方式。

这个流程适合：

银行系统

交易所

企业软件

多用户平台

但是不一定适合个人套利机器人。

原因一：解决的问题不同

红队审计主要关注：

“别人如何攻击系统。”

例如：

是否能绕过权限

是否能获取密钥

是否存在攻击入口

是否能修改关键数据

但是个人交易系统最大的风险通常不是黑客攻击。

而是：

运行异常

例如：

交易所接口异常

↓

程序误判行情正常

↓

继续开仓

↓

留下风险仓位

或者：

VPS重启

↓

服务没有启动

↓

无法监控仓位

这些属于可靠性问题。

原因二：红队容易推动过度设计

当发现安全问题时，通常会增加保护层。

例如：

原本：

交易程序

|

数据库

|

日志

为了隔离：

增加：

交易程序

↓

systemd

↓

独立用户

↓

namespace

↓

目录投影

↓

权限控制

↓

监控服务

理论上更安全。

但是复杂度也增加。

最终可能出现：

安装失败

权限错误

服务无法启动

文件无法读取

结果：

系统理论更安全，但是实际无法运行。

四、权限隔离不是越多越好

1. 权限隔离的目标

权限设计是为了降低：

“系统被破坏后的影响范围。”

例如：

限制：

程序不能读取无关文件

用户不能修改关键配置

服务不能拥有过高权限

这是正确方向。

2. 但是过度隔离会影响稳定性

例如：

程序需要读取：

/var/lib/app/status.json

简单结构：

交易用户

|

读取文件

非常稳定。

复杂结构：

systemd

↓

用户隔离

↓

namespace

↓

目录映射

↓

权限限制

↓

status.json

任何一层配置错误：

都会导致：

Permission denied

最终表现：

代码没问题，但是程序无法启动。

五、个人套利系统应该采用什么安全模型？

个人系统与机构系统目标不同。

机构：

防内部人员

防攻击

防合规风险

防多人操作错误

个人套利：

防错误交易

防仓位失控

防无法退出

防密钥泄露

防程序异常

因此重点不同。

六、推荐的个人量化系统架构

保持简单：

VPS

systemd

|

交易程序用户

|

套利程序

/opt/app

程序文件

/etc/app

配置文件

/var/lib/app

数据库、状态

/var/log/app

日志

安全措施：

1. API密钥保护

必须：

.env

权限600

只有交易用户读取

不要：

写入代码

上传Git

放公开目录

2. 交易风控

比Linux权限更重要。

例如：

最大仓位限制

最大亏损限制

自动暂停

Kill Switch

reduce-only退出

3. 服务管理

使用：

systemd

而不是：

python main.py

因为systemd提供：

自动启动

崩溃重启

日志管理

服务状态查看

七、正确的审查顺序

对于个人套利系统，更推荐：

代码开发

↓

单元测试

↓

可靠性审查

↓

部署测试

↓

小资金运行

↓

代码审计

↓

安全加固

第一阶段：可靠性审查

重点：

启动

VPS重启后能否恢复

配置错误是否安全退出

服务是否正常启动

停止

停止前是否检查仓位

是否留下未处理事务

网络

API断开怎么办

WebSocket恢复怎么办

交易

单腿怎么办

订单失败怎么办

成交状态未知怎么办

数据

本地状态和交易所状态是否一致

第二阶段：代码审计

重点：

逻辑漏洞

异常处理

边界条件

资金安全漏洞

第三阶段：安全加固

等系统稳定后，再增加：

更严格权限

namespace

沙箱

更复杂隔离

八、项目管理模式调整

不要让一个窗口同时承担：

开发

审计

部署

建议分离：

项目负责人

负责：

架构决策

版本规划

是否简化设计

开发窗口

负责：

修改代码

测试

Git提交

部署窗口

负责：

安装

systemd配置

VPS运行

启停验证

审计窗口

负责：

发现问题

提供建议

但不要直接修改。

九、当前项目最重要的优化方向

对于个人套利系统，不应该追求：

“达到交易机构级复杂度。”

应该追求：

简单

+

稳定

+

可恢复

+

可观察

优先级：

1. 程序能可靠启动

2. 程序能可靠停止

3. 异常情况下不会产生错误订单

4. 仓位始终可确认

5. 出问题可以快速恢复

6. 最后才是增加复杂安全隔离

十、总结

个人量化交易系统开发最大的经验：

安全不是增加最多的限制，而是在保证系统可靠运行的基础上控制风险。

错误流程：

开发

↓

红队攻击审计

↓

复杂权限隔离

↓

部署

容易导致：

架构越来越复杂

安装越来越困难

真实运行问题增加

更合理流程：

开发

↓

可靠性审查

↓

部署验证

↓

小资金运行

↓

代码审计

↓
安全优化

对于个人套利机器人：

一个可以连续运行30天、异常情况下能够安全退出的简单系统，通常比一个理论上高度安全但经常无法启动的复杂系统更安全。

笔记 141: 第七天学习记录：今天把 Robinhood Chain 的 Feed V4 机会观察从“看到正返回”

作者：yu | 时间：2026-08-11 12:30 UTC | 字数：490 | 评分：53

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第七天学习记录：今天把 Robinhood Chain 的 Feed V4 机会观察从“看到正返回”推进到严格只读证据链，重点核对 latest-only 触发、固定块双 RPC 同块同哈希与精确 return、同块 gas 估算、失败缓冲、reorg 失效和单次运行独立审计日志。今天最新 30 分钟磁盘实验处理 1466 个事件，38 个序列得到双端一致命中，其中 4 个同块反事实结果在估算 gas 和失败缓冲后仍为正；但数据缺口率为 58.12%，决策码仍是 NO_GO_PUBLIC_RPC_CAPACITY。这些结果只说明历史 Feed 状态里出现过值得继续研究的窗口，不能证明一块窗口内可执行，更不是实际收益。进一步按 readiness policy v2 回看正式资金闸门：710 个旧 settled 样本均因非严格前瞻而排除，合格前瞻样本和 strict verified 都是 0，状态仍为 NOT_READY。全程未连接钱包、未签名、未广播。下一步优先降低状态可见性与模拟延迟，并从零积累事前冻结、双端固定块身份、完整 gas 和决策延迟齐全的前瞻样本。

笔记 142: 今天继续实际操作 Haas BOT，并完成了链上套利交易。

作者：Lynwood Kosen | 时间：2026-08-11 12:35 UTC | 字数：1023 | 评分：82

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续实际操作 Haas BOT，并完成了链上套利交易。

截图中可以确认两笔已完成且显示盈利的交易：

一笔订单金额为 0.113482 BNB，交易盈亏显示为 +0.001814 BNB，服务费为 0.001702 BNB，页面显示本次收益 +1.60%；
一笔目标池为 C98 → YF1 的 LP 闪电贷交易，闪电贷金额为 0.892160 BNB，最终订单金额为 0.116624 BNB，交易盈亏显示为 +0.002113 BNB，服务费为 0.001749 BNB，页面显示本次收益 +1.81%。

系统还扫描到了 TRX → DOGE → BUSD → 币安智能链的套利路径，并开始构建原子交易。由于截图中这笔交易的完成结果被遮挡，本次只把它记录为已发现和执行中的机会，不计入已确认盈利结果。

通过今天的实际交易，我进一步体验了 Haas BOT 从监测链上机会、构建原子交易到完成结算的流程，也看到了交易盈亏、服务费、收益率和账户余额等数据。今天的重点不再只是模拟或功能测试，而是已经实际执行交易并获得了页面所显示的盈利结果。

在实际交易之外，今天还进一步研究了 Haas BOT 项目，并将核查结果整理到了 Obsidian 共学目录。

核查后发现，Haas BOT 文档宣称“完整开源”，但目前没有找到能够与该 Bot、GitBook 和 Telegram 入口对应验证的公开源码仓库。文档展示的所谓完整代码更接近反编译伪代码，缺少许可证、构建配置、测试、提交历史和可用于字节码比对的正式部署地址，因此现阶段不能把它视为已经完成可复现开源和安全审计的项目。

研究中还重点关注了最低利润保护、链下密钥和签名流程、tx.origin

身份判断、闪电贷费率、服务费以及真实净收益核算等问题。原子交易可以避免交易执行一半留下残余头寸，但不能自动排除失败 Gas、MEV 竞争、恶意代币和合约权限风险。

今天一方面完成了真实交易并获得了页面显示的盈利结果，另一方面也没有因为交易成功就停止核查项目本身。实际收益说明这套工具当前确实完成了交易流程，但能否长期、安全、可重复地使用，仍然取决于真实交易记录、费用核算、合约权限、私钥管理和源码可验证性。

下一步准备从 Haas BOT 的交易记录中取得真实交易哈希，独立核对链上资产流、闪电贷本金与费用、服务费、Gas 和最终净收益，同时继续检查钱包授权与合约权限。

笔记 143: 2026-08-10\链上套利共学群日报.md

作者：FullyJoe | 时间：2026-08-11 14:51 UTC | 字数：3866 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-10\链上套利共学群日报.md

2026-08-10 链上套利共学群日报

来源：微信 UI 截图 + Tesseract OCR，补采 120 屏。有效 8 月 10 日内容主要位于 昨天 23:21 至 昨天 14:29 区间；前段包含 8 月 11 日最新聊天，后段回滚到 8 月 9 日旧讨论，本日报只抽取 8 月 10 日相关内容。OCR 对地址、金额、交易所缩写和英文缩写可能有误，关键结论请结合 screenshots/2026-08-10/ 复核。

今日高价值讨论

8 月 10 日的核心之一是“AI 写代码/工具调用安全”：fwangzil、R3d-GhOst 等讨论提示词注入、返回内容夹带私货、公开 swap 函数导致资产被转走等问题。EthanlxI 的测试中出现过 5U 损失，说明套利脚本不是只要能跑就行，安全边界必须先设计。Solana 上出现一笔 Orca -> Meteora 的原子套利复盘：lynch 识别出用 1086.534769 USDC 买入 922.322706 EURC，随后换出 18,361.633282 USDC，净增加约 17,275.098513 USDC。这是一条朴素的两次 swap 路径，不涉及 flashloan 或清算。

群内再次强调 mempool/MEV 竞争现实：Mr.Lai 认为“内存池套利别想了”，差几十毫秒可能就夹不进去；节点、私有内存池、打包服务和机房位置都会影响胜率。研究方法开始更系统化：starkxun 分享了 CEX-DEX extracted value、Aave V2 清算摩擦、MEV 暗森林等论文线索，用来理解 searcher 利润、清算延迟和市场结构。

多链/跨链搬砖被作为案例讨论：幻交精灵 - hstarorg 分享 OKX Web3 上的 BSC 代币页面，并提出“这种 case 应该如何做才能赚到”；这类机会要先确认合约、桥、充提和锁定交易限制。

套利学习方法

先做“黑暗森林式安全检查”：任何 AI 生成的交易代码、swap 参数、合约调用和工具返回，都要默认可能出错或被注入，先用极小金额和本地模拟验证。

复盘原子套利时，不要先迷信复杂术语；先拆成每一步 swap、输入输出、池子、同区块前序交易、滑点设置和异常价差来源。

MEV 学习要把“能理解”和“能上生产”分开。学习阶段可以本机跑节点、复现交易、读论文；实盘阶段需要考虑私有内存池、builder、节点延迟、RPC 稳定性和成本。

研究论文很适合用来找问题地图：CEX-DEX 套利集中度、Aave 清算延迟、searcher 垂直整合，能帮助判断普通玩家适合切入哪里。多链套利不要只问“有没有桥”，还要确认目标代币能否充值/提现、是否可锁定交易、主流桥是否支持、是否有足够流动性和安全退出路径。

套利策略 / 路径 / 案例

Solana DEX-DEX 原子套利案例：Orca 用 USDC 买 EURC，再到 Meteora 换回 USDC，异常价差来自同一区块更早交易。关键复盘点：为何池子价格未及时恢复、是否滑点设置异常、机器人如何发现并排序执行。

MEV 基建路线：Jito 延迟、SOL 节点数量、Blockrazor / bloXroute 私有内存池和打包服务被提到。群内共识偏现实：生产级 MEV 不只是代码，还是网络、节点、私有通道和成本竞争。

多链代币搬砖：针对同名/同合约映射代币，先确认 BSC/EVM 地址、跨链桥、是否开充提、锁定交易限制和主流桥支持情况。若不能提到链上或不能跨链闭环，价差不可执行。

清算研究方向：Aave V2

清算摩擦论文显示，并非所有可清算仓位都会立刻被清算，仍可能存在“下一批清算”研究空间，但需要完整数据拉取和状态分类。

API 数据差异：群内提到 REST 与 WebSocket 数据不完全一样，抓 Binance / OKX 合约数据还会遇到限流和接口质量问题。监控系统要保留多源校验。

工具、数据源与合约线索

Solscan：用于复盘 Solana 原子套利交易，OCR 中出现一条 4V85... 开头交易链接，需截图复核完整哈希。

Orca / Meteora：本日 Solana DEX-DEX 原子套利路径涉及的两个 DEX。

Jito：Solana 交易排序/MEV 相关基础设施线索。

Blockrazor / bloXroute：私有内存池和打包服务线索，群内认为成本较高。

OKX Web3：幻交精灵 - hstarorg 分享 BSC 代币页面，可作为多链代币搬砖案例入口。

REST / WebSocket：行情与合约数据抓取接口，二者数据口径可能不同。

arXiv / 学术论文：starkxun 提到用 Google、arXiv、AI 搜论文，但不能全信，需要回到数据和代码验证。

Aave V2 Main Market：清算摩擦论文的数据对象，适合用于研究清算机会延迟和 stale borrower。

群成员观点摘录

fwangzil：提示词注入和返回内容夹带私货不需要很高技术，工具链本身也可能成为风险源。

R3d-GhOst：返回里多塞点私货就行，不一定需要高技术；这强化了 AI 工具输出审计的重要性。

EthanlxI：测试时 AI 使用公开 swap 函数，导致被其他人转走 5U；也建议小白选定一个策略，用小金额自己实践，亏赚都当作理解流程。

lynch：识别出 Orca -> Meteora 的原子套利，并提醒不能泄气，想法好也可能有机会。

Mr.Lai：认为内存池套利很难，几十毫秒差距都可能夹不进去；学习阶段可自己电脑跑节点，生产环境难度另说。

gjhbus：问到“上生产怎么搞”，触发了节点、RPC、私有服务成本的讨论。

幻交精灵 - hstarorg：分享 OKX Web3 代币页面，并建议把具体 case 当研究对象，分析怎样才能赚到。

starkxun：分享 CEX-DEX、Aave 清算、MEV 暗森林等论文线索，并提醒 arXiv/AI 搜到的资料也不能全信。

小技巧与踩坑提醒

AI 生成交易脚本必须先查权限和资金流向：公开函数、approve、swap 参数、接收地址、滑点和回调都要逐项审。

原子套利不一定复杂。两次 swap 也可能构成原子套利，核心是同一交易/同一区块内完成闭环并锁定利润。

MEV 实盘不要低估基础设施成本：节点、私有通道、打包服务、延迟优化和稳定性都可能比策略代码更贵。

监控接口要交叉验证：REST、WebSocket、交易所页面、链上状态可能不同步，不要单源触发实盘。

多链搬砖最怕“价差能看见，路径走不通”：不能充提、桥不支持、交易被锁、流动性不足都会让价差变成陷阱。

读论文是找地图，不是找圣杯。论文结论要转成可验证数据字段和脚本，再决定是否进入策略池。

待验证问题

Solscan 上 4V85... 交易完整哈希是什么？Orca/Meteora 两个池子的前后价格和同区块前序交易如何变化？

Ethan1x1 测试中 5U 被转走的具体原因是什么：公开 swap 函数、授权问题、接收地址错误，还是交易被抢？

Blockrazor / bloXroute / Jito 的成本、接入方式和普通学习者可用性如何？

OKX Web3 分享的 BSC 代币是否真的存在可跨链搬砖路径？主流桥、充提、锁定交易和池子深度分别如何？

CEX-DEX extracted value 论文中“720 笔套利、19 个主要 searcher、前三占约四分之三”的口径如何，是否能复现实验？

Aave V2 清算摩擦论文里的 stale borrower 是否能转化为可监控清算候选池？

明日学习 / 实操清单

复盘 Solana 原子套利交易：整理交易哈希、两次 swap、池子价格、前序交易、净利润和滑点。

做一张 AI 交易脚本安全检查表：approve、接收地址、公开函数、滑点、回调、私钥/API key、模拟和最小金额测试。

设计多链代币搬砖 checklist：合约地址、桥、充提、锁定状态、池子深度、手续费、滑点、到账时间。

收集并阅读 starkxun 提到的三类论文，先摘出数据字段和可复现实验，不急着写交易逻辑。

对 REST 和 WebSocket 同币种行情做一次对比，记录延迟、盘口差异和异常断流情况。

原始资料索引

raw/2026-08-10_wechat_ocr_combined.md

logs/2026-08-10_微信截图OCR采集日志.md

screenshots/2026-08-10/